

ORACLE METHODSM

APPLICATION IMPLEMENTATION

PROCESS AND TASK REFERENCE

Applications Global Service Line

Volume 2

Release 3.0.0

August, 1999

Part Number A75151-01

ORACLE®

Application Implementation Method (AIM) Process and Task Reference, Volume 2
Release 3.0.0
Part No. A75151-01

Copyright © 1998, 1999, Oracle Corporation. All rights reserved.

Authors: Steve Buchan, Gary Born, Gary Burns, Bruce Dehner, Linda Goossens,
Erik Jarlstrom, Josee Lafortune, Jim Lange, Maurizio Papini, Fred Walker
Contributors: Jay Ashbridge, Julie Corbett, Darron Corfield, Sanjay Narang, Suzanne Wade
Field Reviewers: Ulf Akerbery, John Bard, Rodney Bates, Andrew Clay, Dean Douglas, Dennis Gates,
Jerry Hancock, Steve Machon, Craig Martell, Dennis McDermott,
Lawrence von Bargaen
Project Editor: Theresa Merenkov
Editors: Mie Jarlstrom, Mary Sanders

In memory of a great friend and colleague, Josee, who would not let us forget the silent
constituents — the end users — who must live and work in the environments and
systems we construct and implement.

The Programs (which include both the software and documentation) contain proprietary information of Oracle Corporation; they are provided under a license agreement containing restrictions on use and disclosure and are also protected by copyright, patent, and other intellectual and industrial property laws. Reverse engineering, disassembly, or decompilation of the Programs is prohibited.

The information contained in this document is subject to change without notice. If you find any problems in the documentation, please report them to us in writing. Oracle Corporation does not warrant that this document is error free. Except as may be expressly permitted in your license agreement for these Programs, no part of these Programs may be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose, without the express written permission of Oracle Corporation.

If the Programs are delivered to the U.S. Government or anyone licensing or using the programs on behalf of the U.S. Government, the following notice is applicable:

Restricted Rights Notice Programs delivered subject to the DOD FAR Supplement are "commercial computer software" and use, duplication, and disclosure of the Programs, including documentation, shall be subject to the licensing restrictions set forth in the applicable Oracle license agreement. Otherwise, Programs delivered subject to the Federal Acquisition Regulations are "restricted computer software" and use, duplication, and disclosure of the Programs shall be subject to the restrictions in FAR 52.227-19, Commercial Computer Software - Restricted Rights (June, 1987). Oracle Corporation, 500 Oracle Parkway, Redwood City, CA 94065.

The Programs are not intended for use in any nuclear, aviation, mass transit, medical, or other inherently dangerous applications. It shall be the licensee's responsibility to take all appropriate fail-safe, backup, redundancy, and other measures to ensure the safe use of such applications if the Programs are used for such purposes, and Oracle Corporation disclaims liability for any damages caused by such use of the Programs.

ORACLE, Oracle Designer, Oracle Developer, SQL*Plus, SQL*Loader, SQL*Net, CASE*Method, ORACLE Parallel Server, PL/SQL, Pro*C, SQL*Module are registered trademarks of Oracle Corporation, Redwood City, California.

AIM Advantage, CDM Advantage, EMM Advantage, PJM Advantage, Oracle Cooperative Applications, Oracle Financials, Oracle Alert, Oracle Manufacturing, Oracle Inventory, Oracle Bills of Material, Oracle Engineering, Oracle Capacity, Oracle Commissions, Oracle Master Scheduling, Oracle MRP, Oracle Work in Process, Oracle General Ledger, Oracle Assets, Oracle Order Entry, Oracle Cost Management, Oracle Payables, Oracle Receivables, Oracle Personnel, Oracle Payroll, Oracle Purchasing, Oracle Quality, Oracle Sales and Marketing, Oracle Service, and Application Object Library are trademarks of Oracle Corporation, Redwood City, California.

Oracle is a registered trademark, and Oracle Method is a service mark of Oracle Corporation. Microsoft and MS-DOS are registered trademarks and Windows, Word for Windows, PowerPoint, Excel, and Microsoft Project are trademarks of Microsoft Corporation. Visio is a trademark of Visio Corporation. Project Workbench and Project Bridge Modeler are registered trademarks of Applied Business Technology. All other company or product names mentioned are used for identification purposes only and may be trademarks of their respective owners.



Preface

The Application Implementation Method *Process and Task Reference* provides detailed descriptions of the tasks and deliverables of the Application Implementation Method (AIM). AIM defines a set of organized and flexible process steps that guide a project team through the Application Implementation process.

The material in this reference includes a description of AIM tasks and deliverables. In addition, guidelines on prerequisites; approach and techniques; roles and responsibilities; deliverable components; component descriptions; audience, distribution, and usage; completion criteria; and deliverable template information are provided. This reference is provided to assist in expediting and standardizing all of the tasks executed and deliverables produced in an Application Implementation project.

This reference, and the Application Implementation Method itself, are part of Oracle MethodSM—Oracle’s integrated approach to technical and functional delivery.

Audience

The Application Implementation Method *Process and Task Reference* is written for implementers, developers, team leaders, project team members and project leaders. Implementers, project team members, and team leaders use this reference in detail to execute tasks and create deliverables in AIM. Team leaders and project leaders use this reference as an overview to better understand the nature of tasks and deliverables in order to better manage the execution of tasks and creation and completion of deliverables.

How the Reference Is Organized

This reference consists of an introduction followed by a chapter on each of the processes of the Application Implementation Method. Due to the size of this reference, it is divided into three volumes. Volume 1 contains chapters 1 through 4; volume 2 contains chapters 5 through 8; and volume 3 contains chapters 9 through 11 and the appendices.

Introduction: The introduction presents a brief overview of processes and phases. It also provides conventions used within the reference.

Process Chapters: Each process chapter consists of two main sections: an overview, and detailed task guidelines. The overview section provides the following for each process:

- **Process Flow** — depicts the flow of tasks for this process
- **Approach** — describes the approach for this process
- **Tasks and Deliverables** — lists the tasks executed and the deliverables produced. This table also provides the responsible role and the type of task. Task type definitions can be found in the Glossary at the end of the Application Implementation Method Handbook. Task types are:
 - SI — singly instantiated task (standard task)
 - MI — multiply instantiated task
 - MO — multiply occurring task

- IT — iterated task
- O — ongoing task
- **Objectives** — describes the objectives of the process
- **Deliverables** — lists and defines the deliverables for the process
- **Key Responsibilities** — lists and defines the key roles for the process
- **Critical Success Factors** — lists the success factors for the process
- **References and Publications** — lists any references and publications for the process

The task detail section provides the following for each task:

- **Optional Criteria** — where applicable, criteria specifying under what conditions the task, or some of its task steps, should be executed
- **Prerequisites** — lists the prerequisites for the task and their source
- **Task Steps** — lists the steps of the task
- **Approach and Techniques** — describes the approach and suggested techniques for the task
- **Linkage to Other Tasks** — identifies which downstream tasks use the deliverable produced as input
- **Role Contribution** — provides the role contributions for the task
- **Deliverable Guidelines** — lists the guidelines and deliverable components for the task and describes each section of the deliverable document
- **Tools** — lists the template and tools to use when creating the deliverable for each task

Appendix A: This appendix provides an index which cross-references the AIM tasks in version 2.0 and version 3.0.

Appendix B: Appendix B provides a description of roles used in the Application Implementation Method.

How to Use this Reference

The *Application Implementation Method Process and Task Reference* provides the detailed task and deliverable information that makes up the Application Implementation Method. Each task produces a deliverable and these deliverables are described in this reference. Each task has an identifier (task ID) which makes it easy to locate. The task ID corresponds directly to the deliverable ID of the task that creates the deliverable.

All users of this reference should read the Introduction to better understand tasks, deliverables, and processes.

Oracle recommends that users of all of the AIM handbooks, and the Application Implementation Method itself, take advantage of AIM training courses provided by Oracle University. In addition to the Application Implementation Method handbooks and training, Oracle Consulting or one of Oracle's Implementation Services Partners can also provide experienced AIM consultants, automated work management tools customized for AIM, and Application Implementation Method deliverable templates.

Conventions Used in this Manual

We use several notational conventions to make this reference easy to read.

Capitalization

Names of tasks, deliverables, process and phases are always give title capitals. For example, Design Audit Facilities task, System Data Model deliverable, Technical Architecture process and Build phase.

Abbreviations and Acronyms

Occasionally, it is necessary to use abbreviations and acronyms when adequate space is not available. The Glossary lists meanings of all acronyms and abbreviations.

UPPERCASE TEXT

Uppercase text is used to call attention to command keywords, object names, filenames, and so on.

Italicized Text

Italicized text indicates the definition of a term or the title of a manual.

Bold Text

Bold text is designed to attract special attention to important information.

Attention

We sometimes highlight especially important information or considerations to save you time or simplify the task at hand. We mark such information with an attention graphic, as follows:



Attention: Since project team training occurs simultaneously with this task, some recommendations (or decisions) from training may be implemented in the mapping environment. In this case, these training inputs become predecessors to this task.

For More Information

Throughout the reference we alert you to additional information you may want to review. This may be a task section, appendix, manual reference, or web site. We highlight these references with an easy-to-notice graphic. Here is an example:



Reference: For more information about content for the Solution Design presentation, review the Critical Success Factors, page 3-7.



Web Site: You can find further information on Oracle's Home Web Page <http://www.oracle.com/>

Suggestions

We provide you with helpful suggestions throughout the reference to help you get the most out of the method. We highlight these suggestions with an illuminated light bulb. Here is an example of a suggestion:



Suggestion: Verify your backup and recovery plan with your hardware and software vendors.

Warning

Considerations that can have a serious impact on your project are highlighted by a warning graphic. Read each warning message and determine if it applies to your project. Here is an example:



Warning: Any time you insert data directly into Oracle Application tables, you run the risk of corrupting the database. Oracle strongly discourages inserting data directly into Oracle tables that are not designed as an Open Interface.

Optional Criteria

Where applicable, optional criteria specifying under what conditions the task, or some of its task steps should be executed is highlighted by a delta symbol graphic. Here is an example:



If your project includes *process change*, you should perform this task.

Related Publications

Books in the Application Implementation Method suite include:

- *AIM Method Handbook*
- *AIM Process and Task Reference*, volume 1
- *AIM Process and Task Reference*, volume 2 (this book)
- *AIM Process and Task Reference*, volume 3
- *AIM FastForward Add-In Handbook*

You may also refer to the following Project Management Method (PJM) suite of reference books:

- *PJM Method Handbook*
- *PJM Process and Task Reference*

Obtaining Additional AIM Advantage Licenses

Each key member of your project team should have a licensed copy of AIM Advantage installed on their workstation. Key members are those individuals who will be leading areas of the implementation project and generating key deliverables.

Oracle provides AIM Advantage licenses to select Oracle Applications Implementation Partners and Oracle Consulting for their project staff. Customers should also equip their key project team personnel with licensed copies of AIM Advantage. This facilitates improved understanding of the implementation process by all team members, as well as improved overall cross-project team productivity.

Additional copies of AIM Advantage may be obtained from the Oracle Direct Marketing or telesales group in your country, or you can contact your local Oracle sales representative.

EMM Advantage and Oracle Application Upgrades

EasiPath Migration Method (EMM Advantage), Oracle's packaged methodology for application upgrades, is complementary to the Applications life-cycle of installation and subsequent upgrades. Produced by Oracle Corporation, EMM Advantage is available to help you structure and manage your Oracle Applications upgrade project.

EMM Advantage includes:

- EasiPath Migration Method (EMM) — a proven, structured approach used successfully worldwide by Oracle consultants
- Project Management Method (PJM) — a standardized Oracle approach to project management

The EMM Advantage toolkit, in combination with your skills, experience, and business knowledge, helps promote a higher-quality migration and lead you to business results faster. It is available from the Oracle Direct Marketing or telesales group in your country, or you can contact your local Oracle sales representative.

Your Comments are Welcome

Oracle Corporation values and appreciates your comments as an Oracle AIM user and reader of the reference. As we write, revise, and evaluate our documentation, your comments are the most valuable input we receive. If you would like to contact us regarding comments on this or other Oracle Method manuals, please use the following address:

email: aiminfo.us@us.oracle.com

Contents

	Introduction	xix
	What is a Task?	xx
	What is a Deliverable?.....	xxiii
	What is a Process?	xxvii
CHAPTER 1	Business Process Architecture (BP).....	1-1
	BP.010 - Define Business and Process Strategy (Optional)	1-17
	BP.020 - Catalog and Analyze Potential Changes (Optional)	1-27
	BP.030 - Determine Data Gathering Requirements (Optional).....	1-31
	BP.040 - Develop Current Process Model (Optional)	1-36
	BP.050 - Review Leading Practices (Optional).....	1-44
	BP.060 - Develop High-Level Process Vision (Optional).....	1-50
	BP.070 - Develop High-Level Process Designs (Core).....	1-57
	BP.080 - Develop Future Process Model (Core).....	1-62
	BP.090 - Document Business Procedures (Core).....	1-81

CHAPTER 2	Business Requirements Definition (RD)	2-1
	RD.010 - Identify Current Financial and Operating Structure (Core).....	2-9
	RD.020 - Conduct Current Business Baseline (Core).....	2-17
	RD.030 - Establish Process and Mapping Summary (Optional).....	2-29
	RD.040 - Gather Business Volumes and Metrics (Core).....	2-36
	RD.050 - Gather Business Requirements (Core)	2-43
	RD.060 - Determine Audit and Control Requirements (Core)	2-56
	RD.070 - Identify Business Availability Requirements (Core).....	2-66
	RD.080 - Identify Reporting and Information Access Requirements (Core).....	2-73
CHAPTER 3	Business Requirements Mapping (BR)	3-1
	BR.010 - Analyze High-Level Gaps (Core).....	3-14
	BR.020 - Prepare Mapping Environment (Core).....	3-21
	BR.030 - Map Business Requirements (Core).....	3-28
	BR.040 - Map Business Data (Core)	3-49
	BR.050 - Conduct Integration Fit Analysis (Optional).....	3-55
	BR.060 - Create Information Model (Optional).....	3-63
	BR.070 - Conduct Reporting Fit Analysis (Core)	3-76
	BR.080 - Test Business Solutions (Optional).....	3-83
	BR.090 - Confirm Integrated Business Solutions (Core).....	3-96
	BR.100 - Define Application Setups (Core)	3-101
	BR.110 - Design Security Profiles (Core)	3-107

CHAPTER 4	Application and Technical Architecture (TA)	4-1
	TA.010 - Define Architecture Requirements and Strategy (Core)	4-22
	TA.020 - Identify Current Technical Architecture (Optional).....	4-39
	TA.030 - Develop Preliminary Conceptual Architecture (Optional).....	4-49
	TA.040 - Define Application Architecture (Optional)	4-64
	TA.050 - Define System Availability Strategy (Core).....	4-81
	TA.060 - Define Reporting and Information Access Strategy (Optional).....	4-94
	TA.070 - Revise Conceptual Architecture (Optional).....	4-111
	TA.080 - Define Application Security Architecture (Optional)	4-118
	TA.090 - Define Application and Database Server Architecture (Optional).....	4-129
	TA.100 - Define and Propose Architecture Subsystems (Optional)	4-147
	TA.110 - Define System Capacity Plan (Core).....	4-159
	TA.120 - Define Platform and Network Architecture (Core).....	4-181
	TA.130 - Define Application Deployment Plan (Optional).....	4-190
	TA.140 - Assess Performance Risks (Core).....	4-198
	TA.150 - Define System Management Procedures (Core).....	4-207
CHAPTER 5	Module Design and Build (MD)	5-1
	MD.010 - Define Application Extension Strategy (Optional).....	5-19
	MD.020 - Define and Estimate Application Extensions (Optional).....	5-30

MD.030 - Define Design Standards (Optional)	5-47
MD.040 - Define Build Standards (Optional)	5-57
MD.050 - Create Application Extensions Functional Design (Optional)	5-67
MD.060 - Design Database Extensions (Optional)	5-76
MD.070 - Create Application Extensions Technical Design (Optional)	5-85
MD.080 - Review Functional and Technical Designs (Optional)	5-93
MD.090 - Prepare Development Environment (Optional)	5-101
MD.100 - Create Database Extensions (Optional)	5-108
MD.110 - Create Application Extension Modules (Optional)	5-112
MD.120 - Create Installation Routines (Optional)	5-121

CHAPTER 6 Data Conversion (CV) 6-1

CV.010 - Define Data Conversion Requirements and Strategy (Optional)	6-16
CV.020 - Define Conversion Standards (Optional)	6-30
CV.030 - Prepare Conversion Environment (Optional)	6-37
CV.040 - Perform Conversion Data Mapping (Optional)	6-42
CV.050 - Define Manual Conversion Procedures (Optional)	6-51
CV.060 - Design Conversion Programs (Optional)	6-59
CV.070 - Prepare Conversion Test Plans (Optional)	6-73
CV.080 - Develop Conversion Programs (Optional)	6-81
CV.090 - Perform Conversion Unit Tests (Optional)	6-89

	CV.100 - Perform Conversion Business Object Tests (Optional).....	6-93
	CV.110 - Perform Conversion Validation Tests (Optional).....	6-97
	CV.120 - Install Conversion Programs (Optional)	6-101
	CV.130 - Convert and Verify Data (Optional).....	6-106
CHAPTER 7	Documentation (DO)	7-1
	DO.010 - Define Documentation Requirements and Strategy (Core)	7-13
	DO.020 - Define Documentation Standards and Procedures (Optional).....	7-29
	DO.030 - Prepare Glossary (Core).....	7-38
	DO.040 - Prepare Documentation Environment (Optional)	7-42
	DO.050 - Produce Documentation Prototypes and Templates (Optional)	7-49
	DO.060 - Publish User Reference Manual (Optional).....	7-57
	DO.070 - Publish User Guide (Optional).....	7-65
	DO.080 - Publish Technical Reference Manual (Optional).....	7-74
	DO.090 - Publish System Management Guide (Optional).....	7-82
CHAPTER 8	Business System Testing (TE).....	8-1
	TE.010 - Define Testing Requirements and Strategy (Core)	8-17
	TE.020 - Develop Unit Test Script (Optional).....	8-34
	TE.030 - Develop Link Test Script (Optional)	8-43
	TE.040 - Develop System Test Script (Core).....	8-52
	TE.050 - Develop Systems Integration Test Script (Optional).....	8-61

TE.060 - Prepare Testing Environments (Core)	8-68
TE.070 - Perform Unit Test (Optional).....	8-77
TE.080 - Perform Link Test (Optional)	8-86
TE.090 - Perform Installation Test (Optional)	8-93
TE.100 - Prepare Key Users for Testing (Core).....	8-99
TE.110 - Perform System Test (Core).....	8-105
TE.120 - Perform Systems Integration Test (Optional).....	8-119
TE.130 - Perform Acceptance Test (Core)	8-129

CHAPTER 9 Performance Testing (PT)..... 9-1

PT.010 - Define Performance Testing Strategy (Optional)	9-25
PT.020 - Identify Performance Test Scenarios (Optional)	9-38
PT.030 - Identify Performance Test Transaction Models (Optional)	9-51
PT.040 - Create Performance Test Scripts (Optional).....	9-61
PT.050 - Design Performance Test Transaction Programs (Optional)	9-67
PT.060 - Design Performance Test Data (Optional).....	9-73
PT.070 - Design Test Database Load Programs (Optional).....	9-84
PT.080 - Create Performance Test Transaction Programs (Optional)	9-90
PT.090 - Create Test Database Load Programs (Optional)	9-95
PT.100 - Construct Performance Test Database (Optional)	9-100
PT.110 - Prepare Performance Test Environment (Optional)	9-105
PT.120 - Execute Performance Test (Optional)	9-117

	PT.130 - Create Performance Test Report (Optional)	9-124
CHAPTER 10	Adoption and Learning (AP)	10-1
	AP.010 - Define Executive Project Strategy (Optional)	10-22
	AP.020 - Conduct Initial Project Team Orientation (Core)	10-30
	AP.030 - Develop Project Team Learning Plan (Core).....	10-37
	AP.040 - Prepare Project Team Learning Environment (Core).....	10-44
	AP.050 - Conduct Project Team Learning Events (Core)	10-49
	AP.060 - Develop Business Unit Managers' Readiness Plan (Optional).....	10-55
	AP.070 - Develop Project Readiness Roadmap (Core)	10-62
	AP.080 - Develop and Execute Communication Campaign (Core)	10-71
	AP.090 - Develop Managers' Readiness Plan (Optional)	10-78
	AP.100 - Identify Business Process Impact on Organization (Optional).....	10-88
	AP.110 - Align Human Performance Support Systems (Optional)	10-93
	AP.120 - Align Information Technology Groups (Optional).....	10-105
	AP.130 - Conduct User Learning Needs Analysis (Optional).....	10-112
	AP.140 - Develop User Learning Plan (Core).....	10-117
	AP.150 - Develop User Learningware (Optional).....	10-127
	AP.160 - Prepare User Learning Environment (Core).....	10-133
	AP.170 - Conduct User Learning Events (Core)	10-139
	AP.180 - Conduct Effectiveness Assessment (Core).....	10-146

CHAPTER 11	Production Migration (PM).....	11-1
	PM.010 - Define Transition Strategy (Core)	11-15
	PM.020 - Design Production Support Infrastructure (Core).....	11-22
	PM.030 - Develop Transition and Contingency Plan (Core).....	11-32
	PM.040 - Prepare Production Environment (Core)	11-43
	PM.050 - Set Up Applications (Core).....	11-50
	PM.060 - Implement Production Support Infrastructure (Core)	11-57
	PM.070 - Verify Production Readiness (Core)	11-61
	PM.080 - Begin Production (Core)	11-70
	PM.090 - Measure System Performance (Optional).....	11-74
	PM.100 - Maintain System (Core)	11-82
	PM.110 - Refine Production System (Optional)	11-89
	PM.120 - Decommission Former Systems (Optional).....	11-94
	PM.130 - Propose Future Business Direction (Optional).....	11-100
	PM.140 - Propose Future Technical Direction (Optional).....	11-106
APPENDIX A	Task Cross-Reference	A-1
	AIM Version 2.0 Tasks Cross-Referenced with AIM Version 3.0 Tasks	A-2
	AIM Version 3.0 Tasks Cross-Referenced with AIM Version 2.0 Tasks	A-10

APPENDIX	B	AIM Roles.....	B-1
		Role Descriptions.....	B-2
		Glossary	



Introduction

Processes, tasks, and deliverables are the basis of Oracle's Application Implementation Method. They are the building blocks from which project managers construct Application Implementation projects. The *AIM Process and Task Reference* provides the details of every process that plays a part in the Application Implementation Method and every task included in each process. As an introduction, this section provides a brief overview of the concepts of *process*, *task*, and *deliverable*, and describes the information in this reference that is given for each.

What is a Task?

A **task** is a unit of work that results in the output of a single deliverable. Tasks are the most elementary unit of work that one would put into a project plan — they provide the basis of the work breakdown structure. A task produces a single, measurable deliverable and is usually assigned to be the responsibility of a single team member (although many others may play contribution, review, and approval roles). Project progress is usually measured by the successful completion of tasks.

Task IDs and Names

Each task in AIM has a unique ID. That ID is composed of the alphabetical ID of the process in which the task plays a part (more on this later), followed by a three digit sequence number, that usually increments by 10. The sequence number generally reflects the order in which tasks are to be completed. For example, you can be reasonably sure that task BP.020 will be completed well in advance of BP.090 within the Business Process Architecture (BP) process.

Note that the task ID does not reflect the strict order or phase in which the task occurs within a project. A project manager may combine tasks and processes in different ways to meet the needs of different development approaches. Therefore tasks may have different sequences, relative to each other, in different types of Application Implementation projects.

Each task also has a unique name. A task name always consists of a verb (such as *create...*, *determine...*, *design...*), followed by an object. In most cases the object is the name of the deliverable that the task produces. You will find that the text always refers to task names with title case letters, for example, Develop Current Process Model (BP.040).

Task Information

Each task in AIM has a task guideline. If you know a task's ID, it is easy to find the guideline for that task in the *AIM Process and Task Reference*. Locate the process chapter by the first part of the ID, and then locate the

task within the chapter by using the numerical sequence number part of the task ID.

If you only know the name of a task, you can use Appendix A to find the ID. Appendix A contains an alphabetical listing of tasks by task name. It also contains an alphabetical listing of tasks by deliverable name.

Optional Criteria

Tasks are divided into two types — Core and Optional. A core task must occur on every implementation. An example of a core task is Setup Applications (PM.050). An optional task is performed only if the project requirements mandate its use. An example of an optional task is Design Database Extensions (MD.060), which only needs to occur on projects where application extensions that drive database changes will be developed.

Many of the tasks in the *AIM Process and Task Reference* have criteria that define when the task or some of the task steps should be executed. The optional criteria, where applicable, is located just below the task description. In the case of optional task steps, the delta symbol (Δ) will appear to the left of the task step ID.

Prerequisites

Each task assumes that certain things (such as information, programs, and hardware platforms) have been previously produced or compiled, and are available for use. In most cases, these **prerequisites** of the task are specific deliverables of previous tasks. In some cases, they are expected from the client business organization.

Each task guideline lists that task's prerequisites. Under each prerequisite you will find an indication of which components or specific information within the prerequisite is used in the task. The text also indicates how you will use that component or information when carrying out the task.

Task Steps

Many tasks may be broken down into smaller units of work called **task steps**. In some cases, a task guideline may indicate a suggested set of task steps to follow. Many times, the team member responsible for the task (the “owner” of the task) will want to specify the task steps. The task owner will want to base those steps upon techniques that are appropriate to the overall development approach and the tools and resources that are available to the project. Any set of task steps that reaches the deliverable is acceptable as long as it includes adequate quality assurance steps.

From time to time, the reader will see a task step that has a delta symbol to the right of the task step ID. This indicates that the task step may be optional. The reader should consult the task optional criteria located just below the task description for advice regarding when a particular task step may be optional. If no delta symbol is present, then the task is assumed to be recommended as mandatory.

Role Contribution

In addition to the task owner, many other project team members may spend time on a task. Each of these team members will be fulfilling a particular **role**. Their responsibilities may be to contribute information, analyze, document, set up, review, administer, or control. The effort they spend on the task may be significant or nominal.

Each task guideline provides a suggested list of roles that may play a part in completing the task. Next to each role is an indication of the percentage of the total task effort that that role may contribute. These are suggestions that can be used in planning — actual role contributions will depend on the task steps that the task owner assigns.

What is a Deliverable?

AIM is a deliverable-based method. This means that each task produces one or more deliverables, or output, whose quality you can measure. Deliverables can have many formats, such as documents, schedules, program code, or test results.

Each deliverable in AIM is **recognizable** (it has a specific name and ID) and **measurable**. Each deliverable has the same ID as its corresponding task. Each deliverable also has a unique name, which the text always refers to using title case letters. An example is the Current Process Model (BP.040). If you know the name of a deliverable, you can find its ID, and the name of its corresponding task, by using Appendix A.

An AIM deliverable can be further broken down into smaller units called **deliverable components**. For example, the deliverable Current Process Model (BP.040) contains the following deliverable components:

- Enterprise Process Model
- Core Process Models
- Process Flow Diagrams
- Process Step Catalog
- Event Catalog
- Performance Measures

A **deliverable definition** is a specification of the content, organization, and usage of the product from an AIM task. This reference attempts to specify a standard set of deliverable definitions for the Application Implementation Method.

Dependencies between Application Implementation Method tasks are based on AIM deliverables. Each task requires specific deliverables from prior tasks before it can commence. A single task may impact many subsequent tasks in the current and future phases, based on the deliverable it produces.

The *AIM Process and Task Reference* lists the prerequisite deliverables for each task in the task guidelines, and also indicates this information on the process flow diagram at the start of each chapter. The *AIM Method Handbook* does the same in the diagram at the start of each phase chapter. This makes it easy for a project team member or manager to

verify which deliverables must be collected as input to a particular task, before that task can be started.

Project Deliverables

You identify your deliverables using the project workplan. Each task in the project workplan should produce a single unique deliverable. As you tailor the tasks in your workplan, you need to tailor your deliverables as well.

When you begin preparing the project workplan using AIM routes (work breakdown structures), each Application Implementation Method task initially refers to the name of its corresponding deliverable. As you create or revise the tasks in your workplan, make sure that your deliverable names are unique and meaningful. For example, if you create a separate instance of an AIM task for multiple project teams, you would append a qualifier, such as the team name, to the deliverable name for each new task as well.

Project tasks and dependencies can also be tailored based on the prior availability of project deliverables. If a deliverable is already available prior to a project or phase, the task that normally produces it can be reduced to a “review” task. In some cases, it can be eliminated.

Deliverable Review

The production of deliverables is a way to measure the progress of a project. You verify successful completion of a deliverable by performing a quality review. The quality review should be based on the quality criteria specified for the AIM deliverable definition in this reference. You can also establish alternate or additional criteria, such as those required by the business management. Whatever the case, make sure that the completion criteria for each deliverable are clearly understood by the entire project staff.

Deliverable Documentation

Project deliverables can take many formats. Paper and electronic formats are the most common, but other formats include computer hardware and software (for example, System Test Environment), and

even human beings (for example, Skilled Users). In many cases, you will want to produce not only the project deliverable itself, but also a record or representation of that deliverable that may be easier to review, record, and signoff. For example, in addition to producing the actual Skilled Users deliverable, document the learning events that were actually attended by each student, including an indication of which users were not prepared as expected.

You should keep in mind that in many cases, the document only **represents** the project deliverable, or only documents the parts or aspects of the deliverable that are most relevant to communicate. Much more information can often be required to actually meet the quality criteria of the deliverable. In some cases you may not need to produce a document at all. The production of the document alone should not be the goal.

Deliverable Control

You should determine the level of control of each project deliverable as either **controlled** or **uncontrolled**. You control a deliverable and its corresponding documentation in order to protect its integrity and manage its approval and revision.

As a rule, every key deliverable of the project should be controlled. You control the content of the deliverable using configuration control procedures, to restrict access and changes to the deliverable to only those authorized. You also track each version of the deliverable over time and reconstruct any previous version as needed.

You control documentation for a deliverable using document control procedures, to define how documents are prepared, approved and distributed. A controlled document is assigned a version number and its date and distribution list is clearly indicated. You may also want to number each copy of the document with a copy number. As authorized changes are made to the contents of the document, new versions are periodically created and sent out to the original distribution list (at a minimum). You also include a log of changes within each version. If you had numbered copies, you may also want to request that superseded copies be returned.

A deliverable may be uncontrolled because it will not be updated or will be shortly replaced by a controlled deliverable. Changes to an uncontrolled deliverable would not go through the project's configuration control process. However, you should still review the

deliverable for quality and retain a copy of the deliverable in the project's repository.

In the case of uncontrolled documents, such as meeting minutes or memos, document control requirements can be reduced. You will not need to associate a version number with an uncontrolled document. You may not even need to formally distribute it, although a copy should always be retained in the project library.



For more information on configuration control and document control, see the *Project Management Method Process and Task Reference*.

What is a Process?

Major project objectives are achieved by processes. A **process** is a series of **tasks** that results in one or more critical project deliverables. The tasks in a process are usually highly dependent upon one another, and often span much of the project. For example, the Data Conversion process begins early in the development life cycle by defining the scope of the conversion project. This is followed by designing and building the programs and tools for conversion. After testing, the data is converted and available for production.

Figure I-1 shows the processes that are a part of AIM and their relative durations.

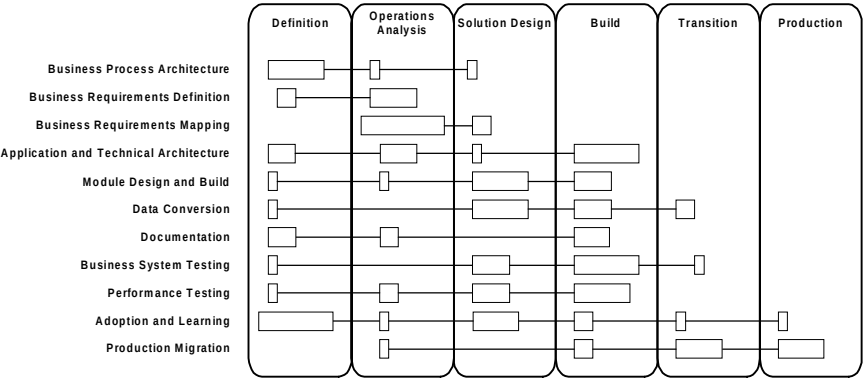


Figure I-1 AIM Context

Process Guidelines

Each chapter of the AIM Process and Task Reference is devoted to a single process. The first part of each chapter gives guidelines on the process as a whole. It shows the relationships of tasks within the process, lists the critical deliverables of the process, and provides guidance on the skills needed to execute the process.

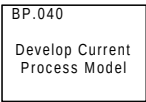
The process guidelines do not indicate exactly where each task falls in the project task plan, since this may vary by the development approach chosen. For more information on choosing and structuring a development approach using Application Implementation Method processes, see the AIM *Method Handbook*.

Process Flow Diagrams

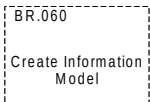
Each process is represented by a process flow diagram. This diagram shows the tasks that are part of the process, and the dependencies between those tasks. It also shows the key deliverables of the process, and the roles that are responsible for each task. The “Approach” section that follows explains the reason behind the organization of the tasks in the process flow diagram.

One thing to keep in mind is that a process flow diagram may indicate that two tasks are strictly dependent upon one another (task “B” may not begin until task “A” has completed) when, in fact, the two tasks will most likely overlap in real life. An example is in the Documentation process. The task Produce Documentation Prototypes and Templates (DO.50) is a predecessor task to Publish User Reference Manual (DO.060). Ideally, all problems would be analyzed before making any changes to the application so that changes to the modules could be made efficiently. However, this is rarely the case in the course of a project, where schedule demands usually require that application changes be made as soon as possible.

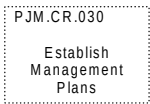
The following describes the symbols that are used in the process flow diagrams.



Core Task. This shows a core task that is contained in AIM. The text provides the AIM ID and the task name.



Optional Task. This shows an optional task that is contained in AIM. The text provides the AIM ID and the task name.



External Task. This symbol depicts a task that should be performed, but is not contained in the same process.



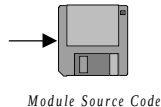
Decision. A decision indicates that there are two or more possible branches to the process flow, depending upon the outcome of the stated question. A decision symbol does not indicate another task -- all work that is done in order to indicate a particular outcome is included in previous tasks.



Process Flow. An arrow between two tasks signifies that the task at the end of the arrow generally should not start until the previous task has been completed. Example: you should not start the task *Map Business Requirements (BR.030)* until you have finished the task, *Gather Business Requirements (RD.050)*. In some cases, it may be desirable to overlap such tasks in an actual project plan.

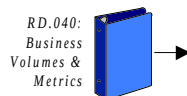


Key Document Deliverable. This represents an important textual output of the process. It includes the name of the deliverable.



Key Software Deliverable. This represents an important software product of the process.

Other symbols are used for key deliverables, as appropriate.



Required Prerequisite. This represents a key input deliverable for a task. The name of the deliverable and the ID of the task that produces the deliverable are given. Different symbols may be used to represent the medium of the deliverable.



Agent Role. Each diagram is divided horizontally into sections or *agent channels*. Each agent channel is labeled with a role. All tasks within that section are the responsibility of that project role.



Attention: Required Prerequisites and Deliverables do not correspond to the agent channel in which they are drawn.

Module Design and Build (MD)

This chapter describes the Module Design and Build process.

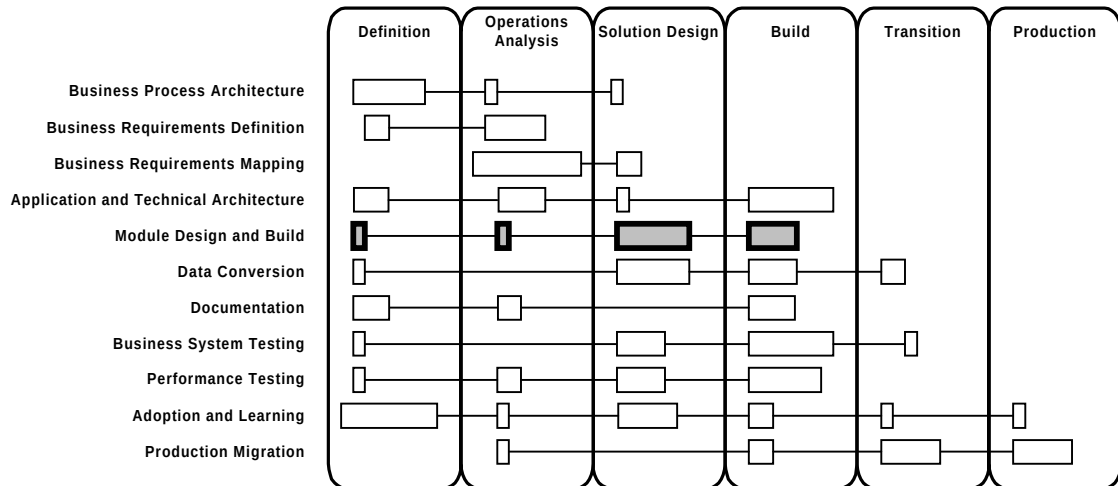


Figure 5-1 Module Design and Build Context

Process Flow

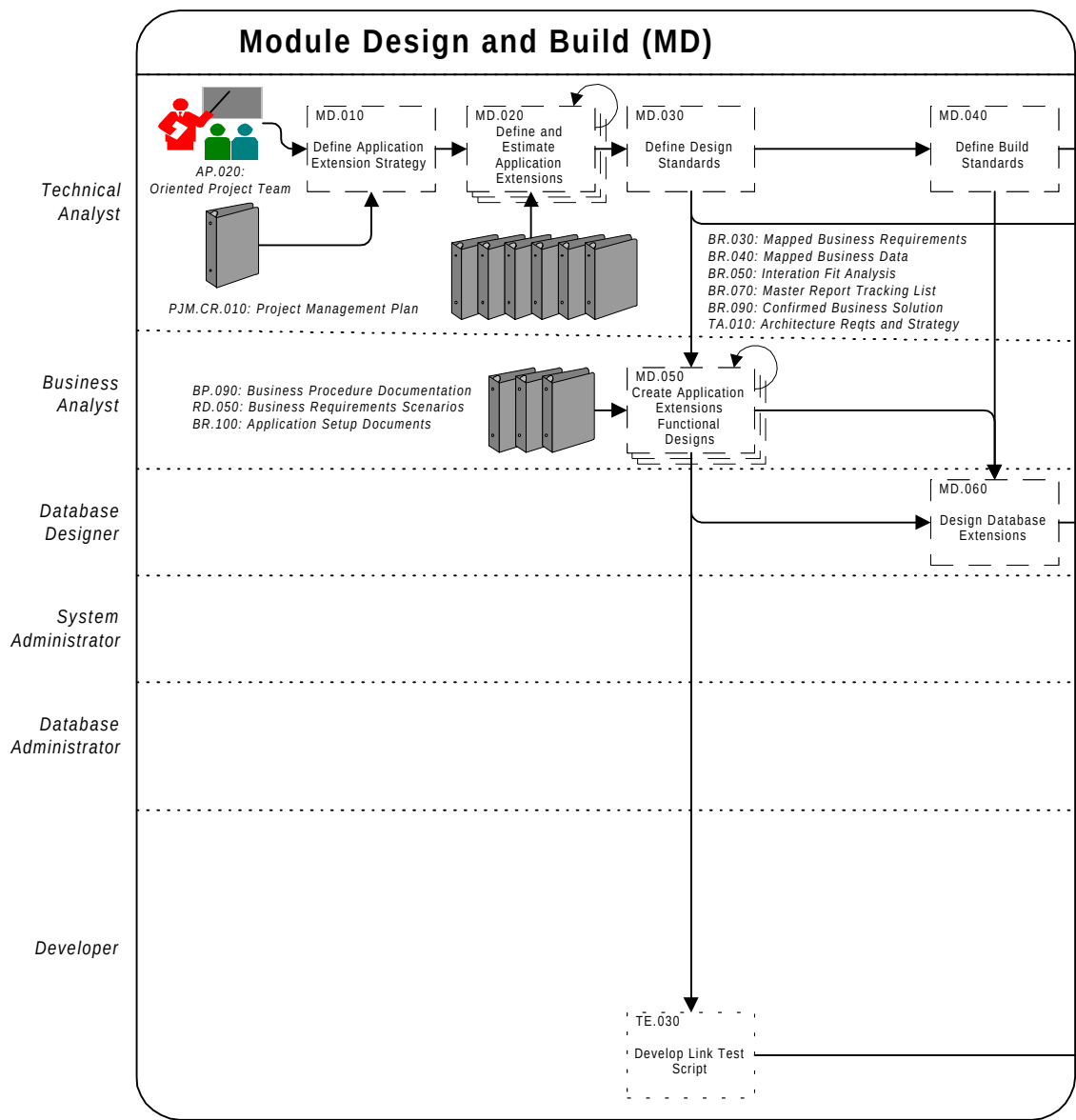


Figure 5-2 Module Design and Build Process Flow Diagram

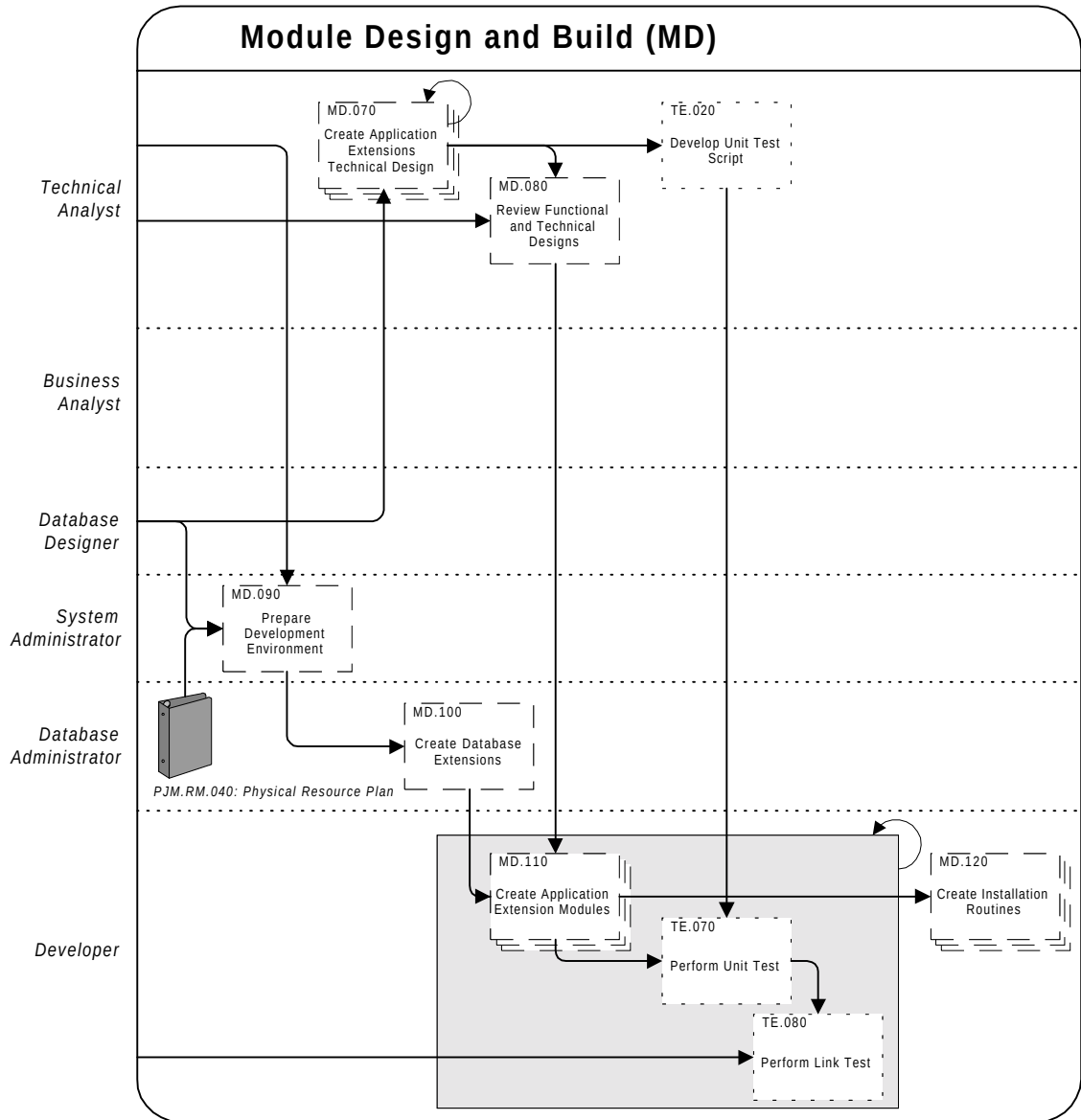


Figure 5-2 Module Design and Build Process Flow Diagram (cont.)

Approach

The objective of Module Design and Build is to focus on the design and development of customizations to satisfy functionality gaps identified during Business Requirements Mapping (BR). There are three approaches to extending the functionality of the applications:

- **Modification** — changes to the base Oracle Applications code
- **Extension** — new forms, reports, programs, tables, interfaces and triggers that add functionality without changing the base application code
- **Configurable Extension** — addition of functionality through flexfields, alerts, and other configuration options provided by the Applications

Module Design and Build tasks are only required if the project team identifies gaps that cannot be satisfied with an acceptable combination of application features, manual steps, and procedural changes. Many projects begin with the goal of using the applications in their *vanilla* configuration, with no customizations. However, even configurable extensions, such as flexfields and alerts should be designed, implemented, and tested with the same rigor as other customizations.



Attention: The terms *customization* and *application extension* are used interchangeably to refer to a custom program approach to a business requirement. However, a customization may have many components and each is referred to as a *module*.

Strategy Selection

An appropriate customization strategy would normally be defined during the generation of the Project Management Plan (PJM.CR.010) and communicated to the project team. The acceptable level of customization and the associated design constraints often affect mapping decisions.

First decide whether to consider customization. Prohibiting customizations produces the fastest and lowest cost implementation, however, this may force you to realign your business policies and practices to fit the applications, more than you may desire. Permitting or encouraging customizations leads to a longer and more expensive

implementation, but may give users exactly what they want. Unfortunately, *any* customizations increase the effort each time you upgrade the applications (including patches), as the customizations sometimes need to be reengineered each time.

If customization is allowed (or you discover that you ultimately require it), you must select the tools, define the process, and implement appropriate control mechanisms.

Functionality Gaps

Each functionality gap the project team identifies during Business Requirements Mapping (BR) represents a potential customization. The sequence of steps that takes you from requirements to completed customizations are as follows:

1. The project team documents requirements in the form of Business Requirements Scenarios (RD.050) and identifies requirements that are not fully supported by the Applications (gaps).
2. For each gap, the mapping team prepares a Business Requirements Mapping Form (BR.030).
3. Business and technical analysts consider several alternatives to address each issue and document the best approaches in Application Extension Definition and Estimates (MD.020). Some of those approaches may require customizations. Requirements for custom reports and interfaces are also consolidated in Application Extension Definition and Estimates.
4. Management, users, and the project team review the proposed customizations and approve those that represent significant benefit at a reasonable cost.
5. A database designer creates an integrated Database Extensions Design (MD.060) to document the database extensions required to support all customizations.
6. Technical analysts write an Application Extensions Functional Design (MD.050) and an Application Extensions Technical Design (MD.070) for each customization. One customization may include multiple modules.
7. Technical analysts create a Link Test Script (TE.030) for each customization and a Unit Test Script (TE.020) for each module.

8. Developers create Module Source Code (MD.110) for each module that comprises a customization.
9. Testers perform a unit test (TE.070) and a link test (TE.080) on the custom modules.

The figure below shows the deliverables that support the identification, specification, construction, and testing of customizations and how they relate to one another.

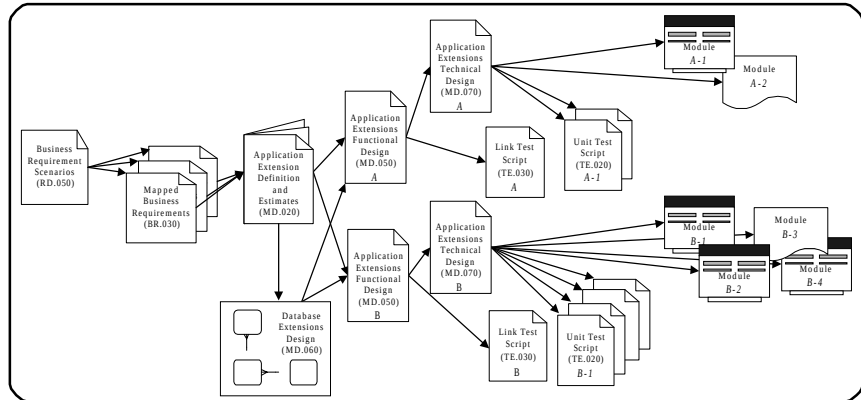


Figure 5-3 Deliverables Supporting Two Sample Customizations A and B

Oracle Designer

Computer Aided Software Engineering (CASE) tools like Oracle Designer can both simplify and complicate the process of designing and building customizations. If you have many customizations or complex requirements, a CASE tool provides a shared repository of information that is easy to modify as requirements change. If you have very few customizations, you may not be able to justify the additional software costs, learning expenses, and administrative overhead required to use CASE tools productively. However, you can use Oracle Designer to facilitate the customization design and development in numerous ways:

- Take advantage of Oracle Designer's shared repository for all design information.
- Generate a large portion of the technical design as reports from the Oracle Designer database.
- Design an integrated data model showing both standard and custom entities.
- Analyze how multiple modules use shared tables.

- Perform impact analysis to determine what modules are affected by database changes in a new release of Oracle Applications.
- Generate Data Definition Language (DDL) scripts to create custom database objects.
- Use the Applications Oracle Designer database from Oracle development as a starting point for extensions.
- If you already use Oracle Designer for other custom development, your developers can continue to use the same tools.
- Generate custom forms and reports from design information stored in the repository.



Warning: The code you generate from Oracle Designer may require modification to be fully compliant with Oracle Applications standards.

Oracle Designer does *not* support the following features:

- support for elaborate textual design information
- smooth integration with word processing tools
- automatic configuration of applications setups or other features (such as menus, responsibilities, flexfields, and alerts)

Upgrades

Upgrades to the Applications will affect each type of customization (modification, extension, and configurable extension) differently. Every time Oracle releases a patch or upgrade, you must analyze the changes and decide how to migrate your customizations.

Modification

Modifications to standard applications code should be avoided because they can lock you into a particular release and preclude Oracle Support from supporting the altered features.

Extension

Adding functionality with extensions is the preferred technique and can address most requirements. Many approaches that appear to require modifications can be implemented with extensions instead. For example, instead of adding a zone to a form, you can build a new form

and link it to an existing form. Database triggers can often be used to implement changes to processing logic.

You can avoid most of the upgrade problems associated with code changes by building extensions, but you must still analyze the effects of database changes if your custom extensions access standard applications tables. In addition, the Applications may implement a new business rule that uses an existing database column in a different way.

Configurable Extension

The safest technique is to use the features that Oracle has built into the Applications for customization. Upgrading the Application automatically preserves the flexfields and alerts. However, you must still consider the effect of database changes on these modules.



Reference: Kodjou, Paule. *Architecting Your Applications Environment for the Development and Preservation of Customizations*. OAUG Conference Proceedings, Spring 1996.

Good Design Documents

Design documents are the primary communication mechanism from the designer to users and are the blueprint from which developers will build custom modules. After the implementation of the customization, they also serve as documentation. Design documents must be unambiguous and leave no question unanswered.

A complete design consists of three major deliverables:

- Application Extensions Functional Design (MD.050)
- Application Extensions Technical Design (MD.070)
- Database Extensions Design (MD.060)

Application Extensions Functional Design (MD.050)

The Application Extensions Functional Design describes the overall functionality that a customization provides and how the new components (modules) integrate with the underlying business processes. It also includes detailed descriptions of each form, report, and concurrent program from the user's perspective. The users must be able to understand the terms used to describe all business logic. Diagrams and examples can help clarify complex issues. The format

and style are similar to the standard Oracle Applications reference manuals.

Application Extensions Technical Design (MD.070)

The Application Extensions Technical Design includes all of the details that a developer needs to build the required modules. This level of detail is important, even if the same person performs both the designer and developer roles, because the documentation will be part of the Technical Reference Manual (DO.080) for the new system. You can use Oracle Designer or other CASE tools effectively to capture the technical specifications for custom modules, particularly the integration points with the database.

Database Extensions Design (MD.060)

The Database Extensions Design is a single model of the database extensions that supports all required extensions. A database designer works closely with technical analysts to define the database structures and how they relate to the standard application database objects.

Scope Control

Scope creep (a gradual increase in scope) with no control mechanism can be a major challenge with custom development. Users like bells and whistles and developers enjoy adding them; however, during mapping, the entire project team must keep in mind the objective of minimizing customizations.

Someone accountable for the project schedule and budget should approve the work estimates included in the Application Extension Definition and Estimates (MD.020). Thereafter, any proposed changes that would increase the estimated work must be pre-approved. Likewise, approval of the Application Extensions Functional Design (MD.050) effectively freezes the functionality described therein. After design approval, a formal Change Request (PJM.CR.060) must be submitted by any users or team members who desire new functionality.



Attention: The Control and Reporting Strategies, Standards, and Procedures (PJM.CR.020) describe the formal change request process for the project.

Keep track of all requested changes and other unexpected conditions that affect the time required to design and build custom extensions.

This helps you predict and control time overruns, and is useful when analyzing estimated versus actual effort.

Performance Considerations

An often overlooked aspect of custom design is the performance of the code. Due to time pressures, the emphasis may be on code that processes correctly and performance issues are often overlooked. However, the resulting inefficiencies may cause performance problems in production.

Technical analysts involved in designing custom extensions and the developers that build them must be familiar with performance tuning techniques for the tools they are using. Designers are responsible for designing the customizations with performance in mind and documenting potential performance issues in the Application Extensions Technical Design (MD.070). This can be challenging, because following the guidelines for upgrading customizations does not always lead to designs that optimize performance.

Developers must work closely with the team conducting performance testing to make sure that custom modules with performance risks are included in the performance tests, as documented in Performance Testing (PT).

Integration with Business System Testing

Testing is a vital aspect of producing quality programs. Although the testing tasks are documented in Business System Testing (TE), some of the tasks are highly integrated with Module Design and Build.

After the designer completes the Application Extensions Functional Design (MD.050), the Link Test Script (TE.030) can then be prepared for the customization. After the designer completes the Application Extensions Technical Design (MD.070) the Unit Test Script (TE.020) should be written. These are then available to the developer to use to test the programs. Having test plans available while building the modules also helps the developer construct the programs with the correct logic.

The developer should execute both unit and link tests before turning the code over to someone else for testing. From a project planning and staffing standpoint, the developer tests the code during Create Application Extension Modules (MD.110), while others perform the

formal testing tasks. The original estimates should contain time for error correction and retesting. A good *rule of thumb* is to schedule target completion of design, build, and testing tasks at 75 to 80 percent of their total scheduled duration. Developers tend to use all time allocated to complete tasks regardless of the complexity of the work. For more information, see Business System Testing (TE).

Tasks and Deliverables

The tasks and deliverables of this process are as follows:

ID	Task Name	Deliverable Name	Required When	Type*
MD.010	Define Application Extension Strategy	Application Extension Strategy	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	SI
MD.020	Define and Estimate Application Extensions	Application Extension Definition and Estimates	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	MI, IT
MD.030	Define Design Standards	Design Standards	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	SI
MD.040	Define Build Standards	Build Standards	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	SI
MD.050	Create Application Extensions Functional Design	Application Extensions Functional Design	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	MI, IT

ID	Task Name	Deliverable Name	Required When	Type*
MD.060	Design Database Extensions	Database Extensions Design	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	SI
MD.070	Create Application Extensions Technical Design	Application Extensions Technical Design	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	MI, IT
MD.080	Review Functional and Technical Designs	Approved Designs	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	SI
MD.090	Prepare Development Environment	Development Environment	Project includes <i>customizations</i> to standard functionality, <i>interfaces</i> with external systems, or <i>performance testing</i>	SI
MD.100	Create Database Extensions	Custom Database Objects	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	SI
MD.110	Create Application Extension Modules	Module Source Code	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	MI
MD.120	Create Installation Routines	Installation Routines	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	MI

*Type: SI=singly instantiated, MI=multiply instantiated, MO=multiply occurring, IT=iterated, O=ongoing. See Glossary.

Table 5-1 Module Design and Build Tasks and Deliverables

Objectives

The objectives of Module Design and Build are:

- Design customizations to satisfy business needs not met with the standard applications.
- Design application extensions that you can easily maintain and upgrade to future releases of the applications.
- Build modules according to the design specifications.
- Develop automated functions and detailed instructions to install customizations in the Testing Environments (TE.060) and Production Environments (PM.040).

Deliverables

The deliverables of this process are as follows:

Deliverable	Description
Application Extension Strategy	A document that describes how the project responds to Application extensions requests.
Application Extension Definition and Estimates	This document summarizes the business need that Oracle Application features cannot meet and proposes application extensions to meet that business need.
Design Standards	Defines the standards which will be followed when designing Application extensions.
Build Standards	Defines the standards that developers will follow in building the Application extensions.
Application Extensions Functional Design	A description of the custom modules expressed in user terms.

Deliverable	Description
Database Extensions Design	A model and definition of the new and changed database objects and their relationships to standard application database objects.
Application Extensions Technical Design	Specifications for the program modules with sufficient technical detail that developers can develop modules.
Approved Designs	Management-approved designs of the functional and technical designs for the application extensions. This approval indicates management's agreement to proceed with development.
Development Environment	A working environment that includes all servers, applications, infrastructure, and development tools required to develop and test application extensions.
Custom Database Objects	New tables, indexes, views, sequences, grants, and synonyms required to support the customizations.
Module Source Code	Forms, reports, SQL, PL/SQL, C, and other code for the new modules. For configurable extensions such as flexfields and alerts, this represents the online implementation of these system features.

Deliverable	Description
Installation Routines	Operating system shell scripts, SQL Loader, or terminal keystroke files to install and configure customizations in the production environment. This may also include not easily scripted manual procedures for online setup.

Table 5-2 Module Design and Build Deliverables

Key Responsibilities

The following roles are required to perform the tasks within this process:

Role	Responsibility
Business Analyst	Provide detailed requirements to the designer. Review the functional designs.
Business Line Manager	Describes the problems the business unit faces and requirements for the new system. Review and approve application extension designs.
Client Project Manager	Review and approve strategy, policies, and designs.
Client Staff Member	Assist in the definition of design and build standards and development environment preparation.

Role	Responsibility
Database Administrator	Install and configure database software for the development environment. Create the various databases required during the development life-cycle (for example, the data dictionary, unit testing, system testing, training); and maintain database access controls. Additionally, provide consultation on performance; monitor growth and fragmentation of the development database; and provide database backup and recovery.
Database Designer	Design the database schema to support customizations.
Developer	Build the custom modules and perform initial unit tests.
IS Manager	Advise the development team on current organization standards. Participate in the definition of design and build standards.
Project Manager	Advise on the overall plan for the implementation. Verify availability of appropriate resources and facilities. Manage and schedule Module Design and Build activities according to their position on the critical path.
Project Sponsor	Provide direction on customization strategy and final approval of proposed customizations.
System Administrator	Prepare the development environment for build activities.

Role	Responsibility
Technical Analyst	Define, estimate, and design customizations; take responsibility for the overall customization strategy and standards.
User	Provide requirements details to the designer. Review the functional designs.

Table 5-3 Module Design and Build Key Responsibilities

Critical Success Factors

The critical success factors of Module Design and Build are as follows:

- understanding of the functional requirements
- product and technology expertise to propose appropriate customizations to satisfy business requirements
- expertise in the selected development tools
- knowledge of the capabilities and limitations of the available technology
- accurate estimating and planning
- restriction and control of changes
- properly configured development environment
- updating of technical and functional designs to reflect what was actually coded to achieve the design requirements

References and Publications



Reference: Kodjou, Paule. *Architecting Your Applications Environment for the Development and Preservation of Customizations*. OAUG Conference Proceedings, Spring 1996.



Reference: Kirwin, Ed. *The Fine Line Between Pleasure and Pain—Creating and Managing Oracle Applications Modifications*. OAUG Conference Proceedings, Spring 1995.



Reference: Sanders, Mike. *Modifying Oracle's Package Applications with No Software Changes*. OAUG Conference Proceedings, Spring 1994.



Reference: Woodhall, Sara. *Custom Development with Oracle Applications and Network Computing Architecture*. An Oracle White Paper, September 1998.

MD.010 - Define Application Extension Strategy (Optional)

In this task, you prepare a strategy document that describes how your project responds to customization requests.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Application Extension Strategy**. It outlines the policies and procedures governing the process of extending the functionality of Oracle Applications.

Prerequisites

You need the following input for this task:



Project Management Plan [initial complete] (PJM.CR.010)

The Project Management Plan describes the high-level customization approach and may include a list of known customization requirements. It also includes the quality assurance process for custom designs and programs.



Oriented Project Team (AP.020)

Since the Oriented Project Team has been exposed to the project, they are able to contribute to the development of the Application Extension Strategy.

Task Steps

The steps of this task are as follows:

No.	Task Step	Deliverable Component
1.	Review background materials.	
2.	Describe the purpose, background, scope, and application of the Application Extension Strategy.	Introduction
3.	Define the customization policy.	Customization Policy
4.	Determine the design tools you will use.	Design Tools
5.	Determine the development tools you will use.	Development Tools
6.	Describe the design and development process.	Development Process
7.	Describe the approach for identifying alternative answers to functionality gaps.	Mapping Approach
8.	Define the approach to estimating.	Estimating Approach
9.	Describe the testing process.	Testing Process

No.	Task Step	Deliverable Component
10.	Describe the upgrade procedure.	Upgrade Procedures
11.	Seek acceptance of the deliverable from the project sponsor or steering committee.	Acceptance Certificate (PJM.CR.080)

Table 5-4 Task Steps for Define Application Extension Strategy

Approach and Techniques

The Application Extension Strategy influences the types of responses to business issues the project team considers and proposes during Map Business Requirements (BR.030), the satisfaction of your users, and ultimately the final cost and duration of your project. Your basic options are as follows:

- no customization
- extensions only
- customization allowable

Regardless of your strategy, you need to provide some guidelines to direct the project team in the application of the options they have.

No Customization

If you decide that you will not customize the applications, your strategy is simple. However, this also means that your only option to add new features to the product is to submit requests to Oracle Corporation.

Query Tools

If you plan to use a user reporting and query tool, your customization strategy should describe it and explain storage and catalog procedures for user-developed reports and catalogs. Some query tools require significant setup and maintenance. You must also deal with database changes in new releases, just as you would with any other custom reports.

Flexfields

Technically, flexfields are customizations, although fully supported by Oracle. Descriptive flexfields can vary in complexity from a few non-validated fields on a form to context-sensitive flexfields with complex validation rules. If your strategy includes the use of flexfields, emphasize the importance of standards and careful documentation.

Extensions Only

If you limit customizations to reports and other pure extensions, your strategy should make a distinction between extensions and modifications. Extensions add modules but do not change any code in the base application. Modifications change code in the application and require significant analysis during upgrades. Because it is additive, incorporating a new field into a form or report may be considered an extension, but this is a pure modification that should be avoided.

When you build new components and integrate them with the applications, you take on the responsibility of maintaining and supporting the new components for your users. A formal help desk can make sure that help requests and problems are routed to the appropriate group for resolution (internal help desk versus Oracle Support). For more information, see Implement Production Support Infrastructure (PM.060).

Customization Allowable

When all types of customizations are permitted, your strategy should provide guidelines for when each type is appropriate. A modification should only be considered when the business need is vital, there are no procedural workarounds, and all other alternatives have been exhausted.

Whenever you modify a standard application component, treat the modified module as if it is a custom component that you have designed and built from scratch. The original source and executable code must remain in its original location. The storage of the modified version must be in a custom directory structure and registered in Application Object Library (AOL) as part of a custom application.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C /Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

When designing and building application extensions, it is essential that all dates be entered, stored, and processed using the full four-digit year for compliance with Century Date standards. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Upgrades

The biggest challenge with any type of customization is upgrading to a new release of the base application. You must design customizations so that the impact of upgrades is minimal. You must also define the process to follow when you perform an upgrade.



Suggestion: Using Oracle's EasiPath Migration Method (EMM) will help you successfully migrate your application extension to new Oracle Application releases. EMM Advantage is available from the Oracle direct marketing organization in your country.

Preserving Custom Components

To prevent an applications upgrade from deleting some of your customizations, implement them in a way that insulates them from upgrades. A summary of the specific techniques is listed below:

Custom Code	Store source and executable code for new and modified modules in a separate directory structure.
Database Objects	Name all database objects with a unique prefix that does not conflict with any current or reserved Oracle product prefixes. Create one or more separate database users to hold custom database objects.
Program Logic	Use views for all database access.
Application Objects	Create a new custom application in AOL and register all objects in this application. This includes forms, tables, responsibilities, menus, Oracle IDs, alerts, report security groups, messages, and help text.

In special cases you must replace existing modules with customized versions to implement custom functionality. For example, to implement zooms in web-client forms, you must add code to a special code library provided with the applications.

Analyzing the Impact of Upgrades

Some of the possible impacts an upgrade or patch can have on various types of customizations are as follows:

Custom Reports	The underlying tables or views may change.
Custom Forms	The underlying tables, views, and shared library functions may change.

Any Modified Modules	The standard module may change and you must reapply your changes to the new version of the module or choose to keep your version and implement improvements to your code.
Database Triggers	The underlying tables may change. The business rules that cause the trigger to fire may change. The business rules that act on the data in tables you update may change.
Alerts	The underlying tables may change. The business rules that cause the trigger to fire may change. The business rules that act on the data in tables you update may change.
Interfaces	The underlying tables may change. The business rules that act on the data in the tables you update may change.
Custom Menus	Oracle may eliminate functions that are included in your menus or add functions that you need to include.
Custom Responsibilities	Oracle may eliminate menus or add new ones that affect your responsibilities.
Process Navigator	Oracle may eliminate functions that are included in your process navigator definitions or add new functions that you may need to include. These definitions need to be revalidated with subsequent releases.
Workflows	As workflows navigate the user from screen to screen, business function to business function, Oracle may change workflow destinations. Workflows must be revalidated with subsequent releases.

Any Extension

Oracle may add support for the functionality. You must decide how to migrate your data and procedures to the new standard functionality.

Linkage to Other Tasks

The Application Extension Strategy is an input to the following tasks:

- BR.030 - Map Business Requirements
- TA.010 - Define Architecture Requirements and Strategy
- MD.020 - Define and Estimate Application Extensions
- MD.040 - Define Build Standards

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	50
Business Analyst	25
Project Manager	25
Client Project Manager	*
Project Sponsor	*

Table 5-5 Role Contribution for Define Application Extension Strategy

* Participation level not estimated.

Deliverable Guidelines

Some information in the Application Extension Strategy overlaps with the Project Management Plan (PJM.CR.010) and the Testing Requirements and Strategy (TE.010). However, the strategy content found in those deliverables is rather general in nature. The Application Extension Strategy contains greater detail and is directed specifically at developers.

Do not attempt to describe standards in the Application Extension Strategy because Design Standards (MD.030) and Build Standards (MD.040) are separate deliverables. The strategy focuses on policy, scope, techniques, and procedures.

After you write the Design Standards (MD.030) and Build Standards (MD.040), you may wish to combine them with the Application Extension Strategy and publish the set as an *application customization developer's guide*. Make each deliverable a chapter of the consolidated document. This provides a single document that new developers on the project can read and reference.

This deliverable should address the following:

- guidelines regarding customization policy
- use of automated tools to aid design and development
- description of the development procedures to be employed
- guidelines for the identification of functionality gaps
- process to nominate and seek approval for proposed customizations
- estimating guidance
- requirements for testing
- procedures for upgrading the applications during the course of the project

Deliverable Components

The Application Extension Strategy consists of the following components:

- Introduction
- Customization Policy
- Design Tools
- Development Tools
- Development Process
- Mapping Approach
- Estimating Approach
- Testing Process
- Upgrade Procedures

Introduction

This component documents the purpose, background, scope, and application of the Application Extension Strategy.

Customization Policy

This component states the general policies to be followed during the customization process when determining which requirements can be satisfied using standard features of the application versus requirements that can only be addressed by building new modules or by modifying existing ones.

Design Tools

This component identifies and describes the design tools that will be used during the customization process.

Development Tools

This component identifies and describes the development tools that will be used during the customization process.

Development Process

This component describes the process that all designers and developers will follow to develop changes to Oracle Applications.

Mapping Approach

This component establishes guidelines for identifying and classifying functional gaps in the standard applications. Gaps can be broadly classified as either information that the applications do not store or functions they do not perform.

Estimating Approach

This component lists all of the potential customization needs and quantifies the necessary development effort.

Testing Process

This component identifies and describes the testing activity on the resulting Application extensions.

Upgrade Procedures

This component lists and describes all of the steps needed to preserve the functionality of the customized modules after upgrades to new releases of the applications.

Tools

Deliverable Template

Use the Application Extension Strategy template to create the deliverable for this task.

Use the Acceptance Certificate (PJM.CR.080) template to document acceptance of the Application Extension Strategy.

MD.020 - Define and Estimate Application Extensions (Optional)

In this task, when the approach to addressing a business issue determined during Map Business Requirements (BR.030) requires custom modules, you must estimate the effort required to complete them.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Application Extension Definition and Estimates**. It summarizes the business need that Oracle Applications features cannot meet and proposes an alternative approach for satisfying the need that includes a combination of custom modules, manual workarounds, and existing application features. It also includes work estimates for designing, building, and testing the alternative approach.

Prerequisites

You need the following input for this task:

☐ **Mapped Business Requirements (BR.030)**

The Mapped Business Requirements describe the detailed requirements of business needs that standard functionality does not satisfy.

☐ **Mapped Business Data (BR.040)**

The Mapped Business Data provides a detailed mapping of legacy data elements to the target Oracle Applications.

☐ **Integration Fit Analysis (BR.050)**

The Integration Fit Analysis identifies the interfaces needed to meet integration requirements. If Conduct Integration Fit Analysis was not

performed, this deliverable will not exist. (See the task description for BR.050 for more information on when this task should be performed.)

☐ **Master Report Tracking List (BR.070)**

The Master Report Tracking List identifies new reports or report modifications needed to meet reporting requirements.

☐ **Confirmed Business Solutions (BR.090)**

Confirmation and approval of an alternative approach to satisfying a business issue involving customizations should be obtained before defining and estimating customizations, as specified by the project policies defined in the Application Extension Strategy (MD.010).

☐ **Architecture Requirements and Strategy (TA.010)**

The Architecture Requirements and Strategy provides key requirements that underpin the design of the new information systems architecture.

☐ **Application Extension Strategy (MD.010)**

The Application Extension Strategy provides guidance on the approach and extent of customization. This deliverable also contains the project policy and processes for nominating and seeking approval to create an extension. If Define Application Extension Strategy was not performed, this deliverable will not exist. (See the task description for MD.010 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review detailed requirements.	
2.	Determine potential approaches to addressing the business issue.	

No.	Task Step	Deliverable Component
3.	Review alternatives with analysts and users.	
4.	Select preferred approach.	
5.	Review estimating guidelines.	
6.	Document estimating assumptions.	Introduction
7.	Describe the custom components required for the customization.	Solution Section
8.	Estimate the effort to design, build, and test customizations.	Solution Section
9.	Estimate the maximum number of resources that could work concurrently on each development task.	Solution Section
10.	Summarize the customizations to the applications.	Master Customization Worksheet
11.	Review final approaches involving customizations and estimates with analysts, users, and management.	
12.	Secure approval as specified in the Application Extension Strategy (MD.010).	

Table 5-6 Task Steps for Define and Estimate Application Extensions

Approach and Techniques

Approaches to Satisfying Business Issues

Gaps can be broadly classified as either information that the applications do not store, or functions they do not perform.

Information Gaps

Information gaps can be further broken down into missing business objects, missing entities, missing data elements, and missing relationships.

Business Objects

Examples of new business objects are service contracts, shipping containers, or material consignees. They may have a many-to-one, one-to-many, or many-to-many relationship with existing business objects. These require new tables to hold the information and provide the proper association with other objects. A single business object may be a collection of multiple entities (for example, a service contract may have a single header and multiple service items).

Business Entities

Business objects may consist of entities. For example, the sales order object consists of a sales order header entity and a sales order line entity. Each logical business entity is usually implemented as a table in the database. If you have a set of information about an existing business object that can occur multiple times for each object, you have identified a missing entity. An example of this is shipping rates associated with a shipping method. The application supports shipping methods, but you need to store multiple rates for each method to support automated shipping method selection.

Data Elements

Data elements are attributes of a supported business entity such as customers or inventory items. Descriptive flexfields can usually satisfy this need.

Relationships

Missing relationships (such as associating a customer with preferred suppliers) are actually a class of missing data elements and a descriptive flexfield can usually satisfy this need. However, if the relationship is many-to-many, the situation may require a new table to store the intersecting relationship.

Basic data modeling techniques are helpful to clarify the requirements. Keep in mind that new tables will require custom forms to enter the information. Descriptive flexfields often lead to report customization requirements.

Functionality Gaps

Functionality gaps can vary in scope from missing business rules in a function that is supported, to missing functions or even missing systems.

Business Rules

If the gap is at the business rule level and business procedural changes cannot address the situation, determine whether an event triggers invocation of the rule. If so, an alert or database trigger may suffice. If the required logic is part of a function that executes as a concurrent program, you may be able to create a new program that runs before or after the existing program. You can combine standard and custom concurrent programs using report sets.



Reference: *Application Object Library Reference Manual.*

You can use views to dynamically transform the representation of data in standard tables so that standard application functions operate on the altered data to produce a new result. For example, if you wanted the cost rollup process in Oracle Cost Management to use a different accumulation rule, you could use a view of a Bills of Material table to present altered values for the columns included in the calculation. You have not modified the standard tables, nor the cost rollup program, but you have implemented a new processing rule.



Reference: Mercer, David. *Views that Masquerade as Tables*. OAUG Conference Proceedings, Spring 1995.

Oracle Applications include a number of special PL/SQL routines specifically designed to allow you to add your own custom logic to adjust the processing logic of standard functions. For example, if you need to modify the information that the MRP process in Oracle Master Scheduling / MRP collects during the snapshot phase of the planning process, you can add logic to the PL/SQL stored procedure called `Mrp_user_defined_snapshot_task`. This procedure is an empty procedure that the MRP process calls before beginning the detailed planning process. Thus, you can alter the inputs to MRP without changing any of the internal MRP code.



Attention: Consult your *Application Technical Reference Manuals* (TRMs) for more information on this and other supported customization hooks.

Functions

You can supplant missing functions with standalone forms, reports, or concurrent programs. You can integrate custom forms with standard forms using links.

Systems

Missing systems or large collections of related functions may require a supplemental Custom Development Method (CDM) or Custom Development Method Fast Track (CDM-FT) subproject to design, build, test, and integrate Application extensions with the core Oracle Applications. Another alternative is to procure a third-party package that addresses the requirements.

Timing

You do not need to wait until all mapping is complete to begin defining and estimating customizations. You can begin writing parts of the Application Extension Definition and Estimates as soon as you identify a gap and propose a custom approach. You will identify some gaps early during Gather Business Requirements (RD.050), while others may not surface until you begin testing business procedures.

Estimating Guidelines

For each business requirement not fully satisfied by the standard Oracle Applications, summarize the amount of effort you estimate it will take to build customizations that close the functionality gaps.

Identify Components

In order to accurately estimate the effort, you must first identify all of the custom elements, which can include any of the following:

- new or modified forms
- new or modified reports
- new or modified programs (SQL*Plus, PL/SQL, Pro*C)
- database triggers
- user exits
- SQL*Loader scripts
- standard report submission parameters
- alerts
- new tables
- descriptive flexfields
- custom workflows

Some relatively simple requirements actually translate into several components to implement correctly.

Assign Complexity

For each component, rank the complexity as very easy (VE), easy (E), moderate (M), or complex (C). For estimating purposes, consider stored procedures, database triggers, user exits, and SQL*Loader scripts as programs. Treat alerts as reports, unless they serve primarily as database triggers, in which case you should treat them as programs. Classify descriptive flexfields and setting up standard report submission parameters as form modifications. Basic guidelines for ranking each type of module are listed in the following tables.

Form

Rating	New	Modified
Very Easy	Low risk, single-block form with 8 or fewer columns. No special functional logic required.	Minor change such as changing form text or navigation. No changes to form processing or underlying table structure. Also, simple descriptive flexfield definitions are classified as Very Easy form modifications.
Easy	Single or multiple block (2-3 blocks) with 20 or fewer columns. Minor functional logic (simple edits, cross edits, simple calculations, totals, or subtotals) required.	Changes to form processing (field validations, formats) or adding fields. Descriptive flexfields with lookup table validation or cross-validation.
Moderate	Single or multiple block (2-3) with greater than 20 columns. Significant functional logic (edits, calculations, calling other forms, flexfield validations).	Many new fields, logic, or table structures are being redesigned and built.
Complex	Multiple block (3 or more) with more than 20 columns. Requires extensive or complex functional logic, one or more user exit calls (user exits should be estimated separately as programs). Navigation or display logic which is unusual for Oracle Forms or Application Object Library.	Major changes to form processing, many additional fields or additional zones, changes to base tables, and so on. Rarely done due to complexity and risk. Usually better to start over with a new custom form.

Table 5-7 Complexity Guidelines for a Custom Form



Attention: The design philosophy is based on an object-oriented paradigm where a single gateway form allows you to perform any function you need for a given business object. If you are designing a new form for a new business object, estimate the gateway form and each subfunction as separate forms.

Report

Rating	New	Modified
Very Easy	Simple report consisting of one SQL statement. Minimal formatting.	Changes to the format only.
Easy	Some formatting and processing logic of one or two tables.	Changes some formatting, adding one or two columns with little or no changes in processing logic.
Moderate	Several tables queried (perhaps master-detail) and significant processing logic or formatting.	Many changes to report format and or reported data. Perhaps accessing additional tables.
Complex	Complex processing logic and report formatting. Multiple table reporting hierarchy or cross-tabulation.	Major changes to report format and processing. Often better to begin fresh with a new report.

Table 5-8 Complexity Guidelines for a Custom Report

Program

Rating	New
Very Easy	Script which operates on a single table. A database trigger that inserts a row into another table would be an example.
Easy	Updates to 2-3 tables with minimal conditional logic or looping.
Moderate	Updates to 3 or more tables with some conditional logic, calculations, and looping.
Complex	Updates to 5 or more tables with sophisticated conditional logic, calculations, and looping.

Table 5-9 Complexity Guidelines for a Custom Program



Attention: The rating of Pro*C or Java programs should be one level higher than other types of programs due to the inherent complexity of linking and debugging.

Use your own judgment for menu and table complexity.

Calculate Base Estimates

Consult your Application Extension Strategy (MD.010) for the proper estimating parameters for your project. It should have a table listing raw design and build numbers for each combination of type and complexity. Calculate the total effort in person days for design and build by multiplying the number of modules of each type by the base estimates.

If you think a new form, report, or program is very complex (more difficult than the complex rating), use an appropriate estimate based on your past experience. Although the base metrics and guidelines are useful, they are not a substitute for experience. Sometimes a relatively minor customization requires significant testing because it is in a complex business area or requires significant preliminary setup to test (such as material requirements planning).

If you are working with a pre-production release of the applications or are planning to implement new development tools and technology, increase the default estimating factors to allow for the learning curve and potential instability of your environment.

Make sure you identify all custom components. If new tables are a requirement, you probably need new forms to maintain them (unless they are interface tables). Each report and concurrent program requires standard report submission parameters or a custom launch form.

Calculate Extended Estimates

For simplicity, the base metrics in the Application Extension Strategy (MD.010) supply only raw design and build numbers. However, you must extend these to estimate the effort of the other development tasks. Use your totals to calculate the effort for other tasks according to the formulas in the following table.

Task	Estimating Formula
Module Design and Build Process	
Create Application Extensions Functional Design (MD.050)	.5 * design
Create Application Extensions Technical Design (MD.070)	1 * design
Create Application Extension Modules (MD.110)	.7 * build
Create Installation Routines (MD.120)	.1 * build
Business System Testing Process	
Develop Unit Test Script (TE.020)	.1* design
Develop Link Test Script (TE.030)	.25 * design
Perform Unit Test (TE.070)	.3 * build
Perform Link Test (TE.080)	.35 * build

Table 5-10 Formulas to Extend Base Estimates to Other Development and Testing Tasks

Recommended Staffing Limits

For each development task, indicate the maximum number of resources that could reasonably work on the modules of the customization simultaneously. This is purely a judgment call. If a single customization requires several forms and reports, you might be able to divide the design and development work between two or three resources without losing efficiency.

Project Planning

After management has approved the customizations, add new tasks to the project plan using your calculated estimates as the basis for work effort. If multiple people will perform the design and build, you may

want to divide the build task into subtasks for each component of the customization, so that you can assign resources individually and perform accurate resource leveling.

Linkage to Other Tasks

The Application Extension Definition and Estimates are an input to the following tasks:

- MD.030 - Define Design Standards
- MD.050 - Create Application Extensions Functional Design
- MD.060 - Design Database Extensions
- MD.080 - Review Functional and Technical Designs

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	70
Project Manager	20
Business Analyst	10
Business Line Manager	*
Project Sponsor	*
Client Project Manager	*

Table 5-11 Role Contribution for Define and Estimate Custom Extensions

* Participation level not estimated.

Deliverable Guidelines

Use the Application Extension Definition and Estimates to describe and estimate all modifications, extensions (including interfaces) and configurable extensions. Typically, you create an Application Extension Definition and Estimates for each major business area or process plus one each for interfaces, reports, and custom support systems. Management must then decide whether the benefits of the customization are worth the time and expense (now and during upgrades) to build and maintain it.

This deliverable should address the following:

- description of the business requirement
- the proposed approach for satisfying the business requirement
- the estimated effort to design, build, and test the proposed approach
- the recommended staffing



Attention: Data Conversion (CV) includes tasks to design, build, and test data conversion programs.

The approaches outlined in the Application Extension Definition and Estimates should be brief — no longer than one or two pages each. Include enough detail to identify all the custom components and make it clear how the components work together, but do not attempt to write a complete design document.

Use the Mapped Business Requirements (BR.030) to derive and summarize the requirements for each alternative. Describe the total approach to satisfying the business issue and include basic references to the following elements:

- standard Oracle Applications features
- manual procedures
- custom extensions

In your summary, communicate the overall approach and present one or more alternatives to filling these functionality gaps. Each approach may have pros and cons as well as time and cost differences.

Management, analysts, and users may not be able to choose the preferred approach until you provide detailed estimates for each option.

It is acceptable to use a mixture of functional and technical language in the Application Extension Definition and Estimates. The goal is to communicate concisely the nature of the customization. Include the following information:

- the amount of effort required to analyze, design, build, and test custom code
- the time required to upgrade the customizations to a future release of Oracle Applications
- a summary of the total effort

As a summary, create a list of all custom approaches to business issues in the document. This list (master custom extension worksheet component) shows the following information:

- unique identifier
- brief description of requirement
- assignment and staffing lists
- subtotal of all customization design, development, and build work
- subtotal of upgrade estimates
- target due date

The subtotals become an input to planning the detailed design, build, and testing tasks.

Deliverable Components

Application Extension Definition and Estimates consists of the following components:

- Introduction
- Solution Section
- Master Customization Worksheet

Introduction

This component summarizes the business requirements that are not addressed by the Oracle Applications with a recommended approach for satisfying each requirement.

Solution Section

This component specifies the name of the business issue you are addressing and a unique identifier for each issue. The unique identifier should come from the BRS (RD.050) or BRM form (BR.030). Additional business issues can be inserted by using the Microsoft® Word copy/paste menu options to repeat the solution section.

Estimating Worksheet

The Solution Section includes an estimating worksheet (it is a standard Microsoft Word table, not an embedded Excel worksheet). The estimating section is used to estimate the number of person days required to implement the approach and the table describes the modules and assigns complexity ratings. Position your cursor in the *type* or *rating* column and press Ctrl-L to select from a list of customization types and complexity ratings, respectively. The template automatically inserts the estimating factors for design, build, and upgrade activities from the table in the Introduction section. When you finish, double click on the red button to update the totals. The following table shows one custom form with base and extended metrics.

Module	Type	Rating	Design	Build	Upgrade
Name of module	Form - New	M	3	5	1.6
SUBTOTALS			3	5	1.6

Double-click here to update totals

	Design	Build	Test
Create Application Extension Functional Design	2.25		
Create Application Extension Technical Design	1.8		
Develop Link Test Scripts			.75
Develop Unit Test Scripts			.3
Create Application Extension Modules		5	
Perform Unit Tests			.5
Perform Link Tests			1.25
Create Installation Routines		.5	
TOTALS	4.0	5.5	2.8
GRAND TOTAL			12.3
TOTAL UPGRADE			1.9

Table 5-12 Sample Application Extension Definition and Estimates Estimating Worksheet

Recommended Staffing

In the last part of the Solution Section, recommended staffing levels are entered. The maximum *reasonable* number of people who could work on each development phase simultaneously is entered as well. The project manager uses this information to plan the customization activities and schedule resources, as shown in the table below:

Development Task	Functional	Technical
Create Application Extension Functional Design	1	
Create Application Extension Technical Design		1
Develop Link Test Script	1	
Develop Unit Test Script		1
Create Application Extension Modules		1
Perform Unit Tests		1
Perform Link Tests	2	
Create Installation Routines		1

Table 5-13 Sample Application Extension Definition and Estimates Staffing Recommendation

Master Customization Worksheet

This component maintains a high-level record of all customizations to the applications.

Tools

Deliverable Template

Use the Application Extension Definition and Estimates template to create the deliverable for this task.

MD.030 - Define Design Standards (Optional)

In this task, you define the standards that designers will follow when designing customizations.

Clear and detailed design standards help make sure that all designs are in a consistent format and include the appropriate level of detail. Standards enforce a high level of quality.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Design Standards**. These standards help make sure that the designs are high quality, portable across multiple platforms, have a consistent look and feel, are easy to maintain, and are compatible with future versions.

Prerequisites

You need the following input for this task:

☐ **Project Management Plan [initial complete] (PJM.CR.010)**

The Project Management Plan includes general documentation and review standards that also apply to design documents.

☐ **Application Extension Definition and Estimates (MD.020)**

The Application Extension Definition and Estimates provides information on the application extensions that need to be designed.

☐ **Oracle Applications Object Library Reference Manual**

The *Oracle Applications Object Library Reference Manual* provides general guidelines for designing customizations to the Applications.

Oracle Applications User Interface Standards

Oracle Applications User Interface Standards describes the standards for building forms using Oracle Forms.

Organization Specific Internal Standards

Consider any standards that your organization has already defined for custom development.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Determine source of standards.	
2.	Review existing Oracle and organization standards.	
3.	Describe the purpose, background, scope and application for the Design Standards.	Introduction
4.	Summarize the design standards philosophy.	Overview
5.	Define design document components.	Design Document Components
6.	Define topical essay standards.	Topical Essay Standards
7.	Define form cosmetic standards.	Form Cosmetic Standards
8.	Define report cosmetic standards.	Report Cosmetic Standards

No.	Task Step	Deliverable Component
9.	Define database design standards.	Database Design Standards
10.	Define interface standards (messages, error codes, and dialog interaction).	Interface standards (Messages)
11.	Define the naming standards for custom objects.	Naming Standards
12.	Review standards with the project team.	
13.	Configure Oracle Designer preferences (if applicable).	Configured Oracle Designer Preferences
14.	Seek acceptance from the client project manager.	Acceptance Certificate (PJM.CR.080)
15.	Secure acceptance that the Design Standards include criteria for compliance with Century Date standards.	Acceptance Certificate (PJM.CR.080) (See the Tools section for information on modifications to the standard Acceptance Certificate template.)

Table 5-14 Task Steps for Define Design Standards

Approach and Techniques

The Design Standards allow you to document customizations in a way that clearly communicates the features and functionality to both users and developers. They also help make sure that new modules have a consistent look and feel and integrate well with the standard Oracle Applications.

Each standard you define should provide a specific design, development, documentation, or learning benefit. Good standards help you do the following:

- Explain functionality to current and potential users.
- Train users.
- Support users when they have questions or problems.
- Manage the development of customizations.
- Produce high-quality program modules.
- Work efficiently and cost-effectively.
- Port application modules to different hardware and software platforms.
- Upgrade customizations to new releases of the applications.

What Makes a Good Standard?

A good standard has the following qualities:

- **unambiguous** — clearly communicates the standard and is easy to read and understand
- **consistent** — consistent with existing standards and your development philosophy; it is self-consistent as well
- **easy to use** — leverages existing standards and tools and increases your productivity; it is not awkward or impractical

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the

implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

When designing and building application extensions, it is essential that all dates be entered, stored, and processed using the full four-digit year for compliance with Century Date standards. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Existing Standards Use

Before creating new standards, review all other design and development standards sources. You may have defined design and development standards for other internal projects, but they are not likely to include important issues relating to Oracle Applications. Any update of current organization standards must reflect the latest Oracle development tools and Application Object Library standards. If you plan to deploy new technologies such as pen-based, hand-held terminals or world wide web interfaces, you may wish to consider standards from external sources.

Remember that one of your implementation objectives is to provide an approach to the business issue that is easy to understand and use. The designer of a program may not be the developer. Therefore it is important to clearly communicate design specifications. Standards help communicate these ideas effectively by establishing a common format that is complete and easy to understand.

Do not duplicate what Oracle has already defined in standard documentation. The *Application Object Library Reference Manual* includes a chapter titled, "Product Customization Standards," that provides basic standards and guidelines for customizing Oracle Applications. *Oracle Applications User Interface Standards* includes detailed standards for using the Oracle® Forms development tools.

Cosmetic standards help provide the same look and feel of the applications. To users, maintaining the same format and style is important. Use Oracle design and development standards to maintain the same appearance of new or modified applications. These standards not only provide cosmetic guidelines, but development standards for building custom modules as well.



Reference: *Oracle Applications Object Library Reference Manual.*



Reference: *Oracle Applications User Interface Standards.*



Attention: *Oracle Applications User Interface Standards* only addresses standards for building forms with Oracle Designer; it does not include report standards.

The Project Library

Store the completed design documents in the project library. Update the site library table of contents to include new materials.

In a multiple-site implementation, a program office develops these standards and helps make sure that each site uses, augments, and refines the standards as the project progresses. Standards are especially helpful in multiple-site situations, where the transference or reuse of design and development work can take place.

Project Team Presentation

Handing out a standards document to the team may not be enough to ensure conformance or even understanding. Plan to hold a learning session and include all analysts, designers, and developers. Those who will be reading the design documents must also understand the standards.

Oracle Designer

If you are using Oracle Designer to design custom modules and plan to generate default forms and reports, this task includes a step to configure the preferences information that determines the layout and logic of default modules.



Reference: *Oracle Designer Documentation.*

Linkage to Other Tasks

The Design Standards are an input to the following tasks:

- MD.040 - Define Build Standards
- MD.050 - Create Application Extensions Functional Design
- MD.060 - Design Database Extensions
- MD.070 - Create Application Extensions Technical Design
- MD.090 - Prepare Development Environment
- CV.020 - Define Conversion Standards
- CV.060 - Design Conversion Programs
- PT.050 - Design Performance Test Transaction Programs
- PT.070 - Design Test Database Load Programs

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	100
Client Staff Member	*
IS Manager	*

Table 5-15 Role Contribution for Define Design Standards

Deliverable Guidelines

Use the Design Standards to describe the standards you adopt for designing future customizations and extensions to Oracle Applications.

This deliverable should address the following:

- cosmetic and content standards for design documents
- description of design environment (file system structure and tool)
- naming standards
- user interface and cosmetic standards
- sample documents or templates

When making customizations to Oracle Applications, follow the cosmetic and coding standards of the applications as closely as possible.

Deliverable Components

The Design Standards consist of the following components:

- Introduction
- Overview
- Design Document Components
- Topical Essay Standards
- Form Cosmetic Standards
- Report Cosmetic Standards
- Database Design Standards
- Interface Standards (Messages)
- Naming Standards

Introduction

This component documents the purpose, background, scope, and application of the Design Standards.

Overview

This component summarizes the Design Standards philosophy.

Design Document Components

This component describes the structure of the design documents to be produced.

Topical Essay Standards

This component describes additional standards for writing topical essays (this section may be omitted).

Form Cosmetic Standards

This component describes and lists reference documents to be used in designing application forms.

Report Cosmetic Standards

This component lists the characteristics that make good reports.

Database Design Standards

This component describes the tools and lists the important aspects to be taken into consideration when designing database extensions.

Interface Standards (Messages)

This component describes the process of creating additions to the user interfaces that *look and feel* like the original Oracle Application.

Naming Standards

This component describes the naming standards that should be followed for custom objects.

Tools

Deliverable Template

Use the Design Standards template to create the deliverable for this task.

Use the Acceptance Certificate (PJM.CR.080) template to document acceptance of the Application Extension Strategy.

Century Date Acceptance

To document the review and approval of Design Standards for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) replacing the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Acceptance Certificate should be filed with the deliverable in the project library.

MD.040 - Define Build Standards (Optional)

In this task, you define the standards that developers follow to build customizations to the applications.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Build Standards**. These standards describe the coding standards that the developers must follow in building application extensions. Build standards help make sure that the resulting customizations are of high quality and fully compatible with the standard Oracle Applications with which they are integrated.

Prerequisites

You need the following input for this task:



Application Extension Strategy (MD.010)

The Application Extension Strategy defines the development tools and valid scope of customizations and dictates what standards you need to document. If Define Application Extension Strategy was not performed, this deliverable will not exist. (See the task description for MD.010 for more information on when this task should be performed.)



Design Standards (MD.030)

The Build Standards must support and complement the Design Standards. If Define Design Standards was not performed, this deliverable will not exist. (See the task description for MD.030 for more information on when this task should be performed.)



Organization-Specific Standards

Consider any standards that your organization has already defined for custom development.

Oracle Applications User Interface Standards

This document is an input to Define Design Standards (MD.030) and also includes information relevant to this task.

Oracle Applications Coding Standards

The *Oracle Applications Coding Standards* define the standards followed by developers at Oracle to build Oracle Applications. Your standards should reference these standards as the basis for all development and define only those standards that are different or are not defined within this manual and the *Oracle Applications User Interface Standards*.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review existing Oracle and organization standards.	
2.	Describe the purpose, background, scope and application of the Build Standards.	Introduction
3.	Describe how to setup of the directory structure, development accounts, and how to build and register new programs.	The Development Environment
4.	Define guidelines and templates for file headers.	Common Standards
5.	Define forms coding standards.	Forms Coding Standards
6.	Define report coding standards.	Report Coding Standards

No.	Task Step	Deliverable Component
7.	Define SQL standards.	SQL Standards
8.	Define PL/SQL standards.	PL/SQL standards
9.	Define database trigger standards.	Database Trigger Standards
10.	Define coding standards for any development tools used.	Other Coding Standards
11.	Define guidelines for comments in source and command files.	Comment Standards
12.	Define guidelines for creating installation scripts for customizations, including a list of standard tools used.	Installation Routine Standards
13.	Describe the source code control procedures that will be employed in building Application extensions.	Source Code Control
14.	Review standards with the development team.	
15.	Secure acceptance that the Build Standards include criteria for compliance with Century Date standards.	Acceptance Certificate (PJM.CR.080) (See the Tools section for information on modifications to the standard Acceptance Certificate template.)

Table 5-16 Task Steps for Define Build Standards

Approach and Techniques

The Build Standards allow you to produce high-quality custom modules that are easy to maintain and support. They also allow custom modules to integrate seamlessly with the standard Oracle Applications and take advantage of future enhancements to shared components and libraries.

Cost and Benefits Consideration

Each build standard you define should provide a tangible benefit, such as the following:

- **improved user interface** — makes the user interface more consistent, easier to use and understand, or increases user productivity
- **efficiency of operation** — makes applications run faster or use fewer system resources
- **speedier development** — allows developers to develop, maintain, or upgrade modules more quickly
- **higher quality** — prevents you from introducing avoidable bugs

Standards come at a cost; the more standards you have, the longer it takes to train new developers and to perform quality reviews. They can also affect developer productivity. Consider the cost/benefit tradeoff for all standards you plan to implement.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the

implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

When designing and building application extensions, it is essential that all dates be entered, stored, and processed using the full four-digit year for compliance with Century Date standards. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Existing Standards Use

The development of Oracle Applications is based on strict standards to help provide quality, consistency, ease of use, efficiency, and portability. Follow all standards documented by Oracle Corporation, unless you have a strong reason to deviate.



Warning: Because the behaviors of Oracle forms, the Oracle Applications standard libraries, and the Oracle Applications standards are so tightly linked, a deviation from the standards that appears to be minor may have far-reaching and unpredictable results.

The definition of standards for building graphical forms with Oracle Forms is in the *Oracle Applications Coding Standards Manual*. Your standards should incorporate and extend the standards defined by Oracle.

Oracle has not documented the standards for writing other types of modules, but you can examine the source code provided with the Applications to derive common standards. Source code is provided for:

- Oracle® Reports
- PL/SQL stored procedures and triggers
- SQL*Plus and PL/SQL concurrent programs
- alerts
- flexfields
- messages

Source Code Templates

To reinforce the standards and make it easier for developers to follow them, you can create templates for each module type to serve as a starting point for developers.

Linkage to Other Tasks

Build Standards are an input to the following tasks:

- MD.070 - Create Application Extensions Technical Design
- MD.090 - Prepare Development Environment
- MD.110 - Create Application Extension Modules
- MD.120 - Create Installation Routines
- CV.020 - Define Conversion Standards
- CV.080 - Develop Conversion Programs
- TE.070 - Perform Unit Test
- PT.080 - Create Performance Test Transaction Programs
- PT.090 - Create Test Database Load Programs

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	100
IS Manager	*
Client Staff Member	*

Table 5-17 Role Contribution for Define Build Standards

Deliverable Guidelines

Use the Build Standards to organize the work of the developers by allowing easy retrieval of the information they need.

This deliverable should address the following:

- naming conventions
- standard file headers
- comments
- structure and style
- debugging techniques
- variable naming and usage
- performance improvement techniques
- exception handling
- error messages
- porting considerations

Code Samples

If possible, include samples of programs that follow the standards as an appendix. For modules that do not have a text representation of source code, you can use screen shots or reference a sample file on the development server.

Deliverable Components

The Build Standards consist of the following components:

- Introduction
- The Development Environment
- Common Standards
- Forms Coding Standards
- Report Coding Standards
- SQL Standards

- PL/SQL Standards
- Database Trigger Standards
- Other Coding Standards
- Comment Standards
- Installation Routine Standards
- Source Code Control

Introduction

This component documents the purpose, background scope, and application of the Build Standards.

Development Environment

This component describes how to set up the directory structure, create development accounts, and build and register new programs.

Common Standards

This component provides guidelines and templates for file headers and comments.

Forms Coding Standards

This component lists reference documentation and exceptions to the standards.

Report Coding Standards

This component provides detailed descriptions of common report elements.

SQL Standards

This component defines the standards for use of the SQL language in building customizations for your project by referencing the applicable Oracle documentation and by specifying exceptions to the standards, where appropriate.

PL/SQL Standards

This component defines the standards for use of the PL/SQL language in building customizations for your project by referencing the applicable

Oracle documentation and by specifying exceptions to the standards, where appropriate.

Database Trigger Standards

This component describes structure and style standards in building database triggers and performance optimizations techniques.

Other Coding Standards (Optional)

This component describes additional standards to be used, if any.

Comment Standards

This component describes the guidelines for comments in source and command files.

Installation Routine Standards

This component describes the guidelines to be followed when creating installation scripts for customizations, and includes a list of standard tools that may be utilized.

Source Code Control

This component provides a brief reference to source code control procedures and tools to be used in developing Application extensions.

Tools

Deliverable Template

Use the Build Standards template to create the deliverable for this task.

Century Date Acceptance

To document the review and approval of Build Standards for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) replacing the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the

acceptance criteria established for Century Date support specified by
<Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century
Date Acceptance Certificate should be filed with the deliverable in the
project library.

MD.050 - Create Application Extensions Functional Design (Optional)

In this task, you document the functional features, use, and behavior of required customizations. The Application Extensions Functional Design confirms that you understand user requirements, and allows users to evaluate and approve the resulting features that the new modules will provide.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Application Extensions Functional Design**. It describes each customization in business and user terms. Users and business analysts are the audience for this deliverable; therefore, it must communicate all the features provided by the customization in non-technical terms.

Prerequisites

You need the following input for this task:



Business Procedure Documentation (BP.090)

The Business Procedure Documentation includes the detailed process steps that users will follow to perform their jobs. You only need the narratives corresponding to the processes that have functionality gaps. They should already incorporate steps that correspond to custom modules identified in the Application Extension Definition and Estimates (MD.020).



Business Requirements Scenarios (RD.050)

The Business Requirements Scenarios describe requirements in the context of business flows.

❑ Mapped Business Requirements (BR.030)

The Mapped Business Requirements provide detailed business requirements for functionality gaps.

❑ Application Setup Documents (BR.100)

The Application Setup Documents define setups that may affect the logic and business rules you define. You also need to know what options will be available for list of values in custom forms and standard report submission parameters.

❑ Application Extension Definition and Estimates (MD.020)

The Application Extension Definition and Estimates describes the high-level approach to satisfying a business issue and required custom components to solve each functionality gap. If Define and Estimate Application Extensions was not performed, this deliverable will not exist. (See the task description for MD.020 for more information on when this task should be performed.)

❑ Design Standards (MD.030)

Design Standards define the format and content of design deliverables, plus cosmetic and interface standards for forms and reports. If Define Design Standards was not performed, this deliverable will not exist. (See the task description for MD.030 for more information on when this task should be performed.)



Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review Mapped Business Requirements (BR.030).	
2.	Write the topical essay.	Topical Essay
3.	Document forms.	Form and Report Descriptions

No.	Task Step	Deliverable Component
4.	Document reports.	Form and Report Descriptions
5.	Document concurrent programs.	Concurrent Program Descriptions
6.	Describe the technical approach.	Technical Overview
7.	Review the high-level design with analysts and key users.	
8.	Obtain approval for the Application Extensions Functional Design by the requester.	

Table 5-18 Task Steps for Create Application Extensions Functional Design

Approach and Techniques

Functional versus Technical

The Application Extensions Functional Design supplies the detailed design that defines the specific logic in each custom module. Designs are also required for any modifications to standard modules and definitions of configurable extensions (such as descriptive flexfields).

The first iteration of the design is the Application Extensions Functional Design which includes a topical essay that provides an overview of the customization. It also includes form and report layouts with descriptions of each zone, field, and column heading. The Application Extensions Technical Design (MD.070) provides form, report, and program logic expressed in technical language. The specific content and format definition of both documents is in the Design Standards (MD.030). Together, they constitute the complete detailed design.

Use the Business Procedure Documentation (BP.090) to understand the business process the customization will support. Include the process steps in the topical essay portion of your design and provide additional

detail. You may add entity relationship diagrams, function hierarchies, and data flow diagrams (from Oracle Designer or another tool) to help explain how the business elements fit together.

Avoid specifying table names, column names, program names, or other technical jargon in the Application Extensions Functional Design. Use the business names for these objects, as they are represented on screens, reports, and menus.

User Reference Manual

The format of the Application Extensions Functional Design should be the same as similar components in the Oracle Applications reference manuals. This not only gives the readers a format that they understand and are comfortable with, it also allows you to easily compile the User Reference Manual (DO.060) for custom modules.

As you write, consider your eventual reader as someone who must understand the new functionality, but who has not participated in any mapping sessions or discussions that led to this design. Include useful information on each form field, so that they can read your description while using the form (or reading a report) and know where and how the information they see is used (or how it was derived).

Indicate whether each field is required, if a list of values is available, and if so, the meaning of each possible value. If the requirements call for special validation or enforcement of business rules, explain these features clearly. Ask yourself if what you write would be helpful to someone using the form for the first time.

Oracle Designer

Use Oracle Designer to lay out new forms and reports and then incorporate screen shots into your design document. For simply laying out a basic form, Oracle Forms is as easy to use as your word processor and gives you a jump start on the build tasks.

Test Development Coordination

You write the Link Test Script (TE.030) for a customization based on the functionality described in the Application Extensions Functional Design. When a reviewer of an Application Extensions Functional Design also has the Link Test Script, it reinforces the information in the design and greatly aids in understanding the functionality.

A useful technique is to have the users who requested the customization write the Link Test Script (TE.030) or write additional test scripts. This forces them to think about the features and business rules as they read the document. It also becomes their personal acceptance test against the final modules. This process may reveal requirements that were not discussed or were glossed over during design meetings. For more information, refer to Develop Link Test Script (TE.030).

Linkage to Other Tasks

The Application Extensions Functional Design is an input to the following tasks:

- BR.040 - Map Business Data
- MD.060 - Design Database Extensions
- MD.070 - Create Application Extensions Technical Design
- MD.080 - Review Functional and Technical Designs
- MD.110 - Create Application Extension Modules
- DO.060 - Publish User Reference Manual
- TE.020 - Develop Unit Test Script
- TE.030 - Develop Link Test Script
- TE.050 - Develop Systems Integration Test Script
- PM.030 - Develop Transition and Contingency Plan

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Business Analyst	80
Technical Analyst	20
User	*

Table 5-19 Role Contribution for Create Application Extensions Functional Design

* Participation level not estimated.

Deliverable Guidelines

Use the Application Extensions Functional Design to document the features of a customization in terms that business people can understand. The Application Extensions Functional Design describes the functionality required to satisfy a specific business requirement and identifies the individual components that make up the application extension.

This deliverable should address the following:

- an overview of the business issue and underlying business process the customization is intended to support
- a description of required data entry forms and reports
- a description of the business function the custom program must support
- a description of how the functionality is to be implemented

Deliverable Components

The Application Extensions Functional Design consists of the following components:

- Topical Essay
- Form and Report Descriptions
- Concurrent Program Descriptions
- Technical Overview

Topical Essay

This component introduces the functionality provided by the customization in the context of the underlying business process. The Topical Essay also provides an overview that is easy to read and understand and in a format that is familiar to anyone who has read the Oracle Applications reference manuals. It includes the following elements:

- basic business needs
- major features
- user procedures

The following sections can be added to clarify issues:

- process flow
- examples
- business rules
- charts and tables
- data flow diagram
- entity relationship diagram
- assumptions

Form and Report Descriptions

This component documents form and report descriptions since they are important functional design components (they are the *external* elements that are most visible to system users). These functional design components must fit into the bigger picture of business process flows.

The functional design helps users see this fit and understand the functionality in the context of business processes.

The following features should be considered when creating functional designs:

- report and screen layout size constraints
- complexity and readability of information (field sizes)
- order in which information is presented
- list of values
- hints or text identifying data
- sort options (if any)
- query criteria (if any)
- source of data
- standard layout conventions
- design standards
- mandatory and optional input parameters
- data manipulation rules (create, read, update, and delete)
- naming conventions

The form description format is used to describe descriptive flexfields.

Concurrent Program Descriptions

This component describes the concurrent programs. Concurrent programs are similar to reports (reports are actually a type of concurrent program), but the primary function of a concurrent program is to update data in the database. The output is typically a log of actions performed. The functional description will focus on several factors:

- when to run the program
- launch parameters
- business rules implemented
- log output
- restart procedures

Technical Overview

This component describes the implementation of the functionality; this is most useful when the approach requires a combination of flexfields, alerts, database triggers, or views to achieve the desired results.

A technical overview is also the first component of the Application Extensions Technical Design (MD.070) — so whatever was included in the functional design can be transferred directly to the technical design and expanded on.

Tools

Deliverable Template

Use the Application Extensions Functional Design template to create the deliverable for this task.



Suggestion: Use the Edit/Copy and Edit/Paste Microsoft Word menu options to add form, report, and concurrent program descriptions for each custom module. Include other components only once.

MD.060 - Design Database Extensions (Optional)

In this task you design the new database objects, such as tables, indexes, views and sequences required by the application extensions, and illustrate how the customizations integrate with the existing applications database schema.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Database Extensions Design**. It includes an entity relationship diagram, definitions of new database objects, and a description of how the new modules access these objects.

Prerequisites

You need the following input for this task:



Mapped Business Data (BR.040)

The Mapped Business Data includes a list of attributes for each business object that the Oracle Applications does not support. Descriptive flexfields or new tables can satisfy these requirements.



Application Extension Definition and Estimates (MD.020)

The Application Extension Definition and Estimates includes information on required database extensions to support the defined approaches to satisfying business issues. If Define and Estimate Application Extensions was not performed, this deliverable will not exist. (See the task description for MD.020 for more information on when this task should be performed.)



Design Standards (MD.030)

The Design Standards include a component called Database Design Standards that governs how you design new database objects. If Define

Design Standards was not performed, this deliverable will not exist. (See the task description for MD.030 for more information on when this task should be performed.)

☐ **Application Extensions Functional Design (MD.050)**

The Application Extensions Functional Design includes information on custom database objects, such as database triggers or views, that are required to meet the chosen design alternative. If Create Application Extensions Functional Design was not performed, this deliverable will not exist. (See the task description for MD.050 for more information on when this task should be performed.)

☐ **Oracle Technical Reference Manuals**

The *Oracle Technical Reference Manuals* for the Applications contain database diagrams and table descriptions for each applications product.



Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review customization data requirements.	
2.	List the requirements and database extensions needed to satisfy them.	Overview
3.	Create integrated data model.	Data Model
4.	Define logical database.	Logical Database Design
5.	Define indexes.	Index Design
6.	Define physical database parameters.	Physical Database Design
7.	Create flexfield design.	Flexfield Design

No.	Task Step	Deliverable Component
8.	Review the design with technical analysts.	
9	Secure acceptance that the Database Extensions Design includes criteria for compliance with Century Date standards.	Acceptance Certificate (PJM.CR.080). (See the Tools section for information on modifications to the standard Acceptance Certificate template.)

Table 5-20 Task Steps for Design Database Extensions

Approach and Techniques

The Database Extension Design defines changes to the objects in the database prior to designing the individual modules that make up all of the defined customizations. In reality, it is often a distributed activity where individual designers define the database extension requirements as they are writing the Application Extensions Functional Design (MD.050) or Application Extensions Technical Design (MD.070). However, it is still important to assign one database designer to maintain the overall data model.

Because Oracle Applications ship with a predefined database model, this task is limited to new database objects needed to implement required functionality. If you do not require new database objects, the scope of this document is limited to flexfield design.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

When designing and building application extensions, it is essential that all dates be entered, stored, and processed using the full four-digit year for compliance with Century Date standards. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Custom Development Method

If you have significant database extensions, you may require a more rigorous approach to data modeling and database design. Oracle's Custom Development Method (CDM) and Custom Development Method Fast Track (CDM-FT) include complete processes for database design that you can integrate easily with AIM.

Integration with Standard Tables

When you add tables to implement new business objects and entities, it is important to show how those new tables relate to the existing database schema on an entity relationship diagram. Use a unique box style to clearly distinguish new from existing entities.



Warning: Oracle strongly advises users not to modify existing tables. Such modification will likely lead to failure of future upgrades.

Since you should not modify existing tables, you may need to add new tables, even if your requirement is to add attributes (columns) to an existing entity. The new table will have a one-to-one relationship with the existing table and use the same unique identifier columns. The only time this should be a requirement is when descriptive flexfields cannot accommodate the required attributes.

Oracle Designer

If you are using Oracle Designer and have the Oracle Applications Oracle Designer database, you can use the sharing feature to include Applications entities and relationships on your diagrams.



Reference: Barker, Richard. *CASE*Method Entity Relationship Modeling*. Addison-Wesley, 1990. ISBN: 0201416964.

Descriptive Flexfields

Descriptive flexfields are a type of database extension that do not require any new tables or columns. However, because several modules may use flexfields on the same table for different reasons, it is important to have a single point of contact to manage the flexfield definitions for application entities.

Coordinate with business analysts who need to define flexfield setup parameters as part of Define Application Setups (BR.100).

In a multi-site implementation, this activity is very important because each site may have different requirements for descriptive flexfields and you may need a single definition that satisfies all requirements. You may need a global data registry to fully satisfy your requirements. For more information, see Application and Technical Architecture (TA).

Linkage to Other Tasks

The Database Extensions Design is an input to the following tasks:

- MD.070 - Create Application Extensions Technical Design
- MD.080 - Review Functional and Technical Designs
- MD.090 - Prepare Development Environment
- MD.100 - Create Database Extensions
- CV.040 - Perform Conversion Data Mapping
- CV.060 - Design Conversion Programs
- DO.080 - Publish Technical Reference Manual
- PT.060 - Design Performance Test Data

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Database Designer	60
Technical Analyst	30
Business Analyst	10

Table 5-21 Role Contribution for Design Database Extensions

Deliverable Guidelines

Use the Database Extensions Design when you must capture and document both the logical data representation and the physical database design.

This deliverable should address the following:

- the relationships between new business objects and existing application entities
- the design of new database tables and related views, grants and synonyms
- indexing requirements
- tablespace and sizing requirements
- flexfield designs

Deliverable Components

The Database Extensions Design consists of the following components:

- Overview
- Data Model
- Logical Database Design
- Index Design

- Physical Database Design
- Flexfield Design

Overview

This component lists the requirements and the database extensions necessary to satisfy them.

Data Model

This component provides a graphical representation of new business objects and entities in the form of an entity relationship diagram. Include existing application entities and the relationships between standard and new entities.

Logical Database Design

This component transforms the business data model into the specific tables and columns required to support the requirements. The logical database design also transforms relationships into foreign key columns or intersection tables. If your custom approach to satisfying business issues involves views, grants, and synonyms, you must include definitions for each.

Because of the nature of business requirements mapping, you may actually start with the logical design, since your analysis is already at a very granular level and work backwards to construct the Data Model to provide a graphical business view.

Index Design

This component identifies when indexes are needed on newly defined tables and columns. The index design also identifies the type of index required. Your decisions will be based on the specific data usage by custom modules and anticipated data volume.

Physical Database Design

This component identifies the specific Oracle tablespace and sizing parameters for each table and index. The growth and usage properties of the data should be considered; for large tables with high query loads, disk striping strategies should be considered.

Flexfield Design

This component records common flexfield definitions across business areas.

Some flexfield requirements may not surface until module designers are preparing the Application Extensions Functional Design (MD.050) or Application Extensions Technical Design (MD.070) documents. This component acts as an ongoing repository of information throughout the design process.



Suggestion: If you do not have other database extensions, you can rename this deliverable *Flexfield Design*, since that will be the primary content. Your primary input will be the Application Extension Definition and Estimates (MD.020) and the Mapped Business Data (BR.040).

Tools

Deliverable Template

Use the Database Extensions Design template to create the deliverable for this task.

Century Date Acceptance

To document the review and approval of Database Extensions Design for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) replacing the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Acceptance Certificate should be filed with the deliverable in the project library.

Oracle Designer

Use Oracle Designer to create the integrated data model and transform it into the logical database design. You can also specify the index design and physical database design. Build your document by printing Oracle Designer reports and assembling them for review and approval.

Use Oracle Designer to generate data definition languages (DDL) scripts from the information in the Oracle Designer database. If you later need to make changes to the database definitions, you should modify the information in Oracle Designer first and then generate new DDL scripts for future installations.

Visio

You can use the Oracle Entity Relationship Diagramming template in Visio to create your integrated data model.

MD.070 - Create Application Extensions Technical Design (Optional)

In this task, you document the technical specifications for modifications, extensions, and configurable extensions.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Application Extensions Technical Design**. It describes the technical requirements for each program module that comprises an application extension. It provides the technical specifications needed by the developer to build the customization and serves as the source for technical documentation needed for maintenance and update of the modules.

Prerequisites

You need the following input for this task:



Mapped Business Data (BR.040)

The Mapped Business Data includes information on business objects and related attributes that must be supported by the Application Extensions Technical Design.



Design Standards (MD.030)

Design Standards define the format and content standards for design deliverables. If Define Design Standards was not performed, this deliverable will not exist. (See the task description for MD.030 for more information on when this task should be performed.)



Build Standards (MD.040)

Build Standards define the standards for building custom modules. Some standards may impact the Application Extensions Technical Design. If Define Build Standards was not performed, this deliverable

will not exist. (See the task description for MD.040 for more information on when this task should be performed.)

 Application Extensions Functional Design (MD.050)

The Application Extensions Functional Design describes the functionality that the technical design must support. Every feature and business rule described in the functional design must have supporting logic detailed in the technical design. If Create Application Extensions Functional Design was not performed, this deliverable will not exist. (See the task description for MD.050 for more information on when this task should be performed.)

 Database Extensions Design (MD.060)

The Database Extensions Design identifies the new, custom database objects and describes how the new modules access these objects. If Design Database Extensions was not performed, this deliverable will not exist. (See the task description for MD.060 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review Application Extensions Functional Design (MD.050).	
2.	Describe the high-level approach.	Technical Overview
3.	Define detailed program logic for modules (forms, reports, and programs).	Form Logic; Concurrent Program Logic; Database Design
4.	Document integration issues.	Integration Issues
5.	List installation requirements.	Installation Requirements

No.	Task Step	Deliverable Component
6.	Document any additional information that may be helpful during the implementation of the customization.	Implementation Notes
7.	Update Application Extensions Functional Design (MD.050), as needed.	
8.	Update Database Extensions Design (MD.060), as needed.	

Table 5-22 Task Steps for Create Application Extensions Technical Design

Approach and Techniques

Technical Details

The Application Extensions Technical Design documents the technical specifications for every form, report, program, database trigger, flexfield, and all other components of a customization. The combination of functional and technical designs must communicate everything necessary for a developer to code and test all modules of the final customization. It also serves as the technical documentation for the customization that allows the information systems staff or future consultants to understand and make changes to the custom modules.

Oracle Designer

If you are using Oracle Designer, much of the design can be entered directly into Oracle Designer, but you may need to combine Oracle Designer reports with descriptive text you create outside of the tool into a final document for distribution and review. You can find the specific guidelines on the format and content of the design document, including the use of Oracle Designer or other CASE tools, in the Design Standards (MD.030) for your project.

Prototyping

Prototyping is a technique that uses development tools to interactively design custom modules. It is different from the prototyping technique used in the mapping process. During mapping, prototyping uses the standard applications to confirm and demonstrate support for business processes. With custom design, prototyping uses partially created code to design and illustrate new functionality.

Prototyping components of the approach can be a good way to review and validate the concepts with users. This is most useful with forms, since users can interact with the *design* instead of merely reading a static description. Prototypes may not include all of the required logic, but can illustrate the basic look and feel. If an Oracle Designer Generator will be used to build the final forms, or other customizations, you can also generate preliminary prototypes easily.

Prototyping is useful for components such as database triggers, when you are not sure if a proposed trigger will produce the desired result. You may use the prototype for testing purposes, then refine it in the final version.

Upgrades

Unlike standalone custom applications, extensions to packaged applications must be designed so that they can be easily migrated to future releases of the base product. Fortunately, Oracle Applications include several features that facilitate upgrades.

Configurable Extensions

Configurable extensions such as flexfields, and alerts are automatically retained when you upgrade to new application releases . Leverage these features when designing customizations. For example, instead of modifying a form by adding a new section, define a new form with the required information and build a link between the two forms. Use descriptive flexfields whenever you need additional data elements, instead of modifying standard tables.

Custom ORACLE IDs

An ORACLE ID identifies a registered database user that is in Application Object Library. Each standard Oracle Application has a corresponding ORACLE ID. Define all custom tables in an ORACLE ID you create for your custom application.

Use a prefix (such as CSTM or an appropriate acronym for your organization) to distinguish custom ORACLE IDs and tables from standard ones. For example, if you create a custom Purchasing form that updates a custom table, you would create the table in the CSTMPO ORACLE ID and name the table CSTMPO_HEADER_DETAILS.

Grants and synonyms allow your forms and other custom programs to access the custom table.



Attention: In release 10.6 and beyond, all Applications access tables from the APPS ORACLE ID. This special ORACLE ID primarily contains synonyms to access tables in other ORACLE IDs. You also need grants from your custom ORACLE ID to APPS and corresponding synonyms in APPS so your forms and programs can access your custom tables.



Reference: *Application Object Library Reference Manual.*

Database Triggers and Post-Processors

If a change in the standard processing logic must take place, consider using a database trigger or post-processor that runs after a standard program to implement the required logic. This is an extremely powerful technique that allows you to implement special rules, without changing any standard application code.

Oracle Designer

One of the best ways to use Oracle Designer is to capture custom module definitions. You can define basic module information and the module to table relationships (create, read, update, and delete). You can even capture this information at the column level. This allows you to run reports to determine what custom modules are affected by changes to tables and columns during an upgrade.



Suggestion: Oracle publishes a database changes manual with each major Applications upgrade. Use this manual, together with the information in your Oracle Designer repository, to perform impact analysis.

Linkage to Other Tasks

The Application Extensions Technical Design is an input to the following tasks:

- MD.080 - Review Functional and Technical Designs
- MD.110 - Create Application Extension Modules
- TE.020 - Develop Unit Test Script
- TE.030 - Develop Link Test Script
- TE.050 - Develop Systems Integration Test Script
- PM.030 - Develop Transition and Contingency Plan

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	80
Business Analyst	10
Developer	10

Table 5-23 Role Contribution for Create Application Extensions Technical Design

Deliverable Guidelines

Use the Application Extensions Technical Design to provide a road map for building and testing custom system modules. Technical design components are very specific; they use a technical language to describe the construction of form and program logic, database entities, and system integration.

This deliverable should address the following:

- technical specifications for the application extensions

Deliverable Components

The Application Extensions Technical Design consists of the following components:

- Technical Overview
- Form Logic
- Concurrent Program Logic
- Integration Issues
- Database Design
- Installation Requirements
- Implementation Notes

Technical Overview

This component introduces the Application Extensions Functional Design as a complete detailed design.

Form Logic

This component describes navigation logic, table and view usage and provides a summary of each zone, the fields within each zone, and any special logic required.

Concurrent Program Logic

This component lists the calling arguments, log output, table and view usage, program logic and other considerations for a custom report or other concurrent program.

Integration Issues

This component describes any integration issues associated with implementing the application extension.

Database Design

This component summarizes new and changed database objects.

Installation Requirements

This component documents the installation scripts for installing the application extension.

Implementation Notes

This component describes how the application customization was developed and implemented.

This is a technical task that requires highly specialized skills. You may need to subcontract external resources to help you with this task.

Tools

Deliverable Template

Use the Application Extensions Technical Design template to create the deliverable for this task.



Suggestion: Use the Microsoft Word Edit/Copy and Edit/Paste menu options to add new components for each custom module. Include other components only once. Add the implementation notes component after building the customization. For specific information on each component of the design document, consult the Design Standards (MD.030) for your project.

Oracle Designer

Oracle's suite of CASE products provides the features you need to support the development of customizations. Use Oracle Designer to document database extensions and record module-to-table cross references for upgrade impact analysis.

MD.080 - Review Functional and Technical Designs (Optional)

In this task, you set up a design review meeting between business analysts, key users, technical analysts, and developers. The goal is to secure final acceptance of the complete designs.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Approved Designs**. These designs provide management approval of the functional and technical designs for the application extensions. This approval indicates management's agreement to proceed with development.

Prerequisites

You need the following input for this task:



Application Extension Definition and Estimates (MD.020)

The Application Extension Definition and Estimates contains the estimated work effort to complete build and testing. You need to confirm or adjust these estimates based on information learned during the design tasks. If Define and Estimate Application Extensions was not performed, this deliverable will not exist. (See the task description for MD.020 for more information on when this task should be performed.)



Application Extensions Functional Design (MD.050)

The Application Extensions Functional Design is the primary document that business analysts and users review. If Create Application Extensions Functional Design was not performed, this deliverable will not exist. (See the task description for MD.050 for more information on when this task should be performed.)

❑ Database Extensions Design (MD.060)

The Database Extensions Design identifies the new, custom database objects and describes how the new modules access these objects. If Design Database Extensions was not performed, this deliverable will not exist. (See the task description for MD.060 for more information on when this task should be performed.)

❑ Application Extensions Technical Design (MD.070)

The Application Extensions Technical Design provides the details required by developers. If Create Application Extensions Technical Design was not performed, this deliverable will not exist. (See the task description for MD.070 for more information on when this task should be performed.)

Optional

You may need the following input for this task:

❑ Mapped Business Requirements (BR.030)

The Mapped Business Requirements describe the original requirements for the customizations. You may need to reference these requirements to resolve issues.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Distribute designs prior to meeting.	Review Comment List (PJM.QM.020)
2.	Present an overview of each design.	
3.	Address issues and questions.	

No.	Task Step	Deliverable Component
4.	Walk through the detail with developers.	
5.	Update designs as needed.	
6.	Obtain approval from developers and requester.	Acceptance Certificate (PJM.CR.080)
7.	Secure acceptance that the Approved Designs include criteria for compliance with Century Date standards.	Acceptance Certificate (PJM.CR.080) (See the Tools section for information on modifications to the standard Acceptance Certificate template.)

Table 5-24 Task Steps for Review Functional and Technical Designs

Approach and Techniques

Scope of Review

The Review Functional and Technical Designs task can cover one or more customizations for a common business area. The number of designs to cover in one meeting depends on the complexity of the designs, the audience, and the completion time of designs. When parallel design and build activities are taking place, reviews usually cover individual designs so build can begin as soon as you secure approval.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C / Date support rather than Year2000 or

Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

When designing and building application extensions, it is essential that all dates be entered, stored, and processed using the full four-digit year for compliance with Century Date standards. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Functional and Technical Content Division

Since a complete design includes both the Application Extensions Functional Design (MD.050) possibly revised and the Application Extensions Technical Design (MD.070), the design review must cover both areas. Due to the mixture of the audience, you may wish to cover the functional aspects first with everyone, then allow non-technical participants to leave while you discuss technical details. You could also schedule two separate meetings, although developers will benefit from the functional discussion.

Linkage to Other Tasks

The Approved Designs are an input to the following tasks:

- MD.110 - Create Application Extension Modules
- MD.120 - Create Installation Routines
- DO.080 - Publish Technical Reference Manual
- TE.070 - Perform Unit Test
- TE.080 - Perform Link Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	40
Business Analyst	30
Developer	30
Business Line Manager	*
User	*

Table 5-25 Role Contribution for Review Functional And Technical Designs

* Participation level not estimated.

Deliverable Guidelines

Use the Approved Designs to represent an acknowledgment that all relevant parties involved have reviewed the design documents and agree on the approach, content, and functionality described therein. This deliverable serves as a formal record of the parties' agreement and authorizes the project to move forward to subsequent build and test activities.

This supporting document should address the following:

- design reviewer comments
- acceptance of the designs and approval to proceed to development

The parties usually note any desired changes and agree on a course of action to implement those changes. Include the following information in the meeting notes:

- name of deliverable being confirmed
- type and status of deliverable
- objectives of deliverable
- agreements (expressed in terms of planned future actions or policy changes)
- key decisions
- key assumptions
- exceptions or references to components requiring changes
- control (signatures of approvers and initiators, dates, revisions, and so on)
- future direction

You can file this document as an appendix to the complete design to clarify any issues encountered later.

You can prepare a separate Acceptance Certificate (PJM.CR.080) for each design or use the signature area on the front of the documents (if you use an Acceptance Certificate, you should delete the signature area from the documents).

The Acceptance Certificate should be signed by the following individuals:

- business analyst who identified the requirements
- user (a representative of the people who will be using the new functionality)
- developer who will code the modules (or an alternate representative who can confirm that the design contains adequate technical detail)

Deliverable Components

Use the following Project Management Method (PJM) templates to support the deliverable for this task:

- Review Comments List (PJM.QM.020)
- Acceptance Certificate (PJM.CR.080)

Review Comments List (PJM.QM.020)

This deliverable template is used to document comments and required actions identified during the review. A Review Comments List can be included with the design documents when they are distributed prior to the review session.

Acceptance Certificate (PJM.CR.080)

This deliverable template is used to prepare the final design Acceptance Certificate.

Tools

Deliverable Template

A deliverable template is not provided for this task. Use the following Project Management Method (PJM) templates to support the deliverable for this task:

- Review Comments List (PJM.QM.020)
- Acceptance Certificate (PJM.CR.080)

Century Date Acceptance

To document the review and approval of Approved Designs for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) replacing the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Acceptance Certificate should be filed with the deliverable in the project library.

MD.090 - Prepare Development Environment (Optional)

In this task, you establish a platform and software environment that supports custom development.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or *performance testing*, you should perform this task.

Deliverable

The deliverable for this task is the **Development Environment**. It is a working environment that includes all servers, applications, infrastructure, and development tools required to develop and test application extensions.

Prerequisites

You need the following input for this task:



Physical Resource Plan (PJM.RM.040)

The Physical Resource Plan outlines the plan for all computer environments needed to support the implementation, including the development environment.



Application Setup Documents (BR.100)

The Application Setup Documents include the required setups to support the business processes.



Architecture Requirements and Strategy (TA.010)

The Architecture Requirements and Strategy identifies the application and technical architecture requirements for the Development Environment.

☐ **Design Standards (MD.030)**

Design Standards describe design tools that you must install and configure in the development environment. If Define Design Standards was not performed, this deliverable will not exist. (See the task description for MD.030 for more information on when this task should be performed.)

☐ **Build Standards (MD.040)**

Build Standards describe development tools that you must install and configure in the development environment. If Define Build Standards was not performed, this deliverable will not exist. (See the task description for MD.040 for more information on when this task should be performed.)

☐ **Database Extensions Design (MD.060)**

The Database Extensions Design defines custom database objects required for customizations. You do not create these database objects during this task, but you may need sizing information to allocate database space. If Design Database Extensions was not performed, this deliverable will not exist. (See the task description for MD.060 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review Architecture Requirements and Strategy (TA.010) to understand the strategy for deployment of project environments in general, and the development environment in particular.	

No.	Task Step	Deliverable Component
2.	Update the introduction component to reflect the development environment hosting and environment sharing strategy per the Architecture Requirements and Strategy (TA.010).	Introduction
3.	Update all of the tables in the database tier, applications tier and desktop client tier sections of the environment component to reflect the configuration of all servers within the database, applications and desktop client tiers.	Environment - Development
4.	List any other software applications needed to support design and build.	Other Applications
5.	Update the automated development tools component to reflect the configuration for each conversion tool.	Automated Development Tools
6.	Describe the configuration of any other software applications that are required to support the project.	Other Applications
7.	Describe the configuration of any other software tools that are required to support the design and build activities of the project.	Automated Development Tools
8.	Set up the Development Environment.	

No.	Task Step	Deliverable Component
9.	Install the automated development tools.	

Table 5-26 Task Steps for Prepare Development Environment

Approach and Techniques

The Prepare Development Environment task describes procedures either to verify completeness of previously installed environments or to perform the installation of the development environment for the first time.

The purpose of this deliverable is to confirm that an adequate Development Environment exists to support program development. Downstream testing tasks, such as the unit test (TE.070) and the link test (TE.080) may also be performed in the Development Environment.

Installations

All installations should follow the Optimal Flexible Architecture (OFA) standard.



Reference: Millsap, Cary V. “Optimal Flexible Architecture.” Oracle Magazine, Vol. IX, Nos. 5 and 6, 1995.

The Physical Resource Plan (PJM.RM.040) prepared early in the project outlines the required systems for the entire project, but you may need to reevaluate the plan and consider new issues at this time.

Multiple Environments

The Development Environment is typically very volatile since programs are constantly changing and there may be temporary test data. Also, with the availability of database triggers and stored procedures, the only way to unit test some programs is to install them in the database. Therefore, the development database must be separate from any other testing environment — particularly if any mapping or testing is taking place simultaneously.

Interface programs may be developed and tested in the Development Environment or in a separate environment. Interface testing may involve loading large batches of data and then deleting the data and starting over. If testing the interfaces could disrupt other development activities, create a separate environment, as documented in Prepare Testing Environments (TE.060).

Software Tools

Create the directory structures to hold custom source code and register custom applications in Application Object Library. You need a script or documented procedure to create this structure on each environment that requires custom extensions, potentially including the (Business System) Testing, Performance Test, User Learning, and Production Environments.

Implement the source code control software and verify that it functions as indicated in the Build Standards (MD.040). Install the design and development tools and verify that they function as required.

Upgrades

Throughout the course of the implementation, you may implement major and minor upgrades. The Development Environment may be the first environment where the application of an upgrade takes place in order to test and update customizations.

Linkage to Other Tasks

The Development Environment is an input to the following tasks:

- MD.100 - Create Database Extensions
- MD.110 - Create Application Extension Modules
- PT.080 - Create Performance Test Transaction Programs
- PT.090 - Create Test Database Load Programs

Role Contribution

The percentage of total task time required for each role follows:

Role	%
System Administrator	50
Database Administrator	25
Technical Analyst	25
Client Staff Member	*

Table 5-27 Role Contribution for Prepare Development Environment

* Participation level not estimated.

Deliverable Guidelines

Use the Development Environment deliverable to document all of your installation and configuration steps. To maintain the integrity and performance of the system, complete a thorough log documenting the changes and updates to all environments.

Execute queries against system tables and capture the output to document tablespaces, database files, and users. Use available worksheets and checklists to plan, track, and verify the installation process.

This deliverable should address the following:

- available disk space
- estimated load for each application
- tablespace requirements for the Oracle RDBMS
- user accounts
- a checklist for verifying the installation and configuration of the system

Deliverable Components

The Development Environment template consists of the following components:

- Introduction
- Environment - Development
- Other Applications
- Automated Development Tools

Introduction

This component describes the purpose for the Development Environment and the detailed configuration approach taken to implement the environment.

Environment - Development

This component describes the configuration for the database tier, applications tier, and desktop client tier servers in detail. It also describes the configuration of the hardware platforms on which these server environments execute.

Other Applications

This component describes the configuration of any other software applications that are required to support the project.

Automated Development Tools

This component describes the configuration of any other software tools that are required to support the development activities of the project.

Tools

Deliverable Template

Use the Development Environment template to create the deliverable for this task.

MD.100 - Create Database Extensions (Optional)

In this task, you create new tables, indexes, views, grants, and synonyms to support custom module development.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Custom Database Extensions**. It consists of the custom database object creation scripts required to support approved application extensions.

Prerequisites

You need the following input for this task:



Database Extensions Design (MD.060)

The Database Extensions Design defines the custom database objects that you must create. If the design is held in Oracle Designer, then you need privileges to access the information. If Design Database Extensions was not performed, this deliverable will not exist. (See the task description for MD.060 for more information on when this task should be performed.)



Development Environment (MD.090)

The configured Development Environment provides the database and applications installation. If Prepare Development Environment was not performed, this deliverable will not exist. (See the task description for MD.090 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review database extension design.	
2.	Create new database users.	
3.	Register new ORACLE IDs.	
4.	Build or generate database object creation scripts.	Database Object Creation Scripts
5.	Run database object creation scripts.	New Database Objects
6.	Confirm database objects.	
7.	Update design documents, as needed, to reflect final development.	

Table 5-28 Task Steps for Create Database Extensions

Approach and Techniques

Separate ORACLE IDs

Create custom database objects in custom ORACLE IDs (an Oracle user name registered within Application Object Library) so that custom objects are not mixed with standard database objects.



Reference: “Customization Standards,” *Application Object Library Reference Manual*.

Grants and Synonyms

Include scripts that create the database grants and synonyms required to allow other ORACLE IDs to access the custom tables. Since custom tables, views, indexes, and sequences exist in a custom ORACLE ID, you need grants and synonyms to allow custom forms and concurrent programs to access the tables.

Database Extensions Creation During Program Construction

If you are not using Oracle Designer to store your new database designs, you can wait to create tables, views, and synonyms until you are coding the modules that require them. However, create scripts for every database object you define, so that you can easily recreate them in other environments.

Linkage to Other Tasks

The Custom Database Objects are an input to the following tasks:

- MD.110 - Create Application Extension Modules
- TE.060 - Prepare Testing Environments
- PT.100 - Construct Performance Test Database

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Database Administrator	80
Database Designer	20

Table 5-29 Role Contribution for Create Database Extensions

Deliverable Guidelines

The deliverable for this task is a complete set of database object creation scripts. The completion criteria for this deliverable includes:

- set of object creation scripts
- script code consistent with design and build standards
- all tables, indices, views, and columns created correctly
- all objects accessible from the APPS ORACLE ID (Release 10.6 and beyond)

This deliverable should address the following:

- database object creation scripts
- creation of tables, views, and columns
- accessibility of all objects from the APPS Oracle ID

Tools

Deliverable Template

A deliverable template is not provided for this task.

Oracle Designer

In Oracle Designer (or other CASE tools that support Oracle), you can create the necessary database objects by simply selecting the appropriate *create database* option in the tool.

MD.110 - Create Application Extension Modules (Optional)

In this task, you produce the modules to support customizations to the Applications. You also perform the first round of testing as part of this task.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Module Source Code**. It consists of the actual program code for the approved application extensions identified in Approved Designs (MD.080).

Prerequisites

You need the following input for this task:

☐ **Build Standards (MD.040)**

The Build Standards define the standards for building custom modules. If Define Build Standards was not performed, this deliverable will not exist. (See the task description for MD.040 for more information on when this task should be performed.)

☐ **Application Extensions Functional Design (MD.050)**

The Application Extensions Functional Design describes the functionality that the technical design must support. Every feature and business rule described in the functional design must have supporting logic detailed in the Application Extensions Technical Design (MD.070). If Create Application Extensions Functional Design was not performed, this deliverable will not exist. (See the task description for MD.050 for more information on when this task should be performed.)

☐ **Application Extensions Technical Design (MD.070)**

The design documents provide the information you need to code the custom modules. If Create Application Extensions Technical Design was not performed, this deliverable will not exist. (See the task description for MD.070 for more information on when this task should be performed.)

☐ **Approved Designs (MD.080)**

Approved Designs provides management approval of the functional and technical designs for the application extensions. If Review Functional and Technical Designs was not performed, this deliverable will not exist. (See the task description for MD.080 for more information on when this task should be performed.)

☐ **Development Environment (MD.090)**

The configured Development Environment provides the operating system directories, tools, database, and applications you need to develop and debug program modules. If Prepare Development Environment was not performed, this deliverable will not exist. (See the task description for MD.090 for more information on when this task should be performed.)

☐ **Custom Database Objects (MD.100)**

Custom tables, views, and sequences are referenced in the Application Extensions Technical Design (MD.070). If Create Database Extensions was not performed, this deliverable will not exist. (See the task description for MD.100 for more information on when this task should be performed.)

Optional

You may need the following input for this task:

☐ **Unit Test Script (TE.020)**

The Unit Test Script provides guidance on how to test individual application extensions. If Develop Unit Test Script was not performed, this deliverable will not exist. (See the task description for TE.020 for more information on when this task should be performed.)

 Link Test Script (TE.030)

The Link Test Script defines link tests that test application extensions as part of a business flow. If Develop Link Test Script was not performed, this deliverable will not exist. (See the task description for TE.030 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review detailed design documents.	
2.	Code modules.	Source Code
3.	Configure non-coded modules in the Application Object Library.	
4.	Register custom modules in the Application Object Library.	
5.	Perform initial unit tests.	
6.	Configure custom menus and report security groups to access custom forms and reports.	
7.	Perform initial link tests.	
8.	Update design documents, as needed, to reflect final development.	

No.	Task Step	Deliverable Component
9.	Add implementation notes section to Application Extensions Technical Design (MD.070).	

Table 5-30 Task Steps for Create Application Extension Modules

Approach and Techniques

The specific approach you follow depends on the development tools you are using and the Build Standards (MD.040) defined for your project.

Testing

Creating a source module is an iterative process. You code a portion of the module, test, apply bug fixes, and retest. Then you add more functionality and repeat the process. When you have incorporated all required functionality and believe that no defects remain, you formally submit the module for unit (TE.070) and link (TE.080) testing.

The figure below illustrates the iterative nature of coding and unit testing.

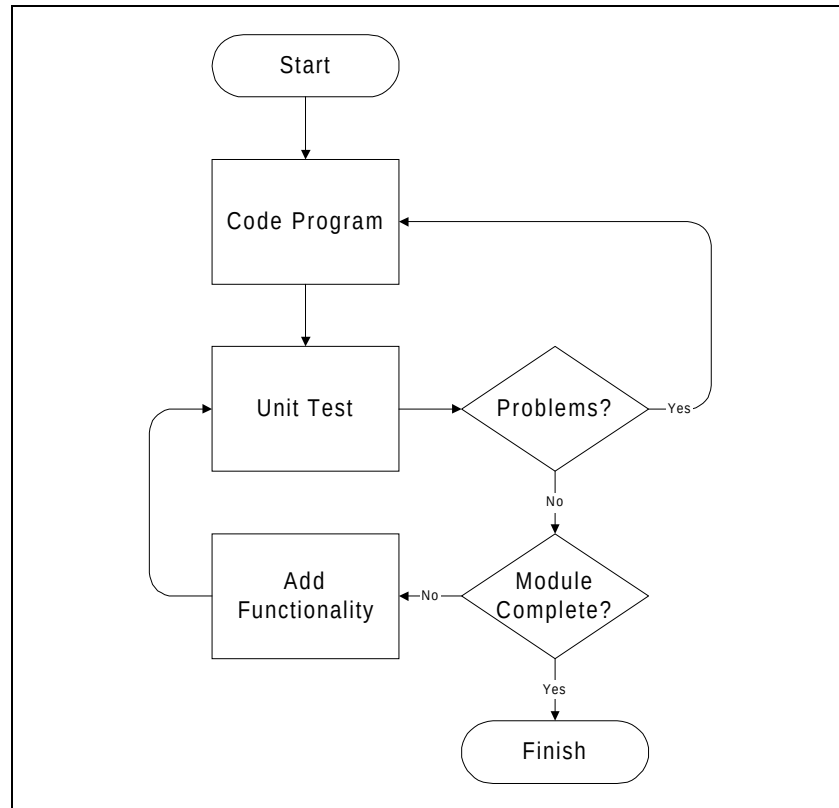


Figure 5-4 The Incremental Code and Unit Test Cycle

When you turn the code over to someone else for testing, you should be confident that your code will pass formal unit (TE.070) and link (TE.080) testing. For more information, see Business System Testing (TE).

Design Review

If you did not design the custom modules, meet with the designer and review the design documents in detail. Although you may have participated in a design review during Review Functional and Technical Designs (MD.080), it included a larger audience and its primary purpose was to secure approval. It may not have covered the detail necessary to prepare for coding.

Oracle Designer Generator

If you used Oracle Designer to design custom forms and reports, you can generate the first-cut version automatically. You can then add additional logic, as required to each module. Part of your development process will be to configure generator preferences and special rules for each module.



Warning: To successfully generate forms for Oracle Applications, the version of Oracle Forms must be the same as used by the particular release of the Oracle Applications.

You can incorporate support for the following features in Oracle Forms by properly configuring the default form template and module specification in Oracle Designer:

- descriptive flexfields
- who columns
- inter-block navigation buttons

You may need to modify the generated forms or create a module-specific attached library to add support for the following elements:

- properly formatted window titles
- pop-up calendar on date fields
- current row indicator
- message dictionary for messages
- alternate regions

Linkage to Other Tasks

The Module Source Code is an input to the following tasks:

- MD.120 - Create Installation Routines
- TE.060 - Prepare Testing Environments
- TE.070 - Perform Unit Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Developer	90
Technical Analyst	10

Table 5-31 Role Contribution for Create Application Extension Modules

Deliverable Guidelines

The deliverable for this task is the Module Source Code for application extensions. When developing the custom module source code, you should give special consideration to the following issues:

- Does code adhere to coding standards?
- Does code support the business purpose described in the Application Extensions Functional Design?
- Did you use appropriate tools?
- Have you updated documentation with technical or design changes you identified during development?
- Did you place source code under version control?
- Does the code meet all performance, maintenance, and integration requirements?

The creation of custom programs also covers modifications to standard Oracle Applications products. Follow these guidelines when developing modifications:

- Code is protected from standard Oracle Applications upgrades.
- Code does not alter the behavior of standard functions and features other than specified in design source code.
- Code is compatible with other integrated subsystems.
- Software patches can be applied to modified code.



Suggestion: Patches can overwrite many hours of development work and systems can crash and delete soft copies of custom code. For safety reasons, regularly print a list of the custom code and always include a list of the final module names, with annotations, in the implementation notes component of the Application Extensions Technical Design (MD.070).

This deliverable should address the following:

- the development of module source code for application extensions using the standards defined in the Build Standards (MD.040)

Tools

Deliverable Template

A deliverable template is not provided for this task.

Oracle Designer

Use the Oracle tools to develop custom forms and reports. The Application Extension Strategy (MD.010) for your project defines the specific tools you will use.

Application Object Library

Use Application Object Library to directly define the following customizations:

- menus
- responsibilities
- descriptive flexfields
- alerts
- profile options
- custom messages
- custom help text

MD.120 - Create Installation Routines (Optional)

In this task, you develop automated functions and detailed instructions to install customizations in the testing and production environments.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is a set of **Installation Routines**. These routines include documented instructions for installing all of the custom modules in the testing and production environments. Not all customizations can be installed with an automated script, so the instructions may include manual steps as well.

Prerequisites

You need the following input for this task:



Build Standards (MD.040)

Follow the Build Standards when building SQL*Plus, PL/SQL, and other scripts to automate custom module installation. If Define Build Standards was not performed, this deliverable will not exist. (See the task description for MD.040 for more information on when this task should be performed.)



Approved Designs (MD.080)

Approved Designs are functional and technical design documents with any updates identified during coding. If Review Functional and Technical Designs was not performed, this deliverable will not exist. (See the task description for MD.080 for more information on when this task should be performed.)

❏ Module Source Code (MD.110)

Module Source Code are completed custom modules configured in the development environment. If Create Application Extension Modules was not performed, this deliverable will not exist. (See the task description for MD.110 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Determine the installation technique for each module.	
2.	Confirm proper installation and configuration of modules in the Development Environment (MD.090).	
3.	Export data for components supported by a seed data loader.	Seed Data ASCII Files
4.	Create PL/SQL scripts for supported application programming interfaces (APIs).	PL/SQL Scripts
5.	Write lookup seed data population scripts.	Seed Data SQL Scripts
6.	Initialize the test environment.	
7.	Test discrete installation steps.	

No.	Task Step	Deliverable Component
8.	Package scripts and commands into an operating system script.	Master Install Routine
9.	Document manual steps.	Installation Instructions
10.	Refresh the Test Environment.	
11.	Test final installation process.	
12.	Place finished routines under source control.	

Table 5-32 Task Steps for Create Installation Routines

Approach and Techniques

The objective of Create Installation Routines is to create an installation process for each customization that a system administrator can reliably execute to install the required modules and supporting setups into any destination environment. You should strive for fully automated installation, but manual steps are acceptable as long as the instructions are clear.

Environment Preparation

Before you can install individual custom modules into a new environment, you must prepare the target environment for customizations. You can automate some of the steps, but you must perform others manually. To prepare a new environment you must perform these actions:

1. Create a directory structure for each customization / custom application.
2. Add environment variables for the root directory structures to the application environment file.
3. Register customizations as custom applications with Application Object Library.

4. Shut down and restart the concurrent manager.
5. Create custom database users.
6. Register custom ORACLE IDs.



Suggestion: If you have a master setup instance that you plan to import into future environments, you can preconfigure the master setup instance with the required custom applications.

Module Installation

Although you can simply copy some types of source and executable code to the proper destination directory, most Applications extensions require you to register the modules in Application Object Library or perform some other configuration. For example, to install a custom report you need to perform the following actions:

1. Copy the source and executable report files to the proper custom directory.
2. Register the executable file in Application Object Library.
3. Create custom value sets for program arguments.
4. Register the concurrent program and arguments.
5. Create a custom report set.
6. Add new report to custom report set.
7. Add other standard reports to report set.
8. Associate report set to a custom responsibility.

Although you can perform all steps manually in each target environment, this is tedious and error prone.

Program Files

Installing the actual source and executable program files is the easy part. Use one of the following techniques to move or install these files:

- direct copy over the network
- archive and restore utilities
- Oracle® Installer

Application Object Library

You can use several techniques to register the proper information in Application Object Library:

- supported open interface
- supported command-line utility
- supported PL/SQL application programming interface (API)
- custom SQL*Plus scripts
- keyboard capture and playback
- manual entry

Test Environment Refresh

In order to debug your installation routines, you may need to delete information from Application Object Library tables and then re-execute your scripts. The easiest way to do this is simply refresh the entire environment or specific tables from a backup. This also guarantees that your routines will work properly in a completely generic target environment.

Linkage to Other Tasks

The Module Source Code is an input to the following task:

- TE.090 - Perform Installation Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Developer	100

Table 5-33 Role Contribution for Create Installation Routines

Deliverable Guidelines

Use the Installation Routines deliverable to package multiple commands and scripts for a customization into a single, easily executed operating system script (such as a UNIX Bourne shell script). You should be able to re-execute all scripts without errors, even if custom modules are already installed.

Apply the same quality criteria to installation routines as you do to other custom modules. Include standard headers and use comments liberally. Follow build standards for SQL*Plus scripts and PL/SQL procedures.

This deliverable should address the following:

- installation routines for all customizations

Deliverable Components

The Installation Instructions consist of the following component:

- Installation Instructions

Installation Instructions

This component provides detailed step-by-step instructions to install and set up each single system.

Tools

Deliverable Template

Use the Installation Instructions template to create the supporting deliverable for this task (the written instructions that accompany your installation routines).

Seed Data Loaders

Oracle Applications include the following command-line utilities to load seed data from text files:

- Function Security Loader
- Message Dictionary Generator
- User Profile Loader
- User Profile Value Loader



Attention: Consult your Oracle Applications documentation for detailed instructions on using these utilities.

Function Security Loader

The Function Security Loader allows you to move function security data (menus) between the database (where its use is for runtime operation) and a text file representation (where its use can be for distribution). Specifically, you can:

- Download database information to a text file.
- Upload (merge) the information in a text file to the database.

With these download and upload capabilities you can easily propagate function security information that is defined in one database to other databases. The text file version of function security data is also useful for bulk editing operations. In this case, a text editor can accomplish the task more efficiently than a form.

Message Dictionary Generator

The Message Dictionary Generator is a concurrent program that transfers Oracle Applications Message Dictionary messages between three different kinds of message repositories: the database, readable text files, and binary runtime files.

The supported transfer options follow:

- Database to Runtime file (.msb)
- Database to Script file (.msg)
- Script file (.msg) to Database

- Script file (.msg) to Runtime file (.msb)
- Runtime file (.msb) to Database
- Runtime file (.msb) to Script File (.msg)

For installation routine development you generally use only the *Database to Script file* to create the installation script. Your master installation script can then use the *Script file to Database* and *Database to Runtime file* options to install the messages.

User Profile Loader

The User Profile Loader is a concurrent program that can move Oracle Applications user profile information between database and text file representations. Specifically, you can:

- Download database information to a text file.
- Upload (merge) the information in a text file to the database.

With these download and upload capabilities you can easily propagate custom user profile information that is defined in one database to other databases. The User Profile Loader only moves the profile options themselves, not the associated values.

User Profile Value Loader

The User Profile Value Loader is a concurrent program that can move Oracle Applications user profile values between database and text file representations. Specifically, you can:

- Download database information to a text file.
- Upload (merge) the information in a text file to the database.

These download and upload capabilities allow you to easily propagate user profile settings from one database to another. The User Profile Value Loader only moves the profile values; it does not create profile options that do not exist in the destination database.

PL/SQL APIs

Oracle Applications include the following PL/SQL Application Programming Interfaces to facilitate loading or migrating configuration information:

- Concurrent Program API
- Concurrent Manager API
- Descriptive Flexfield API
- Descriptive Flexfield Value Set API

Concurrent Program API

The Concurrent Program API allows you to register concurrent programs using a PL/SQL script. It is implemented as a PL/SQL package. The procedures in this package implement the functionality provided by the corresponding Application Object Library forms.

The procedures in the package allow you to:

- add and delete executables
- add and delete concurrent programs
- add and delete program parameters
- add and delete incompatibility rules

Procedures also exist to:

- add and delete request groups
- add and delete request sets
- add and delete request set parameters
- add and remove concurrent programs to and from request sets
- add and remove request sets to and from request groups

The concurrent program API cannot export information from the database so you must construct PL/SQL scripts manually to call the API functions.

Descriptive Flexfield API

The Descriptive Flexfield API allows you to create descriptive flexfield definitions, contexts, and segments. It is implemented as a PL/SQL package. The procedures and functions in this package allow you to:

- determine whether a flexfield already exists
- register and delete flexfields
- enable flexfield columns
- create and delete segments
- create and delete contexts

For customizations, you generally only manipulate segments and contexts. Use caution when using this package, since it performs slightly less validation than the corresponding Application Object Library forms.

The Descriptive Flexfield API cannot export information from the database, so you must construct PL/SQL scripts manually to call the API functions.

After loading flexfield information, compile the flexfield by running the Flexfield Compiler concurrent program.

Flexfield Value Set API

The Flexfield Value Set API allows you to create flexfield value sets for use in descriptive flexfields, key flexfields, and concurrent program parameters. It is implemented as a PL/SQL package. The procedures and functions in this package allow you to:

- determine if a value set exists
- create, delete, and replace value sets
- support all validation types
- add events for special and pair validation types

The Flexfield Value Set API cannot export information from the database so you must construct PL/SQL scripts manually to call the API functions.

Data Conversion (CV)

This chapter describes the Data Conversion process.

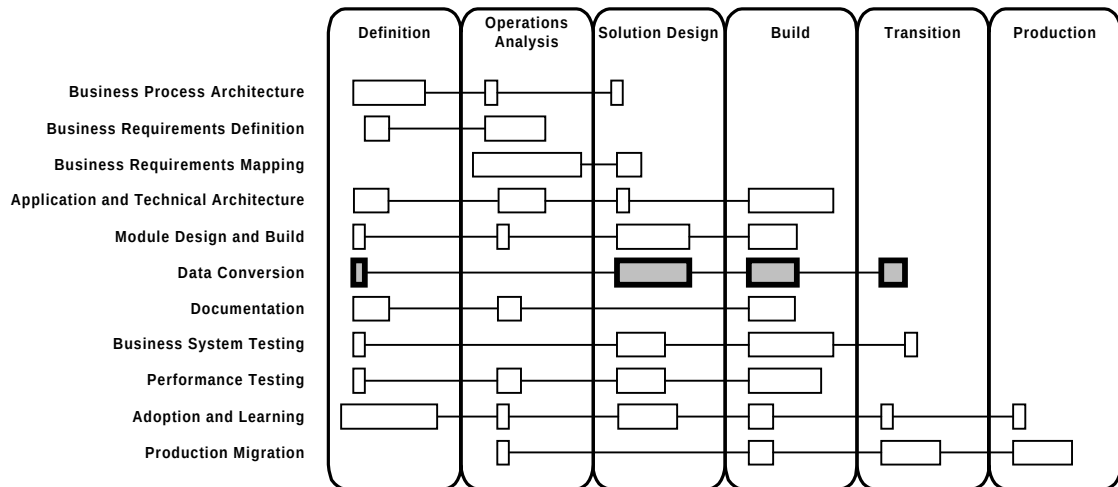


Figure 6-1 Data Conversion Context

Process Flow

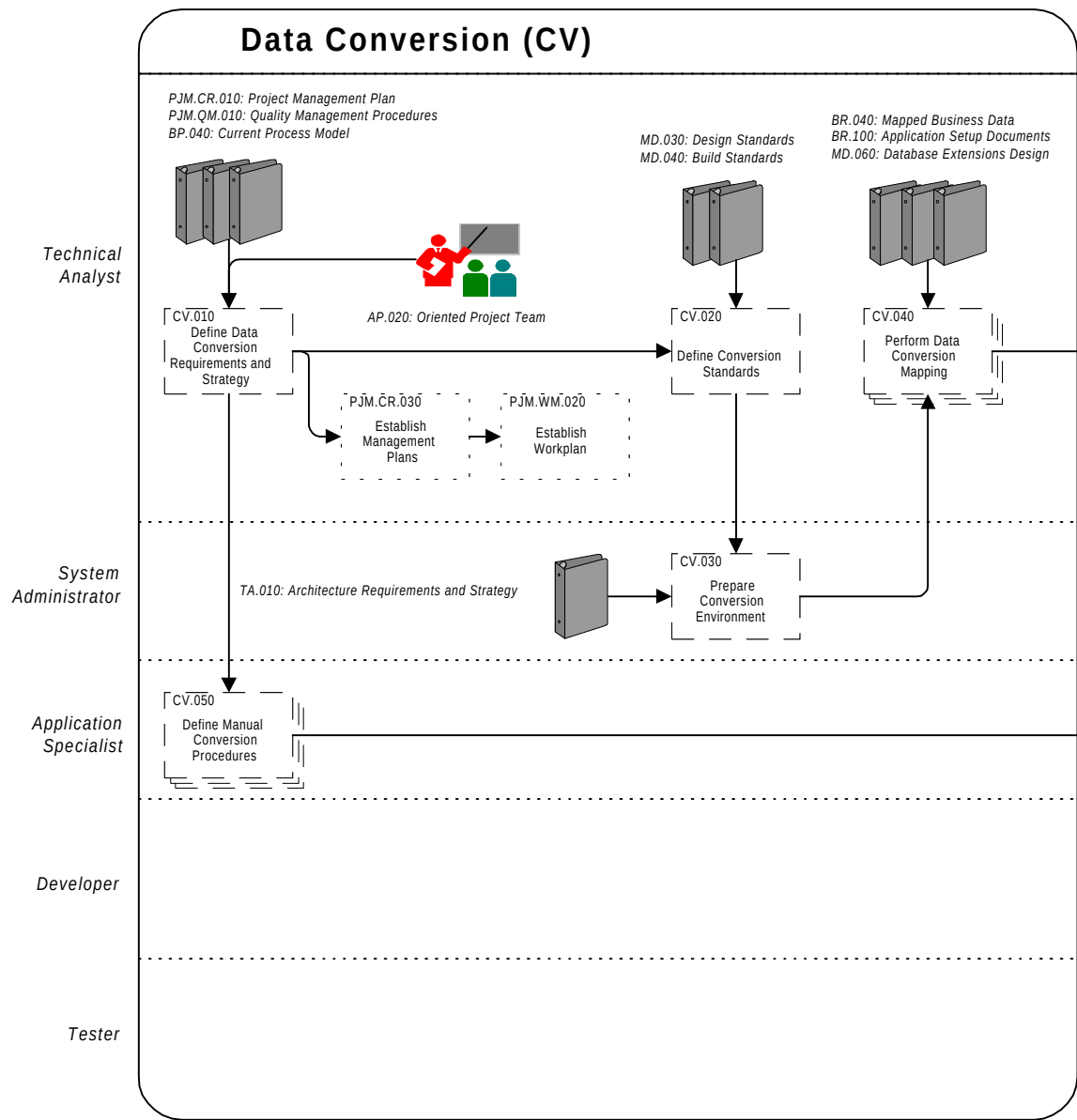


Figure 6-2 Data Conversion Process Flow Diagram

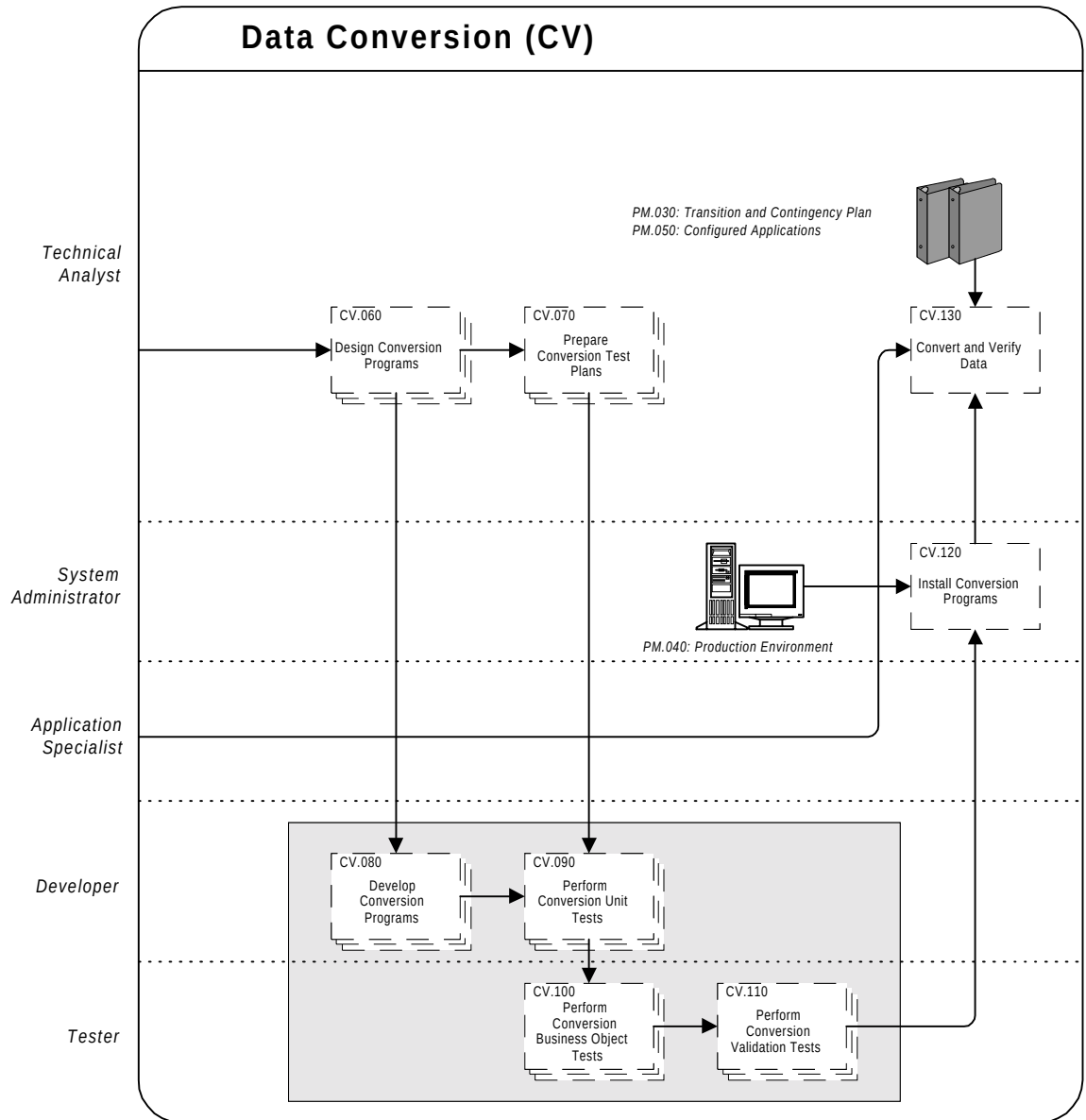


Figure 6-2 Data Conversion Process Flow Diagram (cont.)

Approach

The objective of Data Conversion is to convert and test all necessary legacy data for the operation of the new system. The first step is to explicitly define the data as business objects to be converted, along with their source systems. A business object is a physical or logical object of significance to a business. Examples of business objects include customers, vendors, items, invoices, credit memos, orders, and payments. You may need this data for business system testing, performance testing, acceptance testing, and user learning, as well as for production.

The project team then determines an overall strategy to meet the conversion requirements, defining both automated and manual methods. The process includes designing, coding, and testing any conversion modules that are necessary, as well as the conversion itself.

Conversion Needs

Analyze conversion needs in conjunction with input from the implementing organization’s users, based on the following categories of information:

Configuration/ Setup Data Required in Target Application	Identify setup data (required data in the configuration of the target application) that needs to be converted from the legacy system (for example, item categories and category sets for an inventory control system). The target application requires this data; enter this data during the application system configuration.
---	--

Master Data

Identify business objects that are referenced by, or have a relationship with, the documents and transactional data you will be converting. This usually represents physical objects, people, or businesses. For example, if you want to convert sales orders, you must first convert items and customer data referenced on the sales order. This data differs from setup data because it may or may not be required by the target application during the configuration steps. However, the data is fundamental to the business functions supported by the legacy systems being converted.

Documents

Business objects that represent documents such as sales orders, requisitions, purchase orders, and invoices usually reference both master data and setup information. Do not confuse fundamental business documents with reports that you generate from existing application data, such as a balance sheet or income statement.

Transactions

Transactions represent changes or actions taken against other business objects. Most business objects, such as inventory items, purchase orders, and sales orders, have a history of transactions, but you may not need to convert all past history. Consider consolidating all historical transactions so that the new system only stores the final status. Also consider audit and reporting requirements.

Evaluate each type of data for applicable status (open, closed, canceled, or void), historical postings (weeks, months, or years), and current or future periods.

The Data Conversion Requirements and Strategy (CV.010) helps you analyze and define your conversion requirements.

The amount of effort involved with conversion greatly depends on the condition of the data and the knowledge of the data structures in legacy systems and the Oracle Applications. If data anomalies or an unusual number of exceptions exist in the current system, recommend data

clean-up to the project sponsor. For example, delete obsolete or incorrect part numbers.

Conversion Approaches

Consider these conversion approaches when developing your conversion strategy:

- **Manual Conversion** — In some instances, it makes sense to convert a given business object manually. This decision is primarily driven by the volume of data to be converted and the complexity of the programmatic conversion. In a manual conversion, use the target application forms to key the legacy data into the target application.
- **Traditional Programmatic Conversion** — This conversion approach uses traditional 3GL and 4GL code to convert legacy system business objects to the target application. If the volume of data being converted to the target system is too great to manually key the data into the target application forms, then conversion programs can be written to move and validate the data to the target application tables.
- **Programmatic Conversion using Automated Tools** — Instead of writing traditional conversion programs to perform the conversion, automated conversion tools can be used to facilitate the build and testing of conversion programs. Consider these tools as a means of reducing the time, risk, and expense of performing the conversion.
- **Automated Data Entry.**— This is a variation of manual conversion that takes advantage of special tools that can simulate keyboard entry of information that is stored in a file. The data is entered through the application forms and is subjected to the same edit and validation checks that occur during manual entry; however, the data entry process is fully automated.

These four conversion approaches are referenced throughout this chapter.

Interface Tables

When converting legacy business objects programmatically, do not directly populate Oracle Application tables. Legacy data should first be moved to interface tables, where the data can be manipulated before

being moved to the production tables. Interface tables can either be standard Oracle tables or custom-developed tables.

Standard Oracle interface tables are provided as part of Oracle open interfaces, also known as Application Program Interfaces (API). Open interfaces are powerful, flexible tools that allow you to capture data from your legacy applications, define necessary format conversions, and direct data to your Oracle products, usually with minimal programming. When available, Oracle open interfaces are usually the preferred choice for converting legacy business objects.

Where no open interface is available to support the conversion of a business object, you can write traditional SQL code to create custom-developed interface tables. If you are using an automated conversion tool, you can extend the Oracle production tables to include processing columns for processing rules. The temporary custom-developed interface tables provide a place for you to apply all the rules you have defined for a specific business object, before moving the data into the production tables.



Reference: *Oracle Financials Open Interfaces Guide.*



Reference: *Oracle Manufacturing, Distribution, Sales and Service Open Interfaces Manual.*

Data Cleanup and Normalization

Data cleanup refers to the transformation of data from its current state to a predefined, standardized format. Normalization is a technique to eliminate data redundancy. In the context of an Oracle Applications implementation, the need for data cleanup and normalization of legacy systems data is directly related to the degree to which legacy data meets the structure and data validation requirements of the target applications environment.

The quality, or cleanliness, of the data in the legacy systems can have a significant impact on the scope of the conversion effort. Dirty or inconsistent data can cause serious problems, even system failures, if not corrected prior to the data being loaded into the production tables.

When defining data conversion requirements, it is important to determine the quality of the legacy data to be converted and develop a plan for correcting any data deficiencies that exist. Some data cleanup requirements may be hard to identify. For example, where cost center

or department numbers have been changed and reused over the years to represent different organizational entities, it may not be apparent that historical information cannot be attributed to the entity a given cost center or department value represents today. This type of situation calls for data cleanup prior to conversion and /or thoughtful planning in structuring the conversion of any information associated with the cost center or department business object.

Define legacy systems data that must be combined or adjusted to match the structure of Oracle Applications data. For example, the legacy systems may store vendor addresses as separate vendor records with the same vendor name. In Oracle Purchasing, each vendor has one master record for the name and other common information with related address records stored in a different table.

Determine if the data cleanup will take place in the legacy environment or in the new Oracle Applications environment. As a general rule, legacy data that does not meet the target application environment data validation requirements must be cleaned up before conversion. Only legacy data that meets data validation requirements should be considered for post-conversion cleanup / normalization.

Minimum Data Requirements

Oracle Applications keep track of a tremendous quantity of information about numerous business entities. During conversion data mapping, it may become apparent that the Oracle Applications capture and track many more attributes of the business objects being converted than does the legacy system from which the data is being taken. In such cases, it is necessary to identify default values to populate the Oracle tables with required data that cannot be found in the source system. Document commonly used data defaults so that all conversion team members use the same default values consistently.

Tasks and Deliverables

The tasks and deliverables of this process are as follows:

ID	Task Name	Deliverable Name	Required When	Type*
CV.010	Define Data Conversion Requirements and Strategy	Data Conversion Requirements and Strategy	Project includes <i>programmatic data conversion</i> or <i>manual data conversion</i>	SI
CV.020	Define Conversion Standards	Conversion Standards	Project includes <i>programmatic data conversion</i>	SI
CV.030	Prepare Conversion Environment	Conversion Environment	Project includes <i>programmatic data conversion</i>	SI
CV.040	Perform Conversion Data Mapping	Conversion Data Mapping	Project includes <i>programmatic data conversion</i>	MI
CV.050	Define Manual Conversion Procedures	Manual Conversion Procedures	Project includes <i>manual data conversion</i>	MI
CV.060	Design Conversion Programs	Conversion Program Designs	Project includes <i>programmatic data conversion</i>	MI
CV.070	Prepare Conversion Test Plans	Conversion Test Plans	Project includes <i>programmatic data conversion</i>	MI
CV.080	Develop Conversion Programs	Conversion Programs	Project includes <i>programmatic data conversion</i>	MI
CV.090	Perform Conversion Unit Tests	Unit-Tested Conversion Programs	Project includes <i>programmatic data conversion</i>	MI
CV.100	Perform Conversion Business Object Tests	Business Object-Tested Conversion Programs	Project includes <i>programmatic data conversion</i>	MI

ID	Task Name	Deliverable Name	Required When	Type*
CV.110	Perform Conversion Validation Tests	Validation-Tested Conversion Programs	Project includes <i>programmatic data conversion</i>	MI
CV.120	Install Conversion Programs	Installed Conversion Programs	Project includes <i>programmatic data conversion</i>	SI
CV.130	Convert and Verify Data	Converted and Verified Data	Project includes <i>programmatic data conversion or manual data conversion</i>	SI

*Type: SI=singly instantiated, MI=multiply instantiated, MO=multiply occurring, IT=iterated, O=ongoing. See Glossary.

Table 6-1 Data Conversion Tasks and Deliverables

Objectives

The objectives of Data Conversion are:

- Convert essential business information from the legacy system to the new applications.
- Verify that converted data is accurate and supports the needs of the business.

Deliverables

The deliverables of this process are as follows:

<u>Deliverable</u>	<u>Description</u>
Data Conversion Requirements and Strategy	The conversion requirements, related scope, objectives, approach, and the strategy for implementing the defined conversion requirements.
Conversion Standards	The standards the conversion team follows when performing Data Conversion tasks.

Deliverable	Description
Conversion Environment	The hardware and software environment required to support the design and build activities of the Data Conversion process.
Conversion Data Mapping	The mapping of the legacy system files and data elements to the target application tables and columns.
Manual Conversion Procedures	The plan for converting legacy business objects via manual data entry.
Conversion Program Designs	The designs that detail the program logic and rules coded in the conversion programs.
Conversion Test Plans	The conversion test plans to be followed for conversion unit, business object, and validation tests.
Conversion Programs	The actual conversion code needed for data conversion using the appropriate tools.
Unit-Tested Conversion Programs	Conversion programs that operate without error and according to the predefined conversion unit test specifications.
Business Object-Tested Conversion Programs	Conversion programs for each business object that, when run end-to-end, result in converted business objects that satisfy the pre-defined business object test specifications.
Validation-Tested Conversion Programs	Conversion programs that produce converted business objects that function correctly in the target applications system.

Deliverable	Description
Installed Conversion Programs	Installed conversion programs and automated conversion tools used to convert the legacy system data.
Converted and Verified Data	The converted data in the production database that has been reviewed and verified. Use this data in the production environment and only modify it through a well-controlled set of upgrade procedures.

Table 6-2 Data Conversion Deliverables

Key Responsibilities

The following roles are required to perform the tasks within this process:

Role	Responsibility
Application Specialist	Provide knowledge and guidance regarding application functionality.
Business Analyst	Identify and document source systems for conversion. Perform Conversion Validation Tests (CV.110).
Client Project Manager	Assist in the allocation of client time to the manual conversion procedures.
Client Staff Member	Provide information on legacy data for conversion. Advise on availability of personnel, hardware, and software. Support data conversion. Execute the conversion programs and conversion software during final conversion (if required).

Role	Responsibility
Database Administrator	Prepare and support the data conversion environments.
Developer	Code conversion modules. Develop conversion unit test specifications and perform the conversion unit test. Perform the data conversion, if required.
IS Manager	Provide information on legacy system standards and the operational requirements and capacities of the current platforms.
Project Manager	Advise on the overall plan of the implementation. Verify availability of the system administrator, database administrator, and appropriate facilities. Manage and schedule data conversion, accounting for its position on the critical path.
System Administrator	Support the data conversion and provide assistance as needed for accessing the legacy systems.
Technical Analyst	Perform conversion mapping and design conversion modules.
Tester	Help with conversion business object and validation testing to verify data correctness.
User	Provide information on data to be converted.

Table 6-3 Data Conversion Key Responsibilities

Critical Success Factors

The critical success factors of Data Conversion are as follows:

- clear understanding of legacy system data
- adequate documentation of existing data definition
- adequate quality of legacy system data
- adequate business expertise to assure resulting data quality
- clear business understanding of historical versus current business needs
- clear business understanding of audit/reconciliation requirements
- complete and document application configuration and design
- development of conversion execution and transition plan
- well documented data exception handling procedures
- well defined data reconciliation and error handling procedures
- realistic expectations for conversion cutover window of time during production migration

References and Publications



Reference: Blaylock, Anne and Zerkow, Walt. *Oracle Magazine: Data Conversion from Legacy Systems to Oracle Applications*. January / February 1995.



Reference: Blaylock, Anne and Zerkow, Walt. *Automating the Conversion Process*. OAUG, Fall 1994.



Reference: Blaylock, Anne and Zerkow, Walt. *Integrating Oracle Applications with Legacy and Third Party Systems*. IOUW, Fall 1995.



Web Site: You can find further information on Oracle Corporation's Enterprise Data Management System (EDMS) at <http://www.oracle.com/EDMS/index.html>



Web Site: You can find further information on Smart Corporation's SMART DB Workbench at <http://www.smartdb.com>



Web Site: You can find further information on Evolutionary Technologies, Inc. Extract Tool at <http://www.evtech.com>



Web Site: Information regarding other automated tools to facilitate Data Conversion can be found at http://www.oracle.com/methods/aim/3.0/r_cv.html

CV.010 - Define Data Conversion Requirements and Strategy (Optional)

In this task, you define the scope of the conversion project, conversion objectives and approach, and prepare a strategy for converting information from the legacy systems to the new application environment. Part of defining the scope is documenting your conversion requirements at the application and business object level.

Additionally, this task provides a roadmap for performing the conversion of data from the legacy system to the new Oracle system and defines the task steps and resources needed to fulfill this strategy.

△ If your project includes either *programmatic data conversion* of legacy business objects, *manual data conversion* of legacy business objects, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Data Conversion Requirements and Strategy**. It is the basis for all other conversion project deliverables and should be signed by a designated approver. Use this deliverable to record your conversion requirements, track any scope changes that may impact the conversion project, and record the strategy for meeting the conversion requirements. The strategy includes a discussion of the recommended conversion method, tools, tasks, high-level conversion environment requirements, and testing procedures.

Prerequisites

You need the following input for this task:

❑ **Project Management Plan [initial complete] (PJM.CR.010)**

The Project Management Plan documents the scope, objectives, and approach for the overall implementation project.

☐ **Quality Management Strategies, Standards, and Procedures (PJM.QM.010)**

The Quality Management Strategies, Standards, and Procedures documents change management procedures and quality management standards.

☐ **Current Business Baseline (RD.020)**

The Current Business Baseline provides information on legacy systems that are the source for business objects you are converting to the Oracle Applications.

☐ **Oriented Project Team (AP.020)**

Since the Oriented Project Team has been exposed to the project, they are able to contribute to the development of the Data Conversion Requirements and Strategy.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review existing materials and the Current Process Model (BP.050) and conduct interviews (if needed).	
2.	Describe the purpose of the Data Conversion Requirements and Strategy	Introduction
3.	Document included and excluded conversion project scope and background information for legacy systems.	Scope

No.	Task Step	Deliverable Component
4.	List the objectives of conversion and critical success factors.	Objectives
5.	Describe the conversion approach, key inputs, resource requirements, organization, risks and contingencies.	Approach
6.	Describe the conversion approach you will follow to meet the conversion scope and objectives.	Conversion Strategy
7.	Prepare Conversion Process Flows for each target application to which you are converting legacy data.	Conversion Process Flows
8.	List the tool and deliverable naming standards to be followed for conversion.	Project Standards
9.	Identify the key business objects and data translations for data cleanup, as well as data normalization and reduction requirements.	Data Cleanup Process
10.	Describe the high-level conversion testing strategy.	Testing Strategy
11.	Outline the conversion project's acceptance criteria used to measure the successful completion of the defined conversion tasks.	Acceptance Criteria

No.	Task Step	Deliverable Component
12.	Describe the issue tracking system that will track and resolve conversion project issues.	Issue Tracking Procedures
13.	Define the version control standards for conversion programs and other conversion deliverables.	Version Control Procedures
14.	Document how conversion project scope changes will be managed.	Change Management
15.	Inform the project team of the quality system that will govern this conversion project.	Quality Management
16.	Define the conversion requirements at the application and business object level.	Conversion Requirements
17.	Document the specific selection criteria for each business object you are converting.	Business Objects Conversion Selection Criteria
18.	Review the Data Conversion Requirements and Strategy with the designated approver and secure acceptance.	Acceptance Certificate (PJM.CR.080)

No.	Task Step	Deliverable Component
19.	Identify any material changes to project scope and associated task estimates with the project manager and update the Project Management Plan as appropriate.	Project Management Plan (PJM.CR.010)

Table 6-4 Task Steps for Define Data Conversion Requirements and Strategy

Approach and Techniques

Data conversion can be a major component of a project (for some implementations, the conversion effort may be organized as a subproject) and is critical to the success of the implementation project. Identify data to be converted in conjunction with both the user and technical staff to help verify that the data to be converted is meaningful. It is important to obtain samples of the conversion data. When determining the legacy data for conversion, keep in mind external audit requirements that the data must meet and accurately record the business rules of your organization that you will translate to the target applications.

When discussing the conversion approach, you should consider the number of conversion cycles that will be performed before the final conversion. At a minimum, you should complete the full conversion testing cycle and use converted legacy data during Business System Testing (TE). Your goal is to determine the length of time it will take to convert all of the legacy data to the target applications before the final cutover to production. To set expectations correctly, include the conversion programs in Performance Testing (PT).

Make sure the conversion team and the overall implementation team agree on the conversion strategy.

Update the overall implementation workplan and estimates after completing the Data Conversion Requirements and Strategy. In addition, revisit the Project Management Plan (PJM.CR.010) to include

appropriate elements of the Data Conversion Requirements and Strategy.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C / Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Programmatically converted legacy data must be translated to the appropriate century date state before being uploaded to the production tables. Manually converted legacy data must be keyed into the data entry forms using four digits for the year, where supported.

Linkage to Other Tasks

The Data Conversion Requirements and Strategy is an input to the following tasks:

- TA.010 - Define Architecture Requirements and Strategy
- CV.020 - Define Conversion Standards
- CV.030 - Prepare Conversion Environment
- CV.040 - Perform Conversion Data Mapping
- CV.050 - Define Manual Conversion Procedures
- CV.060 - Design Conversion Programs
- PM.030 - Develop Transition and Contingency Plan

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	60
Application Specialist	30
Project Manager	10
Client Staff Member	*
User	*

Table 6-5 Role Contribution for Define Data Conversion Requirements and Strategy

* Participation level not estimated.

Deliverable Guidelines

Use the Data Conversion Requirements and Strategy to identify the scope, requirements, approach, and strategy for performing your data conversion.

This deliverable should address the following:

- overview of the data conversion effort
- scope of the conversion effort
- description of the legacy systems that will be the source of the business objects to be converted
- definition of tasks and deliverables
- organization of the conversion team
- considerations, such as constraints and assumptions, related to the data conversion effort
- approach to follow for the conversion of legacy data

- development standards
- data cleanup, testing, and validation strategy
- issue and change management procedures
- conversion requirements, and legacy data selection criteria

Deliverable Components

The Data Conversion Requirements and Strategy consists of the following components:

- Introduction
- Scope
- Objectives
- Approach
- Conversion Strategy
- Conversion Process Flows
- Project Standards
- Data Cleanup Process
- Testing Strategy
- Acceptance Criteria
- Issue Tracking Procedures
- Version Control Procedures
- Change Management
- Quality Management
- Conversion Requirements
- Business Objects Conversion Selection Criteria

Introduction

This component describes the purpose of the Data Conversion Requirements and Strategy and provides a description of the conversion deliverables audience, as well as the benefits of following the AIM data conversion approach. Other AIM process teams, whose tasks relate to the conversion tasks, should be included in your audience. Review the

network diagrams in the *AIM Method Handbook* to identify the AIM processes with tasks related to conversion tasks.

Scope

This component describes each legacy system that you plan to convert to the new applications and answers questions regarding the basic characteristics of the implementation project and the conversion effort.

This component documents the conversion scope for the following areas:

- **General** — lists the legacy systems that will be converted to the Oracle Applications and those that will not be converted
- **Platforms** — documents the hardware, operating systems, and software versions on which the legacy applications and the Oracle Applications operate
- **Connectivity** — documents the connectivity / networking options available for the conversion
- **Tools** — lists the automated tools that will be used or acquired for the conversion

Objectives

This component documents the objectives of data conversion and describes the critical success factors for achieving those objectives.

Approach

This component lists the tasks and deliverables produced for the conversion, the roles assigned to each of the tasks and the key inputs required. It describes the conversion team organization and indicates which team members are responsible for each conversion role, and includes an organization chart to illustrate the conversion team organization.

The physical resource requirements are listed as well as information on the software and hardware environment, network file transport facilities, staffing, and specialty tool requirements. Both existing and planned environments should be described in this component. If the implementing client staff members use tools, such as spreadsheets or PC-based databases that affect the conversion strategy, how and when to use these tools during the conversion should be documented.

This component also lists data conversion constraints and assumptions. A constraint is a condition or decision that impacts the conversion process and may limit the work you can perform. An example of a constraint is “The conversion of live data must take place within a 48-hour window during the cutover to production.”

An assumption is a condition or decision that you require to be true in order to accomplish your conversion goals. An example of an assumption is “Data will be converted as is — no redesign or data cleanup is required.”

In addition, this component documents the constraints and assumptions to which the conversion project must adhere in the following areas:

- **Schedule/Critical Milestones** — lists the conversion project milestones; these milestones are not limited to the specific conversion tasks within AIM
- **Required Resources** — lists the roles required for this project and any known restrictions on resource availability
- **Business Constraints** — lists the business constraints with which the conversion project must comply, such as budget, department schedules, and other initiatives that could affect the project
- **Technical Standards and Architecture** — lists expected hardware, software, and limitations of disk space or availability

Finally, this component lists metrics to consider when determining the overall level of effort for the conversion. You should prepare a conversion estimate to determine the number of days allocated for the conversion. This component helps you and the project manager understand the level of complexity of the conversion.

For each complexity factor listed here, include metrics for each business object being converted. The complexity factors listed below are not meant to be exhaustive; add additional complexity factors as appropriate for your environment.

Examples of conversion complexity factors follow:

- periods of data for legacy data being converted
- volumes of data being converted
- normalization of legacy data
- data translations

Conversion Strategy

This component describes the general approach that will guide the conversion and documents how to use automated tools to support the conversion.

Conversion Process Flows

This component documents the conversion process flow for each target application to which you are converting data. These process flows document related application business object prerequisites as well as the conversion sequence of master and transactional business objects.

Project Standards

This component documents the project standards, including the standards for application development tools, conversion tools, source control, version control, system management tools, and deliverable naming conventions.

The standards mentioned in this component should be specific to the conversion. If no specific standards are applicable to the conversion effort, then this component should refer the reader to the overall AIM project standards.

Data Cleanup Process

This component lists the legacy system business objects that require data cleanup. Cleanups are performed as a pre-conversion or a post-conversion activity.

Decide whether to perform data cleanup in the legacy system prior to conversion or to clean up the legacy data using the target application environment after loading the data into destination tables. An example of data that must be cleaned up is a customer that has been entered multiple times in slightly different ways. XYZ Corporation, XYZ Co. and Xyz Company may all represent the same customer entity, but due

to the difference in the way these values were entered (varying use of upper/lower/mixed case, terminology and abbreviations) they are treated as separate entities by the system.

In addition, list key data translations that are part of data cleanup. An example of a data translation is translating all occurrences of YES to Y. Code data translations into the interface programs or perform translation in the interface or temporary table prior to loading the data into the production tables.

Testing Strategy

This component describes the procedures to follow when testing the conversion programs and testing how the converted data functions in the target application. An example of a testing procedure is the comparison of record counts between the legacy and the target application.

Acceptance Criteria

This component states the conditions and acceptance criteria that converted data must meet before users and management considers the conversion successful. In addition, this component describes how you will verify that converted data is accurate and ready for production use.

Issue Tracking Procedures

This component explains issue management and issue resolution procedures. In most cases this component simply explains the overall project-level issue management and resolution procedures. However, if specific issue management considerations exist for conversion, they should be documented.

Version Control Procedures

This component documents the version control procedures to be followed for the development of the conversion modules.

Change Management

This component documents any additional change management guidelines that are necessary to manage conversion project scope changes. This component also documents a high-level explanation of the overall project change management guidelines. Change management is an effective way to manage requested changes that impact the scope of the conversion project.

Quality Management

This component documents additional quality management guidelines specifically required to manage the quality of the conversion.

Conversion Requirements

This component lists the business objects that you plan to convert. For each object indicate the legacy and target application, whether the conversion will be manual or automated, whether an open interface is available, the expected volume of data, and how many times you expect to execute the conversion process.

Business Objects Conversion Selection Criteria

This component explains the conversion selection criteria for each business object that you are converting. Selection criteria include date ranges, status codes, and any other attributes you can use to identify the subset of information you wish to migrate to the new system.

Tools

Deliverable Template

Use the Data Conversion Requirements and Strategy template to create the deliverable for this task.

Use the Acceptance Certificate (PJM.CR.080) to document the management approval.

Automated Conversion Tools

In addition to traditional coding tools, consider including automated conversion tools when producing the conversion approach component of this deliverable. Using these tools has several advantages:

- The automated conversion tools reduce the need for traditional programming skills.
- The automated conversion tools can be used to build reusable templates for multi-site implementations.

Several vendors offering automated conversion tools include:

- Oracle Corporation: Enterprise Data Management System (EDMS)
- Smart Corporation: SMART DB Workbench
- Evolutionary Technologies: ETI Extract

When employing automated conversion tools, the content of the Conversion Program Designs (CV.060) and the functionality of the resulting Conversion Programs (CV.080) can vary depending on the conversion tool you use.



Web Site: You can find further information on Oracle Corporation's Enterprise Data Management System (EDMS) at <http://www.oracle.com/EDMS/index.html>



Web Site: You can find further information on Smart Corporation's SMART DB Workbench at <http://www.smartdb.com>



Web Site: You can find further information on Evolutionary Technologies, Inc. Extract Tool at <http://www.evtech.com>



Web Site: Information regarding other automated tools to facilitate Data Conversion can be found at http://www.oracle.com/methods/aim/3.0/r_cv.html

CV.020 - Define Conversion Standards (Optional)

In this task, you create the Conversion Standards that the conversion team will follow when performing conversion tasks.

This task documents the file structure and naming conventions for the legacy and target systems, as well as for any automated tools you are using to facilitate the conversion.



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Conversion Standards**. It documents the legacy system, target system, and automated tool conversion standards.

Prerequisites

You need the following input for this task:

☐ **Quality Management Strategies, Standards, and Procedures (PJM.QM.010)**

The Quality Management Strategies, Standards, and Procedures document the quality policies for the overall implementation project.

☐ **Design Standards (MD.030)**

The Design Standards contain rules and assumptions to which the designers must adhere when designing custom programs. If Define Design Standards was not performed, this deliverable will not exist. (See the task description for MD.030 for more information on when this task should be performed.)

Build Standards (MD.040)

The Build Standards provide standards and guidelines for the developers to use when building custom programs. If Define Build Standards was not performed, this deliverable will not exist. (See the task description for MD.040 for more information on when this task should be performed.)

Data Conversion Requirements and Strategy (CV.010)

The Data Conversion Requirements and Strategy documents the conversion requirements that the Conversion Standards must address. If Define Data Conversion Requirements and Strategy was not performed, this deliverable will not exist. (See the task description for CV.010 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review existing internal organization and project standards.	
2.	Review recommended standards for selected automated tools.	
3.	Describe the purpose and audience for the Conversion Standards.	Introduction
4.	Document conversion standards for writing conversion programs to extract data from the legacy system.	Conversion Standards for Legacy Systems

No.	Task Step	Deliverable Component
5.	Document conversion standards for creating programs in the target environment.	Conversion Standards for Target Systems
6.	Document specific automated tool standards for the conversion process.	Conversion Standards for Automated Tools
7.	Document conversion standards for developing conversion deliverables.	Conversion Standards for Conversion Deliverables
8.	Review standards with the conversion team.	
9.	Secure acceptance that Conversion Standards. Include guidance regarding compliance with Century Date standards.	Acceptance Certificate (PJM.CR.080) (See the Tools section for information on modifications to the standard Acceptance Certificate template.)

Table 6-6 Task Steps for Define Conversion Standards

Approach and Techniques

The Define Conversion Standards task documents standards that are specific to the conversion process. If the organization already has a preferred file structure and naming conventions on the legacy system, minimize the impact by continuing to use these standards. The Oracle target system standards should describe the file structure and naming conventions for the Conversion Environment (CV.030). Make these standards complementary to other AIM project standards. The standards for automated tools should detail the file structure used for each of the automated tools that you will use on the conversion project.

You may be able to begin development of Conversion Standards prior to the completion of Design Standards (MD.030) and Build Standards (MD.040), but you should consider any applicable standards defined in

those deliverables for common tools, such as SQL and PL/SQL, when preparing the Conversion Standards.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Programmatically converted legacy data must be translated to the appropriate century date state before being uploaded to the production tables. Manually converted legacy data must be keyed into the data entry forms using four digits for the year, where supported.

Linkage to Other Tasks

The Conversion Standards are an input to the following tasks:

- CV.030 - Prepare Conversion Environment
- CV.060 - Design Conversion Programs
- CV.080 - Develop Conversion Programs

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	90
IS Manager	10

Table 6-7 Role Contribution for Define Conversion Standards

Deliverable Guidelines

Use the Conversion Standards deliverable to document and understand the conversion standards that govern the legacy and target system environments.

This deliverable should address the following:

- standards to be followed in extracting data from the legacy systems
- standards to be followed in uploading data to the target systems
- standards to be followed for automated conversion tools
- standards for preparation of conversion deliverables

Deliverable Components

The Conversion Standards consist of the following components:

- Introduction
- Conversion Standards for Legacy Systems
- Conversion Standards for Target Systems
- Conversion Standards for Automated Tools
- Conversion Standards for Conversion Deliverables

Introduction

This component describes the purpose of the Conversion Standards.

Conversion Standards for Legacy Systems

This component describes the file system and naming conventions of the legacy systems for each business object being converted. Do not assume that each legacy system has the same file structure.

Conversion Standards for Target Systems

This component describes the file structure for the conversion programs written on the target system / environment. You may want to refer to the target application database structure. This component also documents the file naming conventions for the files associated with each conversion task in the conversion process.

For example, the Design Conversion Programs (CV.060) task may have the following file naming convention: *invitmds.doc* for *inventory item design*. Also consider the importance of version control when defining your file naming rules.

Conversion Standards for Automated Tools

This component documents the file structure in this conversion effort so that conversion team members know where to store files and how to name them. Keep in mind that each tool you use to facilitate the conversion effort inherently has its own file structure.

Conversion Standards for Conversion Deliverables

This component documents the following standards to be used when developing conversion deliverables:

- writing standards
- table standards
- style standards

Tools

Deliverable Template

Use the Conversion Standards template to create the deliverable for this task.

Century Date Acceptance

To document the review and approval of Conversion Standards for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Acceptance Certificate should be filed with the deliverable in the project library.

CV.030 - Prepare Conversion Environment (Optional)

In this task, you create the Conversion Environment, including the hardware platforms, servers and other software required to support the design and build activities of conversion.



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Conversion Environment**. It is a working applications system environment that includes all of the servers, applications, and development tools required to develop and test conversion programs.

Prerequisites

You need the following input for this task:

☐ **Business Volumes and Metrics (RD.040)**

The Business Volumes and Metrics provides an inventory of key transaction and data volumes for preparing the Conversion Environment.

☐ **Architecture Requirements and Strategy (TA.010)**

The Architecture Requirements and Strategy identifies the application and technical requirements for the conversion environment.

☐ **Data Conversion Requirements and Strategy (CV.010)**

The Data Conversion Requirements and Strategy defines which business objects are subject to conversion. It also specifies the method of conversion and tools used for the conversion, which in turn helps determine the variables required for building the conversion estimate and workplan. If Define Data Conversion Requirements and Strategy was not performed, this deliverable will not exist. (See the task

description for CV.010 for more information on when this task should be performed.)

 Conversion Standards (CV.020)

Refer to the Conversion Standards before establishing the file structure for the conversion environment. If Define Conversion Standards was not performed, this deliverable will not exist. (See the task description for CV.020 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review Architecture Requirements and Strategy (TA.010) to understand the strategy for deployment of project environments in general, and the conversion environment in particular.	
2.	Update the introduction component to reflect the conversion environment hosting and environment sharing strategy per the Architecture Requirements and Strategy (TA.010).	Introduction
3.	Update all of the tables in the database tier, applications tier and desktop client tier sections of the environment component to reflect the configuration of all servers within the database, applications and desktop client tiers.	Environment - Conversion

No.	Task Step	Deliverable Component
4.	List any other software applications needed to support data conversion.	Other Applications
5.	Update the automated conversion tools component to reflect the configuration for each conversion tool.	Automated Conversion Tools
6.	Set up the conversion environment.	
7.	Install the conversion programs and automated conversion tools.	

Table 6-8 Task Steps for Prepare Conversion Environment

Approach and Techniques

Install and configure the Conversion Environment so that the conversion design and build tasks can be completed without affecting the work performed in other areas of the project. This usually requires the establishment of an entirely separate instance of the database and application software.

The conversion requires an environment where you can load legacy data into and purge data from the application tables multiple times. If you do not set up a separate conversion environment, you run the risk of not being able to control the converted data adequately. Attempting to perform the conversion activities in the same environment used for other project activities may adversely affect those environments.

There is a risk associated with setting up a separate conversion environment. If the other implementation instances update the application configuration, the changes must also be applied to the Conversion Environment to help make sure that the conversion tests mimic the eventual production instance. It is very important that the conversion programs be designed and tested in an environment that has the same application configuration as the production environment.

Linkage to Other Tasks

The Conversion Environment is an input to the following tasks:

- CV.040 - Perform Conversion Data Mapping
- CV.080 - Develop Conversion Programs
- CV.090 - Perform Conversion Unit Tests
- CV.100 - Perform Conversion Business Object Tests
- CV.110 - Perform Conversion Validation Tests

Role Contribution

The percentage of total task time required for each role follows:

Role	%
System Administrator	60
Database Administrator	30
Technical Analyst	10

Table 6-9 Role Contribution for Prepare Conversion Environment

Deliverable Guidelines

The deliverable for this task is the Conversion Environment, which is used in subsequent tasks for the development and testing of conversion programs. To support the proper execution of this task, the Conversion Environment deliverable document should also be completed. Use the Conversion Environment template to plan and document your Conversion Environment and record your final configuration.

This deliverable should address the following:

- conversion environment hosting and environment sharing arrangement
- conversion environment database, applications, and desktop client server architecture and configurations

- automated tool requirements

Deliverable Components

The Conversion Environment template consists of the following components:

- Introduction
- Environment - Conversion
- Other Applications
- Automated Conversion Tools

Introduction

This component describes the purpose for the Conversion Environment and the detailed configuration approach taken to implement the environment.

Environment - Conversion

This component describes the configuration for the database tier, applications tier, and desktop client tier servers in detail. This component also describes the configuration of the hardware platforms that these server environments execute on.

Other Applications

This component describes the configuration of any other software applications that are required to support the project.

Automated Conversion Tools

This component describes the configuration of any other software tools that are required to support the conversion activities of the project.

Tools

Deliverable Template

Use the Conversion Environment template to create the supporting deliverable for this task.

CV.040 - Perform Conversion Data Mapping (Optional)

In this task, you map the data elements from the legacy systems to the target Oracle Applications.

In addition to identifying the data sources and target tables and columns, you also identify data defaults, validation, processing, translation, filter, and foreign key rules. The conversion mapping and the extract file layout are prerequisites for designing the conversion programs.



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Conversion Data Mapping**. It records the detailed data mapping and extract file layout for each legacy data element you plan to convert to Oracle. Prepare one deliverable for each business object you are converting.

Prerequisites

You need the following input for this task:

 Mapped Business Data (BR.040)

The Mapped Business Data maps the legacy business object to the Oracle business object at the field / attribute level. You will extend the Mapped Business Data deliverable in Perform Conversion Data Mapping (CV.040) by adding specific table and column names.

 Application Setup Documents (BR.100)

The Application Setup Documents provide information on standard values entered to support the *list of values* feature in the target applications. These values may be needed when selecting appropriate default values for required data elements that are not found in the legacy systems.

Database Extensions Design (MD.060)

The Database Extensions Design describes the database design for any custom extensions defined in the Application Extension Definition and Estimates (MD.020). If Design Database Extensions was not performed, this deliverable will not exist. (See the task description for MD.060 for more information on when this task should be performed.)

Data Conversion Requirements and Strategy (CV.010)

The Data Conversion Requirements and Strategy documents the conversion requirements. Identify the conversion requirements and then identify the Oracle tables that need to be populated. If Define Data Conversion Requirements and Strategy was not performed, this deliverable will not exist. (See the task description for CV.010 for more information on when this task should be performed.)

Conversion Environment (CV.030)

The Conversion Environment provides a working computer system with all databases, applications, and development tools for use in mapping legacy business object data elements to the target Oracle Applications. If Prepare Conversion Environment was not performed, this deliverable will not exist. (See the task description for CV.030 for more information on when this task should be performed.)

Oracle Application Open Interface Manuals

The Oracle Application Open Interface Manual describes how to populate the standard interface tables and provides information about how to execute the interface programs.

Oracle Application Technical Reference Manuals

The Oracle Application Technical Reference Manuals for the Oracle target applications help you understand which Oracle tables and columns the legacy data map to.



Attention: Oracle Technical Reference Manuals are available separately from Oracle Corporation. They may not be included in the standard Application Documentation set.

Optional

You may need the following input for this task:

☐ Existing Reference Material

Gather any existing reference material to identify and gain an understanding of the existing source systems and data structures that require data conversion.

☐ Oracle Designer Repository

If available, the Oracle Designer Repository provides the application entity relationships, including foreign key relationships.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Describe the purpose of the Conversion Data Mapping.	Introduction
2.	Select the appropriate <i>Oracle Application Technical Reference Manual</i> for the business object you are preparing this mapping deliverable for, and look up the reference information about the Oracle Application table relationships. Enter this reference information into the table provided in the Conversion Data Mapping template.	Application Business Object Reference Information

No.	Task Step	Deliverable Component
3.	Identify the business object being converted, the target application, and the Oracle table name you are mapping to.	
4.	Determine the legacy file system structure, data type, data element, and naming conventions.	
5.	Complete the conversion mapping table included in the Conversion Data Mapping template.	Conversion Mapping
6.	Determine what the legacy system file layout should look like.	Extract File Layout
7.	Identify the legacy data that requires review for cleanup prior to conversion.	Data Cleanup
8.	Identify the legacy data that must be combined or adjusted to match the data structure of the target Oracle Application.	Data Normalization

Table 6-10 Task Steps for Perform Conversion Data Mapping

Approach and Techniques

This task maps the legacy source business objects and attributes to the target application tables and columns. As a starting point for this task, use the Mapped Business Data (BR.040), which maps legacy system business objects and attributes to corresponding objects and attributes in the new applications. The Mapped Business Data (BR.040) deals with the business data elements that the users interact with (such as the field

names on screens and reports). You extend this document by adding specific Oracle target application table and column names.

The conversion map is an important deliverable, because developers use the conversion map and the Conversion Program Designs (CV.060) to develop the conversion code. Regardless of whether you are using traditional coding methods or an automated conversion tool, the data map is the basis for the conversion design, build, and execution tasks.

Use the following technique when mapping conversion data:

- Identify the Oracle target tables that you need to populate for each business object. If the target application has standard application programming interfaces (APIs), determine whether you can use them. If no API exists for the business object you are converting, map the legacy data elements to a custom-defined interface table based upon the application table. The interface table you build should not include referential integrity constraints (even if the target table does). This allows you to manipulate the data before validating and moving the data to the target application tables.
- Determine the source of each business object's data elements. For example, for the business object called Inventory Items, the business may have more than one source for its item master.
- Determine for each business object, the key attributes for populating the target application tables.
- Map each legacy data element to an Oracle column. If there is no column to store the data element in, consider storing the data element in a descriptive flexfield.
- Identify any required Oracle columns that have not been mapped yet and assign default values to them.
- Define any validation that needs to occur before the data is loaded into the Oracle production tables.
- Define the selection criteria for each business object. Examples of selection criteria are date ranges and specific status codes.
- Note any processing, translation, filter, foreign key, or derivation rules applied to the data element during the conversion.

When preparing the extract file layout, specifically state whether you need a fixed length or variable length format. The conversion tool you use may dictate this decision.

Linkage to Other Tasks

The Conversion Data Mapping is an input to the following tasks:

- CV.050 - Define Manual Conversion Procedures
- CV.060 - Design Conversion Programs
- CV.070 - Prepare Conversion Test Plans
- CV.080 - Develop Conversion Programs
- TE.040 - Develop System Test Script

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	80
Application Specialist	20
Client Staff Member	*

Table 6-11 Role Contribution for Perform Conversion Data Mapping

* Participation level not estimated.

Deliverable Guidelines

Use the Conversion Data Mapping deliverable to document the conversion mapping for a specific business object. For each business object, copy the corresponding Mapped Business Data (BR.040) and supply additional technical information.

This deliverable should address the following:

- data mapping from legacy to target systems
- extract file format and content for extracting legacy data
- legacy data cleanup and normalization requirements

Deliverable Components

Conversion Data Mapping consists of the following components:

- Introduction
- Application Business Object Reference Information
- Conversion Mapping
- Extract File Layout
- Data Cleanup
- Data Normalization

Introduction

This component describes the purpose, use and audience for the Conversion Data Mapping deliverable.

Application Business Object Reference Information

This component lists reference information about the Oracle Application table relationships. The table provided indicates whether the business object is a candidate for manual or programmatic conversion and whether an existing standard API can be employed. At a minimum, this component should be used as a reference when preparing the data mapping for each business object.

Conversion Mapping

This component consists of the same table included in Mapped Business Data (BR.040). To create this deliverable component, copy the Mapped Business Data (BR.040) document and transform it into conversion mapping, or simply copy and paste only the mapping table from the Mapped Business Data (BR.040) to your new document. Complete the mapping for each legacy business object being converted by entering the missing information in the table. If there are required fields for which there are no corresponding legacy data elements, specify a default value. The conversion map should also identify a place to store those legacy data elements that do not map to an Oracle column. In many cases these data elements can be stored in a descriptive flexfield.

Extract File Layout

This component provides a table that details the extract file layout from the legacy system. When preparing the extract file layout, it is

important to document the relative position of a specific data element for the legacy system. For example, document that the customer name is always going to be in position five through 25 in the flat file. Depending on the tool you choose to use for the conversion, it may be necessary to have a client staff member provide a fixed length ASCII flat file. When instructing them on the required format, it is important to document whether the extract file should be fixed or variable length, the record length, end of record, and the field and end of file delimiters. Remember to choose a delimiter character that does not conflict with a UNIX reserved character.

Data Cleanup

This component defines data that requires visual identification and inspection for cleanup prior to conversion.

Data Normalization

This component defines data that must be combined or adjusted to match the structure of Oracle Applications data. For example, the legacy system may store different vendor addresses as separate vendor records with the same vendor name. In Oracle Purchasing, each vendor has one master record for the name and other common information with related address records stored in a different table.

Tools

Deliverable Template

Use the Conversion Data Mapping template to create the deliverable for this task.

Automated Conversion Tools

If you are using an automated conversion tool, you may be able to perform the mapping task directly in the tool. Refer to the documentation for the tool you are using for information on its capabilities.



Web Site: You can find further information on Oracle Corporation's Enterprise Data Management System (EDMS) at <http://www.oracle.com/EDMS/index.html>



Web Site: You can find further information on Smart Corporation's SMART DB Workbench at <http://www.smartdb.com>



Web Site: You can find further information on Evolutionary Technologies, Inc. Extract Tool at <http://www.evtech.com>



Web Site: You can find further information on Oracle's Designer product at <http://www.oracle.com>

CV.050 - Define Manual Conversion Procedures (Optional)

In this task, you define the plan to convert the business objects that require manual conversion.

The resulting procedure provides a detailed guide for manually converting data to successfully meet conversion project milestones.



If your project includes *manual data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Manual Conversion Procedures**. These procedures define how to manually convert a business object. In addition, they discuss the impact, if you do not successfully convert business objects in the required time frame. You should produce one deliverable for each manually converted business object. Use this deliverable as a roadmap for those who will be performing the manual conversion.

Prerequisites

You need the following input for this task:

☐ **Data Conversion Requirements and Strategy (CV.010)**

The Data Conversion Requirements and Strategy identifies manually converted business objects. If Define Data Conversion Requirements and Strategy was not performed, this deliverable will not exist. (See the task description for CV.010 for more information on when this task should be performed.)

☐ **Legacy Data Cleanup**

Legacy data identified in the Data Conversion Requirements and Strategy (CV.010) for pre-conversion data cleanup should meet the data cleanup, data reduction, and normalization requirements.

Optional

You may need the following input for this task:

 Conversion Data Mapping (CV.040)

The Conversion Data Mapping describes the detailed data mapping and extract file layout for each legacy data element you plan to convert to Oracle. If Perform Conversion Data Mapping was not performed, this deliverable will not exist. (See the task description for CV.040 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Describe the purpose and audience for the Manual Conversion Procedures.	Introduction
2.	Describe the manually converted business object.	Business Object Description
3.	List the data elements for this business object in the legacy system and identify those that will be converted, and those that will not be converted to the Oracle target application.	Data Elements to be Converted / Not Converted
4.	Identify the legacy data that requires review for cleanup and entry and the data that must be combined or adjusted to match the structure of the target Oracle Application.	Legacy Data Cleanup and Normalization

No.	Task Step	Deliverable Component
5.	Identify the person responsible for the manual conversion of the business object and indicate who will be performing the manual conversion. In addition, identify the instance requiring manual data conversion and indicate the conversion priority dates per conversion business object.	Conversion Responsibility and Timeline
6.	Complete a worksheet that shows the navigation path for manually inputting the legacy data into Oracle using the Oracle target application forms and the values to be entered.	Conversion Procedures
7.	Secure acceptance that the Manual Conversion Procedures include considerations for compliance with Century Date standards.	Acceptance Certificate (PJM.CR.080) (See the Tools section for information on modifications to the standard Acceptance Certificate template.)

Table 6-12 Task Steps for Define Manual Conversion Procedures

Approach and Techniques

Manually converting legacy data is the appropriate decision in many cases. Although manually converting data is technically less complicated, it may take more time. Therefore, it is critical to avoid underestimating the manual conversion effort and, as a result, delaying the production date. There is also a risk of introducing keystroke errors.

Consider manually converting data for business objects with low data volumes. You need to weigh the time and expense of keying data into

the target application forms versus the time and expense of programmatically converting the data.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Manually converted legacy data must be keyed into the data entry forms using four digits for the year, where supported.

Screen Scraper/Testing Tools

In addition to the option of entering the legacy data into the Oracle target application via manual data entry, consider the option of using an automated screen scraper or testing tool to load the data. A screen scraper is a utility that can read a text file and automatically send keystrokes to a data entry form to simulate user input. These tools allow you to partially automate a conversion without writing conversion programs.

Timing

During the production conversion, the users or data entry personnel will manually enter the legacy data if you are not using a screen scraper or testing tool. You can begin manual data conversion of static business objects while developers are coding and testing automated conversion programs. This way, you minimize the amount of manual conversion required during production cutover.

Manually Converted Data and Business System Testing

You must decide how much manually converted data you require for Perform System Test (TE.110) and Perform Systems Integration Test (TE.120). With an automated conversion, you can simply run the conversion programs in both the testing environment and the production environment. For manually converted data, you must reenter the information in both environments. Fortunately, a subset of data is usually adequate for testing purposes.

Linkage to Other Tasks

The Manual Conversion Strategy is an input to the following tasks:

- CV.130 - Convert and Verify Data
- TE.110 - Perform System Test
- TE.120 - Perform Systems Integration Test
- PM.030 - Develop Transition and Contingency Plan

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Application Specialist	70
Business Analyst	30
Client Project Manager	*
Client Staff Member	*

Table 6-13 Role Contribution for Define Manual Conversion Procedures

* Participation level not estimated.

Deliverable Guidelines

Use the Manual Conversion Procedures deliverable to document an easy-to-follow plan. Use it as a guide for performing the manual conversion for a given business object. Prepare a separate document for each business object you are converting manually.

This deliverable should address the following:

- the business object to be converted manually
- the legacy data elements to be converted, and those that will not be converted
- data cleanup and normalization
- responsibility and schedule for manual conversion
- navigation and data entry guidance for users

Deliverable Components

The Manual Conversion Strategy consists of the following components:

- Introduction
- Business Object Description
- Data Elements to be Converted / Not Converted
- Legacy Data Cleanup and Normalization
- Conversion Responsibility and Timeline
- Conversion Procedures

Introduction

This component describes the purpose, use, and audience for the Manual Conversion Procedures deliverable.

Business Object Description

This component describes the business object that requires manual conversion.

Data Elements to be Converted/Not Converted

This component lists the data elements within the legacy files for the business object that will be converted to the Oracle Application and identifies those that will not be converted (do not assume that all data elements for this business object need to be converted to Oracle).

Legacy Data Cleanup and Normalization

This component defines data that requires visual identification and inspection for cleanup and entry and data that must be combined or adjusted to match the structure of Oracle Applications data. For example, the legacy system may store different vendor addresses as separate vendor records with the same vendor name. In Oracle Purchasing, each vendor has one master record for the name and other common information with related address records stored in a different table.

Conversion Responsibility and Timeline

This component lists the person who is responsible for the success of this manual conversion effort, as well as the names of the individual users who will be entering and verifying the data. It also documents the scheduled start and end dates for the manual conversion of this business object. In addition, this component states the impact of not meeting these deadlines on the overall conversion and implementation project.

Conversion Procedures

This component documents the navigation path that instructs the user how to navigate to the Oracle form and zone to manually enter the data. This component also includes a table that lists each Oracle field to be populated, the legacy data element needed to populate the field, whether there is a *list of values* for the field, the default value if necessary, and any notes that would help the user.

Tools

Deliverable Template

Use the Manual Conversion Procedures template to create the deliverable for this task.

Century Date Acceptance

To document the review and approval of Manual Conversion Procedures for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Acceptance Certificate should be filed with the deliverable in the project library.

Automated Tools



Suggestion: A screen scraper or testing tool with key stroke emulation may be an alternative to keying the legacy data directly into the Oracle Application forms. The screen scraper tool reads the data extract flat file and automates the key strokes required to populate the Oracle form. By using such a tool, the data is subject to the form-level and package-level validations of the Oracle Applications.



Web Site: Information regarding automated tools to facilitate Data Conversion can be found at http://www.oracle.com/methods/aim/3.0/r_cv.html

CV.060 - Design Conversion Programs (Optional)

In this task, you design and document the conversion programs.

Completion of this task provides the developer with the necessary information for writing an accurate conversion program.



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Conversion Program Designs**. These designs define the key assumptions, rules, and logic needed to create the conversion modules. Prepare one deliverable for each business object you are converting.

Prerequisites

You need the following input for this task:



Design Standards (MD.030)

The Design Standards contain rules and assumptions the designers must adhere to, when designing custom programs. If Define Design Standards was not performed, this deliverable will not exist. (See the task description for MD.030 for more information on when this task should be performed.)



Database Extensions Design (MD.060)

The Database Extensions Design contains definitions of all database objects and the constraints they are subject to. If Design Database Extensions was not performed, this deliverable will not exist. (See the task description for MD.060 for more information on when this task should be performed.)

Data Conversion Requirements and Strategy (CV.010)

The Data Conversion Requirements and Strategy provides the overall framework for converting each business object to the Oracle Application module. If Define Data Conversion Requirements and Strategy was not performed, this deliverable will not exist. (See the task description for CV.010 for more information on when this task should be performed.)

Conversion Standards (CV.020)

The Conversion Standards contain rules and assumptions that the conversion project team must follow. If Define Conversion Standards was not performed, this deliverable will not exist. (See the task description for CV.020 for more information on when this task should be performed.)

Conversion Data Mapping (CV.040)

The Conversion Data Mapping provides the mapping of the legacy data elements to the Oracle target application tables and columns and provides the data defaults and references to the rules defined in the Conversion Program Designs. If Perform Conversion Data Mapping was not performed, this deliverable will not exist. (See the task description for CV.040 for more information on when this task should be performed.)

Optional

You may need the following input for this task:

Detailed Existing System Data Model

Obtain any Existing System Data Models that may provide the data structure of the existing system.

Oracle Application Technical Reference Manuals

Refer to the Oracle Application Technical Reference Manuals to determine table constraints, indexes, views, and so on.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Describe the purpose and audience for the Conversion Program Designs.	Introduction
2.	Document any conversion assumptions that affect the design of the conversion programs.	Conversion Assumptions
3.	Describe the Oracle tables that will be populated during the conversion, and the order in which the tables need to be populated.	Approach
4.	Refer to Conversion Data Mapping (CV.040) to identify the conversion rules to describe in this deliverable.	
5.	Document the processing rules to design in the conversion programs.	Processing Rules
6.	Document the translation rules that need to be designed into the conversion programs.	Translation Rules
7.	Document the filter rules to design in the conversion programs.	Filter Rules
8.	Document the foreign key rules to design in the conversion programs.	Foreign Key Rules

No.	Task Step	Deliverable Component
9.	Document the derivation rules to design in the conversion programs.	Derivation Rules
10.	Define the logic associated with the default values documented in Conversion Data Mapping (CV.040).	Default Values
11.	Document the logic required for the download or extract program.	Download Program Logic
12.	Document the logic required for the interface table creation program.	Interface Table Creation Program Logic
13.	Document the logic required for the upload program logic.	Upload Program Logic
14.	Document the logic required for the translation program logic.	Translation Program Logic
15.	Document the logic required for the interface / validation program logic.	Interface / Validation Program Logic
16.	List the programs and any associated extract files created for each business object.	Conversion Program Modules
17.	Secure acceptance that Conversion Program Designs include criteria for compliance with Century Date standards.	Acceptance Certificate (PJM.CR.080) (See the Tools section for information on modifications to the standard Acceptance Certificate template.)

Table 6-14 Task Steps for Design Conversion Programs

Approach and Techniques

Using the Data Conversion Requirements and Strategy (CV.010) as a starting point, identify and design the conversion modules needed to transfer data from the old system or other outside sources into the new system. In general, the approach and technique you use for designing your conversion programs is impacted by the method you choose for coding your conversion programs. During the conversion process, you may use traditional coding methods, using tools such as PL/SQL and SQL*Loader, or automated conversion tools. Regardless of the methods you choose for coding your conversion programs, you must determine where to document the conversion program design.

You may want to create a logical flow diagram of the conversion approach for a business object to help the reader visualize the flow of the conversion.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Programmatically converted legacy data must be translated to the appropriate century date state before being uploaded to the production tables.

Download/Extract Programs

Typically, the organization's information systems support staff is responsible for providing extract files for conversion testing and conversion production loading. However, depending on the tool that you are using to upload the data to Oracle, you may have to be very specific about how you want the extract file to look. An example of the logic that you need to document is: *There must be multiple records per order in the extract file for order lines.*

Depending on the conversion requirements, the data extraction may be performed multiple times. If this is the case, you may want to consider using one of Oracle's gateway products to provide direct legacy system access.

Interface Tables

During conversion, do not directly populate Oracle Applications tables. The legacy data should first be moved to one or more of the standard or custom-developed interface tables. The number of interface tables required for the conversion of a business object depends on the complexity of the business object being converted. The level of complexity is not necessarily driven by data volume, but is more dependent on the number of processing, translation, filter, and foreign key rules. Many of the Oracle Applications have standard APIs that provide interface tables to load the legacy data before executing the interface programs.

You can write traditional SQL code to create these tables, or if you are using an automated conversion tool, you can extend the Oracle production tables to include processing columns for processing rules. The temporary tables provide a place for you to apply all of the rules you have defined for a specific business object, before moving the data into the production tables.

Uploading Data

There are many tools available for uploading the data to the Oracle tables. The interface tables should already exist (in the case of a standard API) or should have been created before the upload module is run. Other loader options include the following:

- Use the loader facilities offered as standard functionality in automated conversion tools.
- Upload legacy data directly from a spreadsheet or PC database to the Oracle database tables using available ODBC drivers and SQL*Net or Net8.

Translation Programs

Once you create and load the interface tables, create and execute special logic to manipulate the data to attain the correct format which the new Oracle Application needs to operate. Execute this data manipulation before moving the data to the Oracle production tables. You can write traditional code using tools such as SQL*Plus or PL/SQL or use an automated conversion tool.

Interface/Validation Programs

Typically, the level of validation required must be consistent with the validation being performed by any form or package logic. Therefore, it is important to remember that these interface programs not only move the data to the production tables but also perform all required data validation. For example, if the order entry form validates that the customer on an order is already a customer that exists in the order entry system, then your interface module should do the same. If the level of validation is not the same as the form-level validation, when the converted order is queried in the new Oracle system, the user may get an error message.

An automated conversion tool may also be the best option for writing your interface modules. The tool can be used to perform the necessary lookups required to enforce data validation in other Oracle tables.

Linkage to Other Tasks

The Conversion Program Design is an input to the following tasks:

- CV.070 - Prepare Conversion Test Plans
- CV.080 - Develop Conversion Programs
- CV.130 - Convert and Verify Data
- PM.030 - Develop Transition and Contingency Plan

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	100

Table 6-15 Role Contribution for Design Conversion Programs

Deliverable Guidelines

Use the Conversion Program Designs deliverable to detail the conversion design that will be used by the developers as an input for coding the conversion programs.

This deliverable should address the following:

- assumptions used in developing the conversion program
- approach description
- rules for processing, translations, filtering, foreign keys, derivations, and defaults
- program logic for downloading data, interface table creation, uploading, translation, and validation

Deliverable Components

The Conversion Program Designs consist of the following components:

- Introduction
- Conversion Assumptions
- Approach
- Processing Rules
- Translation Rules
- Filter Rules
- Foreign Key Rules
- Derivation Rules
- Default Values
- Download Program Logic
- Interface Table Creation Program Logic
- Upload Program Logic
- Translation Program Logic
- Interface/Validation Program Logic
- Conversion Program Modules

Introduction

This component describes the purpose, distribution, and quality criteria for the document.

Conversion Assumptions

This component states the name of the legacy system to be converted, the volume of data to be converted, and any exceptions of which the developer needs to be aware.

The conversion assumptions document the cleanup criteria specific to each business object. An example of cleanup or selection criteria could be that no closed sales orders will be converted. In this case, a pre-conversion program is written to make sure that no closed sales orders are brought into the new application. For some business objects it makes most sense to clean up or merge the data after the conversion

takes place. This component documents whether the cleanup will take place before or after conversion.

Approach

This component discusses the approach used in data conversion design. The following sections should be completed:

- **Interface Tables** — lists the Oracle tables for mapping and populating, the legacy file names to be extracted, and any other file names that are important to the conversion of this business object
- **Dependencies** — lists any other dependencies that may affect the conversion (or example, this is the appropriate place to list module and configuration prerequisites specific to the conversion of the business object)

If an automated conversion tool is being used to create templates or map documents, the directory location should be documented in this component.

Processing Rules

This component lists processing rules and assigns codes, such as CUSPR1, to each processing rule. This component also documents the source and target application table and field to which the rule applies. This component is used to explain each rule in the table.

- **Example** — A processing rule for a general ledger account ID might be that the old account number must first be mapped to the new account number. Accomplish this by building a translation table in Oracle that maps the old account number to the new account number.

Translation Rules

This component provides a table that lists translation rules to be used and assigns a code, such as CUSTR1, to a translation rule. The table should then document the source and target application tables and fields to which the rule applies. Below the table, an explanation of each rule should be provided.

- **Example** — A translation rule for the translation of a payment term stored as “Net 30” in the legacy system to “NET 30 DAYS” in Oracle.

Filter Rules

This component provides a table that describes the filter rules to be used and assigns a code, such as CUSFR1, to each rule. The table should then document the source and target application table and field to which the rule applies. Below the table, an explanation of each rule should be provided.

- **Example** — A filter rule to pad a field in the legacy system with blanks before the conversion of the data element (field) to Oracle. Because of the way the data is being stored in the legacy system, the tool you use may dictate the use of filtering rules.

Foreign Key Rules

This component provides a table that lists the foreign key rules to be used and assign a code, such as CUSFKR1, to each rule. It also documents the source and target application table and field to which the rule applies. Below the table, an explanation of each rule should be provided.

- **Example** — A foreign key rule to select the order internal ID before you can assign an order line to the purchase order. This rule requires that the legacy order numbers be converted before converting the detail lines of the order.

Derivation Rules

This component provides a table that lists the derivation key rules to be used and assign a code, such as CUSDR1, to each rule. This component also documents the source and target application table and field to which the rule applies. Below the table, an explanation of each rule should be provided.

- **Example 1** — A derivation rule for the derivation of accounting code combinations. You may need to derive a new account code structure from an old account code structure.
- **Example 2** — A derivation rule to derive a customer number from an old legacy customer number from two different sources, plus appending a prefix.

Default Values

This component provides a table that lists each default value previously assigned and documented in Conversion Data Mapping (CV.040) and provides an explanation of the logic behind the choice of this default

value. The explanation should address why this default value makes sense for your specific business processing requirements. If you have already documented the default values in detail in Conversion Data Mapping (CV.040), this component is optional.

Download Program Logic

This component defines the logic required for the download or extract programs.

Interface Table Creation Program Logic

This component defines the logic required for the programs that create interface tables for conversion.

Upload Program Logic

This component defines the logic required for the programs that upload the data from the flat file to the Oracle temporary tables.

Translation Program Logic

This component defines the logic required for the programs that perform any required processing, translation, filter, or foreign key rules for conversion.

Interface/Validation Program Logic

This component defines the logic required for the programs that move the data being converted from the temporary tables to the Oracle production tables.

There are some components, such as the Approach component, that may seem repetitive because the component was also addressed in the Data Conversion Requirements and Strategy (CV.010). However, this deliverable is specific to the individual business object being converted. The Data Conversion Requirements and Strategy (CV.010) is a high-level conversion document that addresses the overall approach for performing the conversion for the entire system.

For example, an order entry system being converted may consist of customers, orders, shipment history, and price lists. In this example, the overall conversion approach for the order entry system would be included in the Data Conversion Requirements and Strategy (CV.010), but there would be multiple Conversion Program Designs for the order entry system—one for each of the business objects being converted.

Conversion Program Modules

This component lists the name and planned directory location of all of the programs, and any associated extract files, created for the business object.

Tools

Deliverable Template

Use the Conversion Program Designs template to create the deliverable for this task.

Century Date Acceptance

To document the review and approval of Conversion Program Designs for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Acceptance Certificate should be filed with the deliverable in the project library.

Automated Conversion Tools

In addition to traditional coding tools, you can use the following automated conversion tools:

- Oracle's EDMS
- Smart Corporation's SMART DB Workbench
- Evolutionary Technologies' ETI Extract
- Oracle's Gateway products
- Tools from other Oracle partners



Web Site: You can find further information on Oracle's Gateway products at <http://www.oracle.com>

CV.070 - Prepare Conversion Test Plans (Optional)

In this task, you outline the testing plans for the unit, business object, and validation testing for conversion.

The unit tests confirm that each module successfully completes the task it is designed to perform. For example, a unit test should verify that the download program has extracted the expected number of records from the legacy system. The business object test verifies that the quality of the data converted to the Oracle system is accurate and functions properly in the individual Oracle Application to which it has been converted. Validation testing verifies that the converted legacy data performs accurately within the entire suite of Oracle Applications.



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Conversion Test Plans**. These plans provide plans for conversion unit, business object, and validation tests. Subsequent Data Conversion tasks perform each of these three levels of testing and then record the results as described in the deliverable guidelines. Prepare one deliverable for each business object you are converting.

Prerequisites

You need the following input for this task:



Conversion Data Mapping (CV.040)

Conversion Data Mapping identifies the data elements that should be business object and integration tested. If Perform Conversion Data Mapping was not performed, this deliverable will not exist. (See the task description for CV.040 for more information on when this task should be performed.)

❑ Conversion Program Designs (CV.060)

Design the conversion programs or automated conversion templates so that you can identify the conversion programs that should be unit tested. If Design Conversion Programs was not performed, this deliverable will not exist. (See the task description for CV.060 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Describe the purpose and audience for the Conversion Test Plans.	Introduction
2.	Define the conversion unit test plans and procedures.	Conversion Unit Test
3.	Define the conversion business object test plans and procedures.	Conversion Business Object Test
4.	Define the conversion validation test plans and procedures.	Conversion Validation Test
5.	Secure acceptance that conversion test plans include criteria for Century Date Compliance testing.	Acceptance Certificate (PJM.CR.080) (See the Tools section for information on modifications to the standard Acceptance Certificate template.)

Table 6-16 Task Steps for Prepare Conversion Test Plans

Approach and Techniques

Prepare the Conversion Test Plans for each business object you are converting. For each business object there are several conversion programs that need to be tested.

If you are using an automated conversion tool to facilitate the conversion, testing is still very important. For unit testing, you may be testing the conversion template and load functionality instead of the conversion programs. Adjust your test procedures accordingly to take into account any automated tools you are using.

For both conversion business object and validation testing, you need to identify record counts and plan for the pre-conversion test totals that will be compared to the Oracle totals. A list of test totals that can be compared between the source and target systems follows:

- **Balances** — month-end, year-end balances, and so on
- **Hash Counts** — total number of items, customers, vendors, and so on
- **Valuations** — total value of open orders, receivables, and so on
- **Distributions** — total items per subinventory, locator, and so on

Execute the conversion programs several times to practice the data conversion. This allows the developers responsible for performing the data conversion during production cutover to become familiar with using the conversion routines. By practicing the data conversion several times, you can predict the conversion program performance before the final production load.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Programmatically converted legacy data must be translated to the appropriate century date state before being uploaded to the production tables. In the case of conversion programs, both the program code and converted legacy application data must be checked for compliance with Century Date standards.

Relationship with Other Testing Processes

Converted data is also a critical part of integration, business system, and performance testing. Refer to Business System Testing (TE) and Performance Testing (PT) for more information on these levels of testing. It is imperative that you know, for example, how much time is required to load 50,000 items or to perform a query of these converted items. Therefore, once you have completed the conversion testing tasks, use the converted data, and the conversion programs, in Business System Testing (TE) and Performance Testing (PT).

Use the converted data that has been unit tested, business object tested, and validation tested in the following Business System Testing (TE) tasks:

- Perform System Test (TE.110)
- Perform Systems Integration Test (TE.120)

Determine with the help of the performance testing team, if conversion programs are within the scope of the Performance Testing Process for your project. If included, use the conversion programs in the following Performance Testing (PT) task:

- Execute Performance Test (PT.120)

Linkage to Other Tasks

The Conversion Test Plans are an input to the following tasks:

- CV.090 - Perform Conversion Unit Tests
- CV.100 - Perform Conversion Business Object Tests
- CV.110 - Perform Conversion Validation Tests

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	80
Application Specialist	20

Table 6-17 Role Contribution for Prepare Conversion Test Plans

Deliverable Guidelines

Use the Conversion Test Plans to create a test plan for conversion unit, business object, and validation testing, that can be followed by the testers performing the conversion tests.

This deliverable should address the following:

- unit test plans
- business object test plans
- validation test plans

Deliverable Components

The Conversion Test Plans consist of the following components:

- Introduction
- Conversion Unit Test
- Conversion Business Object Test
- Conversion Validation Test

Introduction

This component describes the purpose, distribution, and quality criteria for the document

Conversion Unit Test

This component documents which conversion programs you are going to test and provides a place for the tester to record the results of the test. Conversion module testing verifies the accuracy of the smallest objects used in the data conversion effort; for example, SQL-Loader and PL/SQL scripts. This approach attempts to verify the logic and accuracy of each object involved in processing conversion data. This is accomplished by reviewing, for example, record counts on completion of each object processed.

Conversion Business Object Test

This component documents the individual data elements you are planning to test during business object testing. In many cases the user sets the level of testing expectations. For business object testing, the users are required to compare how a data element appeared in the legacy system to how it appears and functions in the new Oracle Application. This level of testing does not test the flow of the data within the entire Oracle system, but only how the data appears or functions in an individual application. By having the users complete the majority of the testing, they become increasingly familiar with navigating the new application. If users are going to be the primary testers, they need to be trained to perform the testing tasks. It works best to have a member of the conversion team work with the users designated as testers to develop these test procedures.

When deciding which data elements need validation, consider how a particular data element affects the Oracle Application's functionality. Think about the downstream effect on reports and concurrent processes of an incorrect data element on the target application's performance.

Examples of data elements that are candidates for business object testing follow:

- In Oracle Receivables, verify the customer records so that if invoices are converted, the name on the invoice is the same as the name in the Oracle customer tables. If the addresses are not the same, when a user queries an invoice, an Oracle error will occur because the form verifies that the name on the invoice is the same as the name in the Oracle database.
- Verify all of the *list of values* (LOV) fields in the Oracle forms to make sure that the legacy data elements being converted are valid Oracle lookups.

- In Oracle Accounts Payable, verify the vendor records to confirm that the vendor information that appears on the AP invoice and purchase order is the same as the vendor data stored in the Oracle AP tables.

Conversion Validation Test

This component documents the procedures for performing the conversion validation test. This is the most comprehensive level of conversion testing. These procedures should lay out a plan for the users to follow when they are testing the overall functionality of how the converted legacy data performs in the entire suite of Oracle Applications.

An example of one validation test is to test how an open converted order can be closed, shipped, deplete inventory, and be posted to General Ledger. In most validation tests, testers must compare pre-conversion and post-conversion balances. This requires planning to gather the pre-conversion totals. In addition, this may require that the legacy and new applications run in parallel for a predetermined length of time.

Tools

Deliverable Template

Use the Conversion Test Plans template to create the deliverable for this task.

You may also want to use Systems Integration Test Script (TE.050) and the Performance Test Scripts (PT.040) to document the impact of the converted data on Business System Testing (TE) and Performance Testing (PT).

Century Date Acceptance

To document the review and approval of Conversion Test Plans for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Acceptance Certificate should be filed with the deliverable in the project library.

CV.080 - Develop Conversion Programs (Optional)

In this task, you create the conversion programs that perform all of the functions required to convert legacy business objects to the target Oracle Applications. The conversion of each business object typically involves the creation of five types of programs, including a download program, interface table creation program, upload program, translation program, and an interface and validation program.

The download program is used to extract the data from the legacy system and create an ASCII flat file that can be uploaded to the Oracle tables. The interface table creation program creates tables that store the legacy data before the data is validated and inserted into the production tables of the Oracle Application. The upload program uploads the legacy ASCII flat file data to the interface tables, while the translation program performs any data-required translation, transformation, or manipulation required before moving the data to the production tables. Finally, the interface and validation program performs validation of the data in the interface tables and updates the data into the Oracle production tables.



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Conversion Programs**. These programs are the actual program code for the programs defined in the Conversion Program Designs (CV.060).

Prerequisites

You need the following input for this task:



Build Standards (MD.040)

The Build Standards provide standards and guidelines for the developers to follow in building custom programs. If Define Build Standards was not performed, this deliverable will not exist. (See the

task description for MD.040 for more information on when this task should be performed.)

Conversion Standards (CV.020)

The Conversion Standards outline conversion-specific standards for writing conversion programs in the legacy and target environments, and for the use of automated conversion tools to facilitate the conversion effort. If Define Conversion Standards was not performed, this deliverable will not exist. (See the task description for CV.020 for more information on when this task should be performed.)

Conversion Environment (CV.030)

The Conversion Environment is the environment that supports the design and build activities of the conversion. If Prepare Conversion Environment was not performed, this deliverable will not exist. (See the task description for CV.030 for more information on when this task should be performed.)

Conversion Data Mapping (CV.040)

Conversion Data Mapping provides the mapping of the legacy data elements to the Oracle target application tables and columns, and provides the data defaults and references to the rules defined in the Conversion Program Designs (CV.060). If Perform Conversion Data Mapping was not performed, this deliverable will not exist. (See the task description for CV.040 for more information on when this task should be performed.)

Conversion Program Designs (CV.060)

The Conversion Program Designs provides the information needed to code the conversion modules. If Design Conversion Programs was not performed, this deliverable will not exist. (See the task description for CV.060 for more information on when this task should be performed.)

Optional

You may need the following input for this task:

Physical Database Design

The physical database design provides the data structure of the Oracle target applications. Either the *Oracle Application Technical Reference*

Manuals or the Oracle Applications Oracle Designer Database can serve as the physical database design.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the Conversion Program Designs (CV.060) to understand the requirements and design elements of the conversion programs to be developed.	
2.	Develop download / extract programs.	Download Programs
3.	Develop interface table creation programs.	Interface Table Creation Programs
4.	Develop upload programs.	Upload Programs
5.	Develop translation programs.	Translation Programs
6.	Develop interface and validation programs.	Interface / Validation Programs

Table 6-18 Task Steps for Develop Conversion Programs

Approach and Techniques

Code the conversion programs following the build standards for custom extensions and any supplemental standards defined specifically for conversion activities in the Conversion Standards (CV.020). In many projects, client staff members may perform the final conversion, and therefore may be executing the modules created by a development team member. In this case, provide the client staff members with a conversion execution document that details what each module does and

the order in which the modules should be run to achieve a successful conversion. Use checklists to document conversion prerequisites and post-conversion activities.

As mentioned in the Conversion Program Designs (CV.060), many Oracle Applications have standard interfaces that facilitate the conversion coding effort. In addition, consider using automated conversion tools instead of traditional coding methods. Below are some approaches and techniques for each of the five conversion programming areas.

Download Programs

The client staff members are normally responsible for writing and maintaining the download modules. As with the Conversion Program Designs (CV.060), it is important that you provide them with a file schema that describes how many ASCII files to produce for a given business object, the order of the fields, the delimiter to be used, and so on. Depending on the tool you are using for the conversion and the legacy file structure, you may want to combine data in one of the ways listed below:

- single extract file to populate multiple Oracle tables
- multiple passes of a single extract file to populate multiple Oracle tables
- multiple extract files to populate multiple Oracle tables
- multiple passes of multiple extract files to populate multiple Oracle tables
- single extract files to populate a single Oracle table
- multiple passes of a single extract file to populate a single Oracle table

This decision is largely dependent on the relationship and complexity of the Oracle tables being populated.

Interface Table Creation Programs

Use SQL*Plus to create the interface tables in Oracle. You can build an interface table for each Oracle production table you are going to load. The interface table can be a direct copy of the production table, or can have only the columns within the production table that you are going to load.

You do not want the interface table to contain the constraints that the production tables have. This is because you are performing the data translation in the interface tables prior to validation and insertion. Therefore, if you copy the production table to use as your interface table, you probably want to remove the constraints, grants, and synonyms. For example, if the extract file contains a Y but the database is expecting a numeric value, then you may load the Y into the temporary table and translate the Y to a numeric value of 1 before passing the value to the production table.

If the Oracle Application has a standard API to facilitate the conversion of a given business object, then you do not have to create an interface table. In this case, the Oracle Application provides an interface table that you load the legacy data elements and data defaults into. If however, there is a great deal of data translation that needs to take place before the legacy data is loaded into the Oracle Applications interface tables, then you may want to create an additional translation table. Refer to the *Oracle Applications Open Interfaces Manual* for specific information on how to populate the interface table columns, as well as information on how to execute the concurrent process.

Automated Conversion Tools

If you are using an automated conversion tool, you may be able to use the tool to create the interface tables. Most of the logic for data validation, transformation, and so on may also be built into the tool templates. However, you may need to create interface tables for mapping between old and new data values. Refer to the documentation of the tool you are using for information on its capabilities and use.

Upload Programs

Use a loader tool such as SQL*Loader to load the legacy system data flat file into the temporary tables you have created. If you are using an automated conversion tool, it may provide this functionality. Automated conversion tools allow you to map the legacy data to the Oracle table and columns, perform the required translations and validations, and then load the data into Oracle.

Automated Conversion Tools

If you are using an automated conversion tool, you may be able to define all the validation, transformation, and translation rules in the tool. Refer to the documentation of the tool in use for information on its capabilities and use.

Translation Programs

Create the translation programs using an appropriate programming language, depending on the complexity of the translations and your skills. SQL and PL/SQL are the preferred tools for creating the translation programs. The level of complexity of the translation depends on the legacy system being converted. Is the legacy system normalized? Is new functionality being built into Oracle that was not used in the legacy system? These types of questions directly affect the level of complexity of the translation programs.

Some or most of the data translation can be built on the legacy side, depending on available IS support resources. The Conversion Data Mapping (CV.040) and Conversion Program Designs (CV.060) should provide the developer with all of the necessary logic for the data translations. Some examples of translations follow:

- date formatting
- truncation of values
- concatenation of values
- if/then logic

As mentioned previously, automated conversion tools are an option for building and executing this code.

Automated Conversion Tools

If you are using an automated conversion tool, you may be able to define these translations in the tool. Refer to the documentation of the tool in use for information on its capabilities and use.

Interface/Validation Programs

The complexity of the interface modules is primarily determined by the level of validation required for the data being converted. To guarantee that the data is useable in the new Oracle Application, your interface programs need to perform the same level of validation as the forms of the application. For example, if a form validates that a sales person assigned to a customer must already exist in the system, your interface/validation program needs to perform the same level of validation. Perform a lookup against the table that stores the sales person information to confirm the existence of such data.

If the Oracle Application provides a standard API, execute the interface program as a concurrent process to provide the required level of validation.

The interface modules are typically written in PL/SQL or Pro*C. However, there are several automated tools that build and execute the code to validate and load the production tables.

If you are using an automated conversion tool , you may be able to build most of the validation logic in the tool. Refer to the documentation of the tool you are using for information on its capabilities and use.

Linkage to Other Tasks

The Conversion Programs are an input to the following tasks:

- CV.090 - Perform Conversion Unit Tests
- PT.100 - Construct Performance Test Database

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Developer	100

Table 6-19 Role Contribution for Develop Conversion Programs

* Participation level not estimated.

Deliverable Guidelines

The deliverable for this task is the conversion module program code. For each legacy business object being converted programmatically, this deliverable should address the following:

- legacy data extraction
- creation of interface tables

- uploading data to interface tables
- translation of data in interface tables
- validation and interfacing of data to production tables

Tools

Deliverable Template

A deliverable template is not provided for this task.

Automated Conversion Tools

In place of traditional coding tools, you can use the following automated conversion tools:

- Oracle's EDMS
- Smart Corporation's SMART DB Workbench
- Evolutionary Technologies' ETI Extract
- Oracle's Gateway products



Web Site: You can find further information on Oracle Corporation's Enterprise Data Management System (EDMS) at <http://www.oracle.com/EDMS/index.html>



Web Site: You can find further information on Smart Corporation's SMART DB Workbench at <http://www.smartdb.com>



Web Site: You can find further information on Evolutionary Technologies, Inc. Extract Tool at <http://www.evtech.com>



Web Site: You can find further information on Oracle's Gateway products at <http://www.oracle.com>

CV.090 - Perform Conversion Unit Tests (Optional)

In this task, you test the conversion programs to verify that all programs work without errors and according to the conversion testing specifications pre-defined in the conversion unit testing components of the Conversion Test Plans (CV.070).



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Unit-Tested Conversion Programs**. These programs include conversion program source code that has been tested to verify that the processing logic of each program module functions without errors.

Prerequisites

You need the following input for this task:



Conversion Environment (CV.030)

To test the interface and validation conversion programs, the legacy data needs to be loaded into an Oracle Application configured to meet your business requirements. If Prepare Conversion Environment was not performed, this deliverable will not exist. (See the task description for CV.030 for more information on when this task should be performed.)



Conversion Test Plans (CV.070)

The Conversion Test Plans provide the procedures for the Perform Conversion Unit Test task. If Prepare Conversion Test Plans was not performed, this deliverable will not exist. (See the task description for CV.070 for more information on when this task should be performed.)

 Conversion Programs (CV.080)

This Conversion Programs provide the programs you have to test. If Develop Conversion Programs was not performed, this deliverable will not exist. (See the task description for CV.080 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the Conversion Unit Test component of Conversion Test Plans (CV.070)	
2.	Conduct the conversion unit test for each conversion program module, per the Conversion Test Plans (CV.070)	
3.	Record conversion unit test results in the tables provided in Conversion Test Plans (CV.070).	

Table 6-20 Task Steps for Perform Conversion Unit Tests

Approach and Techniques

Perform the tests as specified in the conversion unit test component of the Conversion Test Plans (CV.070). Complete this component by recording your results.

In most cases the developer who has written the conversion program should also test the conversion program. Since unit testing involves executing conversion code, the primary role for this task should be a developer.

If you are using an automated conversion tool, you should unit test the individual steps in the conversion. For example, you may need to test the following functions:

- Can you test the conversion template?
- Can you connect to the database?
- Can you map the conversion template to the Oracle tables?
- Can you load the legacy data into the Oracle tables?

Linkage to Other Tasks

The Unit-Tested Conversion Programs are an input to the following task:

- CV.100 - Perform Conversion Business Object Tests

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Developer	100

Table 6-21 Role Contribution for Perform Conversion Unit Test

Deliverable Guidelines

The deliverable for this task is the unit-tested conversion program code. Use the unit-tested conversion programs to show that the processing logic of each conversion program functions without errors.

This deliverable should address the following:

- actual conversion unit test results

Use the Conversion Test Plans (CV.070) deliverable to record your test results directly into the space provided.

Tools

Deliverable Template

A deliverable template is not provided for this task.

CV.100 - Perform Conversion Business Object Tests (Optional)

In this task, you test the complete conversion of each business object by executing all conversion modules for the business object in the appropriate sequence and verify that the resulting data is correct.



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Business Object-Tested Conversion Programs**. These programs include conversion programs that have been tested to verify that, when run end to end in the proper sequence, they result in converted business objects that satisfy the pre-determined business object test specifications.

Prerequisites

You need the following input for this task:



Conversion Environment (CV.030)

In order for the testers to verify that the converted data performs correctly in the new Oracle Application, configure and install the Oracle Application to meet the defined business requirements. If Prepare Conversion Environment was not performed, this deliverable will not exist. (See the task description for CV.030 for more information on when this task should be performed.)



Conversion Test Plans (CV.070)

The Conversion Test Plans provide the conversion test procedures to follow when performing the conversion business object test. If Prepare Conversion Test Plans was not performed, this deliverable will not exist. (See the task description for CV.070 for more information on when this task should be performed.)

 Unit-Tested Conversion Programs (CV.090)

Unit-Tested Conversion Programs provide the programs to load the converted data to the Oracle target application production tables so that testers have data to test in the new applications. If Perform Conversion Unit Tests was not performed, this deliverable will not exist. (See the task description for CV.090 for more information on when this task should be performed.)

Optional

You may need the following input for this task:

 Transition and Contingency Plan (PM.030)

The Transition and Contingency Plan describes the sequence in which conversion modules are executed and identifies other conversion modules or reports you can use to check converted data.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the Conversion Business Object Test component of the Conversion Test Plans (CV.070).	

No.	Task Step	Deliverable Component
2.	Conduct the conversion business object test for each business object being converted, per the Conversion Test Plans (CV.070).	
3.	Record conversion business object test results in the tables provided in Conversion Test Plans (CV.070).	

Table 6-22 Task Steps for Perform Conversion Business Object Tests

Approach and Techniques

Test the data elements previously selected for business object testing in the Conversion Test Plans (CV.070) and compare them to the original data state in the legacy system.

Either members of the conversion team or users may be responsible for performing the conversion business object tests. A prerequisite for performing the business object tests is having the conversion modules written and having the legacy data (or at least a representative sample of the data) loaded into the Oracle Applications tables. It is also necessary to have trained users to navigate in the new application and run the necessary reports for comparison of converted data to legacy system data.

Depending on the level of testing, it may be necessary to designate a person whose primary responsibility is to manage the conversion business object testing effort. It is critical that the failures discovered during testing are properly managed and resolved in a timely manner. A business object test case is not complete until all test failures have been resolved. A test case is defined as all business object tests performed for the complete testing of a given business object (for example, customers, invoices, and so on).

Linkage to Other Tasks

The Business Object Tested Conversion Programs are an input to the following task:

- CV.110 - Perform Conversion Validation Tests

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Tester	90
Technical Analyst	10

Table 6-23 Role Contribution for Perform Conversion Business Object Tests

Deliverable Guidelines

The deliverable for this task is the business object-tested conversion program code. Use the Business Object-Tested Conversion Programs to show that, when the conversion programs for a given business object are run in the proper sequence, the resulting converted data meets the requirements for that business object in the target Oracle Application.

This deliverable should address the following:

- actual conversion business object test results

Use the Conversion Test Plans (CV.070) deliverable to record your test results directly into the space provided.

Tools

Deliverable Template

A deliverable template is not provided for this task.

CV.110 - Perform Conversion Validation Tests (Optional)

In this task, you validate that the target applications function correctly with the converted business objects.



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Validation-Tested Conversion Programs**. These programs include conversion programs that have been tested to verify that the resulting converted legacy data performs correctly in the entire suite of Oracle Applications.

Prerequisites

You need the following input for this task:

 Conversion Environment (CV.030)

A properly configured Oracle Applications environment is required for testers to verify that the converted data performs correctly in the new Oracle Application. If Prepare Conversion Environment was not performed, this deliverable will not exist. (See the task description for CV.030 for more information on when this task should be performed.)

 Conversion Test Plans (CV.070)

The Conversion Test Plans provide the conversion test procedures to follow when performing the conversion validation test. If Prepare Conversion Test Plans was not performed, this deliverable will not exist. (See the task description for CV.070 for more information on when this task should be performed.)

 Business Object-Tested Conversion Programs (CV.100)

Business Object-Tested Conversion Programs provide the programs for loading the converted data to the Oracle target application production

tables, so the testers have data to test in the new applications. If Perform Conversion Business Object Tests was not performed, this deliverable will not exist. (See the task description for CV.100 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the Conversion Validation Test component of Conversion Test Plans (CV.070).	
2.	Conduct the conversion validation test for each business object being converted, per the Conversion Test Plans (CV.070).	
3.	Record conversion validation test results in the tables provided in Conversion Test Plans (CV.070).	

Table 6-24 Task Steps for Perform Conversion Validation Tests

Approach and Techniques

Test and compare data elements selected for validation testing in the Conversion Test Plans (CV.070) to the original data state in the legacy system.

Either members of the conversion team or users may be responsible for performing the conversion validation tests. The prerequisites for performing the conversion validation tests are having the conversion modules written and having the legacy data (or at least a representative sample of the data) loaded into the Oracle Applications tables. Trained

users are required to navigate in the new application and run the necessary reports for comparison of converted data to legacy data.

Decide how thorough your validation test will be. Depending on this decision, it may be necessary to designate a person whose primary responsibility is to manage the validation testing effort. It is critical that any failures be properly managed and resolved in a timely manner. A validation test case is not complete until all test failures have been resolved.

The conversion validation test should mimic the entire flow of the converted data through the suite of Oracle Applications. For example, if you convert an open receivables invoice, can you post cash to that invoice? Can you generate aged trial balance reports? Can you post to the general ledger? If you are interfacing Oracle Applications to legacy or third-party applications, test the flow of the converted data through these interface points as well.

After the converted data has passed the Perform Conversion Validation Tests (CV.110), the data is ready for use in the following Business System Testing (TE) tasks:

- Perform System Test (TE.110)
- Perform Systems Integration Test (TE.120)

If performance testing of conversion programs is within the scope of your project's Performance Testing (PT) Process, then the conversion programs are also ready at this point for use in the following Performance Testing (PT) task:

- Execute Performance Test (PT.120)

Linkage to Other Tasks

The Validation-Tested Conversion Programs are an input to the following tasks:

- CV.120 - Install Conversion Programs
- TE.110 - Perform System Test
- TE.120 - Perform Systems Integration Test
- PT.110 - Prepare Performance Test Environment

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Tester	60
Business Analyst	30
Technical Analyst	10

Table 6-25 Role Contribution for Perform Conversion Validation Tests

Deliverable Guidelines

The deliverable for this task is the validation-tested conversion program code. Use the Validation-Tested Conversion Programs to show that the resulting converted data performs correctly in the entire suite of Oracle Applications.

This deliverable should address the following:

- actual conversion validation test results

Use the Conversion Test Plans (CV.070) deliverable to record your test results directly into the space provided.

Tools

Deliverable Template

A deliverable template is not provided for this task.

CV.120 - Install Conversion Programs (Optional)

In this task, you perform and document the installation of the conversion programs in the production environment.

This task presumes that the conversion programs have been tested. If you are using an automated conversion tool, this task requires that you install the software, tested conversion templates, and conversion maps needed for the task Convert and Verify Data (CV.130).



If your project includes *programmatic data conversion* of legacy business objects, you should perform this task.

Deliverable

The deliverable for this task is the **Installed Conversion Programs**. A supporting deliverable, the Installed Conversion Programs Document, is also prepared to assist in the proper execution of this task.

Prerequisites

You need the following input for this task:

 Validation-Tested Conversion Programs (CV.110)

The Validation-Tested Conversion Programs are the tested conversion programs or tool templates and maps that are to be installed in the production environment. If Perform Conversion Validation Tests was not performed, this deliverable will not exist. (See the task description for CV.110 for more information on when this task should be performed.)

 Production Environment (PM.040)

The Production Environment is a fully configured production environment in which to install the conversion software.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review Production Environment (PM.040) to understand the configuration of the Production Environment.	
2.	Describe the purpose of the Installed Conversion Programs and the supporting document.	Introduction
3.	Describe the Production Environment.	Production Environment
4.	Describe the directory structure of the Production Environment.	Directory Structure
5.	Install the conversion programs and automated conversion tool software (if used).	Installation Procedures Checklist
6.	List the location of each conversion program in the Production Environment.	Conversion Programs
7.	List the location of any conversion tools in the Production Environment.	Automated Conversion Software

Table 6-26 Task Steps for Install Conversion Programs

Approach and Techniques

Verify the installation of the necessary conversion software in the production environment prior to performing the conversion of production data. This is particularly important if you are performing the final conversion without the aid of the team that developed and tested the conversion code.

Linkage to Other Tasks

The Installed Conversion Programs are an input to the following task:

- CV.130 - Convert and Verify Data

Role Contribution

The percentage of total task time required for each role follows:

Role	%
System Administrator	90
Technical Analyst	10
Client Staff Member	*

Table 6-27 Role Contribution for Install Conversion Programs

* Participation level not estimated.

Deliverable Guidelines

The deliverable for this task is the installed conversion program code. To support the proper execution of this task, complete the Installed Conversion Programs template. Use the Installed Conversion Programs template to document important information about the conversion programs and other software installed in the production environment, to support the conversion of legacy data.

The Installed Conversion Programs Document template should address the following:

- documentation of the production environment, its directory structure and installation procedures
- location of conversion programs and automated conversion tools in the production environment

Deliverable Components

The Installed Conversion Programs template consists of the following components:

- Introduction
- Production Environment
- Directory Structure
- Installation Procedures Checklist
- Conversion Programs
- Automated Conversion Software

Introduction

This component describes the purpose of the Installed Conversion Programs and the supporting document.

Production Environment

This component describes the hardware platform, database configuration, operating system, and installed Oracle Applications for the production environment.

Directory Structure

This component describes the directory structure for the installed Oracle Applications and custom extensions.

Installation Procedures Checklist

This component documents the procedures for performing the conversion software and program installation.

Conversion Programs

This component lists the location of the conversion programs within the production environment directory structure.

Automated Conversion Software

This component lists the location of any conversion tools within the production environment directory structure.

Tools

Deliverable Template

Use the Installed Conversion Programs template to create the supporting deliverable for this task.

CV.130 - Convert and Verify Data (Optional)

In this task, you convert and migrate the production data from the old system to the new Oracle production environment.

Completion of this task provides data that is ready for production use.



If your project includes either *programmatic data conversion* of legacy business objects, *manual data conversion* of legacy business objects, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Converted and Verified Data**. This data consists of the converted production data. This deliverable, along with the Production Environment, provides everything required to use the applications in production. This includes properly installed and configured hardware, system software, application software, documentation, and converted production data. A supporting deliverable, the Converted and Verified Data Document, is also prepared to assist in the proper execution of this task. Prepare one deliverable for each business object you are converting.

Prerequisites

You need the following input for this task:

 Manual Conversion Procedures (CV.050)

The Manual Conversion Procedures provide the plan for manually converting business objects from the legacy system to the new application system. If Define Manual Conversion Procedures was not performed, this deliverable will not exist. (See the task description for CV.050 for more information on when this task should be performed.)

 Conversion Program Designs (CV.060)

The Conversion Program Designs provide the plans for converting data from the previous system and loading it into the new application

system. They also include any scripts or other software for automating the conversion. If Design Conversion Programs was not performed, this deliverable will not exist. (See the task description for CV.060 for more information on when this task should be performed.)

☐ **Installed Conversion Programs (CV.120)**

Conversion programs must be installed and ready to run. This also applies to installed automated conversion tools. If Install Conversion Programs was not performed, this deliverable will not exist. (See the task description for CV.120 for more information on when this task should be performed.)

☐ **Transition and Contingency Plan (PM.030)**

The Transition and Contingency Plan contains the strategy for converting the data into the production system and then moving the users to the new system with minimal disruption of their work.

☐ **Production Environment (PM.040)**

The Production Environment is a fully configured production environment in which you convert and verify the data.

☐ **Configured Applications (PM.050)**

The Configured Applications provide properly configured application software in which to convert and verify the data.

☐ **Legacy Data Cleanup**

Legacy data identified in the Data Conversion Requirements and Strategy (CV.010) for pre-conversion data cleanup should meet the data cleanup, data reduction, and normalization requirements.

Optional

☐ **System-Tested Applications (TE.110)**

System-Tested Applications have been tested with converted data to verify that the target applications meet documented business requirements.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Verify that legacy data pre-conversion cleanup has been completed.	
2.	Describe the data conversion and verification sequence of events.	Introduction
3.	Record the sequence in which the download programs should be run and additional important information about the programs.	Download Programs
4.	Record the sequence in which the interface table creation programs should be run and additional important information about the programs.	Interface Table Creation Programs
5.	Record the sequence in which the upload programs should be run and additional important information about the programs.	Upload Programs
6.	Record the sequence in which the translation programs should be run and additional important information about the programs.	Translation Programs

No.	Task Step	Deliverable Component
7.	Record the sequence in which the interface / validation programs should be run and additional important information about the programs.	Interface / Validation Programs
8.	Run the conversion programs to convert the legacy system data.	
9.	Verify the converted data.	
10	Secure acceptance that the Converted and Verified Data meets Century Date compliance standards.	

Table 6-28 Task Steps for Convert and Verify Data

Approach and Techniques

The purpose of this task is to complete creation of the production environment by converting the production data for the new system. Once this task is complete, the system is ready for use in production. Setup of the hardware and software is not addressed in this task since Production Environment (PM.040) is a prerequisite for the conversion of legacy data for production use.

Plan the cutover to the production environment in detail. In most cases, the business stops entering information to prevent data from being lost during the transition from the old system to the new one. This is an acceptable approach when the conversion is scheduled to take place over a relatively short period of time, such as a weekend. If, however, the conversion is going to take place over multiple weeks, users may have to enter transactions into both the legacy and target applications.

The completion of this task is on the critical path. The only exception is if the data does not change frequently and the conversion can be performed in advance of actually going to production. In many cases,

data that is non-transactional fits into this category. For example, you may be able to convert customer or vendor master information before converting the transactional data supported by this master data. If this is the case, consider loading the data into the business system test installation and making the business system test installation the same as the production installation.

Schedule conversion of the production data for off-peak hours so that the additional load does not negatively impact the organization's ongoing business.

In many projects the implementing client staff members perform the final conversion, and therefore they may be executing modules created by a development team member. The Transition and Contingency Plan (PM.030) which delineates the conversion order and any pre- and post-conversion activities, provides a roadmap for them to follow in the completion of this task .

It is important that you verify and audit the converted data to make sure that it meets all defined audit requirements. You should have already thoroughly tested the data through the three levels of conversion testing described earlier and the business system test. However, you should also verify the final production data.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Programmatically converted legacy data must be translated to the appropriate century date state before being uploaded to the production

tables. Manually converted legacy data must be keyed into the data entry forms using four digits for the year, where supported.

Linkage to Other Tasks

The Converted and Verified Data is an input to the following tasks:

- TE.130 - Perform Acceptance Test
- PM.070 - Verify Production Readiness

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Analyst	70
Database Administrator	20
Project Manager	5
System Administrator	5
Client Staff Member	*

Table 6-29 Role Contribution for Convert and Verify Data

* Participation level not estimated.

Deliverable Guidelines

The deliverable for this task is the Converted and Verified Data in the production environment. To support the proper execution of this task, complete the Converted and Verified Data template. Use the Converted and Verified Data template to verify that all legacy data identified in Data Conversion Requirements and Strategy (CV.010) has been converted to the target applications and verified by the users.

The Converted and Verified Data template should address the following:

- documentation of the production data conversion

Achieve Converted and Verified Data by successfully executing the conversion programs to perform the following functions:

- Download the data from the legacy system and create a flat file that can be transferred and uploaded to Oracle tables.
- Create Oracle interface tables that can store the legacy data before the data is validated and moved to the production tables of the Oracle Application.
- Load the legacy data to the temporary tables.
- Perform any required data translation, transformation, or cleanup before moving the data to the production tables.
- Interface (validate) and move the data to the Oracle production tables.

Deliverable Components

The Converted and Verified Data consists of the following components:

- Introduction
- Download Programs
- Interface Table Creation Programs
- Upload Programs
- Translation Programs
- Interface/Validation Programs

Introduction

This component describes the purpose, distribution, and quality criteria of the Converted and Verified Data.

Download Programs

This component records the execution sequence of the download programs, the program name, purpose, file location, the developer, date executed, and the resulting flat file created by executing these programs.

Interface Table Creation Programs

This component records the execution sequence of the interface table creation programs, the program name, purpose, file location, the developer, execution dates, results, and any additional comments.

Upload Programs

This component records the execution sequence of the upload programs, the program name, purpose, file location, the developer, execution dates, results, and any additional comments.

Translation Programs

This component records the execution sequence of the translation programs, the program name, purpose, file location, the developer, and any additional comments.

Interface/Validation Programs

This component records the execution sequence of the interface programs, the program name, purpose, location, the developer, execution date, results, and any additional comments.

Tools

Deliverable Template

Use the Converted and Verified Data template to create the supporting deliverable for this task.

Century Date Acceptance

To document the review and approval of Converted and Verified Data for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Acceptance Certificate should be filed with the deliverable in the project library.

Documentation (DO)

This chapter describes the Documentation process.

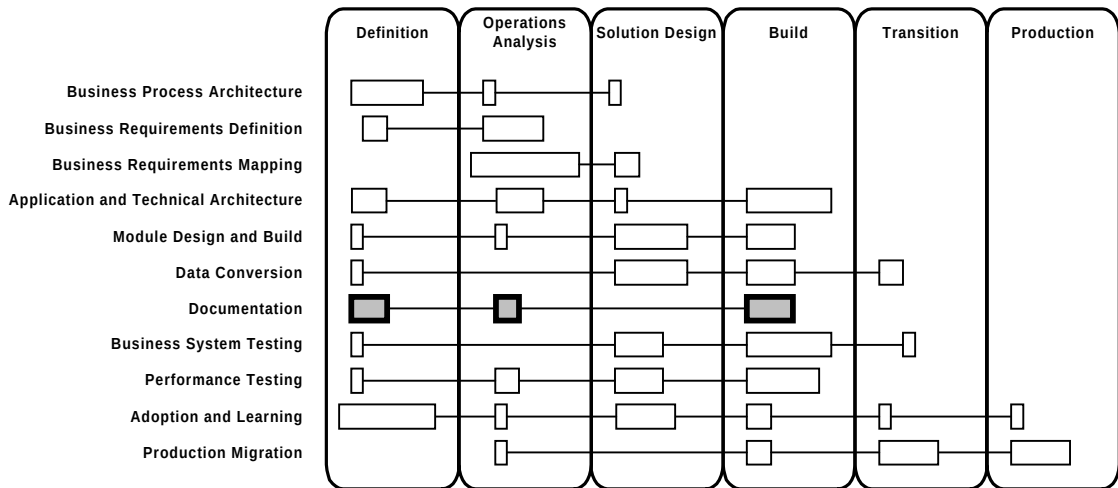


Figure 7-1 Documentation Context

Process Flow

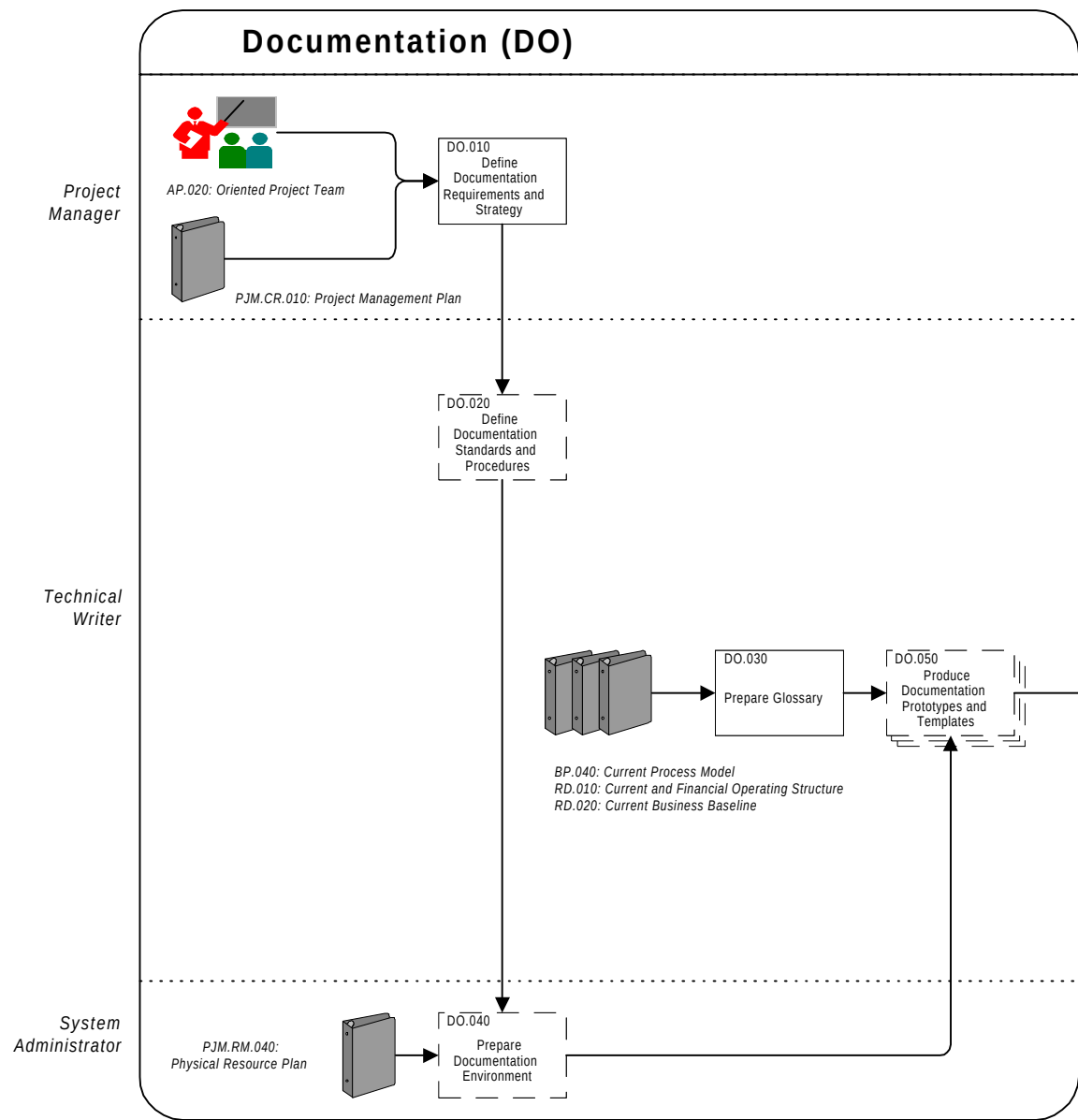


Figure 7-2 Documentation Process Flow Diagram

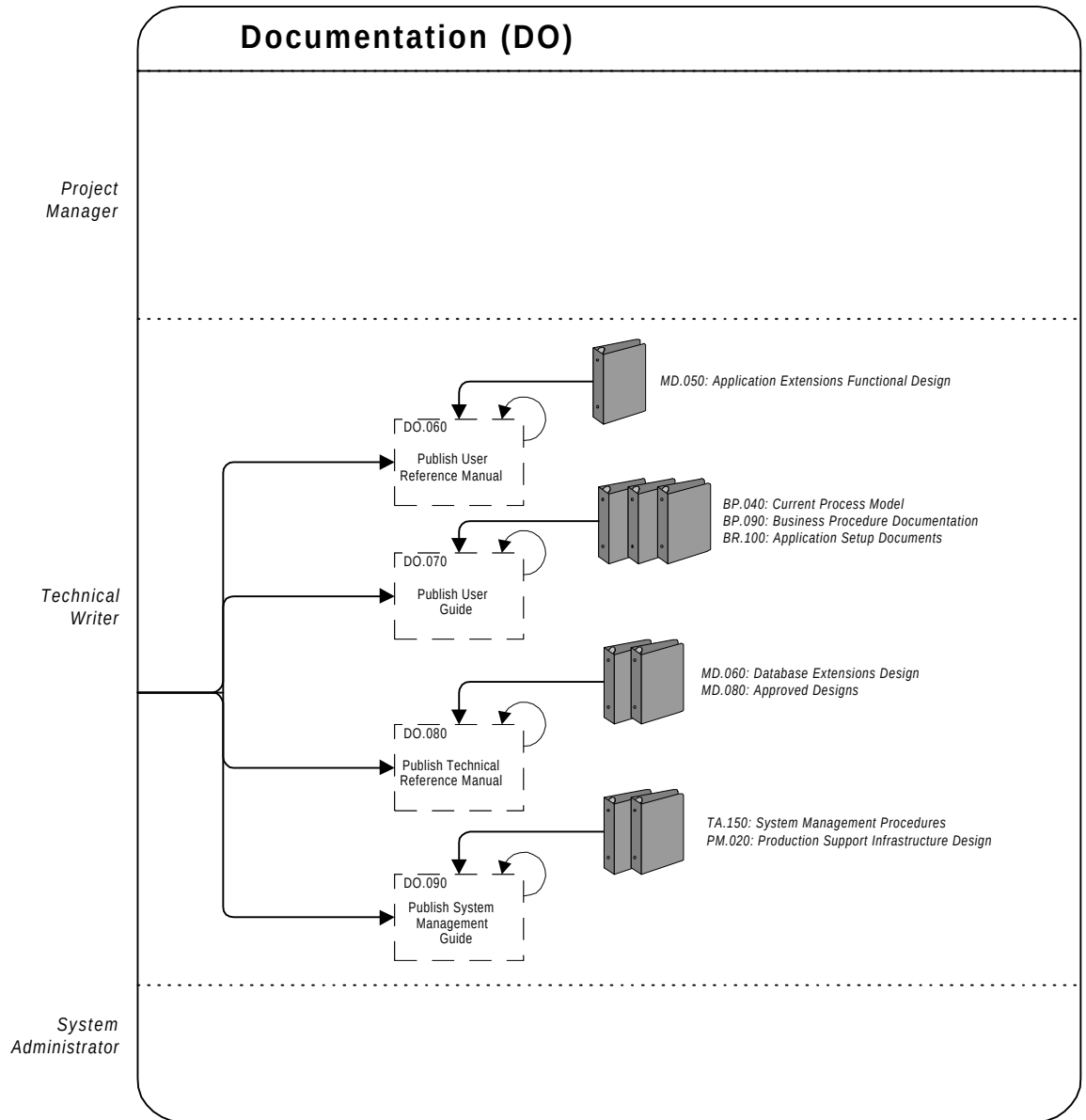


Figure 7-2 Documentation Process Flow Diagram (cont.)

Approach

Oracle Application manuals are the foundation of implementation documentation. The objective of Documentation is to augment the standard Oracle Applications manuals with specific information about the organization’s application extensions and specific business procedures. Using plans, procedures, and detail documents from the project, the writing staff develops supplemental user and technical materials that are tailored to the implementation. Quality documentation is a key factor in supporting the transition to production, achieving user acceptance, and maintaining the ongoing business process.

The Documentation overview figure below illustrates the logical progression of Documentation tasks. Reading from left to right, the diagram indicates that Documentation begins by defining your Documentation Requirements and Strategy (DO.010) to determine which documents you need to produce and your strategy for producing them. Next, you specify the organization’s Documentation Standards and Procedures (DO.020) to capture the expected *look and feel* of the new documentation. Throughout the project, project-specific terms are captured and described in the Glossary (DO.030). After the Documentation Environment (DO.040) is prepared, you are ready to begin building the Documentation Prototypes and Templates (DO.050) of the four main project documents. Ultimately, every documentation task exists to contribute to the publication of the User Reference Manual (DO.060), User Guide (DO.070), Technical Reference Manual (DO.080), and System Management Guide (DO.090).

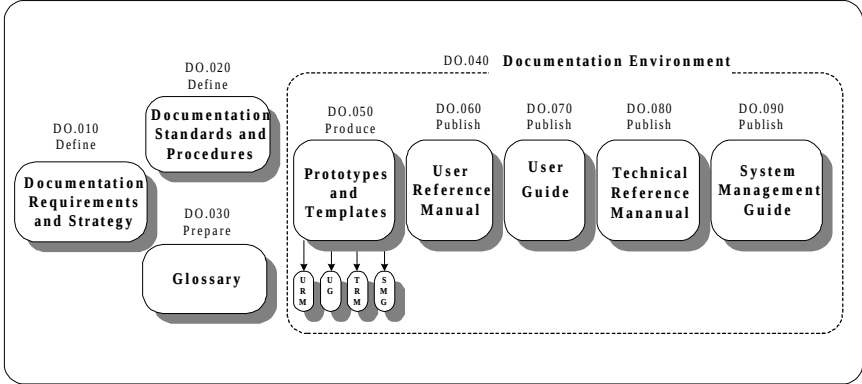


Figure 7-3 Documentation Overview Diagram

Successful project documentation reflects the final application as it is approved for use. It is accurate, concise, and uses diagrams and charts to explain broad or complex concepts. It should be useful to both beginners and experienced users of the system and have a user-friendly tone that is neither too academic nor too technical.

Writing Standards

If you have a writing standards guide, follow it when writing any custom documentation. You can use other appropriate style guides as well.

Prototypes

After you complete the Documentation Requirements and Strategy (DO.010) and define the Documentation Standards and Procedures (DO.020), create a prototype of each document. A prototype consists of a table of contents and a sample chapter. The first purpose for prototyping is to tangibly display your understanding of documentation requirements, standards, and procedures in terms of form and content. The second purpose is to set user expectations. Showing a tangible prototype is far more effective than discussing documentation.

Parallel Work Effort

The project team should begin Documentation tasks whenever the appropriate prototype and the required documentation inputs have been approved. Several required documents may be worked on simultaneously.

Previously Completed Materials

The documentation writers should build on the documents that have been completed during earlier implementation tasks. Earlier documents can be used *as is*, or portions can be incorporated as required.

The following sections can be included in both user and technical documentation.

Table of Contents

Generally, a table of contents is necessary for all documents that are over several pages and cover a variety of subtopics. If a user does not

need to read the entire document from start to finish each time it is referenced, add subtopical headings and create a table of contents.

Introduction

If the customization or subject area includes multiple forms, reports, or programs, consider beginning with an overview of the process explaining how the individual pieces relate to each other.

Appendices

If detailed charts, diagrams, and examples are confusing or cause a loss of continuity in the text, put them in an appendix where they can be referenced.

Glossary

Include any unfamiliar or confusing terms in the glossary. For example, the term *FOB* may not be familiar to all users, or the term *demand* may have a different meaning for each audience. Including these terms in the glossary removes doubt about their meaning.

Index

If the documentation is more than a few pages, an index helps the user locate key topics or words.

Types of Media

Documentation may be published in a number of electronic and published media.

Printed Documentation

The documentation may be produced on printed pages. This has been the standard media in the past for publishing documentation. Some organizations may choose to go *paperless* — if they do, they may not allow any documents to be published in printed form.

Portable Electronic Documents

The documentation may be produced as a portable electronic document using a format such as Adobe's Portable Document Format (pdf). These portable documents may be exchanged among users and read on a variety of computing environments using Adobe's Acrobat Reader. The

key benefit to this approach is that a single publication activity yields a multitude of platforms in which the documents may be read.

Web Pages

The documentation can be published on a web page and can be accessed via a web browser and an URL address.

User Validation

Initial documentation can be used when you execute the task, Perform System Test (TE.110). The initial versions of the documentation should be used to help the user validate the standard implementation and any custom application extensions that have been built. Users should review documentation along with early test results for programs, reports, enhancements, and new forms.

Documentation Testing

If it is to be a valuable resource, documentation must accurately reflect the system. During Business System Testing (TE), applicable documentation should be referenced by the testers as it would be in production. If the documentation is unclear or outdated, appropriate changes must be made. Testing is not complete until the documentation is verified as correctly reflecting the new system.

Tasks and Deliverables

The tasks and deliverables for this process are as follows:

ID	Task Name	Deliverable Name	Required When	Type*
DO.010	Define Documentation Requirements and Strategy	Documentation Requirements and Strategy	Always	SI
DO.020	Define Documentation Standards and Procedures	Documentation Standards and Procedures	Project includes <i>publishing project-specific documentation</i>	SI
DO.030	Prepare Glossary	Glossary	Always	SI
DO.040	Prepare Documentation Environment	Documentation Environment	Project includes <i>publishing project-specific documentation</i>	SI
DO.050	Produce Documentation Prototypes and Templates	Documentation Prototypes and Templates	Project includes <i>publishing project-specific documentation</i>	MI
DO.060	Publish User Reference Manual	User Reference Manual	Project includes <i>customizations to standard functionality or interfaces with external systems</i>	IT
DO.070	Publish User Guide	User Guide	Project includes <i>publishing project-specific documentation</i>	IT
DO.080	Publish Technical Reference Manual	Technical Reference Manual	Project includes <i>customizations to standard functionality or interfaces with external systems</i>	IT
DO.090	Publish System Management Guide	System Management Guide	Project is of <i>medium or high complexity</i>	IT

*Type: SI=singly instantiated, MI=multiply instantiated, MO=multiply occurring, IT=iterated, O=ongoing. See Glossary.

Table 7-1 Documentation Tasks and Deliverables

Objectives

The objectives of Documentation are:

- Produce a reference that shows the users how to use application functionality (User Reference Manual - DO.060).
- Produce a set of procedures for using the application in response to day-to-day business events (User Guide - DO.070).
- Assemble the documents that describe the technical details of the application for the maintenance staff (Technical Reference Manual - DO.080).
- Produce a set of procedures for managing the system (System Management Guide - DO.090).

Deliverables

The deliverables of this process are as follows:

Deliverable	Description
Documentation Requirements and Strategy	A listing of documents that will need to be published for the project and the strategy for producing them.
Documentation Standards and Procedures	A document that identifies how the documentation should <i>look and feel</i> and lists the procedures to be followed when producing documentation.
Glossary	A listing of terms that are unique to the project.
Documentation Environment	The hardware and software environment that allows you to support documentation development.

Deliverable	Description
Documentation Prototypes and Templates	A prototype of each major deliverable that will be produced.
User Reference Manual	A detailed description of the functionality of each custom module and extension.
User Guide	A detailed set of procedures to help the reader respond to specific business events by using the organizations standard and modified application programs.
Technical Reference Manual	Maintenance instructions for database extensions, custom programs, and upgrade considerations.
System Management Guide	A set of procedures for managing the application system.

Table 7-2 Documentation Deliverables

Key Responsibilities

The following roles are required to perform the tasks within this process:

Role	Responsibility
Business Analyst	Identify and document business terms for the Glossary (DO.030).
Business Line Manager	Describe documentation needs and current documentation usage. Agree on documentation standards.

Role	Responsibility
Client Project Manager	Provide input into developing the documentation requirements, strategy, standards, and procedures for the project.
Client Staff Member	Install the Documentation Environment and assist in testing.
Developer	Provide assistance in verifying the documentation is correct and complete. Provide material for the Technical Reference Manual (DO.080).
Project Manager	Obtain agreement with users on deliverables. Approve and procure the Documentation Environment (DO.040).
System Administrator	Revise sections of the System Management Guide (DO.090) based on system operations test scenarios.
System Architect	Provide content for sections of the System Management Guide (DO.090).
Technical Analyst	Identify terms for the Glossary (DO.030) and provide content, as necessary, for the User Reference Manual (DO.060), User Guide (DO.070), and Technical Reference Manual (DO.080).
Technical Writer	Revise sections of the User Guide (DO.070) based on system test scenarios. Design, write, edit, and review documentation deliverables.

Role	Responsibility
User	Provide the definition of business terms for the Glossary (DO.030). Review the documentation and provide feedback.

Table 7-3 Documentation Key Responsibilities

Critical Success Factors

The critical success factors of Documentation are as follows:

- availability of skilled technical writers
- early involvement of technical writers
- user input to the procedures
- project team commitment to the time and resources required to produce documentation
- verification that the documentation accurately reflects the final application implementation

DO.010 - Define Documentation Requirements and Strategy (Core)

In this task, you outline the overall requirements for creating documentation and specify the deliverables to be produced. You also identify the strategy you will use in publishing project-specific documentation.

Deliverable

The deliverable for this task is the **Documentation Requirements and Strategy**. It identifies the list of project-specific documentation that needs to be published and the strategy for publishing this documentation.

Prerequisites

You need the following input for this task:

☐ **Project Management Plan [*initial complete*] (PJM.CR.010)**

The Project Management Plan provides a high-level discussion of the scope for publishing project-specific documentation. It also lists the deliverables specific to this project, and indicates how the project should be organized and run.

☐ **Oriented Project Team (AP.020)**

Prior to defining the Documentation Requirements and Strategy, the project team members attend a project team orientation meeting.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Verify the documentation descriptions in the project-level scope, objectives, and approach as contained within the Project Management Plan (PJM.CR.010). Review the project documentation standards, if available.	
2.	Identify the scope of the documentation, milestone publishing dates, and constraints.	Scope
3.	Discuss the project with the project manager and establish the specific scope and objectives for documentation.	Scope
4.	Identify Documentation objectives and critical success factors.	Objectives
5.	Review the deliverables that will be produced by this process. Review the list of deliverables with client staff members.	Approach
6.	Identify constraints and assumptions that are associated with the process.	Approach
7.	Review risks to the process activities and objectives.	Approach

No.	Task Step	Deliverable Component
8.	Define policies and procedures unique to Documentation.	Approach
9.	Identify the mechanism for publishing documentation.	Documentation Strategy
10.	Establish the procedures for maintaining the Documentation Environment.	Documentation Strategy
11.	Identify potential risks in the strategy outlined.	Documentation Strategy
12.	Identify the documentation requirements at a summary and detailed level.	Documentation Requirements
13.	Identify users for each document and interview them to determine their requirements.	Documentation Requirements
14.	Describe the components of each document. Determine the format for transferring ownership to the user.	Documentation Requirements
15.	Describe the resource requirements needed to support the Documentation process.	Documentation Requirements
16.	Document the Documentation Environment requirements.	Documentation Requirements

No.	Task Step	Deliverable Component
17.	Document the staffing requirements for publishing project-specific documentation.	Documentation Requirements
18.	Identify human and physical resources required to develop and deliver documentation.	Documentation Requirements
19.	Document specific tool requirements for the Documentation process.	Documentation Requirements
20.	Review the draft deliverable with senior management and seek approval.	
21.	Secure acceptance for the Documentation Requirements and Strategy.	Acceptance Certificate (PJM.CR.080)
22.	Identify any material changes to project scope and associated task estimates with the project manager.	

Table 7-4 Task Steps for Define Documentation Requirements and Strategy

Approach and Techniques

Every project must determine if there is a need to publish project specific documentation, or whether standard Oracle Applications documentation meets their requirements. While most organizations are able to implement Oracle Applications with standard Oracle documentation, some organizations determine that they need additional, project-specific documentation based on the review of the organization's overall documentation requirements.

Gather your requirements by interviewing users and asking them questions about their documentation needs. These interview questions should help you determine if the organization is building any application extensions, whether interfaces to third-party or legacy systems are to be used, and whether there are any other compelling reasons for which you may want to build project-specific documentation.

Once you have defined the Documentation Strategy, it should be reviewed and accepted by management before progressing with the remaining documentation tasks.

The advantage of the AIM documentation approach is that it is iterative in nature. By preparing a prototype, and an initial draft version of each document you are able to gather feedback on the document's content and make revisions as necessary. This allows for continuing improvement in the quality of the documents. Project-specific documentation is often used during business system testing, learning, and the ongoing support of the new system.

Required Document List

Review the Project Management Plan (PJM.CR.010) Documentation Requirements. If the Project Management Plan specifies the documents to be produced, use this list and indicate in the Documentation Requirements and Strategy where the list is located in the Project Management Plan (PJM.CR.010).

If the Project Management Plan is not specific, work with the project leader and user management to determine the list of required documents. The following documents have Documentation tasks specifically designed for publishing manuals and guides:

- User Reference Manual (DO.060)
- User Guide (DO.070)
- Technical Reference Manual (DO.080)
- System Management Guide (DO.090)

User Reference Manual (DO.060)

The User Reference Manual explains the functionality of the customizations and extensions to the system users. Each section of the User Reference Manual explains the details of a single system function

and includes information for each field, option, data validation, error message, and so on. It is an important reference for experienced users of the system who need information on infrequently used functions.

User Guide(DO.070)

The User Guide is business oriented and contains the manual-and application-based procedures needed by the users to respond to business events. It is an instruction manual for the business area and is the basis for user learning. New applications users rely on the User Guide to learn how to perform their jobs on the system.

Technical Reference Manual (DO.080)

The Technical Reference Manual describes the custom modules and extensions, including their design and architecture. It should supplement the *Oracle Application Technical Reference Manual* and is used by anyone responsible for future application maintenance.

System Management Guide (DO.090)

The System Management Guide includes all System Management Procedures (TA.150) and is organized by process rather than by system function. System management personnel use it to determine how to backup, recover, and audit the application system.

Scale and Scope of Documentation

Scale of the Documentation Process

The creation of a separate Documentation Requirements and Strategy for Documentation may be different depending on the type of project.

Small- to Medium-Sized Implementations

If documentation is part of a small implementation project, the scope definition, work management, and funding of the Documentation process may be incorporated into the management of the overall project, and this task is greatly simplified. In such cases, you should review the existing Project Management Plan (PJM.CR.010), verify that it is an accurate statement of the Documentation process, and then move to the next task. In some cases you may wish to incorporate many or all of the deliverable components of the Documentation Requirements and Strategy (DO.010) into the Project Management Plan.

Large, Complex Projects

In a large and more complex implementation project, the Documentation process may need to be semi-autonomous because of its number, size, and complexity, and a separate documentation subproject may be needed to provide effective control. In these types of implementation projects, the Project Management Plan (PJM.CR.010) defines the high-level scope and policies, but does not provide enough detail about individual subprojects. This detail would be documented in this deliverable.

Task Complexity Depends on Project Scope

The level of detail required for this task depends on the scope of the implementation project and the Documentation process within it. If the Documentation work is being performed for a small number of applications that need only to fit into existing documentation or information systems strategy, you need only to document the parts of the strategy that are relevant to the limited Documentation scope and proceed to the next task. At the other extreme, if this is an enterprise-level documentation for a large-scale replacement of business systems, or there is no clear documentation or information systems strategy in place already, you may need to go into some detail to document the documentation policies and expectations.

Business Vision

The vision for the new business systems is a key initial ingredient of the design of any documentation, especially where the Documentation process is covering the strategic aspects of the new systems as well as the tactical design. The vision must be documented and understood early in the process. Examples of business visions are listed below:

- Users must be able to access their documentation online over the organization's intranet.
- The corporation will standardize all corporate-wide documentation and allow each site to document their site-specific documentation as they wish.
- The business will offer all documentation on electronic form. Some documentation may not be offered in hardcopy format.

Documentation Policies and Standards

At the heart of every documentation project are directives and policies that are fundamental to the documentation design. In large projects

that are modifying many of the standard application modules, you may be defining these directives and policies for the first time. These are the guiding principles for the design and distribution of the new documentation which might be using new technology for the first time. In projects where the implementation is more limited in scope, this may be a matter of conforming to policies and principles that the information systems department has already established for any new documentation.

In situations where a formal information systems strategy project has preceded the implementation Documentation process, a set of principles, directives, and strategies may already be in place, which you can directly input as requirements to the implementation documentation. The selection of a particular publishing tool or technology may be partly in response to the demands of the particular information systems strategy.

Documentation Scope and Objectives

When setting scope for the Documentation process, it is important to understand that project-specific documentation can be expensive to author, publish, and maintain. Proper scope setting and management of expectations are essential. The project and business managers should specify their high-level goals for preparing documentation and how this project will use the documentation once it is published.



Attention: Documentation may be one aspect of an overall performance quality management program within a project. If this is the case, the scope should indicate how documentation fits into that program.

Staffing Resources

The staff resources needed for the Documentation process are directly dependent upon the type and amount of project-specific documentation being published. Technical writers are the primary resources used for publishing documentation. If the documentation project is large enough, you should consider adding a document administrator to help manage documentation files.

Environments Required

The Documentation team should have access to a Documentation Environment (DO.040) that supports the organization's authoring and

publishing needs. This environment could be as simple as a laptop computer with desktop publishing software, or as complex as integrated publishing tools, platform servers, data storage devices, communication networks, and documentation libraries with version control.

Linkage to Other Tasks

The Documentation Requirements and Strategy are an input to the following tasks:

- DO.020 - Define Documentation Standards and Procedures
- DO.040 - Prepare Documentation Environment
- DO.050 - Produce Documentation Prototypes and Templates
- DO.060 - Publish User Reference Manual
- DO.070 - Publish User Guide
- DO.080 - Publish Technical Reference Manual
- DO.090 - Publish System Management Guide

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Business Analyst	30
Technical Writer	20
Project Manager	50
Business Line Manager	*
Client Project Manager	*

Table 7-5 Role Contribution for Define Documentation Requirements and Strategy

* Participation level not estimated.

Deliverable Guidelines

Use the Documentation Requirements and Strategy to determine the custom documentation that is needed for the implementation project. User requirements for each document are analyzed and an agreement regarding design, format, and content is reached. Every user that relies on documentation as an ongoing resource for their day-to-day operations should be represented during this critical task. You also use the Documentation Requirements and Strategy to establish and document the strategy for the documentation as part of the implementation project.

The Documentation Requirements and Strategy should expand the Project Management Plan (PJM.CR.010) created for the overall implementation project in greater detail for the Documentation process. It should make reference to the main project deliverable where appropriate, and indicate those objectives and approaches that are inherited from the main project. It should not, however, be just a duplication of the Project Management Plan.

This deliverable should address the following:

- the list of requirements for producing documentation
- the list of documents that need to be published
- the strategy used to produce project-specific documentation

Deliverable Components

The Documentation Requirements and Strategy consists of the following components:

- Introduction
- Scope
- Objectives
- Approach
- Documentation Strategy
- Documentation Requirements
- Documentation Synopses

- Interview Preparation
- Documentation Audiences
- Record of Interviews

Introduction

This component provides an introduction for the deliverable. The introduction contains the following sections:

- **Scope, Approach, and Methods** — discusses the objectives and approach to documentation
- **Information Sources** — lists the information sources for the Documentation Requirements and Strategy
- **How to Review** — lists the criteria for reviewing the content of the Documentation Requirements and Strategy
- **Summary of Findings** — summarizes the reasons for the proposed structure and content of the documentation and provides a qualitative summary of the proposed documentation and identifies any problems or concerns that were found
- **Next Steps** — outlines the documentation process

Scope

This component describes the scope of Documentation in as much detail as possible. The scope statement should state exactly how many project-specific documents will be published and the level of content included in these documents. Scope statements can be made in terms of whether certain documentation is in or out of scope for the project. Examples of components that can define the documentation scope include:

- Business Processes
- Functions
- Sites
- Languages
- Application Extensions
- Interfaces to Third-Party or Legacy Systems

In addition to discussing the overall scope of Documentation, this component should also include any assumptions made, the risks inherent in the Documentation process, the expectations for new systems documentation, whether new technology will be used to publish the document, and the relationship of Documentation tasks to other process tasks already underway.

Objectives

This component lists the high-level objectives communicated by the business and project managers. In addition, it should contain a description of the critical success factors for Documentation.

This component includes the following sections:

- Objectives
- Critical Success Factors

Since this is a strategic document, the stated objectives should not be too detailed; however, they should be specific and measurable. Critical success factors associated with meeting the objectives of the process within the context of the overall project are also identified within this component.

Approach

This component describes the method, policies and procedures, project dependencies, and other background for Documentation. It should include a high-level discussion of the approach selected for the Documentation work, the tasks and deliverables involved, the reasons for selection of the approach, and the benefits of the particular method adopted.

The project dependencies section of the component describes the dependencies between tasks in Documentation and tasks in other processes taking place within the overall implementation project.

In relation to the statements about the technical requirements for the documentation subproject, the deliverable should indicate which Documentation Environments is needed to support the documentation subproject. Typically, at least one separate environment is needed for documentation.

If the process uses different policies, procedures, or standards from the main project in any of the key control and reporting areas, the

deliverable should document the differences in detail and explain why they differ. The following are examples of areas where the process typically inherits standards and procedures from the main project:

- Project Management Plan
- issue management and resolution
- change management
- reporting format
- reporting relationship to main project
- acceptance
- project policies and procedures
- process team meetings
- logistics and administrative support

The technical background should describe the technical circumstances affecting the approach to the project. Examples of the points that might be included are:

- implementation sites
- documentation direction
- computing platforms and technical infrastructure
- major system or application requirements
- innovative or unusual technical requirements

Documentation Strategy

This component describes the documentation strategy and includes the means for determining the documents to be authored, published, and produced. It also covers which media will be used to publish documentation. The media could include printed documentation, portable electronic documents, or web pages that can be accessed via a web browser and URL address.

Requirements may be highlighted and discussed because of their criticality in the documentation work, the inherent risk to the project, an innovative or unusual approach to be applied in the project, or for some other implementation-specific reason.

Documentation Requirements

This component describes the human and physical resources required to develop and deliver documentation. Make sure you include user participation as part of your plan. It describes the specific resources needed for Documentation which may include resources in the following areas:

- software
- hardware and networks
- hardware and software delivery schedule
- staff

This component includes the following sections:

- Summarized Requirements
- Detailed Requirements
- Human Resource Requirements
- Software Requirements
- Server Platform and Network Requirements
- Server Platform and Software Delivery Schedule

The Summarized Requirements should summarize the detailed requirements gathered for the business and information systems and their likely impact on the documentation and systems.

The Detailed Requirements should detail the individual business and information systems requirements that affect the documentation. Some of the requirements that may be included are:

- *Application Extensions* — Document all custom extensions built for the new system as standard documentation does not address this new functionality.
- *Interface to Third-Party and Legacy Systems* — Document all custom interfaces that move data between Oracle Applications and third-party and legacy systems.

Documentation Synopses

This component describes each document that is to be produced. This includes the document format and content, as well as the required

format for transferring ownership of the documentation to the users. In addition, the following questions should be addressed:

- Will both soft and hard copy be needed?
- If hard copy is required, how many copies must be provided?
- For soft copy, should a particular text processing package or format be used?
- Are you required to publish the manuals and guides in hypertext markup language (HTML) format?
- Will learning be required for the users to assume ownership of the deliverable?
- What types of edits and reviews must be performed?

Interview Preparation

This component is comprised of a list of questions to help you prepare for interviewing the applicable users of the documentation. Depending on the project, the checklist can be used for internal reference only, or it can be part of the user deliverable. You may want to send this questionnaire to the respondents before the interview to help them prepare for the requirements gathering process.

Documentation Audiences

This component identifies the documentation audience and what they need to know. The degree of overlap and commonality in tasks determines how you group the job roles into documentation audiences.

Record of Interviews

This component is used to indicate who has been interviewed.

Audience, Distribution, and Usage

Distribute and communicate the Documentation Requirements and Strategy to the following:

- project manager who should sign off on the Documentation Requirements and resources needed
- IS manager who should sign off on the Documentation Strategy

- Documentation team members
- other process leaders who have dependent tasks with the Documentation process

Quality Criteria

Use the following criteria to help check the quality of this deliverable:

- Are the project scope and objectives clearly identified?
- Are specific critical success factors and risks documented?
- Does this document take into account the impact of dependent tasks from other processes?
- Are the Documentation Requirements clearly defined?
- Does the strategy discuss existing information systems policies and strategies and indicate how they relate to this project?
- Is the strategy understood by those on the distribution list for this deliverable?
- Are all resource and tool requirements that could impact the Documentation process stated and understood?
- Is the deliverable thorough in capturing different types of business and information systems requirements that are directly relevant to documentation?

Tools

Deliverable Template

Use the Documentation Requirements and Strategy template to create the deliverable for this task.

Oracle Tutor

Oracle Tutor is a procedural development and documentation tool . The tool is also used for user learning development for organizations deploying Oracle Applications. If you have purchased Oracle Tutor, use the predefined standards as defined in the *Tutor Style Guide*.

DO.020 - Define Documentation Standards and Procedures (Optional)

In this task, you specify the standards for all documentation deliverables and determine the *look and feel* of the project documentation. In addition, you also specify the procedures that the team uses to develop documentation.



If your project includes publishing *project-specific documentation*, you should perform this task.

Deliverable

The deliverable for this task is the **Documentation Standards and Procedures**. It identifies how the documentation should *look and feel*. The Documentation Standards and Procedures should also list the procedures to be followed when preparing project-specific documentation.

Prerequisites

You need the following input for this task:



Documentation Requirements and Strategy (DO.010)

The Documentation Requirements and Strategy defines all of your Documentation deliverables. Once these Documentation deliverables have been identified, standards can then be created to define the *look and feel* of each deliverable.



Existing Business Documentation Standards

Obtain the standards that the organization uses for other application systems documentation, if it exists. This prerequisite is only necessary if the project team plans to adopt any existing documentation standards.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Write the front material.	Introduction
2.	Understand any existing documentation standards. If no standards exist, determine what guidelines should be used.	Paragraph and Sentence Structure; Usage; User Input and System Output; Lists; Attention, Suggestion and Warning Icons; Numbers and Operations; Translation and International Audiences; Preferred Terms
3.	Determine the documentation development procedures.	Initial Material; Writing and Editing; Translation Procedures; Downloading; Testing and Change Control; Hard Copy and Reproduction; Backups and Archives
4.	List any outstanding issues and record their resolution.	Open / Closed Issues
5.	Finalize the Documentation Standards and Procedures.	
6.	Secure acceptance of the Documentation Standards and Procedures from the client project manager.	Acceptance Certificate (PJM.CR.080)

Table 7-6 Task Steps for Define Documentation Standards and Procedures

Approach and Techniques

Use the Documentation Standards and Procedures to outline the guidelines on how the documentation will *look and feel*. You can either define your own documentation standards and procedures or you can use an authoring and publishing tool, such as Oracle Tutor, which has documentation standards and procedures built in.

Participants

Participants in this task determine the *look and feel* of the documentation and the process for producing it. Be sure to include the project team members who write the documentation and the users who reference it during their ongoing business processes.

Documentation Standards

If possible, do not start with a blank piece of paper when defining documentation standards because too much time may be lost reaching agreement, documenting the standards, and creating the necessary tools to implement them. There are authoring and publishing tools available that have proven documentation standards built into the software product.

The standards provide a consistent style even though several people may be involved in the writing process. Standards are an important part of the Documentation process since they provide a common *look and feel* to the documents. Documentation standards should address the following:

- **Paragraph and Sentence Structure** — describes the standards for paragraph and sentence structure
- **Usage** — describes the standards for language usage
- **User Input and System Output** — describes the standards for displaying user input and system output in the documents
- **Lists** — describes the standards for using lists
- **Attention, Suggestion and Warning Icons** — describe the standards for using Attention, Suggestion and Warning Icons
- **Punctuation** — describes the standards for punctuation

- **Numbers and Operations** — describes the standards for displaying numbers, mathematical operators, and date formats
- **Translation and International Audiences** — describes the standards for writing documents that will be translated or read by global audiences
- **Preferred Terms** — describes standards for presenting an organization's preferred terms in the documentation
- **Media** — describes printed, portable (pdf), and web documentation

Documentation Procedures

Procedures specify the way that the documentation will be produced. Include an explanation of the following:

- **Initial Material** — identifies the source of the initial content for each deliverable
- **Writing and Editing** — identifies writers and editors for the documents and defines the document review process
- **Translation Procedure** — determines the procedures for translating documents
- **Downloading** — determines whether significant content comes from other systems and how it gets from one system to another
- **Testing and Change Control** — determines the method for verifying that the documentation matches the application implementation and defines the process for applying application changes to the documentation
- **Hard Copy and Reproduction** — determines hard copy requirements and identifies how the documentation is produced, stored, and distributed
- **Backups and Archives** — identifies documents for backup, backup frequency, and restore procedures and in addition, identifies archive requirements for soft and hard copy
- **Review and Approvals** — verifies that the appropriate individuals have reviewed the documentation to meet the requirements for clarity and consistency and have approved the documentation for final publication

Linkage to Other Tasks

The Documentation Standards and Procedures are an input to the following tasks:

- BP.090 - Document Business Procedures
- DO.040 - Prepare Documentation Environment
- DO.050 - Produce Documentation Prototypes and Templates
- DO.060 - Publish User Reference Manual
- DO.070 - Publish User Guide
- DO.080 - Publish Technical Reference Manual
- DO.090 - Publish System Management Guide

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Writer	80
Project Manager	20
Business Line Manager	*
Client Project Manager	*

Table 7-7 Role Contribution for Define Documentation Standards and Procedures

* Participation level not estimated.

Deliverable Guidelines

Use the Documentation Standards and Procedures to define the guidelines for how the documentation will *look and feel* and to guide the quality management of the documentation.

Standards also clarify the expectations being set for the writers. The procedures define the writing process so that all writing is completed at the appropriate time with the required reviews, edits, and updates.

This deliverable should address the following:

- standards for how the published documents will look
- procedures covering how to author and publish documentation

Deliverable Components

The Documentation Standards and Procedures consist of the following components:

- Introduction
- Paragraph and Sentence Structure
- Usage
- User Input and System Output
- Lists
- Attention, Suggestion and Warning Icons
- Numbers and Operations
- Translation and International Audiences
- Preferred Terms
- Initial Material
- Writing and Editing
- Translation Procedures
- Downloading
- Testing and Change Control
- Hard Copy and Reproduction
- Backups and Archives
- Review and Approval Process
- Publication / Deployment

Introduction

This component explains the purpose of the deliverable and areas covered by the document. References to any documentation standards and procedures already defined are included as well.

Paragraph and Sentence Structure

This component documents the many elements of writing, such as guidelines on how to structure a paragraph and a sentence, and provides examples of paragraph and sentence structures.

Usage

This component documents standards on language usage and includes a list of *right* and *wrong* examples of language usage. This resource gives numerous examples of how to use certain text in constructing your document.

User Input and System Output

This component identifies the standards for user input and system output. When developing documentation, use these standards for user inputs and system outputs.

Lists

This component provides guidelines to be used when creating lists. This component addresses numbered lists, bulleted lists, descriptive lists, and others and provides examples.

Attention, Suggestion, and Warning Icons

This component provides guidelines for using attention, suggestion, and warning icons in the documentation. Oracle Method icons for *attention*, *suggestion*, and *warning* can be inserted into a document to draw attention to specific content.

Numbers and Operations

This component provides guidelines for using numbers and operations in the documentation. Examples that show the right and wrong way to present numbers and options are included.

Translation and International Audiences

This component provides standards for writing documentation for international audiences. Specific advice for preparing documentation for international audiences is included.

Preferred Terms

This component lists preferred terms. This list contains words that are preferred in Oracle documentation and specifies the way in which the term should be used. This list also provides the proper spelling and capitalization for the term.

Initial Material

This component identifies the source of the initial content for each deliverable. In some cases, you may have a deliverable from a predecessor task that becomes the initial material for the current task.

Writing and Editing

This component identifies the individuals who will be assigned as subject matter experts, writers, editors, and reviewers of the User Reference Manual (DO.060), User Guide (DO.070), Technical Reference Manual (DO.080), and System Management Guide (DO.090).

Translation Procedures

This component lists the translator, editor, and reviewer assignments. Identify the resources used to translate documents and then obtain the necessary review, approval, and acceptance of the documents.

Downloading

This component indicates how to download documentation from another system. Downloading procedures indicate how to download documentation to and from the documentation library.

Testing and Change Control

This component documents testing and change control for your User Reference Manual (DO.060) and User Guide (DO.070). Any changes made to the system should be reflected in the latest version of the documentation.

Hard Copy and Reproduction

This component defines the hard copy and reproduction requirements for the User Reference Manual (DO.060), User Guide (DO.070), Technical Reference Manual (DO.080), and System Management Guide (DO.090). Depending on the extent of printing required, you may be able to print your documents from a local printer, a network printer, or you may need to take your print job to the print center or an outside printing service.

Backups and Archives

This component defines the backup and retrieval process for the documents produced during Documentation. This component also includes the archival information.

Review and Approval Process

This component defines the organization's standard procedures for reviewing and approving documentation. Documentation review and approval occurs several times during documentation. Secure client acceptance for the documents that have been reviewed and approved.

Publication/Deployment

There are several media types available for publishing / deploying documentation. Document the type of media used and the procedures to be followed.

Tools

Deliverable Template

Use the Documentation Standards and Procedures template to create the deliverable for this task.

Oracle Tutor

Oracle Tutor is a procedural development and documentation tool . The tool is also used for user learning development for organizations deploying Oracle Applications. If you have purchased Oracle Tutor, use the predefined standards as defined in the *Tutor Style Guide*.

DO.030 - Prepare Glossary (Core)

In this task, you document the definition of the terms that may be unfamiliar or confusing to project members in a Glossary. The Glossary provides an alphabetized list of terms and their definitions used to support the development of consistent documentation. Including these terms in the Glossary removes doubt about their meanings.

Deliverable

The deliverable for this task is the **Glossary**. It is a list of terms that are unique to this project. The Glossary can contain technical terms as well as company-specific terms and acronyms.

Prerequisites

You need the following input for this task:

☐ **Current Process Model (BP.040)**

Gather acronyms and terms used in documenting the Current Process Model and include them in the Glossary. If Develop Current Process Model was not performed, this deliverable will not exist. (See the task description for BP.040 for more information on when this task should be performed.)

☐ **Current Financial and Operating Structure (RD.010)**

The Current Financial and Operating Structure includes acronyms and terms that are used during the application implementation. Include these terms in the Glossary.

☐ **Current Business Baseline (RD.020)**

The Current Business Baseline identifies the terms that currently exist and should be included in the Glossary.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Gather and review input documents.	
2.	Identify additional terms to be included in the project Glossary.	
3.	Revise terms where needed or add them to the project Glossary with a recommended definition.	Terms Definition
4.	Write the front material.	Introduction
5.	List any outstanding issues and resolve them.	Open / Closed Issues
6.	Circulate the project Glossary to the team for review and feedback.	
7.	Finalize definitions and publish the project Glossary.	

Table 7-8 Task Steps for Prepare Glossary

Approach and Techniques

Use the Glossary to document terms that are specific to your project. This task involves a review of the Current Financial and Operating Structure (RD.010), Current Business Baseline (RD.020), and Current Process Model (BP.040). Compare the terms that appear in these deliverables to the terms defined in the AIM Process and Task Reference Glossary. If the AIM Process and Task Glossary definition can be used, add the term to the project Glossary and reference the AIM Process and Task Reference Glossary definition.

If the AIM Process and Task Reference definition cannot be used or the term is missing from the AIM Process and Task Reference Glossary, add the term and its definition to the project Glossary. Circulate the project Glossary to everyone involved in the application implementation and hold a review to obtain feedback and reach agreement on the definitions. Remember that terms will continue to be identified and defined throughout the project life cycle.

Linkage to Other Tasks

The Glossary is an input to the following tasks:

- DO.050 - Produce Documentation Prototypes and Templates
- DO.060 - Publish User Reference Manual
- DO.070 - Publish User Guide
- DO.080 - Publish Technical Reference Manual
- DO.090 - Publish System Management Guide

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Writer	80
Business Analyst	15
Technical Analyst	5
User	*

Table 7-9 Role Contribution for Prepare Glossary

* Participation level not estimated.

Deliverable Guidelines

Use the Glossary to describe terms that are used when writing the documentation. The Glossary helps to clarify terms and may contain words that are new to the team due to the new implementation and the subsequent changes in system functionality.

This deliverable should address the following:

- the format for documenting project terms and definitions

Deliverable Components

The Glossary consists of the following components:

- Introduction
- Terms Definition

Introduction

This component explains the types of terms the Glossary contains (for example, acronyms, synonyms, and entities) and documents why the Glossary is necessary.

Terms Definition

This component defines each unique term (make sure that each definition is clear, concise, and accurately represents the meaning you want to convey). In cases where more than one definition is currently in use, you must recognize that not every definition can be included in the Glossary. It confuses communication to have more than one meaning associated with a word. Reach a consensus on a definition as it relates to the current project, regardless of how the term was used previously.

Tools

Deliverable Template

Use the Glossary template to create the deliverable for this task.

DO.040 - Prepare Documentation Environment (Optional)

In this task, you prepare a hardware and software environment that supports documentation development. You must procure, install, and test the environment. At the end of this task, you are ready to begin developing documentation.



If your project includes publishing *project-specific documentation*, you should perform this task.

Deliverable

The deliverable for this task is the **Documentation Environment**. It is the platform and software environment that allows you to support documentation development.

Prerequisites

You need the following input for this task:

☐ **Physical Resource Plan (PJM.RM.040)**

The Physical Resource Plan defines the overall requirements and responsibility for all physical resources and supporting services needed to execute project tasks.

☐ **Documentation Requirements and Strategy (DO.010)**

The Documentation Requirements and Strategy list all of the documents that need to be produced for the project.

☐ **Documentation Standards and Procedures (DO.020)**

The Documentation Standards and Procedures specify the standards for all documentation deliverables and determine the *look and feel* of project documentation. If Define Documentation Standards and Procedures was not performed, this deliverable will not exist. (See the task description for DO.020 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the documentation procedures.	
2.	Review the number of technical writers and development team members assigned to the project. Assess their availability to the project.	
3.	Propose hardware, software, and utilities.	Hardware Proposal; Software Proposal; Utilities Proposal
4.	Procure hardware, software, and utilities.	Hardware Procurement; Software Procurement; Utilities Procurement
5.	Install hardware, software, and utilities.	Hardware Installation; Software Installation; Utilities Installation
6.	Test hardware, software, and utilities.	Hardware Testing; Software Testing; Utilities Testing
7.	Determine contingency plans.	Contingency Plans
8.	Secure acceptance of the Documentation Environment.	Acceptance Certificate (PJM.CR.080)

Table 7-10 Task Steps for Prepare Documentation Environment

Approach and Techniques

Use the Documentation Environment to prepare project documentation and to control access to project documents. The Documentation Environment allows you to have a hardware and software environment that is properly set up to support documentation development.

Documentation Environment Specifications

Make sure that the hardware (platform), software, and utilities accommodate the documentation procedures you have specified. You may have to use existing platform or software (modify your procedures accordingly). If possible, specify an environment that is familiar to the team, thereby reducing the learning curve. A list of critical points that are sometimes overlooked follows:

- screens should be large and high-resolution
- a high-speed printer is required
- quick and accurate file transfer capability is required if writing is to be done in more than one location

If necessary, revisit the Physical Resource Plan (PJM.RM.040) and add any updates to material that relates to the Documentation Environment.

If necessary, revisit the Project Team Learning Plan (AP.030) and add any software to the learning needs for the team involved in the documentation.

Procurement, Installation, and Testing

Consider the potential of significant lead times in procuring hardware and software that is not available off the shelf.

The time required to set up the environment can vary greatly. Plan well in advance if approval, purchasing, or procurement delays are common for the products you need. You should plan to test the environment against the actual procedures you have specified.

Linkage to Other Tasks

The Documentation Environment is an input to the following tasks:

- BP.090 - Document Business Procedures
- DO.050 - Produce Documentation Prototypes and Templates
- DO.060 - Publish User Reference Manual
- DO.070 - Publish User Guide
- DO.080 - Publish Technical Reference Manual
- DO.090 - Publish System Management Guide

Role Contribution

The percentage of total task time required for each role follows:

Role	%
System Administrator	80
Technical Writer	20
Client Staff Member	*

Table 7-11 Role Contribution for Prepare Documentation Environment

* Participation level not estimated.

Deliverable Guidelines

Use the Documentation Environment to formally track the proposal, procurement, installation, and testing of elements involved in creating the Documentation Environment. Sufficient features must be available in the chosen hardware, software, and utilities to support the Documentation Requirements and Strategy (DO.010).

This deliverable should address the following:

- proposal for hardware, software, and documentation production utilities
- hardware, software, and utilities for the Documentation Environment
- hardware, software, and utilities installation for the Documentation Environment
- hardware, software, and utilities testing for the Documentation Environment
- documentation contingencies

Deliverable Components

The Documentation Environment consists of the following components:

- Hardware Proposal
- Software Proposal
- Utilities Proposal
- Hardware Procurement
- Software Procurement
- Utilities Procurement
- Hardware Installation
- Software Installation
- Utilities Installation
- Hardware Testing
- Software Testing
- Utilities Testing
- Contingency Plans

Hardware Proposal

This component documents the detailed information about the proposal for procuring hardware for the Documentation Environment.

Software Proposal

This component documents the detailed information about the proposal for procuring software for the Documentation Environment.

Utilities Proposal

This component documents the detailed information about the proposal for procuring utilities for the Documentation Environment.

Hardware Procurement

This component identifies the issues involved in the procurement of hardware for the Documentation Environment.

Software Procurement

This component identifies the issues involved in the procurement of software for the Documentation Environment.

Utilities Procurement

This component identifies the issues involved in the procurement of utilities for the Documentation Environment.

Hardware Installation

This component identifies the issues involved in the installation of hardware for the Documentation Environment.

Software Installation

This component identifies the issues involved in the installation of software for the Documentation Environment.

Utilities Installation

This component identifies the issues involved in the installation of utilities for the Documentation Environment.

Hardware Testing

This component documents the testing of hardware used in preparing the Documentation Environment.

Software Testing

This component documents the testing of software used in preparing the Documentation Environment.

Utilities Testing

This component documents the testing of utilities used in preparing the Documentation Environment.

Contingency Plans

This component documents the contingency plans for hardware, software, and utilities used in the preparation of the Documentation Environment.



Attention: Confirm that the individuals selected to assist in the environment proposal have a thorough knowledge of the documentation development procedures, as well as the hardware, software, and utilities being considered.

Tools

Deliverable Template

Use the Documentation Environment template to create the supporting deliverable for this task.

DO.050 - Produce Documentation Prototypes and Templates (Optional)

In this task, you build and review a single prototype for each type of documentation deliverable. The results conform to the *look and feel* of each documentation deliverable, as well as present a clear expectation of what will be delivered. You also create templates for later use.



If your project includes publishing *project-specific documentation*, you should perform this task.

Deliverable

The deliverable for this task is the **Documentation Prototypes and Templates**. It is a prototype of each major document that was identified in Documentation Requirements and Strategy (DO.010). The purpose of prototypes and templates is to gain agreement on the expectation of what the final documents will look like.

Prerequisites

You need the following input for this task:

☐ **Documentation Requirements and Strategy (DO.010)**

The Documentation Requirements and Strategy specify which prototypes and templates should be produced.

☐ **Documentation Standards and Procedures (DO.020)**

Follow the Documentation Standards and Procedures when building the prototype documents. If Define Documentation Standards and Procedures was not performed, this deliverable will not exist. (See the task description for DO.020 for more information on when this task should be performed.)

❑ Glossary (DO.030)

The Glossary provides definitions of terms as they specifically apply to the development of documentation. You may want to include the Glossary in your prototype.

❑ Documentation Environment (DO.040)

The Documentation Environment is the hardware, software and utilities that supports the production of the prototypes and templates. If Prepare Documentation Environment was not performed, this deliverable will not exist. (See the task description for DO.040 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Build a prototype for the User Reference Manual (DO.060).	Title Page; Preface; Table of Contents; Chapter Introduction; Topical Essay; Appendix
2.	Build a prototype for the User Guide (DO.070).	Title Page; Preface; Table of Contents; Chapter Introduction; Topical Essay; Appendix
3.	Build a prototype for the Technical Reference Manual (DO.080).	Title Page; Table of Contents; Chapter Introduction; Appendix
4.	Build a prototype for the System Management Guide (DO.090).	Title Page; Table of Contents; Chapter Introduction; Appendix
5.	Review the prototypes with users; discuss and reach agreement.	

No.	Task Step	Deliverable Component
6.	Create a sample User Reference Manual (DO.060) from the prototype sample chapter.	
7.	Create a sample User Guide (DO.070) from the prototype sample chapter.	
8.	Create a sample Technical Reference Manual (DO.080) from the prototype sample chapter.	
9.	Create and test the System Management Guide (DO.090) from the prototype sample chapter.	
10.	Package prototypes and templates for distribution (if necessary).	
11.	Secure acceptance of the Documentation Prototypes and Templates.	Acceptance Certificate (PJM.CR.080)

Table 7-12 Task Steps for Produce Documentation Prototypes and Templates

Approach and Techniques

Use the Documentation Prototypes and Templates to show how the project documentation deliverables will look. The prototypes help set the expectation on how the documents will *look and feel*, and include a listing of what each document contains.

Standard Deliverable Contents

To produce the standard documentation deliverables you need a prototype and template for each of the following deliverables:

- User Reference Manual (DO.060)
- User Guide (DO.070)
- Technical Reference Manual (DO.080)
- System Management Guide (DO.090)

Each prototype should contain a table of contents, indicating the front matter and the chapter titles and organization. They should also include a sample chapter.

There are two ways to create the prototypes:

- cut and paste
- custom prototypes

Cut and Paste

If the user has agreed to use Oracle documentation and the required documents are part of the standard set offered by AIM, use documentation deliverables from a previous project as the basis for the prototypes. Be sure to tailor the deliverables to remove any references to previous organizations. It is often beneficial to personalize the prototypes. For example, if you reference the organization name and the project name directly in the prototypes, it is easier for the users to take ownership of them.

Custom Prototypes

Use this approach for prototypes of deliverables that are not part of the standard documentation set offered by AIM and for projects that do not use the Oracle documentation standards. This approach takes considerably more time than the first and involves building one or more prototype from scratch, using the specified documentation standards for each.

Use documentation from a previous project or advance information from the current project as content for the sample chapter. You should try to use content that is not related to the current project, if possible. The use of current project material often makes it difficult to examine and approve the format, without being distracted by the content itself.

Prototype Review

For each prototype, set up a review schedule for those who will be using the documents. Explain the organization of the document or document set and the front matter that is to be included. Use the sample chapter as a basis for discussing the content of each chapter.

If you have used sample information from the current project, inform the reviewers that you do not intend for the content to be correct, or even meaningful. Make sure that the review focuses on format and style, rather than substance.

Template Creation

You need to create a template for the sample chapter of each prototype. Create the templates using the word processing software designated for documentation development. Each template should be as easy as possible to use. You may want to include instructions within the template itself to indicate the desired content and level of detail for each section.

Testing

Test each template as if you were a technical writer using the template for the first time. Remember that each mistake in a template is repeated in every chapter that uses the template; templates must be error-free. If you involve more than one technical writer, you may want to package the set of templates and sample chapters for easy distribution.

Planning

Planning is affected by the number of manuals required to be prototyped and the documentation requirements of each. In addition, consider whether the prototypes require custom development.

Linkage to Other Tasks

The Documentation Prototypes and Templates are an input to the following tasks:

- DO.060 - Publish User Reference Manual
- DO.070 - Publish User Guide

- DO.080 - Publish Technical Reference Manual
- DO.090 - Publish System Management Guide

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Writer	80
Business Analyst	15
Technical Analyst	5
User	*

Table 7-13 Role Contribution for Produce Documentation Prototypes and Templates

* Participation level not estimated.

Deliverable Guidelines

Use the actual document template to create the document prototype. For example, to create a prototype of the User Reference Manual (DO.060), use the User Reference Manual template. Create prototypes for the following deliverables:

- User Reference Manual (DO.060)
- User Guide (DO.070)
- Technical Reference Manual (DO.080)
- System Management Guide (DO.090)

This deliverable should address the following:

- the basic format and structure of the documents
- the high-level components included in the documents
- the overall *look and feel* of the deliverable documents

Deliverable Components

This task does not have its own deliverable template for creating prototypes and templates. Use the deliverable templates from the following tasks:

- User Reference Manual (DO.060) — create the prototype User Reference Manual
- User Guide (DO.070) — create the prototype User Guide
- Technical Reference Manual (DO.080) — create the prototype Technical Reference Manual
- System Management Guide (DO.090) — create the prototype System Management Guide

If developing custom prototypes seems overwhelming, review similar reference material from other systems to determine their content and organization. Having a baseline as a starting point helps you refine your own prototype design.

Templates are a natural outgrowth of the prototypes. If they are created correctly, they automatically format the document into the appropriate style. This provides the writer additional time to concentrate on the documentation content.

Tools

Deliverable Template

You can select any one of the predefined documentation templates (such as DO.060, DO.070, DO.080, or DO.090) to create your prototype.

Oracle Tutor

Oracle Tutor is an authoring and publishing tool that allows you to create prototypes and templates for documentation. If you have purchased Oracle Tutor, use the *Tutor Style Guide* when generating prototypes of user manuals and guides.

DO.060 - Publish User Reference Manual (Optional)

In this task, you gather material for the User Reference Manual and publish the final version. The User Reference Manual supplements the Oracle Applications user reference material.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **User Reference Manual**. This manual shows users how to use custom application functionality. It also shows users how application extensions enhance standard Oracle Applications functionality.

Prerequisites

You need the following input for this task:



Application Extensions Functional Design (MD.050)

The Application Extensions Functional Design document describes the functional specifications that support each customization. There could be numerous Application Extensions Functional Design documents where there is more than one application extension. If Create Application Extensions Functional Design was not performed, this deliverable will not exist. (See the task description for MD.050 for more information on when this task should be performed.)



Documentation Requirements and Strategy (DO.010)

The Documentation Requirements and Strategy specify the overall requirements for creating documentation and the deliverables to be produced.

☐ **Documentation Standards and Procedures (DO.020)**

Follow these development and editorial standards when writing the User Reference Manual chapters. If Define Documentation Standards and Procedures was not performed, this deliverable will not exist. (See the task description for DO.020 for more information on when this task should be performed.)

☐ **Glossary (DO.030)**

The Glossary defines terms that are specific to a project. You may want to include the Glossary in the User Reference Manual.

☐ **Documentation Environment (DO.040)**

Once the Documentation Environment is set up, the User Reference Manual can be created. If Prepare Documentation Environment was not performed, this deliverable will not exist. (See the task description for DO.040 for more information on when this task should be performed.)

☐ **Documentation Prototypes and Templates (DO.050)**

Use the User Reference Manual prototypes and templates as a guide to writing the chapters and front matter for the User Reference Manual. If Produce Documentation Prototypes and Templates was not performed, this deliverable will not exist. (See the task description for DO.050 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Collect, edit, and collate Application Extensions Functional Design (MD.050) documents.	

No.	Task Step	Deliverable Component
2.	Review the Application Extensions Functional Design (MD.050) documents to understand the functional reasons for each application extension.	
3.	Write the front material and introductions to the chapters.	Title Page; Preface; Contents; Chapters
4.	For each application extension, write a topical essay that explains how to use the new application extension functionality.	Topical Essay
5.	Add any appendices, as needed.	Appendices
6.	List any outstanding issues identified while authoring and publishing this manual and record the resolution of each issue.	Open / Closed Issues
7.	Review the preliminary User Reference Manual for consistency. Make corrections as necessary. Publish a preliminary version of the manual.	
8.	Make a preliminary version of the manual available to system testers.	
9.	Receive documentation feedback from testers. Make corrections to the User Reference Manual.	

No.	Task Step	Deliverable Component
10.	Publish the latest version of the User Reference Manual.	
11.	Secure acceptance of the User Reference Manual.	Acceptance Certificate (PJM.CR.080)

Table 7-14 Task Steps for Publish User Reference Manual

Approach and Techniques

Use the User Reference Manual to explain to users in functional terms how the application extensions work. This task should be executed in parallel with Perform Unit Test (TE.070) and Perform Link Test (TE.080). Review and update the User Reference Manual as each round of unit and link testing is completed for application extensions.

The initial information for building the User Reference Manual comes from the Application Extensions Functional Design (MD.050), which defines application extensions in functional terms. If the contents of the Application Extensions Functional Design (MD.050) are well structured, this content could feed directly into the User Reference Manual. Otherwise, the User Reference Manual must be initially written using the structure of the Application Extensions Functional Design (MD.050) as the basis for the work.

When preparing material for the User Reference Manual, remember that writing the manual is an iterative task and that the quality of the manual improves with each successive round of testing. Although a preliminary User Reference Manual is available for testers to use, testers are encouraged to give constructive feedback to the technical writers. After feedback comments are analyzed, new information can be updated in the User Reference Manual.

Sometimes changes are introduced into the application extension modules as a result of testing or functionality review. These changes must be reflected in the final version of the User Reference Manual.

It is important to edit each initial chapter thoroughly and verify that the sections are written with a similar style and tone, and that errors are revealed before they are perpetuated. Even if the task involves only a

single writer, it is a good idea to engage a professional technical writer to edit several of the initial chapters. One of the last steps is to read the entire User Reference Manual to find and correct inconsistencies. Once these corrections have been made, you can prepare to transfer ownership of the document to the users by formatting the User Reference Manual as required. You may be required to publish your documents in HTML format to make them available via the intranet.

Planning

If you directly involve developers, this task can be executed more quickly. If you plan to use multiple technical writers, it is important to schedule time to review and edit sections soon after they are written. This helps make sure that errors are not repeated in multiple sections.

Include the task of reviewing and revising previous versions of the User Reference Manual directly in the procedures for completing the module test of primary modules. The developers who were involved in detailed design should do the initial review to incorporate additions and identify errors. Technical writers should do the final review and edit.

Linkage to Other Tasks

The User Reference Manual is an input to the following tasks:

- TE.100 - Prepare Key Users for Testing
- TE.110 - Perform System Test
- TE.120 - Perform Systems Integration Test
- AP.140 - Develop User Learning Plan
- AP.150 - Develop User Learningware
- AP.170 - Conduct User Learning Events

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Writer	80
Business Analyst	20

Table 7-15 Role Contribution for Publish User Reference Manual

Deliverable Guidelines

Use the preliminary versions of the User Reference Manual to show the users how the application extensions enhanced standard Oracle Applications functionality. Use the User Reference Manual to provide relevant functional information to the user community.

Once Business System Testing (TE) is complete, publish a final version of the User Reference Manual to provide users with functional information specific to application extensions. The User Reference Manual includes information about custom forms and reports and is an extension of the standard reference manuals provided with Oracle Applications. Describe each visible title, field, option, and transaction type for any Oracle Applications extensions. A user should be able to reference any topic, screen, or report that is available in the production system. The final version of the User Reference Manual should include all changes that were made during unit and link testing.



Attention: The value of the document diminishes in direct proportion to the information that is missing or inaccurate.

This deliverable should address the following:

- how to add content to the User Reference Manual
- how to publish the User Reference Manual

Deliverable Components

The User Reference Manual consists of the following components:

- Preface
- Contents
- Chapters
- Topical Essay
- Appendices

Preface

This component documents how the manual is organized.

Contents

This component provides a format for the table of contents.

Chapters

This component provides boilerplate text that can be used to format each chapter.

Topical Essay

This component provides information about the scope, approach, and methods of the User Reference Manual.

Appendices

This component provides an outline for creating an appendix.

Tools

Deliverable Template

Use the User Reference Manual template to create the deliverable for this task.

Oracle Tutor

Oracle Tutor supports the automatic conversion to HTML format and automatic generation of hypertext links. This simplifies the electronic publication of documents on your intranet.

DO.070 - Publish User Guide (Optional)

In this task, you publish a User Guide that defines a set of detailed procedures for using the applications. This task is often performed in parallel with Perform System Test (TE.110).



If your project includes publishing *project-specific documentation*, you should perform this task.

Deliverable

The deliverable for this task is the **User Guide**. This guide describes each business procedure and provides detailed instructions for using the applications in response to day-to-day business events.

Prerequisites

You need the following input for this task:



Current Process Model (BP.040)

Information from the Current Process Model may be included in the User Guide. If Develop Current Process Model was not performed, this deliverable will not exist. (See the task description for BP.040 for more information on when this task should be performed.)



Business Procedure Documentation (BP.090)

The Business Procedure Documentation further defines process designs to job-level designs, thereby defining how the work is done and laying a foundation for user procedures and user learning (role-based).



Application Setup Documents (BR.100)

The Application Setup Documents define the application configuration needed to support the business processes. These documents may include a number of user-defined codes, system- and application-level parameters, and enabled features.

☐ **Documentation Requirements and Strategy (DO.010)**

Refer to the Documentation Requirements and Strategy for an outline of the overall requirements for creating documentation and a listing of the specific deliverables to be produced.

☐ **Documentation Standards and Procedures (DO.020)**

Follow these development and editorial standards when writing the chapters of the User Guide. If Define Documentation Standards and Procedures was not performed, this deliverable will not exist. (See the task description for DO.020 for more information on when this task should be performed.)

☐ **Glossary (DO.030)**

Use the Glossary to obtain a definition of terms that are used on this project.

☐ **Documentation Environment (DO.040)**

The Documentation Environment includes the hardware and software used to prepare project documentation. If Prepare Documentation Environment was not performed, this deliverable will not exist. (See the task description for DO.040 for more information on when this task should be performed.)

☐ **Documentation Prototypes and Templates (DO.050)**

Use the User Guide prototypes and templates as a guide when writing the chapters and front matter of the User Guide. If Produce Documentation Prototypes and Templates was not performed, this deliverable will not exist. (See the task description for DO.050 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Collect Business Procedure Documentation (BP.090) and Application Setup Documents (BR.100).	
2.	Review the Business Procedure Documentation (BP.090) and Application Setup Documents (BR.100).	
3.	Write the front material and introductions to the chapters.	Title Page; Preface; Contents; Chapters
4.	For each application extension, write a topical essay that explains how to use the new application extension functionality.	Topical Essay
5.	Add any appendices, as needed.	Appendices
6.	List any outstanding issues identified while authoring and publishing the User Guide and record the resolution of each issue.	Open / Closed Issues
7.	Review the User Guide for consistency and make necessary corrections. Publish a preliminary version of the guide.	
8.	Make a preliminary version of the manual available to system testers.	

No.	Task Step	Deliverable Component
9.	Receive documentation feedback from testers. Make corrections to the User Guide.	
10.	Publish the latest version of the User Guide. This may include publishing the document in HTML format.	
11.	Secure acceptance of the User Guide.	Acceptance Certificate (PJM.CR.080)

Table 7-16 Task Steps for Publish User Guide

Approach and Techniques

Use the User Guide to support the day-to-day usage of the new system. It describes each business procedure and highlights any system-specific uses of screens and reports. The User Guide introduces the user to the purpose of procedures (such as *Create a Standard Customer Invoice*) and then explains the detailed steps and options needed to perform the business task or transactions.

Develop and update the User Guide in parallel with the business system test. As business system test scenarios are executed, the team should review the corresponding section of the User Guide for accuracy.

While creating the User Guide, actively involve the users (especially if they have not participated in creating the initial draft). The ongoing success of the application system depends a great deal on the users' ability to follow the correct procedures in response to business events without the assistance of the project team.

Revisit the front matter of the User Guide for possible revisions. Create an index for the document if it is specified in the Documentation Requirements and Strategy (DO.010). Prepare to transfer ownership of the document to the users by formatting the User Guide as required.

The writer should assemble all relevant documents and flow charts prior to writing the guide. The Application Setup Documents (BR.100) are the most useful input, along with the Business Procedure Documentation (BP.090), which actually documents the business new business processes.

When preparing the User Guide, there are several points to remember:

- Project documentation should be brief and concise.
- The tone of the documentation should be simple and straightforward, rather than academic or technical.
- Anticipate questions that the user may ask regarding a procedure, and answer them. As you document these procedures, compare the current and future approach and add more detailed explanations to the areas that may be confusing to a user.
- Write the documentation to the level of the targeted audience. If you are documenting a technical process, include technical details; if you are documenting a business process, only include technical details if they are necessary to convey a concept.
- Use diagrams and charts to explain broad or complex concepts. For example, a paragraph containing many conditions may be more easily understood when the information is presented in a table or chart.

It may be appropriate to refer the reader to another section or document for more information on a subject. Create a symbol or wording style that can be used consistently to identify references and verify that these references remain accurate as the documentation evolves.

Planning

If you are using multiple technical writers, it is important to schedule time to review and edit sections soon after they are written. This helps make sure that information is fresh and errors are not repeated in multiple sections.

You should include the task of reviewing and revising previous versions of the User Guide directly in the procedures for completing the business system test scenarios.

Linkage to Other Tasks

The User Guide is an input to the following tasks:

- TE.100 - Prepare Key Users for Testing
- TE.110 - Perform System Test
- TE.120 - Perform Systems Integration Test
- AP.140 - Develop User Learning Plan
- AP.150 - Develop User Learningware
- AP.170 - Conduct User Learning Events

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Writer	80
Business Analyst	10
Technical Analyst	10

Table 7-17 Role Contribution for Publish User Guide

Deliverable Guidelines

Use the User Guide to document the day-to-day procedures for using the new system. These procedures are responses to day-to-day business events. The User Guide provides learning content and a comprehensive source for answering daily procedural questions for new users.

This deliverable should address the following:

- how to add content to the User Guide
- how to publish the User Guide

Deliverable Components

The User Guide consists of the following components:

- Preface
- Contents
- Chapters
- Topical Essay
- Appendices

Preface

This component identifies how the User Guide is organized.

Contents

This component provides a format for the table of contents.

Chapters

This component provides boilerplate text that can be used to format each chapter.

Topical Essay

This component provides information about the scope, approach, and methods of the User Guide.

Appendices

This component provides an outline for creating an appendix.

The User Guide helps you provide the user community with documentation that reflects all aspects of the business and not just the areas that require system usage. Incorporate customizations into the procedures. Remember that users typically do not distinguish between standard and custom modules. The following list identifies some of the basic information to include in the User Guide:

- pictures of forms and reports, preferably with data recognizable to the reader
- the input of information or materials to process
- the source of input of information or materials

- explanation of standard system, custom, and manual processes
- step-by-step procedures
- description of exception cases
- reversals of any manual or system transactions
- how to complete the procedure
- how to review the transaction
- how transactions are reported
- how to navigate to the system through menus
- how to correct mistakes and reverse undesirable actions



Attention: When assembling the User Guide, include the following components:

- preface
- table of contents
- introduction
- procedures
- graphs or illustrations of the process
- glossary of terms
- indexes
- management override (optional)
- controls and mechanisms that are required at each business process stop

Tools

Deliverable Template

Use the User Guide template to create the deliverable for this task.



Attention: Many of the recommended User Guide sections are not included in the template. You need to create most of the material manually using the standard styles supported by the template.



Suggestion: Divide your User Guide into chapters using multiple chapter components.

The styles used in the template are consistent with the standards used in all Oracle user documentation. If you have customized the User Guide template, use the customized template for this task.

Oracle Tutor

Oracle Tutor is a procedural development and documentation tool. The tool is also used for user learning development for organizations deploying Oracle Applications. If you have purchased Oracle Tutor, use the predefined standards as defined in the *Tutor Style Guide*.

DO.080 - Publish Technical Reference Manual (Optional)

In this task, you assemble the content of the Technical Reference Manual. The size of this task depends on the extent of the technical changes that were made to support the new applications.

This information is intended to be a supplement to the *Oracle Applications Technical Reference Manuals* and other technical documentation. These materials are usually consistent in format and content with Oracle documentation standards.



If your project includes *customizations or interfaces to standard functionality*, you should perform this task.

Deliverable

The deliverable for this task is the **Technical Reference Manual**. This manual describes the technical details of the application for the information technology maintenance staff. The Technical Reference Manual provides technical information regarding application extensions and interfaces.

Prerequisites

You need the following input for this task:

 Database Extensions Design (MD.060)

The Database Extensions Design includes the new tables, indexes, views, and sequences for customizations. If Design Database Extensions was not performed, this deliverable will not exist. (See the task description for MD.060 for more information on when this task should be performed.)

☐ **Approved Designs (MD.080)**

Obtain final signoff of completed designs prior to producing the Technical Reference Manual. If Review Functional and Technical Designs was not performed, this deliverable will not exist. (See the task description for MD.080 for more information on when this task should be performed.)

☐ **Documentation Requirements and Strategy (DO.010)**

The Documentation Requirements and Strategy state what the user expects of the technical documentation.

☐ **Documentation Standards and Procedures (DO.020)**

Follow the documentation standards and development procedures when writing the Technical Reference Manual. If Define documentation Standards and Procedures was not performed, this deliverable will not exist. (See the task description for DO.020 for more information on when this task should be performed.)

☐ **Glossary (DO.030)**

The Glossary includes terms defined specifically for a project.

☐ **Documentation Environment (DO.040)**

The Documentation Environment provides an installed environment for developing documentation. If Prepare Documentation Environment was not performed, this deliverable will not exist. (See the task description for DO.040 for more information on when this task should be performed.)

☐ **Documentation Prototypes and Templates (DO.050)**

The prototype of the Technical Reference Manual is a model for development of the final Technical Reference Manual. If Produce Documentation Prototypes and Templates was not performed, this deliverable will not exist. (See the task description for DO.050 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Interview technical analysts.	
2.	Extract and assemble the detailed components from Database Extensions Design (MD.060), Application Extensions Technical Design (MD.070), Approved Designs (MD.080), and Oracle Designer documents (if applicable).	
3.	Review the detail from Database Extensions Design (MD.060), Approved Designs (MD.080), and Oracle Designer (if applicable).	
4.	Write the front material and introductions to the chapters.	Title Page; Contents; Chapters
5.	Add any appendices, as needed.	Appendices
6.	List any outstanding issues identified while authoring and publishing the Technical Reference Manual and record the resolution of each issue.	Open / Closed Issues
7.	Review the Technical Reference Manual for consistency and make the necessary corrections. Publish a preliminary version of the manual.	

No.	Task Step	Deliverable Component
8.	Make a preliminary version of the manual available to system testers.	
9.	Receive documentation feedback from testers. Make corrections to the Technical Reference Manual.	
10.	Publish the latest version of the Technical Reference Manual.	
11.	Secure acceptance of the Technical Reference Manual.	Acceptance Certificate (PJM.CR.080)

Table 7-18 Task Steps for Publish Technical Reference Manual

Approach and Techniques

Use the Technical Reference Manual to bring together technical documentation prepared specifically for the application extensions.

Developers may have implemented technical changes in the system without updating the appropriate design and build documents. In order to locate all relevant technical information, you may want to interview the technical analysts and developers as well as review change control forms, specification updates, or memos relating to technical issues. In addition, update the Technical Reference Manual to include changes made to the application extensions as a result of Perform Unit Test (TE.070) and Perform Link Test (TE.080).

The most important step in this task is to understand the requirements for the Technical Reference Manual. The Documentation Requirements and Strategy (DO.010) should clearly state what is included in the Technical Reference Manual and describe how the document is to be used. You determine the contents of the document based on this information.

Assemble the following relevant technical material and designs that are being implemented in Module Design and Build (MD):

- designer models and design information
- entity relationship diagrams
- new table definitions
- migration and customization instructions

Compile these documents in a logical format and add any written material needed to guide the user through the manual.

To perform this task, you need a technical writer who understands technical design issues (or alternatively, a technical analyst who writes well).

Planning

If multiple technical writers are used, it is important to schedule time to review and edit sections soon after they are written. This helps make sure that the information is fresh and errors are not repeated in multiple sections. Include the task of reviewing and updating previous versions of the System Management Guide (DO.090) directly into the procedures for executing the business system test.

Linkage to Other Tasks

The Technical Reference Manual is an input to the following task:

- PM.100 - Maintain System

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Writer	70
Developer	10
System Architect	10
Technical Analyst	10

Table 7-19 Role Contribution for Publish Technical Reference Manual

Deliverable Guidelines

Use the Technical Reference Manual to describe the custom modules and extensions that are developed during Module Design and Build (MD). The Technical Reference Manual provides technical information regarding the custom modules and extensions. This information is helpful later when the maintenance staff updates the application custom code or extensions. The content of the document should supplement the *Oracle Applications Technical Reference Manual*.

This deliverable should address the following:

- module descriptions
- profile options
- new or updated seed data
- descriptive flexfields
- value sets
- grants and synonyms
- installation and upgrade
- archiving

- database diagram
- tables, indexes, and sequences
- system performance issues
- glossary of unique terms

Deliverable Components

The Technical Reference Manual consists of the following components:

- Contents
- Chapters
- Appendices

Contents

This component provides a format for the table of contents.

Chapters

This component provides boilerplate text that can be used to format each chapter.

Appendices

This component provides an outline for creating an appendix.

The Technical Reference Manual does not replace the standard *Oracle Applications Technical Reference Manual*, so there is no need to duplicate information found in the standard *Oracle Applications Technical Reference Manual*. Although this manual is directed at technicians, remember that there can be a wide range of knowledge and experience within the technical group.

Tools

Deliverable Template

Use the Technical Reference Manual template to create the deliverable for this task.

The documentation styles provided in the template are identical to the styles used in the *Oracle Applications Technical Reference Manuals*. If you have customized the Technical Reference Manual template, select the customized template for this task.

DO.090 - Publish System Management Guide (Optional)

In this task, you gather material for the System Management Guide and publish the final version. Perform this task in parallel with the execution of the business system test.



If your project is of *medium or high complexity*, you should perform this task.

Deliverable

The deliverable for this task is the **System Management Guide**. This guide is a set of procedures for managing the new application system. The content for this deliverable comes from the System Management Procedures (TA.150).

Prerequisites

You need the following input for this task:

☐ **Project Management Plan [initial complete] (PJM.CR.010)**

The Project Management Plan contains the definition of project strategies, standards, and procedures aimed at assuring the success of the project.

☐ **System Management Procedures (TA.150)**

The System Management Procedures define the operational plans for system contingency situations and include disaster or failure conditions that require preventive and recovery procedures.

☐ **Documentation Requirements and Strategy (DO.010)**

The Documentation Requirements specify user requirements for the System Management Guide as well as the way the guide is to be used and the format for transferring ownership of the System Management Guide to the users.

☐ **Documentation Standards and Procedures (DO.020)**

Follow the documentation standards and development procedures when writing the System Management Guide. If Define Documentation Standards and Procedures was not performed, this deliverable will not exist. (See the task description for DO.020 for more information on when this task should be performed.)

☐ **Glossary (DO.030)**

The Glossary provides definitions of terms that are used on the project.

☐ **Documentation Environment (DO.040)**

Set up the Documentation Environment so you can create prototypes and deliverable documents. If Prepare Documentation Environment was not performed, this deliverable will not exist. (See the task description for DO.040 for more information on when this task should be performed.)

☐ **Documentation Prototypes and Templates (DO.050)**

The Documentation Prototypes and Templates include a prototype and template of the System Management Guide. If Produce Documentation Prototypes and Templates was not performed, this deliverable will not exist. (See the task description for DO.050 for more information on when this task should be performed.)

☐ **Production Support Infrastructure Design (PM.020)**

The Production Support Infrastructure Design identifies the operational infrastructure for managing and maintaining the application environment, database, and network communications in the Production Environment.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Collect and review the System Management Procedures (TA.150).	
2.	Write the front material and introductions to the chapters.	Title Page; Contents; Chapters
3.	For each operational task performed by the information technology staff, document how the task is done and who does it.	Chapters
4.	Add any appendices, as needed.	Appendices
5.	List any outstanding issues identified while authoring and publishing the System Management Guide and record the resolution of each issue.	Open / Closed Issues
6.	Review the System Management Guide for consistency and make the necessary corrections. Publish a preliminary version of the guide.	
7.	Make a preliminary version of the System Management Guide available to information technology staff during system testing.	

No.	Task Step	Deliverable Component
8.	Receive documentation feedback from information technology staff as testing occurs. Make corrections to the System Management Guide.	
9.	Publish the latest version of the System Management Guide.	
10.	Secure acceptance of the System Management Guide.	Acceptance Certificate (PJM.CR.080)

Table 7-20 Task Steps for Publish System Management Guide

Approach and Techniques

Use the System Management Guide to document the set of procedures needed for managing the new application system. The System Management Guide provides relevant system information for the internal support staff. It includes step-by-step procedures for performing daily, nightly, and batch processing (such as report submissions). Some management procedures will define a fixed schedule for maintenance and other routine work.

Collect and review the System Management Procedures (TA.150). The procedures are used to write the System Management Guide for the new business system. Review all documentation for relevance, redundancy, and style. The content of the front matter is specified in Documentation Prototypes and Templates (DO.050).

If the prerequisites of this task are done well, then the approach to this task is simpler; it involves understanding the prototypes and templates for the System Management Guide, and using them to build the sections of the guide (organized by each system management event).

Plan to update the System Management Guide in parallel with executing the system testing. As the Systems Integration Test Script

(TE.050) is developed, the team should review the corresponding sections of the System Management Guide.

Planning

If multiple technical writers are used, it is important to schedule time to review and edit sections soon after they are written. This helps make sure that information is fresh and errors are not repeated in multiple sections. Include the task of reviewing and revising previous versions of the System Management Guide directly in the system management test scenarios. System administrators should do the initial review to incorporate additions and spot errors. Technical writers should perform the final review.

Linkage to Other Tasks

The System Management Guide is an input to the following tasks:

- TE.120 - Perform Systems Integration Test
- AP.150 - Develop User Learningware
- AP.170 - Conduct User Learning Events
- PM.100 - Maintain System

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Technical Writer	70
System Architect	15
System Administrator	15
Client Staff Member	*

Table 7-21 Role Contribution for Publish System Management Guide

* Participation level not estimated.

Deliverable Guidelines

Use the System Management Guide to document relevant system information for the internal support staff. The System Management Guide is also used as a procedural document by the internal support staff. Use the System Management Guide to document a first draft set of procedures for managing the new application system. The System Management Guide explains how and when to accomplish routine operational tasks.

This deliverable should address the following:

- regularly scheduled procedures
- report generation procedures
- exception handling procedures
- database and application monitoring techniques
- system performance statistics
- archiving and purging procedures
- backup and recovery procedures
- periodic tuning techniques
- hardware and software vendor information
- miscellaneous support procedures
- interface management
- electronic data interface
- glossary of unique system terms

Deliverable Components

The System Management Guide consists of the following components:

- Contents
- Chapters
- Appendices

Contents

This component provides a format for the table of contents.

Chapters

This component provides boilerplate text that can be used to format each chapter.

Appendices

This component provide an outline for creating an appendix.

The maintenance staff depends on the accuracy of the information presented in this document. Creating an appendix requires the active involvement of a client staff member during the final stage (especially if client staff members did not participate in the creation of the initial draft). Make sure that all topics are addressed thoroughly.



Suggestion: Because this manual is used frequently, make sure that the users can reference procedures quickly. Use markers and tabs, as in a directory, to separate sections and help users locate information.

Tools

Deliverable Template

Use the System Management Guide template to create the deliverable for this task.

Business System Testing (TE)

This chapter describes the Business System Testing process.

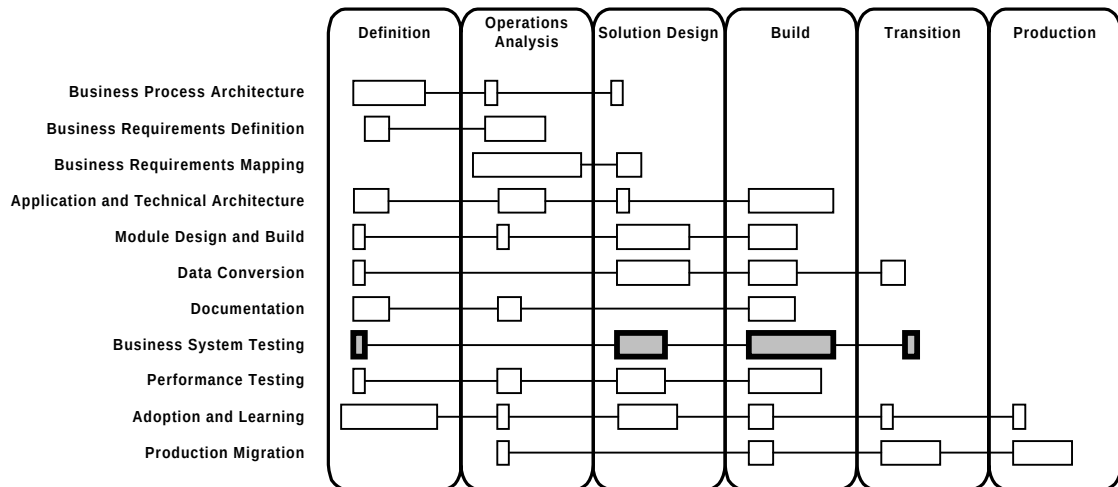


Figure 8-1 Business System Testing Context

Process Flow

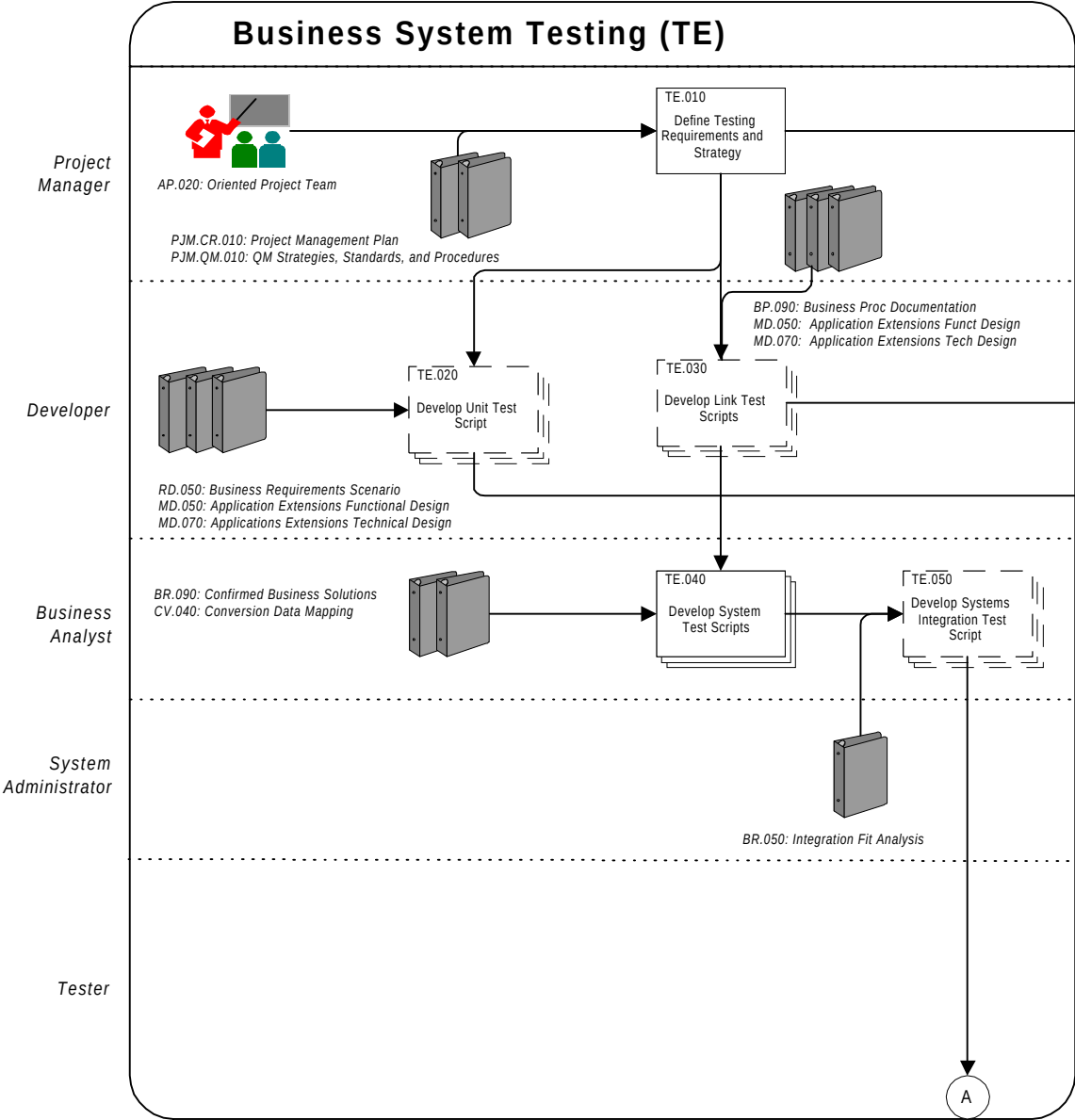


Figure 8-2 Business System Testing Process Flow Diagram

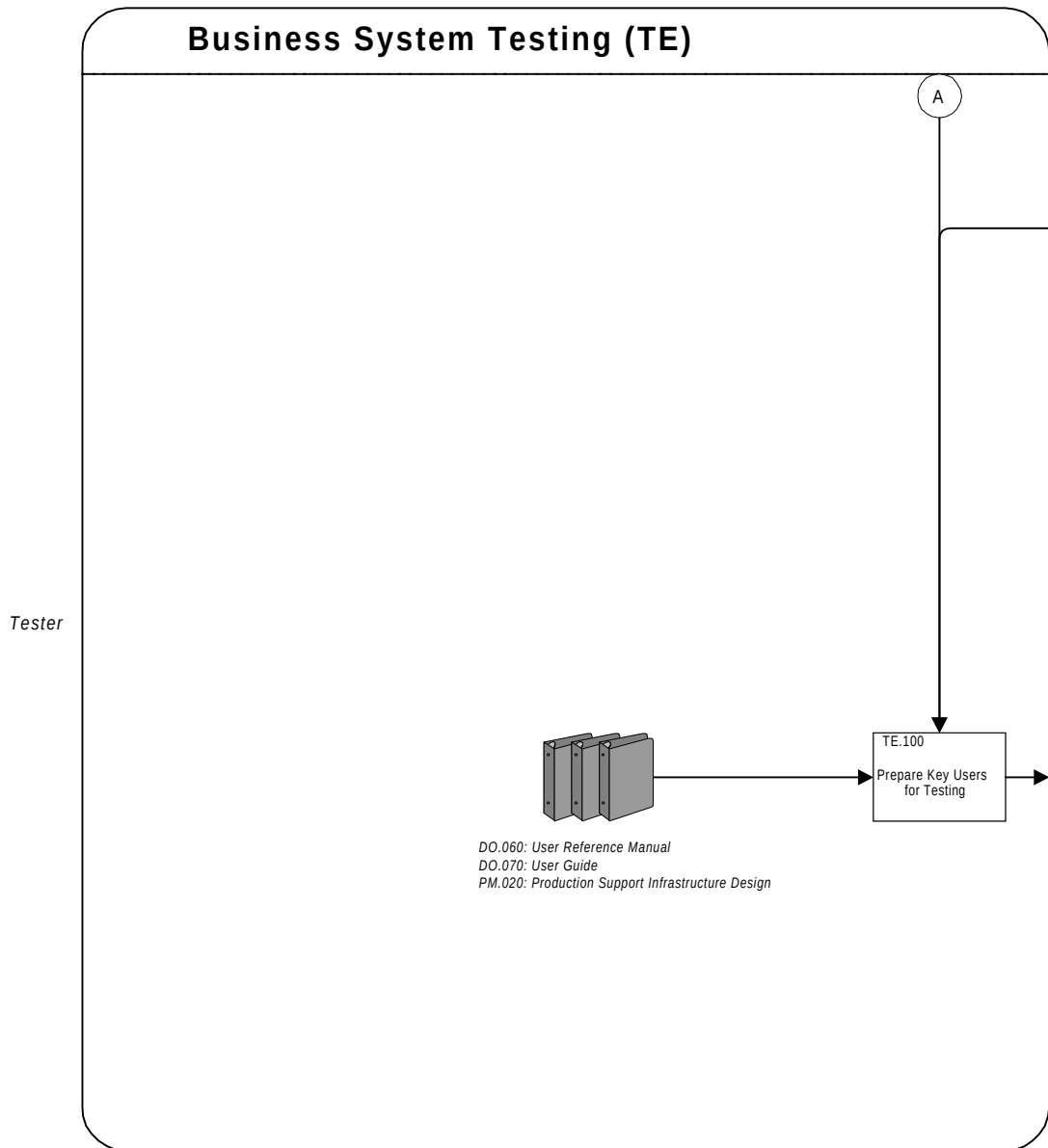


Figure 8-2 Business System Testing Process Flow Diagram (cont.)

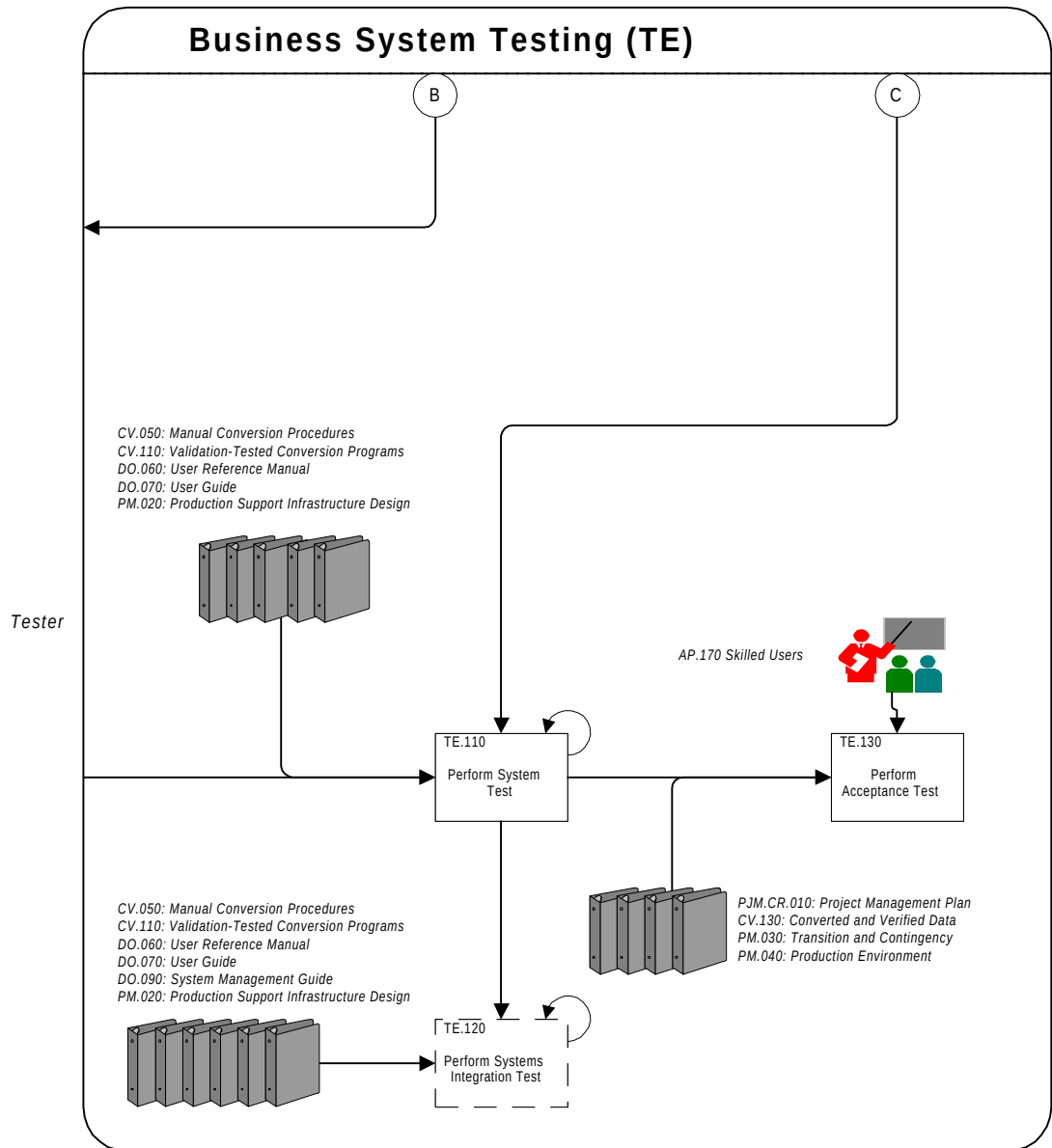


Figure 8-2 Business System Testing Process Flow Diagram (cont.)

Approach

The objective of Business System Testing is to test the quality of all of the elements of the application system. Business System Testing emphasizes a common planning approach for all types of testing and advocates the reuse of deliverable components to test successively larger aspects of the application system.

The following figure shows the logical progression of the Business System Testing tasks. Reading from left to right, the figure shows that the testing process begins with defining the Testing Requirements and Strategy (TE.010). After the Testing Requirements and Strategy is prepared, testers begin developing unit, link, system, and systems integration test scripts (TE.020, TE.030, TE.040, and TE.050). These test scripts are used to guide testers through the various stages of the testing process. While test scripts are being prepared, one or more Testing Environments will be configured (TE.060). These Testing Environments are used by the testers to perform their unit, link, system and systems integration tests (TE.070, TE.080, TE.110, and TE.120). While the system test environment is being configured, key users receive their instructions (TE.100) and the Installation Routines (MD.120) are tested (TE.090) to verify that custom extensions will be properly loaded into the Production Environment (PM.040). Once testing is completed in the test environment, users are supported in their acceptance testing (TE.130) of the new production system.

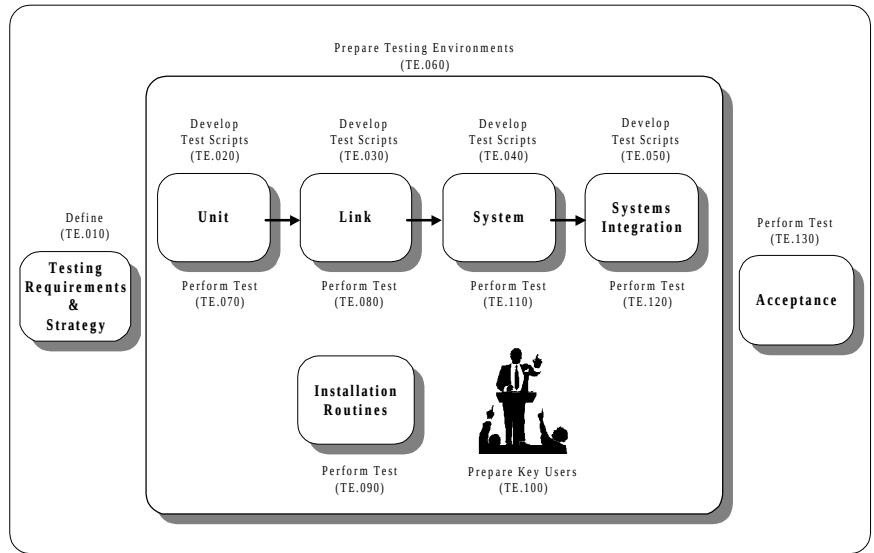


Figure 8-3 Business System Testing Overview Diagram

Business System Testing starts at the smallest element — the unit test (TE.070) — and expands to include the link test (TE.080), the system test (TE.110), and the systems integration test (TE.120). For those environments where there are no custom extensions and no interfaces to legacy or third-party systems, there is no need to perform unit, link, and systems integration testing.

Business System Testing also focuses on preparing for all types of testing early in the project life-cycle, so that you can associate testing scenarios back to business requirements and secure availability of the project resources that were originally involved in the business process design.

Finally, Business System Testing supports utilizing common testing information, including data profiles, to promote testing coordination and minimize duplication of test preparation and execution effort.

The intent of Business System Testing is to provide a formal approach to the challenge of testing. The testing approach is integral to the *entire* implementation effort and is structured to build upon itself. The primary deliverable of the testing process is high quality application systems that include both packaged applications and custom extensions.



Attention: Business System Testing does not address performance testing or the testing of data conversion programs; the Performance Testing (PT) and Data Conversion (CV) processes address these issues.

Planning

Business System Testing includes both functional unit testing and business flow oriented link, system, systems integration, and acceptance testing. You use the Business Requirements Scenarios (RD.050) developed in Business Requirements Definition (RD) to drive business-oriented testing, so that you can trace test results back to the original business requirements. As you define business processes during Business Requirements Mapping (BR), you extend Business Requirements Scenarios (RD.050) with Business Mapping Test Results (BR.080). Each business mapping test result eventually corresponds to a test script that includes a test scenario and data profiles.

The figure below shows an example of created and tested applications extensions, based on the requirements identified during Business Requirements Definition (RD).

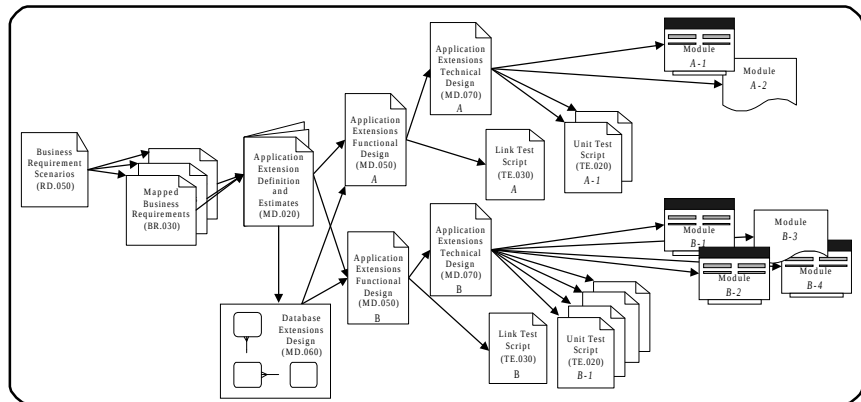


Figure 8-4 Integrated Business System Testing Flow Diagram

Early Introduction of Testing

You should introduce testing concepts early in the project life-cycle. When the project team considers the testing implications early, they define clearer requirements and more comprehensive process flows, and are then able to easily translate business models into test scenarios. This often leads to concurrent consideration or development of test

scenarios during other related activities (such as requirements mapping or process flow development). If you postpone comprehensive test design until later in the implementation, the following could occur:

- Key analysts, who most closely understand the business requirements to be tested, may move on to other projects.
- Opportunities to develop test scenarios during mapping and process development may be lost.
- You may lose the opportunity to capture testing requirements that should have occurred earlier in the project. This can lengthen the project timeline, cause resource conflicts, and require duplication of effort.

Conference Room Pilot

A conference room pilot (CRP) test refers to the technique of setting up a conference room where the testers' workstations are arranged in a particular order (usually by logical processes) and the test scripts are passed down the line from one tester to the next according to the natural flow of the business process. A CRP test usually involves performing a system test to check the validity of application setups, and the integration of business system flows within the target applications system. For more information on the CRP testing technique, see Perform System Test (TE.110).

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Business System Testing should include test scripts and test case scenarios that check for Century Date compliance of customizations and custom interfaces. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards

Tasks and Deliverables

The tasks and deliverables of this process are as follows:

ID	Task Name	Deliverable Name	Required When	Type*
TE.010	Define Testing Requirements and Strategy	Testing Requirements and Strategy	Always	SI
TE.020	Develop Unit Test Script	Unit Test Script	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	MI
TE.030	Develop Link Test Script	Link Test Script	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	MI
TE.040	Develop System Test Script	System Test Script	Always	MI
TE.050	Develop Systems Integration Test Script	Systems Integration Test Script	Project includes <i>interfaces</i> with external systems	MI
TE.060	Prepare Testing Environments	Testing Environments	Always	MI
TE.070	Perform Unit Test	Unit-Tested Modules	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	MI, IT
TE.080	Perform Link Test	Link-Tested Modules	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	MI, IT

ID	Task Name	Deliverable Name	Required When	Type*
TE.090	Perform Installation Test	Tested Installation Routines	Project includes <i>customizations</i> to standard functionality or <i>interfaces</i> with external systems	IT
TE.100	Prepare Key Users for Testing	Prepared Key Users	Always	SI
TE.110	Perform System Test	System-Tested Applications	Always	IT
TE.120	Perform Systems Integration Test	Integration-Tested System	Project includes <i>interfaces</i> with external systems	IT
TE.130	Perform Acceptance Test	Acceptance Test Results	Always	SI

*Type: SI=singly instantiated, MI=multiply instantiated, MO=multiply occurring, IT=iterated, O=ongoing. See Glossary.

Table 8-1 Business System Testing Tasks and Deliverables

Objectives

The objectives of Business System Testing are:

- Establish an approach to testing that naturally leverages business requirements and processes documented during requirements definition activities.
- Introduce and prepare scripts for all testing activities early in the software development and implementation life-cycle, such as during requirements mapping and module design activities.
- Build on and reuse initial testing deliverables, including test scripts and test data, in subsequent testing tasks.
- Confirm that business solutions meet business requirements.
- Confirm that the business system meets Century Date requirements.
- Produce thoroughly tested, quality systems.

Deliverables

The deliverables of this process are as follows:

Deliverable	Description
Testing Requirements and Strategy	The testing requirements, approach and scope of testing activities. It also specifies the testing tools and environment to be used and how to manage defects, as well as how to document acceptance criteria.
Unit Test Script	The test script that is used to test individual application extension modules.
Link Test Script	The test script that is used to test the combination of application extension modules as part of business flow.
System Test Script	The test script that is used to test the target applications' support of business processes including any application extensions.
Systems Integration Test Script	The test script that is used to test the integration of interfaces between the target application system and third-party and legacy systems.
Testing Environments	The hardware and software environment required to support the testing activities.
Unit-Tested Modules	Modules that have been tested according to the Unit Test Script (TE.020) to verify the detailed function of each application extension module.

Deliverable	Description
Link-Tested Modules	Modules that have been tested according to the Link Test Script (TE.030) to verify the detailed interaction between related application extension modules.
Tested Installation Routines	Installation routines that have been tested to verify that application extension modules function properly.
Prepared Key Users	Key users that have been given basic training in participating in Business System Testing.
System-Tested Applications	Applications that have been tested according to the System Test Script (TE.040) to validate that the system meets the defined business requirements and supports execution of business processes.
Integration-Tested System	A system that has been tested to validate the integration between the target application system and other systems.
Acceptance Test Results	The recorded results of the acceptance test performed by key users in order to secure acceptance of the production system.

Table 8-2 Business System Testing Deliverables

Key Responsibilities

The following roles are required to perform the tasks within this process:

Role	Responsibility
Business Analyst	Identify and document test scenarios for the System Test Script (TE.040).
Business Line Manager	Prepare for and participate in the system and acceptance testing activities.
Client Project Manager	Verify availability and commitment of users. Verify availability of platforms, software, and facilities. Develop a sense of teamwork and ownership of the new application. Review and resolve any issues that arise.
Client Staff Member	Provide support for the system test environment and the acceptance test environment. Provide support for the acceptance test execution.
Database Administrator	Install, configure, and prepare the test databases. Verify that all testers have access to the database.
Developer	Develop and execute unit and link test scripts. Provide support during the system, systems integration, and acceptance tests by answering questions and helping resolve potential issues.
Key User	Participate in the acceptance testing activities of the new production system.

Role	Responsibility
Project Manager	Identify the business system testing requirements and strategy that will be used for testing the new system.
System Administrator	Install application software. Provide support for the test databases. Install and configure the platforms and operating system. Install and configure application code. Perform recovery and rollback testing. Verify that application login IDs and responsibilities exist for all users. Create and maintain custom menus. Monitor and administer concurrent managers. Provide support during testing activities.
System Architect	Identify and document test scenarios for the Systems Integration Test Script (TE.050).
Technical Analyst	Provide support for the unit, link, system, systems integration, and acceptance tests.
Tester	Develop the testing strategy. Verify proper installation and configuration of testing facilities, platforms, and software. Verify the availability of a facilities coordinator and system applications and database administrators. Oversee, review, and approve the development of system and systems integration scripts. Manage execution of system and systems integration tests. Review and resolve any testing issues.

Role	Responsibility
User	Verify the application functions properly so that acceptance testing can begin. Conduct acceptance tests and log results.

Table 8-3 Business System Testing Key Responsibilities

Critical Success Factors

The critical success factors of Business System Testing are as follows:

- reliable approach for identifying system requirements
- a project plan that endorses an integrated testing approach
- adequate resources and time for test script development and execution, and support of testing environments
- adequate tools, including properly configured environments and well-trained users, to support test script development and execution
- the development of unit and link test scripts that are solid, integral components of subsequent testing deliverables
- the development of test scripts that are based on business process driven requirements
- reliable, timely (converted) test data for each testing environment
- a thorough execution of all business system and systems integration tests
- independent QA testing and quality sign off on all testing activities
- early notification of managers that their staff (key users) will be involved in testing

TE.010 - Define Testing Requirements and Strategy (Core)

In this task, you identify the Business System Testing requirements and strategy to be used for the testing of the system you are implementing. This also includes the recommended approach to testing, how to manage testing errors, and the criteria for accepting test results.

Deliverable

The deliverable for this task is the **Testing Requirements and Strategy**. It establishes the requirements, strategy, approach and scope of business system testing activities. In addition, it documents the testing tools and testing environment to be used.

Prerequisites

You need the following input for this task:

☐ **Project Management Plan [initial complete] (PJM.CR.010)**

The Project Management Plan provides a high-level discussion of the scope of testing work, testing approach, limited information about testing tools, and how the project should be organized and run.

☐ **Quality Management Strategies, Standards, and Procedures (PJM.QM.010)**

The Quality Management Strategies, Standards, and Procedures provide the Testing Requirements and Strategy with the project's quality criteria and infrastructure.

☐ **Oriented Project Team (AP.020)**

Prior to defining the testing requirements and strategy, the project team members should attend the initial project team orientation.

Optional

You may need the following input for this task:

☐ **Control and Reporting Strategies, Standards, and Procedures (PJM.CR.020)**

The Control and Reporting Strategies, Standards, and Procedures contains procedures that define how problems found during testing will be managed, including identification, analysis, resolution, and escalation procedures.

☐ **Existing Systems Testing Strategy or Policy Documents**

If high-level testing strategy or policy documents exist, use them to provide information about current thinking on the testing requirements and strategy. In situations where a formal information systems strategy project has preceded the testing process, a set of principles, directives, and strategies may already be in place, which you can directly input as requirements to testing here.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the Project Management Plan (PJM.CR.010).	
2.	Verify the testing description in the project-level scope, objectives, and approach contained within the Project Management Plan (PJM.CR.010)	
3.	Identify the scope of the testing, types and iterations of tests, and relationships to other projects and systems.	Scope

No.	Task Step	Deliverable Component
4.	Identify objectives and critical success factors.	Objectives
5.	Identify methods to be used for the testing process. Review the tasks, for this process, that will be used by the testing team.	Approach
6.	Review the deliverables that will be produced by this process.	Approach
7.	Identify constraints and assumptions that are associated with the process.	Approach
8.	Review risks to the process activities and objectives	Approach
9.	Identify procedures for scope control and issue management.	Approach
10.	Review relationships between process tasks / deliverables and other aspects of the overall project.	Approach
11.	Identify the high-level approach for carrying out testing activities.	Testing Strategy
12.	Identify the testing requirements at a summary level.	Testing Requirements
13.	Identify the testing requirements at the detailed level.	Testing Requirements

No.	Task Step	Deliverable Component
14.	Define the resource requirements for the testing activities of the project.	Testing Requirements
15.	Review the draft deliverable with senior management and seek approval.	Acceptance Certificate (PJM.CR.080)
16.	Identify any material changes to project scope and associated task estimates with the project manager.	Project Management Plan (PJM.CR.010)

Table 8-4 Task Steps for Define Testing Requirements and Strategy

Approach and Techniques

Use the Testing Requirements and Strategy to document the testing requirements, testing approach, and the scope of testing involved. The Testing Requirements and Strategy includes a listing of the types and purpose of each testing task as well as an explanation of the testing environments. In addition, it covers the tools used to perform testing.

In the Testing Requirements and Strategy, you establish or provide the following:

- a list of testing requirements
- an overview of the strategy, including relevant background, the testing approach, critical success factors for successful testing, and the risks associated with not performing adequate testing
- an understanding of the type and purpose of each testing task
- an understanding of the deliverables for each testing task
- an overview of the testing tools
- an overview of how problems will be managed
- detailed acceptance criteria for testing

The objective of this task is to prepare a working strategy document that team members can reference during the tasks in Business System Testing. New testing team members can quickly become familiar with the process by reviewing this document; it can also resolve any questions about general information listed in the Project Management Plan (PJM.CR.010).



Warning: The risk of not documenting the specifics of the testing strategy includes a general lack of understanding of each type of testing, nonstandard deliverables, disorganized or unfocused test execution, and redundant work effort due to reinventing test script components and test data.

It is important that the information contained in the Testing Requirements and Strategy be thoroughly documented, clearly communicated and understood throughout the testing team, and constantly updated with any changes to the Project Management Plan (PJM.CR.010).

The testing team should include sample deliverables for some tasks, such as Link Test Script (TE.030) and System Test Script (TE.040), to illustrate how testing deliverables relate to and build upon one another.

You may also decide to communicate the Testing Requirements and Strategy in a formal presentation and obtain acceptance on the approach, tasks, and planned deliverables.

Testing Scope

Testing scope can vary widely, depending on the needs of your project. When determining testing scope you must consider the following:

- the testing tasks you must perform
- the types of testing you need to include in each test
- the degree of customization
- the number of system interfaces

Business System Testing defines six specific test execution tasks. Identify the testing tasks that are appropriate for your project and determine the scope of each.

Perform Unit Test (TE.070)

Test the functionality based on elementary business functions, validations, calculations, error-handling, module security, user interface, help text, and adherence to standards. Unit tests are usually mandatory only for new application extensions. However, unit tests can be valuable if you are implementing a beta or early production release of new applications.

Perform Link Test (TE.080)

Test the linkage and integration between related application extension components.

Perform Installation Test (TE.090)

Test the Installation Routines (MD.120) to verify that their setup steps cover the proper procedures and instructions for setting up the custom extensions in the target application environment. This test is performed as part of Perform System Test (TE.110) as a way of verifying that the instructions can be relied upon when it comes time to set up the production environment.

Perform System Test (TE.110)

Test the entire application by testing the integration between business processes. In addition, test database journaling, security, documentation, manual data, converted data, reconciliation with legacy systems, job streams, backup and recovery, and data archival. Regression testing may occur during this task if application extensions need to be retested in order to validate that prior defects have been corrected.

Perform Systems Integration Test (TE.120)

Test the coexistence and integration of the application system with neighboring applications systems. If there are other systems your application must coexist and interface with, then you need to state your assumptions about these interfaces and decide whether to require a distinct systems integration test.

Perform Acceptance Test (TE.130)

Verify that the new application system meets user acceptance criteria and simulates live production in the production environment (or a suitably configured production-like environment) with users executing

system scripts on recently converted data. If key users participated in system and systems integration testing, acceptance testing can be perfunctory. In addition, document your assumptions for the user acceptance criteria.

Test Types by Task

The specific types of information you need to confirm during each test also influence the scope of testing. The table below provides a sample cross-reference between test types and test tasks.

Test Type	Unit	Link	System	Systems Integration
System Process Steps		X	X	
Validation	X			
Calculation	X			
Error Handling (negative logic)	X			
Database Journaling	X		X	
Security	X	X	X	
Volume Data		X		
Help Text	X			
Checkpoint Restart	X			
User Interface	X			
Report Layout	X			
Screen layout	X			
Business Scenarios		X	X	X
Initial System Documentation			X	
Manual Data Inspection			X	
System Sequences Using Scripted Data			X	
Converted Data Load			X	
Converted data inspection			X	
System Sequences Using Converted Data			X	
Parallel Legacy Reconciliation			X	
Job Stream			X	
Backup and Recovery			X	
Data Archival			X	
Systems Integration Sequences Using Converted Data				X
Database Locking			X	X
Batch Window			X	X
Online Response Time			X	X
Network Stress				X

Table 8-5 Test Types by Task

Converted Data Test Requirements

If there is legacy data to convert, you must consider when you should introduce converted data into the various tests. A system test usually begins with *scripted* data until the application is stable enough to introduce a new variable — converted data. Be sure to cleanse the data before using it in system testing. Also, consider using your application to inspect the converted data, using read-only transactions, before introducing more complex update transactions to the converted data. The Testing Requirements and Strategy should indicate how and when to use conversion data.

System Interfaces

Document and classify each system interface type. The type indicates whether the interface supplies data to the application system (input), receives data from the application system (output), or both supplies and receives data (two way). Use this information to plan the general sequence of the systems integration test.

Testing Approach

The approach that AIM employs for testing is robust and proven in prior implementations. The advantage of the AIM approach is that it ties the requirements to the test scripts. The AIM testing approach has tasks that start by testing the smallest element of the system (unit) and proceeding to other tasks that involve larger pieces of the system. As the testing task ID numbers increase, larger pieces of the system are being tested until the entire integrated system is tested and accepted.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C / Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Business System Testing should include test scripts and test case scenarios that check for Century Date compliance of customizations and custom interfaces. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Testing Tools

Testing tools can support your testing effort and align with the overall testing objective. Testing tools supply some or all of the following features:

- test case management
- test results and reporting
- test defects management
- regression testing
- stress testing

The Testing Requirements and Strategy should determine *what* tools you will use and *how* you intend to use them.

Test Case Management

This is one of the most important tools available to support the life-cycle of Business System Testing. Features of a good tool allow you to create a test case repository for reuse in the development of specific testing scripts. There are numerous tools that allow the testing manager to determine the *state* of each test script and whether the scripts are currently under development or are ready to be executed. Since business system testing should align with business processes, you can develop test scripts by extending and documenting business process flows with this tool. The Testing Requirements and Strategy must clarify how to use test case management tools by specifying tool standards and guidelines.

Test Results and Reporting

Determine the reports required for your project and whether you need to develop additional reports. Standardize the test result classifications (for example, pass, fail, skipped, or not executed).

Test Defects Management

Determine your test defect work flow. Many tools support standard work flow states but also allow customizations. The Testing Requirements and Strategy must clarify and standardize the defect work flow in the tool (for example, open, assigned, fixed, verified, or closed).

Regression Testing

Determine your level of regression testing automation. Regression test automation is now a very realistic option. Current tools feature the ability to record user actions and “learn” GUI objects, capturing this information into test scripts. You can then generalize the scripts to use variable data, enabling the scripts’ execution to be data-driven. If your organization is considering automating its regression testing, clearly define your automation goals and plan extra time for script development and generalization. Place test scripts under configuration control and update the scripts each time the application is updated.

Relationship to Performance Testing

Business System Testing does not address volume and stress testing; these are the subjects of Performance Testing (PT). Coordinate with the Performance Testing team on your project so that you can share resources, business process flows, and automation tools. Depending on how many configurations the Performance Testing team plans to evaluate, you may be able to schedule one of the tests on the same configuration on which you are planning to run the system test.

Linkage to Other Tasks

The Testing Requirements and Strategy is an input to the following tasks:

- TA.010 - Define Architecture Requirements and Strategy
- TE.020 - Develop Unit Test Script
- TE.030 - Develop Link Test Script

- TE.060 - Prepare Testing Environments
- TE.070 - Perform Unit Test
- TE.090 - Perform Installation Test
- TE.100 - Prepare Key Users for Testing
- TE.110 - Perform System Test
- TE.120 - Perform Systems Integration Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Business Analyst	30
Project Manager	30
Technical Analyst	20
Tester	20
Client Project Manager	*

Table 8-6 Role Contribution for Define Testing Requirements and Strategy

* Participation level not estimated.

Deliverable Guidelines

Use the Testing Requirements and Strategy to document the requirements for testing, the project’s approach to testing, and the testing strategy to be deployed. The Testing Requirements and Strategy should expand the Project Management Plan (PJM.CR.010) in greater detail for testing.

This deliverable should address the following:

- testing requirements and strategy
- testing approach

Deliverable Components

The Testing Requirements and Strategy consists of the following components:

- Introduction
- Scope
- Objectives
- Approach
- Testing Strategy
- Testing Requirements

Introduction

This component describes testing and its purpose. It also describes the purpose of the testing strategy and the audience involved in the Business System Testing process.

Scope

This component describes the scope of the testing process for your project in as much detail as possible and explains the testing tasks and the types of tests to perform during each task.

Testing scope statements can be made in terms of whether testing components are in or out of scope for the project. Examples of such components that can define the scope include:

- application extensions
- interfaces to third-party and legacy systems
- applications setup
- test data

Objectives

This component lists the high-level objectives that the business and project managers have communicated about testing. It includes the following sections:

- Objectives
- Critical Success Factors

Since this is a strategic document, the stated objectives should not be too detailed, but they should be specific and measurable.

Critical success factors associated with meeting the objectives of the process within the context of the overall project are also identified within this component.

Approach

This component describes the process, policies and procedures, project dependencies, and the technical background for the testing process. It includes the following sections:

- Tasks
- Deliverables
- Constraints and Assumptions
- Scope Control
- Integration with Other Aspects of the Project

The description of testing process should include a high-level discussion of the approach selected for the testing work, the tasks and deliverables, the reasons for selection of the approach, and the benefits of the particular method adopted.

Testing Strategy

This component describes the strategy for addressing key areas in the testing project. These areas may be highlighted and discussed because of their criticality in the testing work, the inherent risk to the project, an innovative or unusual approach to be applied in the project, or for some other implementation-specific reason.

Testing Requirements

This component documents the detailed and summarized testing requirements. The detailed requirements should detail the individual business system testing requirements that will affect testing. Some of the requirements that may be included are:

- *Application Extensions* — this is a list of application extensions that must be tested for business functionality and compliance to coding standards.
- *Interfaces* — this is a list of interfaces to third-party and legacy systems. This involves data flow either into or out of Oracle Applications.
- *System Setup* — this is a list of business processes that will be tested to verify that the application setup supports functionality as defined in the business requirements.
- *Test Data* — this is a list of data objects needed to perform the tests. This may involve seeded demo data, manually loaded test data, or automatically converted legacy data depending on the requirements.
- *Testing Environments* — this describes the server configuration, network configuration, supporting software, workstation configuration, and database configuration for the testing environments.
- *Testing Tools* — this describes what testing tools, if any, you plan to implement to support the testing activities. In addition, it includes information on how to use the tools or provides a reference to additional documentation.
- *Problem Management* — this documents your problem management process for defect tracking. Problem management uses work flow diagrams to clearly show the proper actions. It develops and documents the defect categories, so that testers can consistently document the type of defects they encounter (for example, functionality, navigation, usability, manual data, conversion data, performance, help text, or test script). In addition, it documents fix priority categories (for example, critical, high, medium, and low).
- *Acceptance Criteria* — this lists the specific acceptance criteria that the team must evaluate for each test and clearly states the criteria and approach for deciding what is an acceptable outcome.

- *Hardware and Networks* — this lists the server platform(s) and networks required to build the testing environment where business system testing is supported.
- *Staff Resources* — this lists the resources, by role, needed to perform the testing tasks.

Following a detailed review, a summary of the requirements should be produced.

Audience, Distribution, and Usage

Distribute and communicate the Testing Requirements and Strategy to the following:

- project manager
- testing team members
- other process leaders who have dependent tasks with the testing process

Quality Criteria

Use the following criteria to help check the quality of this deliverable:

- Are the project scope and objectives clearly identified?
- Are specific critical success factors and risks documented?
- Does this document take into account the impact of dependent tasks from other processes?
- Are the testing requirements clearly defined?
- Does the strategy discuss existing information systems testing policies and strategies and indicate how they relate to this project?
- Is the testing strategy understood by those on the distribution list for this deliverable?
- Are all resource and tool requirements that could impact the testing process stated and understood?
- Is the deliverable thorough in capturing different types of business system testing requirements that are directly relevant to testing?

Tools

Deliverable Template

Use the Testing Requirements and Strategy template to create the deliverable for this task.

TE.020 - Develop Unit Test Script (Optional)

In this task, you develop the script to test individual application extension components. The tests validate that the application extension inputs, outputs, and processing logic function as designed.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Unit Test Script**. It identifies what needs to be unit tested, as well as the steps that are required to complete the test. A unit test script is used to verify that each application extension component (such as a table, form, report, or database trigger) does not include coding errors.

Prerequisites

You need the following input for this task:



Business Requirements Scenarios (RD.050)

The Business Requirements Scenarios capture the original requirements that led to the need for the customization and help you understand how the application extension fits into the larger business process.



Application Extensions Functional Design (MD.050)

The Application Extensions Functional Design describes the functionality and business rules implemented by the application extension. They are used as your primary guide in developing unit test scripts. If Create Application Extensions Functional Design was not performed, this deliverable will not exist. (See the task description for MD.050 for more information on when this task should be performed.)

Application Extensions Technical Design (MD.070)

The Application Extensions Technical Design provides additional details regarding internal logic that can help you establish exhaustive test cases. If Create Application Extensions Technical Design was not performed, this deliverable will not exist. (See the task description for MD.070 for more information on when this task should be performed.)

Testing Requirements and Strategy (TE.010)

The Testing Requirements and Strategy provides the testing approach and testing requirements and strategy for Business System Testing.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the Design Standards (MD.030) and Build Standards (MD.040).	
2.	Incorporate the design and build standards into the Unit Test Checklist.	Unit Test Checklist
3.	Review the Application Extensions Functional Design (MD.050).	
4.	Review the Application Extensions Technical Design (MD.070).	
5.	Review the Business Requirements Scenarios (RD.050) related to the custom application extension.	
6.	Develop the Unit Test Specifications.	Unit Test Specifications

No.	Task Step	Deliverable Component
7.	Develop the Data Profile.	Data Profile
8.	Document the coding defects.	Defect Log
9.	Validate the components of the Unit Test Script.	
10.	Include criteria for Century Date Compliance testing and secure acceptance.	Acceptance Certificate (PJM.CR.080)

Table 8-7 Task Steps for Develop Unit Test Script

Approach and Techniques

Use the Unit Test Script to document the steps needed to test a single application extension component (such as a program, form, report, flexfield, table, or database trigger).

Unit Testing Scope

The unit test is the narrowest scope of testing you will perform. Each unit test script exercises a single program, form, report, flexfield, database trigger, or other custom object. When performed thoroughly, unit testing is one of the biggest contributors to a stable application system and will significantly reduce all downstream testing efforts. Unit testing is a repetitive task; testers execute each unit test numerous times using different combinations of test data as specified in the data profile.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or

Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Business System Testing should include test scripts and test case scenarios that check for Century Date compliance of customizations and custom interfaces. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Cosmetic and User Interface Standards

When defining tests for forms and reports, provide a checklist to validate conformance with cosmetic standards, including informative messages for error conditions. The Design Standards (MD.030) define general cosmetic standards.

Basic Functionality

Your test must verify that everything works as the designer intended. Examine the Application Extensions Functional Design (MD.050) and Application Extensions Technical Design (MD.070) to identify specific business rules and conditional logic. Construct data profiles and test specifications to exercise all possible logic combinations.

Boundary Conditions

Organize your tests to evaluate normal usage cases first, and then test exception or out-of-range cases and boundary conditions. A boundary condition is a combination of input parameters that represent the largest or smallest values permitted. It is much easier to diagnose problems during unit testing, rather than later when many different programs are interacting.

Interface Programs

Interface programs transfer or integrate data from one business application to another. Interface programs often extract data from the source and place them in a temporary state (tables or files), before placing them in the destination environment. The temporary state

allows data validation, collection (batch), and other transformations to occur. Therefore, unit-testing interface programs must consider the discrete components that extract, validate, or transform the data. Define unit tests for each component of an interface separately. Depending on the design, you may need to test interface components for error correction and feedback.

Performance

If the Application Extensions Technical Design (MD.070) identifies performance as a possible issue, or the application extension is part of a business process with high volumes or transaction rates, you should include information in your test scripts so that the tester can monitor performance during the tests. Make sure you define prerequisites for sufficient data volume in the test environment to exercise the application extension adequately. Coordinate your tests with the team performing Performance Testing (PT) tasks.

Restart and Recovery

If the application extension is a concurrent program or other batch job, include tests to confirm that you can rerun the application extension in the event of failure. Look at the design document to determine the restart strategy. Some programs keep track of their progress and complete processing where they left off. Other programs simply roll back all changes when an error occurs and start from the beginning when the program is rerun.

You may need to provide instructions for artificially simulating a database error by renaming a synonym or resizing tables to a smaller size (to trigger *out of extent* errors), before executing the test. The size and number of rollback segments relative to the processing volume and run duration can influence both Business System Testing and the program design. Long-running, high-volume batch programs may need to commit data at periodic checkpoints to avoid *rollback segment to old* errors and to prevent running out of rollback segments. Coordinate with the developer and the people who are planning the physical database design of the production system to structure an appropriate test case.

Custom database triggers that fire when standard application modules insert or update data can also leave data in an invalid state if they do not handle errors properly. Any error in a database trigger must raise an exception that propagates to the program that initiated the

transaction, so that it can roll all changes back and display (or print) an appropriate message. The module that performs the initial update or insert operation should process exceptions generated by a database trigger like any other database error. Construct test cases that will cause the error logic to execute.

Destructive Testing

The most extreme form of testing is when you try to *break* the application extension by providing bad input data or extreme test conditions. For forms, this can include keying invalid fields, entering non-century compliant dates, and invalid field sequences. Stored procedures and SQL scripts should handle invalid parameter values. Include test cases for these extreme conditions or suggest general techniques that allow the testers to be creative.

Unit Test Data

Unit test data is generally contrived since converted data is either not available or is in an unstable state. You may want to use sample documents and data from the legacy application, or have the technical analyst or business analyst help you design the unit test data.

The specific data you need depends on the type of application extension you are testing. For example, if you are designing a test for a form that users employ for entering information, you only need the data required to serve as lookup values. On the other hand, if you are testing a complex report or a batch program that operates on existing data, you need enough data to test all possible logic combinations.

Linkage to Other Tasks

The Unit Test Scripts are an input to the following tasks:

- MD.110 - Create Application Extension Modules
- TE.070 - Perform Unit Test

Role Contribution

The percentage of total task time required for each task follows:

Role	%
Developer	80
Business Analyst	10
Technical Analyst	10

Table 8-8 Role Contribution for Develop Unit Test Script

Deliverable Guidelines

Use the Unit Test Script to document your testing of a single program, form, report, flexfield, database trigger, or other custom object.

This deliverable should address the following:

- functional testing
- testing for cosmetic standards
- specific unit tests to be performed
- data to be used

Deliverable Components

The Unit Test Script consist of the following components:

- Overview
- Unit Test Checklist
- Unit Test Specifications
- Data Profile
- Defect Log



Attention: You must create one Unit Test Script for every custom application extension component in the system.

Overview

This component documents the reasons for the Unit Test Script and the resources needed to develop the Unit Test Script.

Unit Test Checklist

This component defines the categories of tests to validate the execution features of the application extension (for example, field validation, help test, error handling, and appearance). Design standards drive many of these categories, and they are common for all application extensions of the same type. For example, this checklist can be used to evaluate all forms for adherence to cosmetics.

Unit Test Specifications

This component details the test steps necessary to test the unique logic of the application extension component. To evaluate all of the logic combinations, numerous test specification can be created. The goal is to exercise all logic, so the tests do not need to reflect real business situations. In fact, the tests may be very artificial so that they test the maximum amount of functionality in the fewest possible passes.

Data Profile

This component specifies the test data required to execute the unit test specification. Typically, multiple data profiles are developed for execution with each test specification. By defining a unit test specification for each logic path and a data profile for each data combination, the reusability of the tests and the coverage they provide is maximized.

Defect Log

This component facilitates the tracking of each defect the tester uncovers during unit testing, as well as the related *fix* documentation recorded by the owning developer once they correct the error. By definition, the defect log is empty when the test script is created.

Tools

Deliverable Template

Use the Unit Test Script template to create the deliverable for this task.

Century Date Compliance

To document the review and approval of Unit Test Script for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Compliance Certificate should be filed with the deliverable in the project library.

TE.030 - Develop Link Test Script (Optional)

In this task, you develop scripts to test modifications to standard Oracle Applications as well as new application extensions as part of a business flow. This uncovers any integration problems with other application extension components that provide or use the data manipulated by the target modules.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Link Test Script**. It identifies the link tests to be performed. Link tests verify that application extension components are properly linked and no coding errors are generated when components are linked together.

Prerequisites

You need the following input for this task:



Business Procedure Documentation (BP.090)

Review the Business Procedure Documentation for the business processes associated with the custom application extensions. This documentation provides a broader context for the business process and requirements.



Application Extensions Functional Design (MD.050)

The user procedures described in the topical essay component of the Application Extensions Functional Design are your primary guide to the structure and requirements of the link test. If Create Application Extensions Functional Design was not performed, this deliverable will not exist. (See the task description for MD.050 for more information on when this task should be performed.)

❑ Application Extensions Technical Design (MD.070)

Technical details about the application extensions are outlined in the technical design document. Use information from this design document to build the test script for testing the technical aspects of the extension. If Create Application Extensions Technical Design was not performed, this deliverable will not exist. (See the task description for MD.070 for more information on when this task should be performed.)

❑ Testing Requirements and Strategy (TE.010)

The Testing Requirements and Strategy provides the testing approach and testing requirements and strategy for Business System Testing.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the Business Procedure Documentation (BP.090) related to the custom application extensions to be link tested.	
2.	Review the User Reference Manual (DO.060).	
3.	Review the Application Extensions Functional Design (MD.050).	
4.	Review the Application Extensions Technical Design (MD.070).	
5.	Develop the Link Test Specification.	Link Test Specification
6.	Develop the Data Profile.	Data Profile

No.	Task Step	Deliverable Component
7.	Validate the components of the Link Test Script.	
8.	Secure acceptance that link test scripts include criteria for Century Date Compliance testing.	Acceptance Certificate (PJM.CR.080)

Table 8-9 Task Steps for Develop Link Test Script

Approach and Techniques

Use the Link Test Script to check the linkage and integration between two or more application extension components that are part of the same business process. Create one Link Test Script for each application extension, which may consist of several individual application extension components (such as a program, form, report, flexfield, database trigger, or other application extension components). Each Link Test Script directly corresponds with an Application Extensions Functional Design (MD.050); you write one Link Test Script for each Application Extensions Functional Design.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C / Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Business System Testing should include test scripts and test case scenarios that check for Century Date compliance of customizations and custom interfaces. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Link Test Data

If the link test environment is a fresh installation, you need to construct the test data or use converted data from preliminary conversion programs. If the link test environment is a copy of another environment (like the mapping environment), you should design your test data based on information already defined in the database.



Attention: If you use converted data, do not derail your testing effort by spending too much time cleansing your converted data.

Link Test Script

The Link Test Script consists of the Link Test Specification and the Data Profile. The Link Test Specification component details the test steps necessary to thoroughly test the customization. The Data Profile specifies the test data required to execute the Link Test Specification. Link testing includes business flow testing, integrity testing, usability testing, security testing, and volume testing categories.

Business Flow Testing

Business flow testing involves developing the Link Test Script to test the business processes that include the customization. Business flow testing is the primary and most important form of link testing. The Application Extensions Functional Design (MD.050) and Business Procedure Documentation (BP.090) provide a solid basis for developing the test steps and data to test the business flow.

Your tests should also include unusual or extreme business flows, even if they are highly unlikely. Experience has shown that users can and will perform the most unlikely procedures.

Integrity Testing

Any application extension that updates data must be able to handle record locks. Part of your link test should evaluate how the application

extensions respond to this condition. Appropriate design strategies include waiting for locks to be released or simply aborting when a program cannot obtain a lock.

Forms

When two users attempt to update the same record via a form, the first user to make a change should secure a lock while the other user is asked if they want to wait for the lock to be released. Sloppy programming can cause forms to engage a lock when a user simply queries a record, and can even result in a deadlock situation where neither user can release the lock.

Batch Programs

A common error in batch programs is selecting data from tables without a lock at the start of processing, and then updating the original tables with new data at the end. In this scenario, another program can update the table between the original data selection and the final update, resulting in data loss (the middle transaction is lost). This can cause severe referential integrity problems when the middle transaction updates other tables with synchronized data that is not lost.

You can designate concurrent programs as incompatible with themselves and other concurrent programs that manipulate data from the same tables. This can prevent locking problems caused by two update programs running simultaneously. The Application Extensions Technical Design (MD.070) should define this requirement and you should configure the test environment accordingly. Test to make sure that the concurrent manager enforces the rules appropriately.

Testing Techniques

Use these techniques to evaluate lock integrity:

Forms

- Use two sessions to query the same row into identical forms by different users. Attempt to have both users lock the row by modifying a data element. Verify that the form handles the lock properly and releases it when you save the change or clear the form (commit or rollback).

- Make sure that the form does not inadvertently lock the rows of query-only forms or blocks. Query the row on one form while using an update form for the base table (or SQL*Plus) to test the lock condition.

Concurrent Programs

- Run two copies of the same concurrent program simultaneously. One instance should wait for the other to complete or abort with an appropriate error message.
- Run the concurrent program and use a form to lock a row that the program will update. The program should wait a reasonable amount of time or abort with an appropriate message.
- Run the concurrent program and use a form to lock a row that the program reads but does not update. The program should complete without any errors.
- Run the concurrent program and use a form or SQL script to update a row that the program also updates. If you receive a message that the form or SQL*Plus cannot obtain a lock, make sure the program releases its lock when it completes. If the transaction succeeds, verify that both sets of updates are preserved.

Usability Testing

Usability testing does not focus on the accuracy of an application extension, but on user-friendliness. Naturally, users are the best resource for this type of testing; they should be able to use the application extensions as part of a business flow after reading the Application Extensions Functional Design (MD.050) and receiving basic instructions.

If you perform this type of testing, your role is to observe the user's actions and responses to the software. Take notes to document functionality or program behavior that confuses the user or elicits an incorrect response.

Usability testing is a way of refining the design of an application extension and is common when following an iterative design and build approach to custom development. Consider usability testing whenever the user interface or procedures are complex or unusual.

Security Testing

Security link testing involves two areas: user access to application extensions (menus), and application extension access to data (including data residing in both owned and shared tables). You must verify that each user has access to the correct responsibilities, menus, and data to perform his or her job. You must also verify that each application extension has access to both its owned tables as well as any shared tables.

Volume Testing

Volume testing involves simulating live operations with multiple users active on the same program, as well as on conflicting programs. Have multiple users access combinations of the same application extensions and test data while performing the same operations to evaluate any obvious conflicts or performance issues. Coordinate any volume testing with the Performance Testing team on your project.

Linkage to Other Tasks

The Link Test Script is an input to the following tasks:

- MD.110 - Create Application Extension Modules
- TE.040 - Develop System Test Script
- TE.080 - Perform Link Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Developer	80
Business Analyst	10
Technical Analyst	10

Table 8-10 Role Contribution for Develop Link Test Script

Deliverable Guidelines

Use the Link Test Script to define specific link tests that test custom application extensions as part of a business flow.

This deliverable should address the following:

- link tests to be performed
- data used in the link tests
- link testing errors

Deliverable Components

The Link Test Script consists of the following components:

- Overview
- Link Test Specification
- Data Profile
- Defect Log

Overview

This component documents the reasons for the Link Test Script and the resources needed to develop the Link Test Script.

Link Test Specification

This component defines the test script execution and describes the functional features of an integrated set of application extension components. There may be more than one Link Test Specification if it is necessary to test more than one response path through the process the application extension supports.

Data Profile

This component describes the business conditions or seeded data you need to link test the customization. There may be more than one Data Profile if the logic within the application extensions is sensitive to data conditions. In particular, if one application extension calls another, and the called application extension can only accept arguments within a particular range of values, you should have Data Profiles that include data both within and outside the range.

Defect Log

This component facilitates tracking each defect uncovered during link testing, as well as the related *fix* documentation recorded by the owning developer once they have corrected the error. By definition, the default log is empty when the Link Test Script is created.

Tools

Deliverable Template

Use the Link Test Script template to create the deliverable for this task.

Century Date Compliance

To document the review and approval of Link Test Scripts for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Compliance Certificate should be filed with the deliverable in the project library.

TE.040 - Develop System Test Script (Core)

In this task, you develop the script to test the integration of application extensions with Oracle Applications modules. A system test script contains detailed steps which testers follow to verify the system setup and the integrity of custom application extensions for supporting business processes.

Deliverable

The deliverable for this task is the **System Test Script**. It identifies the system tests to be performed to verify that custom application extensions and standard Oracle Applications modules adequately support the organization's business processes.

Prerequisites

You need the following input for this task:

☐ **Confirmed Business Solutions (BR.090)**

Use the Confirmed Business Solutions to establish the business processes that will be tested during Perform System Test (TE.110).

☐ **Conversion Data Mapping (CV.040)**

The Conversion Data Mapping document records the detailed mapping of legacy data to the new system. This information may be useful when completing the data profile component of the System Test Scripts. If Perform Conversion Data Mapping was not performed, this deliverable will not exist. (See the task description for CV.040 for more information on when this task should be performed.)

☐ **Link Test Script (TE.030)**

Group the Link Test Script into more comprehensive system test sequences. If Develop Link Test Script was not performed, this deliverable will not exist. (See the task description for TE.030 task description for more information on when this task should be performed.)

Oracle Applications Documentation Library

Use information found in the standard Oracle Applications documentation to gain knowledge about the functionality of the standard Oracle Applications.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the Business Mapping Test Results (BR.080) and map to test scenarios.	
2.	Review the Link Test Script (TE.030).	
3.	Develop the System Test Specifications.	System Test Specifications
4.	Develop the Data Profile for the system test.	Data Profile
5.	Include a Defect Log to be used during testing.	Defect Log
6.	Develop the System Test Sequences.	System Test Sequences
7.	Validate the components of the System Test Script.	
8.	Secure acceptance that system test scripts include criteria for Century Date compliance testing.	Acceptance Certificate (PJM.CR.080)

Table 8-11 Task Steps for Develop System Test Script

Approach and Techniques

Use the System Test Script to document the steps needed to test the integration of application extensions with the Oracle Applications modules.

System Testing

System testing measures the quality of the entire application system, using system test sequences and scripts. You must create scripts for all business processes based on the Mapped Business Requirements (BR.030). You should be able to reuse the test scripts you created during Test Business Solutions (BR.080); however, the focus of business solution testing is confirming individual business processes, while business system testing focuses on confirming the collective application system. For more information, refer to the Business Mapping Test Results (BR.080) as a basis for business system test specifications.

The system test can include the following types of testing:

- integrated business processes
- manual procedures
- support procedures
- security testing
- initial system documentation inspection
- manual data inspection
- database journaling
- converted data load testing
- converted data inspection
- interface testing (limited to processing data as input from another system, or creating data for use by another system)
- data / transaction reconciliation to the legacy system
- job stream testing (if there is a batch component)
- backup and recovery testing
- data archival testing

- lock testing simulating user load (executing identical scenarios)
- batch window test (on full converted data volumes)

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Business System Testing should include test scripts and test case scenarios that check for Century Date compliance of customizations and custom interfaces. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

System Test Specifications

A test specification is the component of a test script that defines test script execution. Based entirely on a specific business process, it consists of scenario information, process information, and a series of test steps whose order was determined from the process. A test specification communicates the following:

- test steps, corresponding to some, if not all, system process steps, detailing the business testing requirement of a scenario
- the sequential relationship between test steps
- the data profiles involved in each test step
- the actions performed in each test step
- the expected application system responses in each test step

The figure below represents the relationships between the process, process steps, scenarios, and test steps. A business process can have multiple process steps. Likewise, a business process can have multiple scenarios that involve the same business process. The System Test Script should include multiple test steps that evaluate the process steps for each of the scenarios.

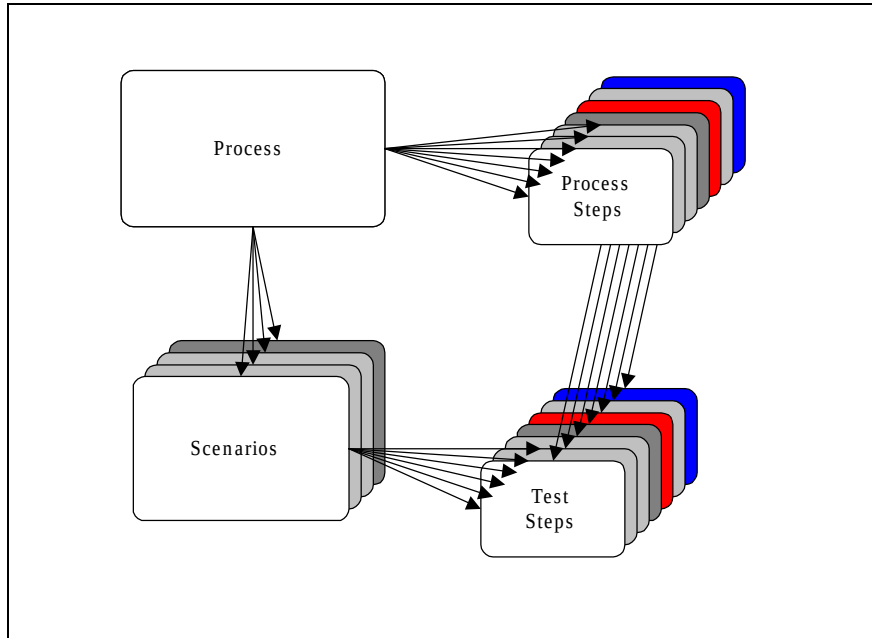


Figure 8-5 Scenario and Test Steps Relationship

System Test Specifications also provide for recording the results of tests — both qualitative (for example, description of outputs) and quantitative (for example, measured cycle time of the actual test).

Data Profiles

Data Profiles describe the business information and conditions required to test the application system. You can determine Data Profiles by performing a careful walkthrough of the steps of the scenario. The inputs into each scenario step include business objects (like *customer* or *master demand schedule*), data conditions (actual values, reference to some other document containing values, or even a screen shot), or business rules (the policy or decision drivers that influence the process step), and type and status information (to clarify the data entity).

Together this profile information for a scenario step creates a business condition (also known as a script).

For the purpose of testing, there are three types of Data Profiles:

- **Preexisting Data Profile** — describes the conditions that must already exist for proper execution of a test
- **User Input Data Profile** — describes the data that the user must enter during test execution
- **Expected Output Data Profile** — describes the data conditions that a successful test execution should produce

A properly named scenario may imply the primary business condition (for example, *schedule a non-stocked domestic finished goods item*).

The figure below illustrates the relationships between scenarios, test steps, and data profiles. It may be appropriate to have specific data profiles that meet the unique requirements of each test step.

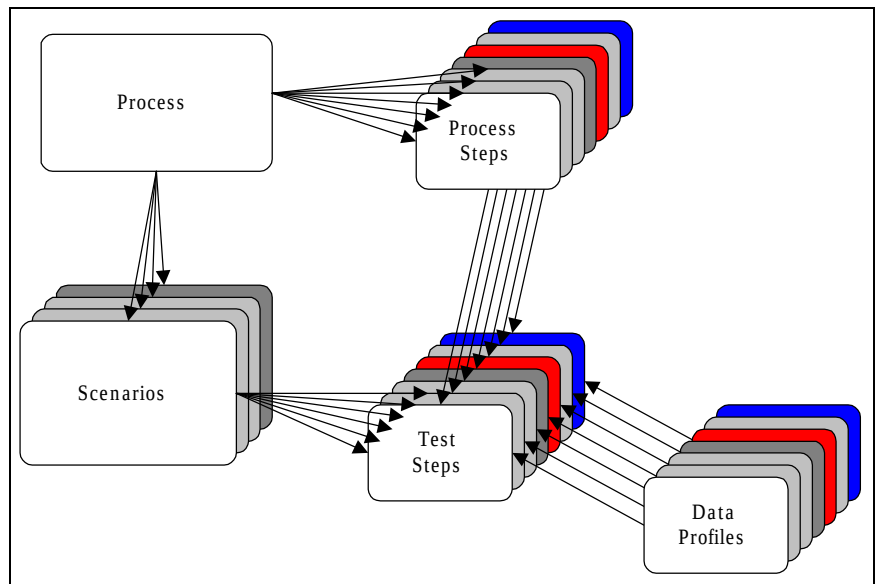


Figure 8-6 Step and Data Profile Relationship

System Test Sequences

Determine the sequence in which the system test specifications should be executed to simulate real-life business transaction processing.

Ideally, an expected output data profile resulting from execution of its associated test specification should serve as the preexisting Data Profile (conditions that must already exist for proper execution of a test) for execution of the next test specification in the test sequence.

Linkage to Other Tasks

The System Test Script is an input to the following tasks:

- TE.050 - Develop Systems Integration Test Script
- TE.100 - Prepare Key Users for Testing
- TE.110 - Perform System Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Business Analyst	40
Tester	30
Developer	20
Technical Analyst	10

Table 8-12 Role Contribution for Develop System Test Script

Deliverable Guidelines

Use the System Test Script to measure the quality of the entire application system.

This deliverable should address the following:

- all system tests to be performed
- the data used in the system test
- system testing errors

Deliverable Components

The System Test Script consists of the following components:

- Overview
- System Test Sequences
- System Test Specifications
- Data Profile
- Defect Log

Overview

This component documents the reasons for the System Test Script and the resources needed to develop them.

System Test Sequences

This component documents the order in which you execute the System Test Specifications to simulate real life business transaction processing.

System Test Specifications

This component defines the test script execution and includes the functional features of the business process. There may be more than one System Test Specification if it is necessary to test more than one response path through the business scenario.

Data Profile

This component describes the business conditions or seeded data necessary in order to system test the business process. There may be more than one Data Profile if it is necessary to test more than one response path through the business process (for example, if you have more than one test specification).

Defect Log

This component facilitates the tracking of each defect uncovered during system testing, as well as the related *fix* documentation recorded by the owning developer once they have corrected the error. By definition, the default log is empty when the System Test Script is created.

Tools

Deliverable Template

Use the System Test Script template to create the deliverable for this task.

Century Date Compliance

To document the review and approval of System Test Script for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Compliance Certificate should be filed with the deliverable in the project library.

TE.050 - Develop Systems Integration Test Script (Optional)

In this task, you develop the test script that validates the integration between your new application system and other third-party and legacy systems.



If your project includes *interfaces* with external systems, you should perform this task.

Deliverable

The deliverable for this task is the **Systems Integration Test Script**. It identifies the detailed steps to be performed to verify the technical and functional integration points between separate systems to confirm that they are able to work together from a process flow and data integration perspective.

Prerequisites

You need the following input for this task:



Integration Fit Analysis (BR.050)

Review the integration fit analysis to gain an understanding of key integration points with external systems. If Conduct Integration Fit Analysis was not performed, this deliverable will not exist. (See the task description for BR.050 for more information on when this task should be performed.)



Application Extensions Functional Design (MD.050)

Review the Application Extensions Functional Design to determine the functional requirements for all interfaces. If Create Application Extensions Functional Design was not performed, this deliverable will not exist. (See the task description for MD.050 for more information on when this task should be performed.)

❑ Application Extensions Technical Design (MD.070)

Review the Application Extensions Technical Design to determine the technical requirements for all interfaces. If Create Application Extensions Technical Design was not performed, this deliverable will not exist. (See the task description for MD.070 for more information on when this task should be performed.)

❑ System Test Script (TE.040)

The System Test Script includes System Test Sequences, System Test Specifications, and Data Profiles for business processes that may integrate with external systems.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Use the Integration Fit Analysis (BR.050) to identify the integration points of the systems to be tested.	
2.	Review the technical and functional designs for all interfaces.	
3.	Develop the Systems Integration Test Specifications.	Systems Integration Test Specifications
4.	Develop the Data Profiles to support the test specifications.	Data Profile
5.	Develop the Systems Integration Test Sequences.	Systems Integration Test Sequences

No.	Task Step	Deliverable Component
6.	Include a Defect Log to be used during testing.	Defect Log
7.	Validate the components of the Systems Integration Test Script.	
8.	Secure acceptance that systems integration test scripts include criteria for Century Date compliance testing.	Acceptance Certificate (PJM.CR.080)

Table 8-13 Task Steps for Develop Systems Integration Test Script

Approach and Techniques

Use the Systems Integration Test Script to test the integration between the target application system and legacy and third-party applications.

Systems Integration Testing

The systems integration test measures the coexistence of your application system with the neighboring application systems in the enterprise with which it must interface. The systems integration test uses test scripts comprised of System Test Sequences from two or more systems that interface. The systems integration test checks the integration points between separate systems to confirm that they are able to work together from a process flow perspective and from a data integration perspective.

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Business System Testing should include test scripts and test case scenarios that check for Century Date compliance of customizations and custom interfaces. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Systems Integration Test Script

The Systems Integration Test Script addresses the following:

- systems integration sequences using converted data
- lock testing simulating user load, executing identical scenarios
- batch window test on full converted data volumes (for example, financial close period, MRP run, and Payroll run)



Suggestion: Since interfaces can be performance bottlenecks, coordinate with Performance Testing (PT) to address potential issues.

Linkage to Other Tasks

The Systems Integration Test Script is an input to the following tasks:

- TE.100 - Prepare Key Users for Testing
- TE.120 - Perform Systems Integration Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Business Analyst	60
System Architect	10
Tester	10
Developer	10
Technical Analyst	10

Table 8-14 Role Contribution for Develop Systems Integration Test Script

Deliverable Guidelines

Use the Systems Integration Test Script to validate the integration between the new application system and legacy and third-party systems.

This deliverable should address the following:

- extending the System Test Script (TE.040) to include external systems data and processing validation
- testing the technical infrastructure using performance-related test scripts which focus on integration points between the systems

Deliverable Components

The System Integration Test Script consists of the following components:

- Overview
- Systems Integration Test Sequences
- Systems Integration Test Specifications

- Data Profile
- Defect Log

Overview

This component documents the reasons for the Systems Integration Test Script and the resources needed to develop it.

Systems Integration Test Sequences

This component documents the order in which The Systems Integration Test Specifications are executed to simulate real-life business transaction processing across and between system boundaries.

Systems Integration Test Specifications

This component defines the test script execution and includes those business processes that span one or more systems. There may be more than one Systems Integration Test Specification for each script if it is necessary to test more than one response path through the business process.

Data Profile

This component describes the business conditions or seed data necessary to test the integration points between systems. There may be more than one Data Profile if it is necessary to test more than one response path through the business process (for example, if there is more than one systems integration test specification).

Defect Log

This component facilitates the tracking of each defect uncovered during systems integration testing, as well as the related *fix* documentation recorded by the owning developer once they have corrected the error. By definition, the default log is empty when the Systems Integration Test Script is created.

Tools

Deliverable Template

Use the Systems Integration Test Script template to create the deliverable for this task.

Century Date Compliance

To document the review and approval of Systems Integration Test Script for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Compliance Certificate should be filed with the deliverable in the project library.

TE.060 - Prepare Testing Environments (Core)

In this task, you install and configure one or more testing environments to support all testing activities.

Deliverable

The deliverable for this task is the **Testing Environments**. These environments include the platforms, software, and utilities used to configure various environments where testing activities are supported.

Prerequisites

You need the following input for this task:

☐ **Architecture Requirements and Strategy (TA.010)**

The Architecture Requirements and Strategy provides information on the number of testing environments to be setup and other relevant information about the testing environments.

☐ **Custom Database Objects (MD.100)**

Data definition language (DDL) scripts are required to create the tables and other database objects for the applications and custom modules. If disk space is limited, adjust the storage parameters before running the scripts. If Create Database Extensions was not performed, this deliverable will not exist. (See the task description for MD.100 for more information on when this task should be performed.)

☐ **Module Source Code (MD.110)**

Load your Module Source Code into each Testing Environment and begin system testing the new source code. If Create Application Extension Models was not performed, this deliverable will not exist. (See the task description for MD.110 for more information on when this task should be performed.)

Testing Requirements and Strategy (TE.010)

The Testing Requirements and Strategy provides the details describing the environments required to support each testing task.

Task Steps

The steps for this task are as follows (repeat these steps for each Testing Environment you create):

No.	Task Step	Deliverable Component
1.	Review Architecture Requirements and Strategy (TA.010) to understand the strategy for deployment for project environments in general, and the testing environments in particular.	
2.	Update the introduction component to reflect the testing environment hosting and environment sharing strategy per the Architecture and Requirements Strategy (TA.010).	Introduction
3.	Update all of the tables in the database tier, applications tier, and desktop client tier sections of the Environment - Testing component to reflect the configuration of all servers within the database, applications, and desktop client tiers.	Environment - Testing
4.	List any other software applications needed to support business system testing	Other Applications

No.	Task Step	Deliverable Component
5.	Update the testing tools component to reflect the configuration for each testing tool.	Testing Tools
6.	Set up the Testing Environment.	
7.	Install the application extension programs and testing tools.	

Table 8-15 Task Steps for Prepare Testing Environments

Approach and Techniques

The Testing Environments are used to support all of the testing activities.

Environment Configuration

Configure the test environments based on the latest application setups from the Application Setup Documents (BR.100). Before you begin testing in each environment, you should:

1. Set up the applications.
2. Decide on the real business data to use.
3. Plan the volume of data (for example, converted data).
4. Verify that all application parameters have been set up to enable transactions.
5. Verify that sufficient business data has been entered to demonstrate application features.
6. Provide access to definition and setup screens to appropriate users.
7. Enable links to non-Oracle or custom systems (if applicable).

8. Make reporting, data query, and testing tools available in the testing environments to verify correct results against the expected results of the test scripts.
9. Record all changes and updates to the test environment in the Test Environment Setup Log template to help you maintain the integrity of the objects that have been installed or updated.

Multiple Environments

You may need to create multiple test environments to accommodate the six types of business system testing:

- unit test
- link test
- installation test
- system test
- systems integration test
- acceptance test

Unit Test (TE.070)

You can unit test application extension components in the development environment or in a dedicated testing environment. For application extension components that access standard application tables, the environment is generally a full installation of Oracle Applications. If custom application extensions only access custom tables, you can unit test against a smaller dedicated database on a workgroup server or workstation.



Warning: If multiple testers will be working in the same environment, someone must manage the data so that multiple unit testers do not over write other people's test data. To accomplish this, predetermine the data value ranges for each unit tester.

Link Test (TE.080)

You can perform link tests in a dedicated environment or in the environment you will eventually use for the system test. In either case, it should be separate from the unit test environment. Since business flows are the basis of link testing, you may want to copy your mapping database and use it for link testing.

Installation Test (TE.090)

During the installation test, you install custom application extensions in the system test environment. This test serves as a dry run of the installation steps in preparation for configuring the production systems.

System Test (TE.110)

The system test environment should be separate from other environments or refreshed (so that it is clean before beginning the test). Since you always perform system testing after unit and link testing is complete, you may be able to reuse an existing environment. However, you need an environment where you can unit and link test application extensions after developers apply corrections to problems discovered in the system test. Regression testing must be done in an environment that is separate from the system and systems integration test environments; however it can be a recycled link test environment. You can determine regression test scope by reviewing the testing strategy as you perform a regression test to stabilize application defects before reintroducing the improved application back into the system test environment. Based on the testing strategy, regression testing may require unit test scripts and selected link test scripts.



Suggestion: You should plan to keep the test environment where you perform regression testing available after cutover to the production environment so that you can retest key business flows after applying future upgrades and patches to the applications.

The system test environment includes the following:

- configured server with application
- configured workstations with application
- installed customizations
- configured database
- loaded manual data
- loaded scripted data
- loaded converted data
- loaded help text (optional)

Systems Integration Test (TE.120)

The systems integration test environment can be separate from all other environments or it can be an extension of the system test environment. The systems integration test environment includes the following:

- configured servers with multiple application systems
- configured workstations with multiple application systems
- installed customizations (including interfaces)
- configured databases
- loaded manual data
- loaded converted data

Acceptance Test (TE.130)

The acceptance test team performs acceptance tests on the production environment or in an environment that mirrors the production environment. The benefit of testing in the true production environment is that users test the servers, organization's networks, and printers in place; nothing must be dismantled, moved, reconfigured, or reinstalled between the acceptance test and production cutover.

If the data conversion and acceptance test tasks need to occur concurrently, then the acceptance test environment and the production installation must be different instances. Otherwise, the acceptance test environment should be the production installation. This gives both the development team and the users the confidence that the acceptance test represents acceptance of the production installation.



Suggestion: Advise the system architect of your testing environment requirements so that they can be included in the architecture strategy.

Import/Export

If you have entered your setups into a master setup environment during Map Business Requirements (BR.030), you export that data and import it into your Testing Environments. If testing uncovers issues that affect setups, you must change the setups in your master environment so that they will be correct for production.

Linkage to Other Tasks

The Testing Environments are an input to the following tasks:

- TA.110 - Define System Capacity Plan
- TE.070 - Perform Unit Test
- TE.080 - Perform Link Test
- TE.090 - Perform Installation Test
- TE.100 - Prepare Key Users for Testing
- TE.110 - Perform System Test
- TE.120 - Perform Systems Integration Test
- PM.110 - Refine Production System

Role Contribution

The percentage of total task time required for each role follows:

Role	%
System Administrator	60
Database Administrator	30
Tester	10

Table 8-16 Role Contribution for Prepare Testing Environments

Deliverable Guidelines

In addition to Testing Environments, this task also produces a supporting deliverable — the Testing Environment Setup Log. The setup log is used to record information that is necessary for preparing and configuring all Testing Environments.

This deliverable should address the following:

- configuration of required servers
- workstation configuration
- database configuration
- manual data load
- scripted data load
- converted data load
- help text data load
- application object load

Deliverable Components

The Testing Environment Setup Log template consists of the following components:

- Introduction
- Environment - Testing
- Other Applications
- Testing Tools

Introduction

This component describes purpose for the testing environment and the detailed configuration approach taken to implement the environment.

Environment - Testing

This component describes the particulars of the testing environment being configured. This information describes many things including what type of testing will be performed, such as, unit/link, system / systems integration, and acceptance.

Other Applications

This component describes the configuration of any other software applications that are required to support the project.

Testing Tools

This component describes the configuration of any other software applications that are required to support the testing activities of the project.

Tools

Deliverable Template

Use the Testing Environment Setup Log template to create the supporting deliverable for this task.

TE.070 - Perform Unit Test (Optional)

In this task, you test application extension components on an individual basis to verify that the inputs, outputs, and processing logic of each application extension component functions without errors. Unit testing is performed in either the development environment or a testing environment.

The goal is to find errors in the smallest unit of software before you logically link it into larger units. If successful, subsequent link testing should only reveal errors related to the integration between application extensions.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Unit-Tested Modules**. These modules include application extension source code that has been tested to verify that the inputs, outputs, and processing logic of each application extension component functions without errors.

Prerequisites

You need the following input for this task:



Build Standards (MD.040)

Use the Build Standards to validate conformance to cosmetic and user interface standards. The Unit Test Script (TE.020) and Application Extensions Functional Design (MD.050) should already encapsulate the standards, but you may find the official reference useful when you discover a nonconformance. If Define Build Standards was not performed, this deliverable will not exist. (See the task description for MD.040 for more information on when this task should be performed.)

☐ **Approved Designs (MD.080)**

Obtain final signoff of completed designs prior to producing the Technical Reference Manual. If Review Functional and Technical Designs was not performed, this deliverable will not exist. (See the task description for MD.080 for more information on when this task should be performed.)

☐ **Module Source Code (MD.110)**

You must have the coded application extension components with the corresponding executable files before you can unit test them. If you are testing incremental functionality, you must code the functions that are subject to testing in this iteration. For some types of application extensions (like descriptive flexfields), the *source code* consists of setup data entered into Oracle Applications forms. If Create Application Extension Modules was not performed, this deliverable will not exist. (See the task description for MD.110 for more information on when this task should be performed.)

☐ **Testing Requirements and Strategy (TE.010)**

The Testing Requirements and Strategy provides the testing approach and testing requirements and strategy for business system testing.

☐ **Unit Test Script (TE.020)**

The Unit Test Script provides the prerequisite seed data, the detailed steps, and the expected results of the test. If Develop Unit Test Script was not performed, this deliverable will not exist. (See the task description for TE.020 for more information on when this task should be performed.)

☐ **Testing Environments (TE.060)**

You may perform unit tests directly in the development environment or in a dedicated unit test environment. In either case, the application extension components you are testing must be accessible in this environment.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Enter or confirm the required setup data.	Completed Form Information
2.	Review the Unit Test Script (TE.020) and complete the form information.	
3.	Using the Unit Test Script, identify the functional items to be tested.	
4.	Using the Unit Test Script, identify the cosmetic items to be tested.	
5.	Perform the unit test and document the unit test results in the <i>actual results</i> column.	Unit Test Specifications
6.	Fix errors in the application extension.	
7.	Repeat the unit tests (if needed).	
8.	Place the corrected application extension under configuration control	Acceptance Certificate (PJM.CR.080)
9.	Secure acceptance of unit test results.	

Table 8-17 Task Steps for Perform Unit Test

Approach and Techniques

The Unit-Tested Modules are the result of performing unit testing on custom application extension components.

The figure below shows an example of how an application extension to the standard Oracle Purchasing module can be comprised of three separate components (form, table, and report). Each of these components is unit tested separately. Later, during Perform Link Test (TE.080), these three components are tested together as part of a link test of the business flow.

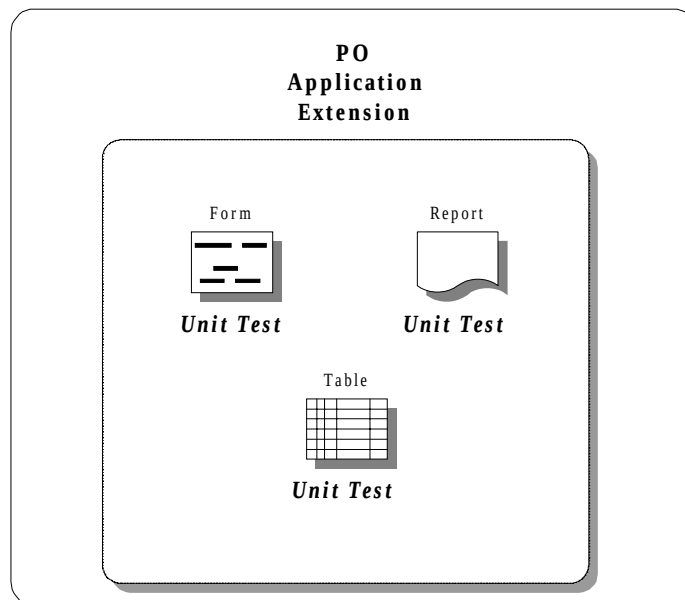


Figure 8-7 Purchasing (PO) Application Extension Components Needing Unit Testing

Test Results Documentation

Use the Unit Test Specifications component from the Unit Test Script (TE.020) to document the actual unit test results. More formal documentation, such as the Application Extensions Technical Design (MD.070) and Technical Reference Manual (DO.080), should be revised to include any fixes to the original custom application extension code and any required changes to design documents.

Follow the Unit Test Script (TE.020) to execute your test. Record actual unit test results, specific data entered, and other comments in the *actual results* column of the Unit Test Specifications component. Describe error situations clearly and write down error codes and messages exactly as they appear so that a developer can easily reproduce and identify them. You can also include screen shots and sample report output to support your results.

Iterative Testing

Unit testing is an iterative process tightly integrated with coding. Before beginning the formal unit tests defined by this task, the developer who codes an application extension performs some preliminary testing of the application extension as part of Create Application Extension Modules (MD.110). For best results, another developer or dedicated tester should perform the final unit test to validate the application extension. A developer codes an application extension or part of it and unit tests it, performs bug fixes, and retests it until the program is free of errors. More functionality is then added and the process is repeated. The following figure illustrates this process.

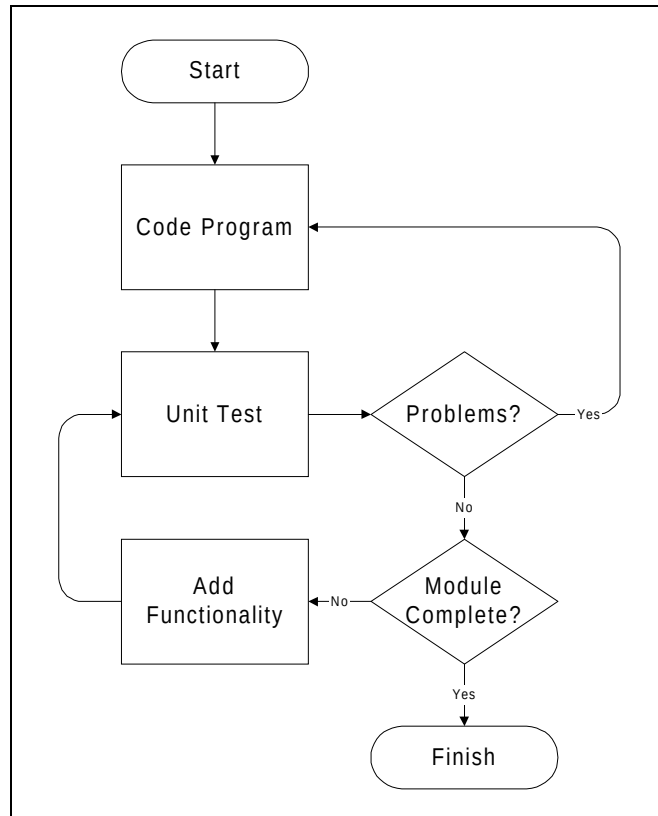


Figure 8-8 The Incremental Code and Unit Test Cycle

Once the developer is comfortable with the completed application extension component, it is turned over to another developer or tester for final unit testing. In each round of unit testing, the tester must communicate defects back to the developer, who fixes the problems and then releases a new version for retesting. Therefore, even though the two tasks are separate in AIM, consider them to be one integrated activity.

Make duplicates of the Unit Test Specifications component (TE.020) before you start, so that you can easily record the results of several test iterations.

Completion Criteria

Testing is complete when you execute the test script and your actual results match the expected results. At the completion of unit testing, you should have:

- fully tested all code for application extension components
- documented test results completely
- all errors completely resolved and documented

With the completion of unit testing, the developer should verify that:

- all errors are corrected
- new functionality is implemented and documented
- design documents are revised as needed



Suggestion: At the end of unit testing, bring in a key user to validate the application extension to identify design and build flaws early so they can be corrected prior to system testing.

Test Results Acceptance

Follow the method outlined in the Project Management Plan (PJM.CR.010) for securing acceptance of test results.

Linkage to Other Tasks

The Unit-Tested Modules are an input to the following task:

- TE.080 - Perform Link Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Developer	80
Technical Analyst	20

Table 8-18 Role Contribution for Perform Unit Test

Deliverable Guidelines

Use the Unit-Tested Modules to show that inputs, outputs, and processing logic of each individual custom application extension functions without errors. Later, these Unit-Tested Modules are combined with other custom application extension components to become Link-Tested Modules (TE.080).

This deliverable should address the following:

- actual unit test results
- problems with unit testing
- individual application extension components that are unit tested and ready for link testing

If you find it necessary to update your Unit Test Script (TE.020), consider renumbering the steps in the original Unit Test Script. The final version of the test script should be filed in the project’s documentation library.

Deliverable Components

The Unit-Tested Modules consist of the following components:

- Completed Form Information
- Unit Test Specifications (from the Unit Test Script - TE.020)

Completed Form Information

This component lists specific information about the form being unit tested (such as the name of the application, form short name, form title, tester's name, and date tested).

Unit Test Specifications

This component records the results of each unit test.

Tools

Deliverable Template

A deliverable template is not provided for this task. Record your results in the columns provided in the Unit Test Specifications component of the Unit Test Script (TE.020).

Query Tools

Sometimes you can only evaluate the results of a single application extension component by examining data in the database (particularly with database triggers and interfaces). SQL*Plus, GUI query tools, or other ad hoc query tools are invaluable for this process.

TE.080 - Perform Link Test (Optional)

In this task, you test several application extension components together as part of a business flow to uncover any integration problems with other application extension components that provide or use the data manipulated by the target component. Link testing is performed in either the development environment or a testing environment.

The scope of each link test typically includes the set of components that support or are affected by a single application extension. An application extension is defined for each gap identified during requirements mapping and is described by a functional design and corresponding technical design document.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Link-Tested Modules**. These modules include application extension source code that has been tested to verify that the links between application extension components and standard Oracle Applications modules function without errors.

Prerequisites

You need the following input for this task:

☐ **Approved Designs (MD.080)**

Obtain final signoff of completed designs prior to producing the Technical Reference Manual. If Review Functional and Technical Designs was not performed, this deliverable will not exist. (See the task description for MD.080 for more information on when this task should be performed.)

☐ **Link Test Script (TE.030)**

The Link Test Script provides prerequisite seed data, detailed test steps, and the expected results of your link test. If Develop Link Test Script was not performed, this deliverable will not exist. (See the task description for TE.030 for more information on when this task should be performed.)

☐ **Testing Environments (TE.060)**

The configured Testing Environments must provide access to all application extension components included in the test and must be seeded with appropriate setup data.

☐ **Unit-Tested Modules (TE.070)**

Custom application extension components must be unit tested before you can perform link testing. If Perform Unit Test was not performed, this deliverable will not exist. (See the task description for TE.070 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Enter and confirm the required setup data.	
2.	Execute the Link Test Script (TE.030).	
3.	Document actual results.	Link Test Specifications
4.	Review the results with the developer.	
5.	Fix errors in the program, setup, or configuration.	

No.	Task Step	Deliverable Component
6.	Update the Application Extensions Functional Design (MD.050) and Application Extensions Technical Design (MD.070), if needed.	
7.	Reregister corrected application extension components under configuration control.	
8.	Secure acceptance of the link test results.	Acceptance Certificate (PJM.CR.080)

Table 8-19 Task Steps for Perform Link Test

Approach and Techniques

The Link-Tested Modules are the result of performing link testing on custom application extension components. The name *link test* comes from testing the connections, or *links*, between application extension components. Link testing should uncover all potential problems with the custom application extension code.

The figure below shows how custom modules are linked together to perform the link test for a Purchasing (PO) application extension.

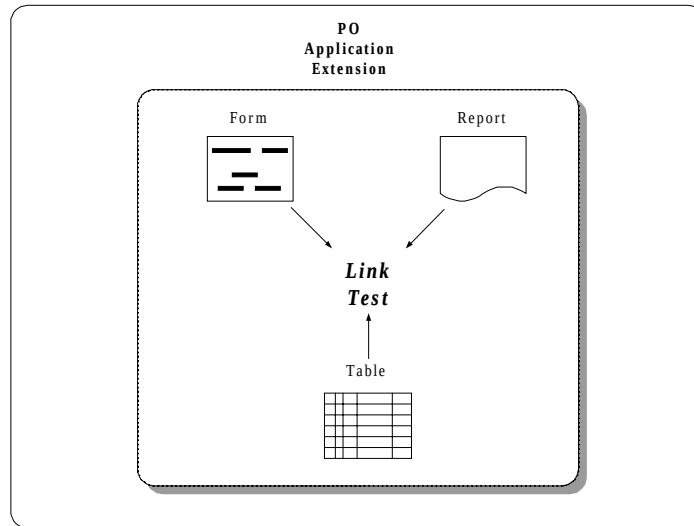


Figure 8-9 Link Testing Components Within a Purchasing (PO) Application Extension

Test Results Documentation

Record actual results, specific data entered, and other comments in the *actual results* column of the Link Test Specifications component of the Link Test Script (TE.030). You can also include screen shots and sample report output to support your results.

Link testing verifies that all components of a custom application extension work together properly and integrate seamlessly with standard application modules. For example, an application extension to support unique purchasing requirements may include a new database table, a new form, and a new report. Each of these custom components would have been tested individually during Perform Unit Test (TE.070). The link test employs a business scenario to test the end-to-end flow of all of the custom application extension components together.

The link test also extends the test to include other standard Oracle Application modules that either precede or follow the steps in the flow that exercise the application extensions. Link testing should incorporate functions that rely on data manipulated by the application extensions. The following figure illustrates this point.

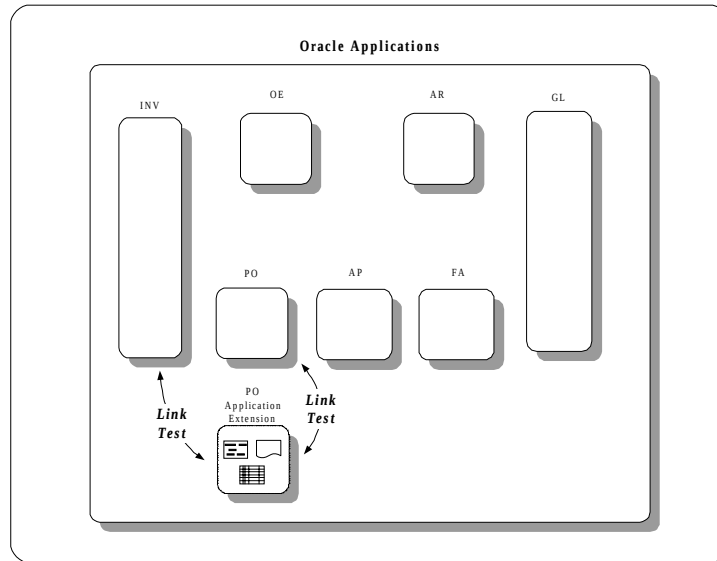


Figure 8-10 Link Testing the Purchasing (PO) Application Extension with Standard Application Modules

Build up to the full link test by testing programs that work together — first in pairs, then in sets of three, four, and so on. Test simple scenarios first and then build more complex cases with planned exceptions to test warning and error trapping. Finally, test the customization in the context of other related application functions. For example, if your customization creates forecast information in Oracle Master Scheduling/MRP, try loading the forecast in a master demand schedule, then run the MRP process and review the output.

Although the already developed Link Test Script (TE.030) primarily dictates your link testing process, you may think of other variations and scenarios that you should test. Test these new scenarios and document each one in a new Link Test Script (TE.030).

Test Results Acceptance

Follow the method outlined in the Project Management Plan (PJM.CR.010) for securing acceptance of link test results.

Linkage to Other Tasks

The Link-Tested Modules are an input to the following task:

- TE.090 - Perform Installation Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Developer	60
Business Analyst	20
Technical Analyst	20
User	*

Table 8-20 Role Contribution for Perform Link Test

* Participation level not estimated.

Deliverable Guidelines

Use the Link Test Specifications component of the Link Test Script (TE.030) to document that the connections or *links* between custom application extensions have been tested and are free from error. The final version of the Link Test Script should be filed with other test scripts in the project documentation.

This deliverable should address the following:

- actual link test results
- problems with link testing
- individual custom application extensions that are link tested and ready for Perform System Test (TE.110)

Deliverable Components

Update the following component from the Link Test Script (TE.030):

- Link Test Specifications

Link Test Specifications

This component (from Link Test Script - TE.030) records the actual results from performing the link tests.

Tools

Deliverable Template

A deliverable template is not provided for this task.

TE.090 - Perform Installation Test (Optional)

In this task, you install application extensions in the system test environment to test the installation routines and refine them for the eventual production environment.



If your project includes either *customizations* to standard functionality, *interfaces* with external systems, or both, you should perform this task.

Deliverable

The deliverable for this task is the **Tested Installation Routines**. These routines provide evidence that the application extensions can be added to the system test environment by following the installation steps found in the Installation Routines (MD.120).

Prerequisites

You need the following input for this task:



Installation Routines (MD.120)

The Installation Routines and supporting installation instructions are your guides to the installation process. If Create Installation Routines was not performed, this deliverable will not exist. (See the task description for MD.120 for more information on when this task should be performed.)



Testing Requirements and Strategy (TE.010)

The Testing Requirements and Strategy provides the testing approach and testing requirements and strategy for business system testing.



Testing Environments (TE.060)

The configured Testing Environments include the database, all applications, seed data, and setups. Initially, these Testing Environments do not include custom application extension components.

❑ Link-Tested Modules (TE.080)

Programs pass link testing before you test the installation process. If Perform Link Test was not performed, this deliverable will not exist. (See the task description for TE.080 for more information on when this task should be performed.)

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the Installation Instructions component of the Installation Routines (MD.120).	
2.	Follow instructions and execute the Installation Routines.	
3.	Document actual results.	Installation Routines
4.	Review the results with the developer.	
5.	Fix errors in the installation routines and instructions.	(Corrected) Installation Routines
6.	Secure acceptance of installation test results	Acceptance Certificate (PJM.CR.080)

Table 8-21 Task Steps for Perform Installation Test

Approach and Techniques

This task serves as a dry run of the installation steps in preparation for configuring the production system. This task is required only if you have developed application extensions.

Use the Tested Installation Routines to prove that the Installation Routines (MD.120) used to install custom application extensions in a system test environment work properly. The figure below shows the approach for using installation routines to load custom application extensions into the system test environment.

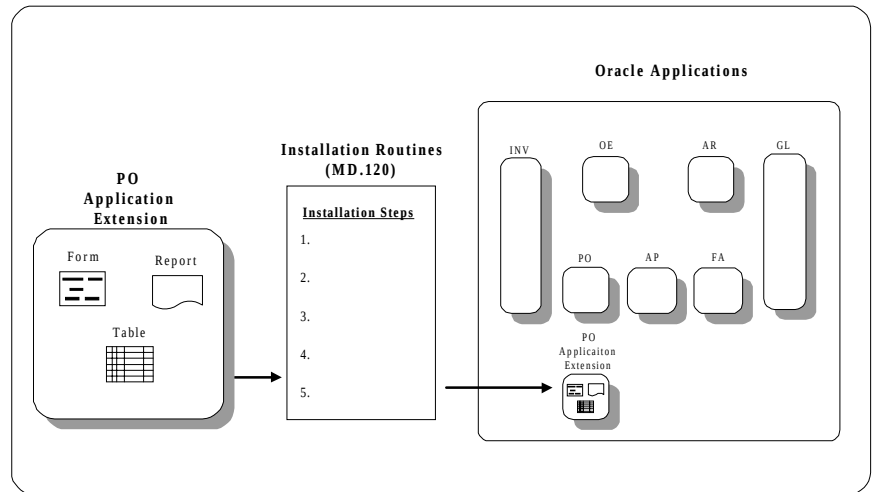


Figure 8-11 Installation Test Diagram



Attention: This task does not cover installation of the Oracle servers, tools, and applications. These steps take place during Prepare Testing Environments (TE.060).

Test Results Documentation

Since there are no formal test scripts for this task, use the Installation Instructions component of the Installation Routines (MD.120) to record your test results. Make comments directly on the Installation Instructions. Review your notes with the developer, so that the Installation Routines and Installation Instructions can be updated accordingly. At the completion of your testing, you should have an accurate set of Installation Routines and Installation Instructions.

One of the responsibilities of the developer is to create and unit test installation routines and procedures to support each customization. A common mistake is to have the developer who coded the custom application extensions also install the customization in the system test environment. This creates a potential problem because the original developer may have transitioned off the project and the system administrator, or someone else, must reinstall the customization without accurate installation instructions.

The approach and techniques for creating Installation Routines are described in Create Installation Routines (MD.120). During testing, you should plan to follow the Installation Instructions without help from the developer unless you run into a problem you cannot resolve. The goal is to be able to install applications extensions by following only the Installation Instructions provided. This may take several attempts.

Environment Preparation

Some aspects of custom application extension installation, such as concurrent program registration, are difficult to undo. If you need to retest the installation process or if one of the steps results in an incorrect configuration, you can restore the target environment from a backup and start over.

Test Results Acceptance

Follow the method outlined in the Project Management Plan (PJM.CR.010) for securing acceptance of installation test results.

Linkage to Other Tasks

The Tested Installation Routines are an input to the following task:

- TE.110 - Perform System Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
System Administrator	80
Developer	20

Table 8-22 Role Contribution for Perform Installation Test

Deliverable Guidelines

The Tested Installation Routines show that you have Installation Routines (MD.120) that can be relied upon when installing application extensions in the production environment. Testing your Installation Routines is done in conjunction with setting up the system test environment to perform the system test.

This deliverable should address the following:

- the adequacy of the Installation Routines

Deliverable Components

Update the following component from Installation Routines (MD.120):

- Installation Instructions

Installation Instructions

This component is produced during Create Installation Routines (MD.120). Any difficulties noted during the installation of applications extensions in the testing environment should be noted directly on the Installation Instructions to reflect the corrected instructions.

Tools

Deliverable Template

A deliverable template is not provided for this task.

TE.100 - Prepare Key Users for Testing (Core)

In this task, you provide basic training to key users participating in Business System Testing. A test environment is used to prepare key users for testing.

Deliverable

The deliverable for this task is **Prepared Key Users**. These users have been trained on the new system and are able to perform the system test for their business process area.

Prerequisites

You need the following input for this task:

☐ **User Reference Manual (DO.060)**

Use the User Reference Manual to obtain organization specific information about the functionality of the application extensions. If Publish User Reference Manual was not performed, this deliverable will not exist. (See the task description for DO.060 for more information on when this task should be performed.)

☐ **User Guide (DO.070)**

A User Guide is prepared and given to key users to help prepare them for system testing.

☐ **Testing Requirements and Strategy (TE.010)**

The Testing Requirements and Strategy provides the testing approach and strategy for Business System Testing.

☐ **System Test Script (TE.040)**

Review the System Test Script with key users prior to executing the system test.

☐ **Systems Integration Test Script (TE.050)**

Review the Systems Integration Test Script with key users prior to executing the systems integration test.

☐ **Testing Environments (TE.060)**

Prior to executing the system test, familiarize key users with the Testing Environments.

☐ **Production Support Infrastructure Design (PM.020)**

Users should review the new internal support procedures so they are familiar with how to handle problems encountered during Business System Testing.

Optional

You may need the following input for this task:

☐ **Business Requirements Scenarios (RD.050)**

The Business Requirements Scenarios may provide input regarding which users should participate in the testing for each business area. If Gather Business Requirements was not performed, this deliverable will not exist. (See the task description for RD.050 for more information on when this task should be performed.)



Task Steps

The steps for this task are as follows:

<u>No.</u>	<u>Task Step</u>	<u>Deliverable Component</u>
1.	Identify key users for the testing process.	
2.	Review the testing requirements and strategy, approach, and techniques.	

No.	Task Step	Deliverable Component
3.	Introduce key users to testing techniques.	
4.	Provide an overview of specific business processes.	
5.	Create or collect sample test data.	
6.	Instruct users about new custom features.	
7.	Review the business system procedures.	
8.	Review the System Test Script (TE.040).	
9.	Review the user support procedures.	

Table 8-23 Task Steps for Prepare Key Users for Testing

Approach and Techniques

This task should communicate any necessary background information and introduce materials that would be helpful for performing testing.

Familiarize users with prior and subsequent process steps so they understand the context of Business System Testing. Discuss the testing in terms of three major sections:

- simple business flows (normal transactions carried across organizational boundaries)
- exceptions (like error correction)
- volume testing (simulating live operation with multiple users active on the same and conflicting programs)

The objective of this task is to equip key users with enough information so they can take a primary role in testing the overall business solution. This select audience must come away from this task with a clear understanding of:

- testing objectives
- testing techniques
- use of testing tools
- system login
- steps to perform the tests
- navigation through the applications
- analysis of results
- definition of terms (for example, test cycles, business system, and conference room pilot)

Formal Test Instruction Class

Discuss the objectives of the test, the test script, and the techniques to validate the business system. Communicate the following testing guidelines:

- Test the standard or discrete functions first.
- Validate interim steps using standard reports and inquiries for each business system test.
- Perform new setup entries (super users only).
- Perform reversing transactions (undo forward).
- Perform transactions and record results.
- Perform adjustments and modifications that update the transactions or entries.
- Perform cancellations (if possible).
- Assess net results of operations and finance.
- Stress the significance of the period close process and the impact throughout the organization.
- Review test analysis techniques (how to interpret the results).

- Incorporate real data (current system documents such as purchase orders, and so on) within test script.
- Link test plans into business system test cycles to represent the process flow.

Linkage to Other Tasks

The Prepared Key Users participate in the following tasks:

- TE.110 - Perform System Test
- TE.120 - Perform Systems Integration Test

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Tester	50
Business Analyst	30
System Administrator	20
Business Line Manager	*
Client Staff Member	*

Table 8-24 Role Contribution for Prepare Key Users for Testing

* Participation level not estimated.

Deliverable Guidelines

Prepared Key Users are knowledgeable about the new system's features and are ready to perform the system test (TE.110) and systems integration test (TE.120).

This deliverable should address the following:

- the training necessary for key users to perform their testing tasks
- instructions on when to use project-specific documentation
- instructions on how to read a test script and how to perform system testing

Tools

Deliverable Template

A deliverable template is not provided for this task.

TE.110 - Perform System Test (Core)

In this task, you test the integration of all business system flows within the target application system, including all standard and custom processes and reports. This task is equivalent to a full conference room pilot (CRP) where the environment simulates the future production environment. The system test is performed in a test environment.

Deliverable

The deliverable for this task is the **System-Tested Applications**. These validate that the new system meets defined business requirements and supports execution of business processes.

Prerequisites

You need the following input for this task:

☐ **Manual Conversion Procedures (CV.050)**

If your system test includes data that is converted manually, the Manual Conversion Procedures provide information about manually converting data that will be used in the system test. If Define Manual Conversion Procedures was not performed, this deliverable will not exist. (See the task description for CV.050 for more information on when this task should be performed.)

☐ **Validation-Tested Conversion Programs (CV.110)**

After the converted data has passed the conversion validation test, the data is ready for use in the system test. If Perform Conversion Validation Test was not performed, this deliverable will not exist. (See the task description for CV.110 for more information on when this task should be performed.)

☐ **User Reference Manual (DO.060)**

The User Reference Manual provides a detailed description of the use of each application extension that you test for validity during the system test. If Publish User Reference Manual was not performed, this

deliverable will not exist. (See the task description for DO.060 for more information on when this task should be performed.)

☐ **User Guide (DO.070)**

The User Guide provides a description of responses to business events that you test for validity during the system test. If Publish User Guide was not performed, this deliverable will not exist. (See the task description for DO.070 for more information on when this task should be performed.)

☐ **Testing Requirements and Strategy (TE.010)**

The Testing Requirements and Strategy provides the testing approach and testing requirements strategy for business system testing.

☐ **System Test Script (TE.040)**

The System Test Script comprises the prerequisite test data and test steps to be executed during the system test.

☐ **Testing Environments (TE.060)**

The Testing Environment should closely resemble the future production environment.

☐ **Tested Installation Routines (TE.090)**

Custom application extension components must be installed in the Testing Environment prior to the start of system testing. If Perform Installation Test was not performed, this deliverable will not exist. (See the task description for TE.090 for more information on when this task should be performed.)

☐ **Prepared Key Users (TE.100)**

Key users trained in specific areas of responsibility should execute the System Test Script (TE.040) using draft versions of procedures.

☐ **Production Support Infrastructure Design (PM.020)**

When you encounter problems during the testing process, you should reference and validate the new internal support procedures.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Generate the test report for system test template and complete the general information sections.	Introduction; System Test Summary; System Test Method; System Test Environment
2.	Review the responsibilities for the system test with the team.	Roles and Responsibilities
3.	Validate the system test environment.	
4.	Execute the system test script.	
5.	Document the detailed test results.	System Test Specifications (TE.040)
6.	Summarize the results of each test sequence.	System Test Results
7.	Review the results with the developer.	
8.	Perform regression tests.	
9.	Determine required actions for problem resolution.	System Test Actions
10.	Make recommendations for applications changes or corrections based on system testing.	System Test Change Recommendations

No.	Task Step	Deliverable Component
11.	Secure acceptance of system test results.	Acceptance Certificate (PJM.CR.080)

Table 8-25 Task Steps for Perform System Test

Approach and Techniques

The purpose of the system test is to test the operation and integration of the business system processes. The figure below illustrates the approach for testing processes that occur between custom application extensions and standard application modules.

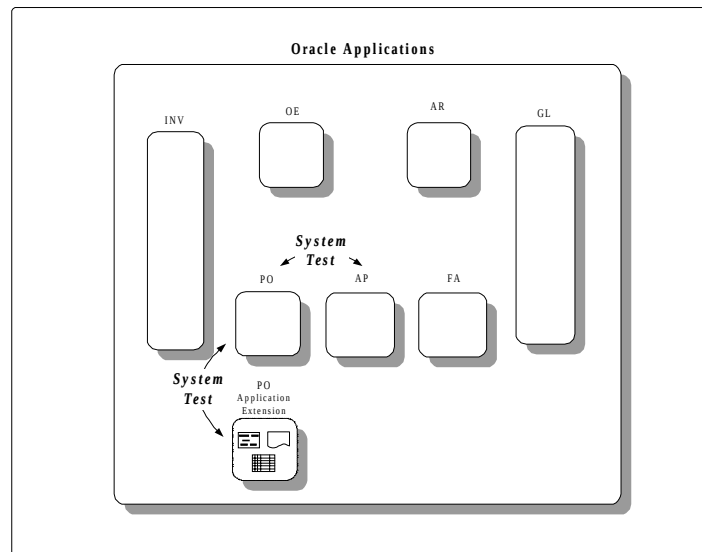


Figure 8-12 System Test Diagram

During Perform System Test, users develop confidence in the programs and the overall usability of the target application system. It is important to test operating routines and procedures as well as computer programs, thereby simulating business operations, not just systems.

The repetitive nature of this task allows the project team to test each business flow and incrementally improve the system until they reach the desired level of success. Test each process flow until you achieve

success. Then integrate the script with each other across process boundaries.

The supporting deliverable for this task is the Test Report for System Test. This report includes a summary of the goals and objectives of the test, the business scope, a description of the test scenarios and processes, the final results, status, and recommendations.

System test results are useful for implementing revisions to the business solutions. The audience for this task is the project team and steering committee. In addition to collecting and organizing the completed System Test Script (TE.040), this deliverable should contain:

- summary (including statistics) of successes of tests organized by process
- test method overview
- critical success factors for certifying the system based on testing
- changes to acceptance criteria based on the new factors or changes to the business system
- impact assessment of changing procedures and organization policy
- actions and assignments relating to changes to procedures and policies
- summary of retraining required to support policy and procedure changes
- highlights of critical failure in code
- actions, assignments, and estimates relating to the rework of code
- summary of programming changes
- summary of repeating select business systems tests based on these changes
- completed test scripts as an appendix
- a list of technical issues communicated and logged with Oracle Support

Successful Business System Testing should verify the following:

- the system successfully supports business operations
- all results are predictable and repeatable
- variations of the test script produces expected results
- data is verifiable through alternative means across the business system
- relevant key performance indicators can be validated
- support procedures intended for production are accurate
- modifications to procedures, policy, and system are fully implemented
- century date compliance

Conduct system testing using the testing guidelines and standards adopted by the project team and referenced in the Project Management Plan (PJM.CR.010).

Relationship to Business Solution Testing

Test Business Solutions (BR.080) and Perform System Test are similar tasks — they are both based on business processes and they both employ the use of test scripts and their components. They differ, however, in the test data used, the level of applications setup validation, and in the user skills and level of documentation required to thoroughly execute the tests.

System testing involves validation of converted data, whereas business solution testing uses manually keyed sample data, because generally converted data is not available at that time. System testing also validates the integration of the entire suite of application configuration setups that were individually tested during business solution testing. System testing includes all custom application extensions that are not available (perhaps not even designed) during business solution testing. System testing, however, does not include testing interfaces to legacy and third-party systems. These interfaces, if they exist, are tested during Perform Systems Integration Test (TE.120).

The skills required for the system testing team are also similar to those required for solution testing, except that system testing requires more discipline with regard to recording details (actual results), while solution testing involves validating that a given process can be

supported. System testing also tends to be more comprehensive from the standpoint that tests repeat iteratively, and for all action items resulting from the tests, you must resolve, retest, and document any changes to application setup for implementation in the production environment.

Testing Teams Organization

Organize system testing teams by assigning specific job roles to individual testers. This prevents testers from inappropriately carrying forward knowledge of past test steps that another role would not possess. Key users should assume roles that match their actual jobs. These users were prepared to participate in testing in the previous task, Prepare Key Users for Testing (TE.100). They should be familiar with both the Oracle Applications modules and custom application extension functionality, as well as the testing process itself.

Always organize testing sessions for efficiency by publishing a thorough agenda that includes:

- session location and duration
- role assignments
- tests to be executed
- activities and sequence
- props and other physical setups that help simulate realism
- expected outputs and criteria for successful closure of the testing session

Conference Room Pilot

A formal conference room pilot (CRP) is the most effective means of executing the system test.

Role Playing

Use role playing and props to simulate business processes. For example, when a purchasing agent prints a purchase order, has it delivered to the *vendor*, who responds by shipping products and *mailing* an invoice. Furthermore, try putting the purchase order into a window envelope. This might reveal that the printed address does not display completely through the window — a subtlety that could easily be overlooked with more casual testing. Even though actions such as these

may only involve passing paper across a table, they are intended to mimic real life as much as possible in an attempt to flush out potential problems with the system.

Keep verbal communication to a minimum, except when it is truly part of the process. For example, avoid the temptation to say to the shipping clerk, "I have just loaded some orders into the system for a Z500-32X configuration. Try shipping them. "The process and test plans should dictate when the shipping clerk checks for orders and picks products to ship. The spoken communication would not take place in real life.

Parallel Process Flows

Where processes overlap or are independent, it is possible to perform process testing in parallel with coordination. For example, processing purchase orders and building items in Work in Process (WIP) can be performed independently. The only coordination is at the points when the processes intersect, such as outside processing of WIP operations or receiving material on a purchase order directly to WIP.

Documented User Procedures

The System Test Script (TE.040) is based on user procedures. The documented user procedures should also be available and used for reference (just as they would be available at each user's desk when the new system is in place).

Converted Data

Convert all or a part of the legacy system data for the system test. If you cannot convert all legacy data for the test, convert a representative sample. For example, if you are converting General Ledger balances, make sure that you include converted journals from each cost center, business unit, division, and so on. If you are converting vendors, make sure that the data includes vendors with multiple sites and multiple contacts, so that you can test full vendor-related functionality and conversion. Keep in mind that the system test serves as the final test of the conversion programs prior to transition.

Multiple Copies of Test Scripts

Typically, you execute the system test more than once. After completing a test execution (iteration), you should have a set of detailed and summary test results that have been recorded on the System Test

Script (TE.040). Between iterations you correct defects found during testing and prepare for the next iteration. When the next iteration is complete, you can compare results from each iteration to measure quality gains or losses, and use this information to manage quality and plan for additional test iterations, if necessary.

The System Test Specifications component of the System Test Script (TE.040) includes blank columns to record actual results and resolution notes. Keep a master copy of the scripts with these blank columns so you can make additional copies for repeated tests. Fill out the results by hand on the forms and file multiple copies of the same test together with an identifying iteration or cycle number. At the completion of the test, update the master soft copy with the final test result notes for each test step. Generate the Test Report for System Test template and summarize the results of performing the system test. Keep the final version of the System Test Script (TE.040) and file them with other project documentation.

Regression Testing

The regression test retests changes made to a modified application extension because of an enhancement or a correction to a coding error. The regression test gets its name from the principle that when the development of a system progresses, validation of that progression requires proof that all prior validations have not regressed.

In regression testing, you retest application extensions in order to validate that prior application extension defects have been corrected. The regression test generally re-executes the entire Unit Test Script (TE.020) and a selection of related Link Test Script (TE.030) to confirm that the overall quality of the software has improved. You perform regression testing in order to answer the questions:

- How do I prove that the defect has been fixed?
- How do I prove that it did not cause another defect somewhere else?

In development environments where libraries of shared objects exist and object sharing is encouraged, be careful to run *where used* reports that expose all application extensions affected by a defect in a common object. Your regression test needs to select test scripts that retest all of the application extensions that invoke the common module.

Determine whether your regression test should run concurrently with system, systems integration, and acceptance tests. Some projects use the regression test to stabilize application defects for the next iteration of the system test. This strategy allows the current system test to proceed to completion while the regression test stabilizes and fixes defects, thus preparing for the next iteration of the system test. Other projects use regression testing as the final test after fixing all faults.

Automated Regression Testing

Determine your level of regression testing automation. Regression test automation is now a very realistic option. Current tools feature the ability to record user actions and document GUI objects, thereby capturing this information into test scripts. You can then generalize the scripts to use variable data, enabling the script execution to be data-driven. If your project is considering automating its regression testing, clearly state your automation goals and be sure to plan extra time for script development and generalization. Test scripts now become a baselined item and you must take care to update the scripts each time the application is enhanced, due to approved change control, scope increase, and so on.

Document the extent of automation you expect to achieve by leveraging regression testing tools. Be careful not to overlook the overhead of automation, as you will need additional development time and skill to perform automated regression testing.

Test Condition Recreation

When attempting to validate that a defect has been fixed, be careful to recreate the same data conditions and sequence of events that exposed the original defect.

Converted Data

When testing with converted data, verify that the data was not the cause of the problem. Application extensions are usually designed to handle data conditions created by the application system they are part of. If the application extensions were not built with an emphasis on error-handling, introducing converted data can cause application extensions to fail.

Support Procedures

When you encounter problems, use the Production Support Infrastructure Design (PM.020) as a guide to test the organization's new internal support procedures. When these procedures do not resolve the problem, call Oracle Support (or other external support) and log the technical issue with them.

Testing Results Summarization

Use the Test Report for System Test template to document summary results from the system test. Log detailed results for each step on the System Test Specifications component of the System Test Script (TE.040). Summarize these results on the Test Report for System Test.

Test Results Acceptance

Follow the method outlined in the Project Management Plan (PJM.CR.010) for securing acceptance of system test results.

Linkage to Other Tasks

The System-Tested Applications are an input to the following tasks:

- CV.130 - Convert and Verify Data
- TE.120 - Perform Systems Integration Test
- PM.040 - Prepare Production Environment

Role Contribution

The percentage of total task time required for each role follow:

Role	%
Tester	70
Business Analyst	10
Developer	10
Technical Analyst	10
Business Line Manager	*

Table 8-26 Role Contribution for Perform System Test

* Participation level not estimated.

Deliverable Guidelines

Use the Test Report for Systems Test template to communicate the results of the system test to organization management, the project sponsor, and other stakeholders. The System-Tested Applications prove the successful operation and integration of the target applications system processes and this report shows the results of performing the system test.

This deliverable should address the following:

- application setup provides the business solution for supporting the future operational and financial needs of the organization
- integration between custom application extensions and standard Oracle Applications modules
- integration among Oracle Applications modules
- validation of converted data

Deliverable Components

The Test Report for Systems Test template consists of the following components:

- Introduction
- System Test Summary
- System Test Method
- System Test Environment
- System Test Results
- System Test Actions
- System Test Change Recommendations
- Roles and Responsibilities

Introduction

This component summarizes the project goals, test configurations, results, and recommendations.

System Test Summary

This component documents the purpose for testing, the scope of testing, and any known requirements and constraints.

System Test Method

This component documents the testing approach, types of tests performed, and some of the key terminology used during system testing.

System Test Environment

This component documents the platforms, operating system, and network configuration setup for the system test. Additionally, it documents the server architecture, application product setup parameters, and technical considerations.

System Test Results

This component documents the test results for each application tested, including testing failures, successes, timing, conclusions, and potential areas for future investigation.

System Test Actions

This component lists the action items necessary to resolve outstanding issues resulting from the testing process.

System Test Change Recommendations

This component identifies any recommendations for applications changes or corrections based on system testing.

Roles and Responsibilities

This component identifies the people who participated in the testing process and document their corresponding roles and responsibilities during testing.



Suggestion: To support your recommendations, attach the System Test Specifications component from the System Test Script (TE.040) with the results to your Test Report for System Test as an appendix.

Tools

Deliverable Template

Use the Test Report for Systems Test template to create the supporting deliverable for this task.

TE.120 - Perform Systems Integration Test (Optional)

In this task, you test the system's integration with other application systems in a production-like environment. The systems integration test is performed in a test environment.



If your project includes *interfaces* with external systems, you should perform this task.

Deliverable

The deliverable for this task is the **Integration-Tested System**. It validates the integration between the target application system and other systems and verifies that the new system meets defined interface requirements and supports execution of business processes.

Prerequisites

You need the following input for this task:



Manual Conversion Procedures (CV.050)

If your system test includes data that is converted manually, Manual Conversion Procedures provide information about manually converting data that will be used in the systems integration test. If Design Manual Conversion Procedures was not performed, this deliverable will not exist. (See the task description for CV.050 for more information on when this task should be performed.)



Validation-Tested Conversion Programs (CV.110)

After the converted data has passed the conversion validation test, the data is ready for use in the systems integration test. If Perform Conversion Validation Test was not performed, this deliverable will not exist. (See the task description for CV.110 for more information on when this task should be performed.)

☐ **User Reference Manual (DO.060)**

The User Reference Manual provides a detailed description of the use of each application extension that you test for validity during the systems integration test. If Publish User Reference Manual was not performed, this deliverable will not exist. (See the task description for DO.060 for more information on when this task should be performed.)

☐ **User Guide (DO.070)**

The User Guide provides a description of responses to business events that you test for validity during the systems integration test. If Publish User Guide was not performed, this deliverable will not exist. (See the task description for DO.070 for more information on when this task should be performed.)

☐ **System Management Guide (DO.090)**

The System Management Guide is a compilation of procedures for operating the application system and interfacing with other systems that you test for validity during the systems integration test. If Publish System Management Guide was not performed, this deliverable will not exist. (See the task description for DO.090 for more information on when this task should be performed.)

☐ **Testing Requirements and Strategy (TE.010)**

The Testing Requirements and Strategy provides the testing approach and testing requirements and strategy for business system testing.

☐ **Systems Integration Test Script (TE.050)**

The Systems Integration Test Script includes test sequences, test specifications, and data profiles. If Develop Systems Integration Test Script was not performed, this deliverable will not exist. (See the task description for TE.050 for more information on when this task should be performed.)

☐ **Testing Environments (TE.060)**

You can use the Testing Environment to perform the integration test or you may choose to have a separate environment to perform systems integration testing.

☐ **Prepared Key Users (TE.100)**

Key users trained in specific areas of responsibility should execute the Systems Integration Test Script (TE.050).

☐ **System-Tested Applications (TE.110)**

Verify that all Oracle Applications modules and application extensions within the target system have been system tested prior to testing integration to external systems.

☐ **Production Support Infrastructure Design (PM.020)**

As you encounter problems during the systems integration test, you should reference and validate the new internal support procedures.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the User Guide (DO.070).	
2.	Review the User Reference Manual (DO.060).	
3.	Review the System Management Guide (DO.090).	
4.	Review the Systems Integration Test Script (TE.050).	
5.	Generate the test report for systems integration test template and document systems integration testing overview, testing summary, and test method.	Introduction; Systems Integration Test Summary; Systems Integration Test Method

No.	Task Step	Deliverable Component
6.	Review responsibilities for the systems integration test with the team.	Test Participants: Roles and Responsibilities
7.	Validate the system's integration test environment.	Systems Integration Test Environment
8.	Execute the Systems Integration Test Script (TE.050).	
9.	Document the detailed test results.	Systems Integration Test Specifications (TE.050)
10	Summarize the test results.	Systems Integration Test Results
11.	Determine required actions for problem resolution.	Systems Integration Test Actions
12.	Make recommendations for applications changes or corrections based on systems integration testing.	Systems Integration Test Change Recommendations
13.	Secure acceptance of the systems integration test results.	Acceptance Certificate (PJM.CR.080)

Table 8-27 Task Steps for Perform Systems Integration Test

Approach and Techniques

The purpose of the systems integration test is to test the operation of the business system across and between application systems. Use this task to develop user confidence in the overall usability of the system, including interfaces to external systems. It is important to test operating routines and procedures as well as computer programs, thereby simulating business operations, not just systems.

The Integration-Tested System indicates that all interfaces between the target applications and the legacy and third-party systems are functioning properly. The figure below illustrates interfaces between Oracle Applications and external systems. The interfaces between systems are the objective of the systems integration test.

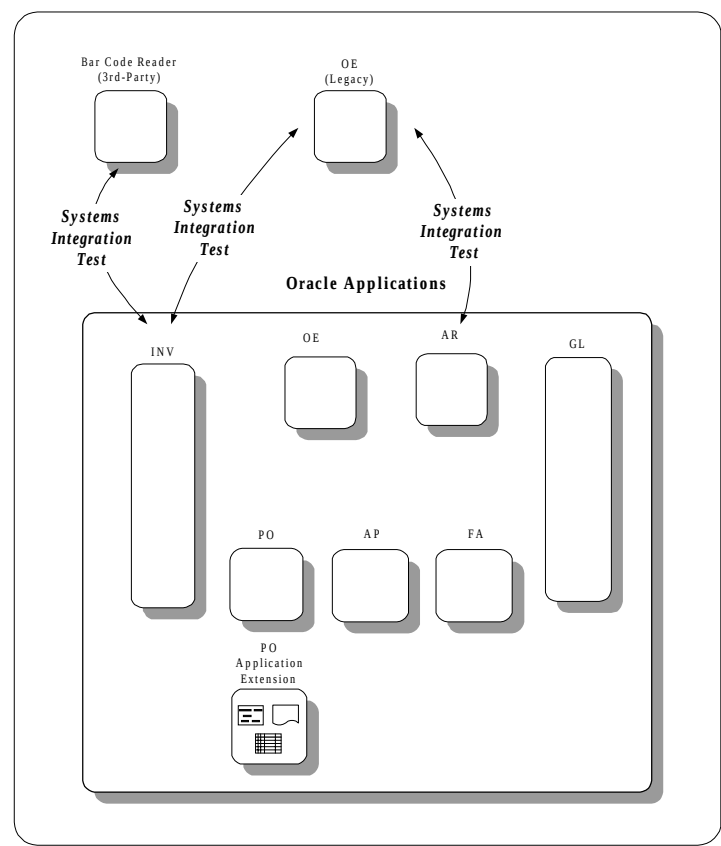


Figure 8-13 Systems Integration Test Diagram

Results from the systems integration test are useful for implementing changes to the interfaces between the application systems. The audience for this task is the project team and steering committee. In addition to executing the Systems Integration Test Script (TE.050), you also generate the Test Report for Systems Integration Test template. This deliverable should contain the following:

- summary (including statistics) of successes of tests organized by process
- testing method overview
- critical success factors for certifying the system based on testing
- changes to acceptance criteria based on the new factors or changes to the business system
- impact assessment of changing procedures and policy
- actions and assignments relating to changes to procedures and policies
- summary of retraining required to support policies and procedures changes
- highlights of critical failure in code
- actions, assignments, and estimates relating to the rework of code
- summary of programming changes
- summary of repeating select business systems tests based on these changes
- completed test scripts as an appendix
- list of open technical issues communicated with Oracle Support

The repetitive nature of systems integration tests allows the project team to test each systems integration flow and incrementally improve the system interfaces until they reach the desired level of success. Test each interface point until you achieve success.

Successful systems integration testing should verify the following:

- The integration points (interfaces) can successfully support business operations across and between application systems.
- All integration test results are predictable and repeatable.

- Variations of the test scripts produce expected results.
- Data are verifiable through alternative means across the business systems.
- The support procedures intended for production are accurate.
- Modifications to the procedures, policy, and system are fully implemented.

Execute the procedures and steps as recorded in the prerequisite deliverables to perform the systems integration test. When the test results comply with the expected results, your job is complete. Typically, you execute the systems integration test more than once. After completing a test execution (iteration), you should have a set of detailed and summary test results. Between iterations, correct defects found during testing and prepare for the next iteration. When the next iteration is complete, you can compare results from each iteration to measure quality gains or losses. Use this information to manage quality and plan for additional test iterations, if necessary.

Testing Results Summarization

Use the Test Report for Systems Integration Test template to document both detailed and summary results from the systems integration test. Log detailed results for each step of the test sequence or test script. Log defects for each step where there is a discrepancy between the actual and expected step results. Summary results use detailed test results to produce summaries by test type and by test iteration.

Test Results Acceptance

Follow the method outlined in the Project Management Plan (PJM.CR.010) for securing acceptance of systems integration test results.

Linkage to Other Tasks

The Integration-Tested Systems are an input to the following tasks:

- TE.130 - Perform Acceptance Test
- AP.160 - Prepare User Learning Environment
- PM.040 - Prepare Production Environment

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Tester	70
Business Analyst	10
Developer	10
Technical Analyst	10

Table 8-28 Role Contribution for Perform Systems Integration Test

Deliverable Guidelines

Use the Test Report for Systems Integration Test template to record and communicate the results of the systems integration test. This report includes a summary of the goals and objectives of the test, the business scope, a description of the test scenarios and processes, the final results, status, and recommendations.

This deliverable should address the following:

- the test results from performing systems integration testing
- the resolution of any testing irregularities found during systems integration testing

Deliverable Components

The Test Report for Systems Integration Test template consists of the following components:

- Introduction
- Systems Integration Test Summary
- Systems Integration Test Method
- Systems Integration Test Environment

- Systems Integration Test Results
- Systems Integration Test Actions
- Systems Integration Test Change Recommendations
- Test Participants: Roles and Responsibilities

Introduction

This component lists the systems integration test goals, test configurations, results, and recommendations.

Systems Integration Test Summary

This component records the purpose for testing, the scope of testing, and any known requirements and constraints.

Systems Integration Test Method

This component documents the testing approach, types of tests performed, and some of the key terminology used during system testing.

Systems Integration Test Environment

This component documents the platforms, operating system, and network configuration. In addition, it addresses the server architecture, application product setup parameters, and technical considerations.

Systems Integration Test Results

This component describes the test results for each application tested, including testing failures, successes, timing, conclusions, and potential areas for future investigation.

Systems Integration Test Actions

This component lists the action items for resolving any outstanding issues that resulted from the systems integration testing.

Systems Integration Test Change Recommendations

This component lists recommendations for applications changes or corrections based on systems integration testing.



Suggestion: To support your recommendations, attach the Systems Integration Test Specifications component from the Systems Integration Test Script (TE.050) with the results to your Test Report for Systems Integration Test as an appendix.

Test Participants: Roles and Responsibilities

This component identifies the people who participated in the systems integration test and their corresponding roles and responsibilities during testing.

Tools

Deliverable Template

Use the Test Report for Systems Integration Test template to create the supporting deliverable for this task.

TE.130 - Perform Acceptance Test (Core)

In this task, you support users in performing their acceptance test of the new production system. The acceptance test is performed in the Production Environment. This task also involves scheduling the acceptance test team, support staff, and user facilities.

Deliverable

The deliverable for this task is the **Acceptance Test Results**. These results provide evidence that the new system meets the acceptance criteria as defined in the Project Management Plan (PJM.CR.010).

Prerequisites

You need the following input for this task:

☐ **Project Management Plan [initial complete] (PJM.CR.010)**

The Project Management Plan provides an overall strategy of testing for the project and an approach for acceptance of test results. It includes the following:

- test management — an overall strategy of testing for the project including test strategy, test stages, and test procedure
- acceptance approach and criteria — the method you will use to secure acceptance and the criteria used to accept the acceptance test results

☐ **Converted and Verified Data (CV.130)**

If there is data to be converted, then the acceptance test should include data that has been converted and verified. If Convert and Verify Data was not performed, this deliverable will not exist. (See the task description for CV.130 for more information on when this task should be performed.)

☐ **Integration-Tested System (TE.120)**

Once the systems integration test is complete, the acceptance test begins. The acceptance test includes various tests. One of these tests includes the validation of all system integration points.

☐ **Skilled Users (AP.170)**

Users receive training on the new system before they perform the acceptance test. The system and database administrators who know how to support the system, as well as designers who can provide functional and technical support for the tests should also receive training.

☐ **Transition and Contingency Plan (PM.030)**

The Transition and Contingency Plan states what to do if the acceptance criterion is not met during the acceptance test.

☐ **Production Environment (PM.040)**

The Production Environment needs to be fully configured and ready for the testers to begin their acceptance test. This includes, platforms, software, test data, and documentation.

Task Steps

The steps for this task are as follows:

No.	Task Step	Deliverable Component
1.	Review the acceptance criteria from the Project Management Plan (PJM.CR.010).	
2.	Select the System Test Script (TE.040) and the Systems Integration Test Script (TE.050) that reflect the acceptance criteria.	

No.	Task Step	Deliverable Component
3.	Review the System Test Script (TE.040) and the Systems Integration Test Script (TE.050).	
4.	Generate the Test Report for Acceptance Test template and document acceptance testing Introduction, test summary, test method, and test environment.	Introduction; Acceptance Test Summary; Acceptance Test Method; Acceptance Test Environment
5.	Review responsibilities for the acceptance test with the team.	Test Participants: Roles and Responsibilities
6.	Back-up the Production Environment for later availability	
7.	Validate the acceptance test environment.	
8.	Execute the acceptance test script.	
9.	Document the test results.	Acceptance Test Results
10.	Summarize the test results.	Acceptance Test Results
11.	Determine required actions for problem resolution.	Acceptance Test Actions
12.	Make recommendations for application changes or corrections based on acceptance testing.	Acceptance Test Change Recommendations
13.	Secure acceptance of acceptance test results.	Acceptance Certificate (PJM.CR.080)

No.	Task Step	Deliverable Component
14	Secure acceptance confirming that the acceptance test included criteria for Century Date compliance testing.	Acceptance Certificate (PJM.CR.080)

Table 8-29 Task Steps for Perform Acceptance Test

Approach and Techniques

Provide support, as needed, to key users while they test the new system for acceptance. The amount of acceptance testing is dependent on the specific acceptance criteria defined in the Project Management Plan (PJM.CR.010). The scope of your acceptance testing may change if users are participating substantially in Perform System Test (TE.110) or Perform Systems Integration Test (TE.120).

Conduct acceptance testing using the testing guidelines and standards adopted by the project team and referenced in the Testing Requirements and Strategy (TE.010). Successful acceptance test results should reflect the acceptance criteria defined in the Project Management Plan (PJM.CR.010).

Century Date Compliance

In the past, two-character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four-character date encoding. As the year 2000 approached, it became evident that a full four-character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

Business System Testing should include test scripts and test case scenarios that check for Century Date compliance of customizations and custom interfaces. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Testing Results Summarization

Record and communicate the results of the acceptance test in the Test Report for Acceptance Test. This report includes a summary of the goals and objectives of the test, the business scope, a description of the test scenarios and processes, the final results, status, and recommendations.

In addition to collecting and organizing the completed acceptance test script, this deliverable should contain:

- summary (including statistics) of successes of tests organized by process
- testing method overview
- critical success factors for certifying the system based on testing changes to acceptance criteria based on the new factors or changes to the business system
- impact assessment of changing procedures and policy
- actions and assignments relating to changes to procedures and policies
- summary of retraining required to support policies and procedures changes
- highlights of critical failure in code
- actions, assignments, and estimates relating to the rework of code
- summary of programming changes
- summary of repeating select business systems tests based on these changes
- completed test scripts as an appendix
- a list of open technical issues that have been communicated to Oracle Support

The acceptance test verifies that the new application system includes all of the required functionality. Key users from the project team previously performed this verification during system testing, but it is a basic requirement for the users to accept the new system.

Contractors

For this task, the role of any contractors on an implementation project is to support the organization's key users while they perform the acceptance test. Contractors should not perform the acceptance test as it is ultimately the organization that must verify, through their testing efforts, that the new system meets the predefined acceptance criteria.

Staff

The system administrators provide support for the acceptance test environment while the users perform the user tests. The staff who use the system most should participate in the acceptance test to gain familiarity with the new system so that they better understand the functionality that is being tested. The acceptance test team members must dedicate themselves to testing.

Technical analysts from the project team should be available to resolve technical and functional issues and questions but they should not participate directly in the acceptance test.

Criteria

Acceptance testing consists of performing the tests and verifying the results against the acceptance criteria specified in the Project Management Plan (PJM.CR.010). The acceptance test may cover any aspect of the new system, including administrative procedures (such as backup and recovery). The acceptance test is a verification — it is not an opportunity for the users to indicate what they might like the system to do.

The tests are successful if they meet the acceptance criteria. Ideally, the acceptance criteria should match what the users think the system should do; however, the acceptance test should only be validating against predefined acceptance criteria, and not what the users wish the new system would do.

The results of all tests are recorded and reviewed as part of the acceptance test process. Logging all tests allows management to assess the completeness of the acceptance test, as well as the results.

Issue Resolution

Set up a central help desk to provide support for the testers and review any issues raised during testing. Staff the help desk with transition team members. The transition team should verify problems before they are accepted as problems. Log and handle verified problems according to the established procedures defined in Problem Management (PJM.CR.050).

Facilities

Conduct the test in a facility that is separate from the users' normal work area so that daily work does not interfere with the test. In addition, group everyone in an isolated facility to allow for easier communication regarding problems and their resolutions.

Scheduling

A successful acceptance test requires dedicated resources for conducting the acceptance test. Schedule the test far in advance so that managers will allow users the time off to perform their acceptance test. By scheduling the acceptance test far in advance, the users can make the necessary adjustments to cover their normal work duties.

Techniques

One technique for performing the acceptance test is to run the new system and the old system in parallel. Enter all data and changes into both systems. Then run reports to verify that the new system provides the same results as the old system.

You may also perform one or more dry-runs (the business practices the cutover to production operations on the new system). System administrators practice data conversion and the users then start using the system as if it were in production. Simulate a full day's production to verify that no issues were overlooked. This test verifies that the staff is prepared to use the new system.

Schedule dry-runs as far in advance as possible so that the business has time to allocate the proper resources.

Test Results Acceptance

Follow the method outlined in the Project Management Plan (PJM.CR.010) for securing acceptance of acceptance test results.

Linkage to Other Tasks

The Acceptance Test Results are an input to the following task:

- PM.070 - Verify Production Readiness

Role Contribution

The percentage of total task time required for each role follows:

Role	%
Tester	50
Business Analyst	10
Technical Analyst	10
Database Administrator	10
Developer	10
System Administrator	10
Business Line Manager	*
Client Project Manager	*
Client Staff Member	*
Key User	*

Table 8-30 Role Contribution for Perform Acceptance Test

* Participation level not estimated.

Deliverable Guidelines

Use the Test Report for Acceptance Test template to validate, from a user standpoint, that the system meets the acceptance criteria developed and documented in the Project Management Plan (PJM.CR.010). This task confirms user confidence in the overall usability of the system.

This deliverable should address the following:

- the results of the acceptance tests
- specific criteria met to validate readiness of production system

Deliverable Components

The Test Report for Acceptance Test template consists of the following components:

- Introduction
- Acceptance Test Summary
- Acceptance Test Method
- Acceptance Test Environment
- Acceptance Test Results
- Acceptance Test Actions
- Acceptance Test Change Recommendations
- Test Participants: Roles and Responsibilities

Introduction

This component summarizes the project goals, acceptance test configurations, results, and recommendations.

Acceptance Test Summary

This component documents the purpose of the acceptance test, the scope of acceptance testing, and any known requirements and constraints.

Acceptance Test Method

This component documents the testing approach, types of tests performed, and some of the key terminology used during acceptance testing.

Acceptance Test Environment

This component documents the platforms, operating system, and network configuration of the new production environment used to perform the acceptance test, including the server architecture, application product setup parameters, and technical considerations.

Acceptance Test Results

This component describes the test results for each application tested, including testing failures, successes, timing, conclusions, and potential areas for future investigation.

Acceptance Test Actions

This component lists the action items for resolving any outstanding issues that resulted from the acceptance testing.

Acceptance Test Change Recommendations

This component documents the recommendations for applications changes or corrections based on systems integration testing.



Suggestion: Attach the documented test specifications and results to the Test Report for Acceptance Test as an appendix to support your recommendations.

Test Participants: Roles and Responsibilities

This component identifies the people who participated in the acceptance testing process and documents their corresponding roles and responsibilities during testing.

Tools

Deliverable Template

Use the Test Report for Acceptance Test template to create the supporting deliverable for this task.

Century Date Compliance

To document the review and approval of the Acceptance Test Results for Century Date compliance considerations, prepare a separate Acceptance Certificate (PJM.CR.080) and replace the standard acceptance language with the following:

The above deliverable has been reviewed by <Company Long Name> and fully meets the Century Date compliance objectives expressed by <Company Short Name> and <Client Project Manager> and passes the acceptance criteria established for Century Date support specified by <Company Long Name>.

After obtaining acceptance and appropriate signatures, this Century Date Compliance Certificate should be filed with the deliverable in the project library.

